Outline

- Handling exceptional cases: errno, assert, longjmp
- Goals of exception handling
- Handlers and exceptions
- Standard exceptions
- Exception safe programming
- C++11 noexcept
- Exception_ptr, nested_exceptions

C Errno

```
#include <errno.h>
#include <stdio.h>   /* fopen */
#include <string.h>  /* strerror */

struct record { ... };
struct record rec;

extern int errno;  /* preprocessor macro: thread-local since C11 */
int myerrno;       /* my custom error code */

FILE *fp;

if ( NULL == (fp = fopen( "fname", "r")) ) /* try to open the file */
{
    fprintf( stderr, "can't open file %s\n", "fname");
    fprintf( stderr, "reason: %s\n", strerror(errno)); /* perror(NULL) */
    myerrno = 1;
}
else if ( ! fseek( fp, n*sizeof(rec), SEEK_SET) ) /* pos to record */
{
    fprintf( stderr, "can't find record %d\n", n);
    myerrno = 2;
}
else if ( 1 != fread( &r, sizeof(r), 1, fp) ) /* try to read a record */
{
    fprintf( stderr, "can't read record\n");
    myerrno = 3;
}
else   /* everything was successful up to now */
{
    ...
}
```

C++ Errno

```
#include <cerrno>
#include <cstdio>   // std::fopen
#include <cstring>  // std::strerror

struct record { ... };
struct record rec;

extern int errno;  /* preprocessor macro: thread-local since C++11 */
int myerrno;       /* my custom error code */

std::FILE *fp;

if ( NULL == (fp = std::fopen( "fname", "r")) ) /* try to open the file */
{
    std::fprintf( stderr, "can't open file %s\n", "fname");
    std::fprintf( stderr, "reason: %s\n", std::strerror(errno)); /* perror(NULL) */
    myerrno = 1;
}
else if ( ! std::fseek( fp, n*sizeof(rec), SEEK_SET) ) /* pos to record */
{
    std::fprintf( stderr, "can't find record %d\n", n);
    myerrno = 2;
}
else if ( 1 != std::fread( &r, sizeof(r), 1, fp) ) /* try to read a record */
{
    std::fprintf( stderr, "can't read record\n");
    myerrno = 3;
}
else   /* everything was successful up to now */
{
    ...
}
```

Iostream error handling

```
void f()
{
    std::ifstream file("input.txt");

    if ( ! file )    /* before C++11: void*, since C++11: bool */
    {
        std::cerr << "file opening failed\n";
        return;
    }

    for( int n; file >> n; )  /* while ( ! cin.fail() ) */
    {
        std::cout << n << '\n';
    }

    if ( file.bad() )
    {
        std::cerr << "i/o error while reading\n";
    }
    else if ( file.eof() )
    {
        std::cerr << "eof reached\n";
    }
    else if ( file.fail() )
    {
        std::cerr << "non-integer\n";
    }
}
```

Assert

```
#include <cassert>   /* assert.h in C */

void open_file(std::string fname)
{
   assert(fname.length() > 0);


   std::ifstream f(fname.c_str());
   . . .
}
```

- Run-time error!

Static assert (C++11)

```
#include <type_traits>

template <typename T>
void swap(T &x, T &y)
{
   static_assert( std::is_nothrow_move_constructible<T>::value, &&
                  std::is_nothrow_move_assignable<T>::value, "Swap may throw" );

   auto tmp = x;
   x = y;
   y = tmp;
}
```

```
#if __STDC_HOSTED__ != 1
#  error "Not a hosted implementation"
#endif

#if __cplusplus >= 202302L
#  warning "Using #warning as a standard feature"
#endif
```

Goals of exception handling

- Type-safe transmission of arbitrary data from throw-point to handler
- Every exceptions should be caught by the appropriate handler
- No extra code/space/time penalty if not used
- Grouping of exceptions
- Work fine in multithreaded environment
- Cooperation with other languages (like C)

Setjmp/longjmp

```c
#include <setjmp.h>
#include <stdio.h>

jmp_buf x;

int main()
{
    int i = 0;

    if ( (i = setjmp(x)) == 0 ) // try
    {
        f();
    }
    else    // catch
    {
        switch( i )
        {
        case  1: handler1(); break;
        case  2: handler2(); break;
        default: fprintf( stdout, "error code = %d\n", i); break;
        }
    }
    return 0;
}
```

```c
// perhaps in another source file

#include <setjmp.h>
extern jmp_buf x;

void f()
{
    // ...
    g();
}

void g()
{
    if ( something_wrong() )
    {
        longjmp(x,2);  // throw
    }
}
```

Setjmp/longjmp

```c
#include <setjmp.h>
#include <stdio.h>

jmp_buf x;

int main()
{
    int i = 0;

    if ( (i = setjmp(x)) == 0 ) // try
    {
        f();
    }
    else    // catch
    {
        switch( i )
        {
        case  1: handler1(); break;
        case  2: handler2(); break;
        default: fprintf( stdout, "error code = %d\n", i); break;
        }
    }
    return 0;
}
```

```c
// perhaps in another source file

#include <setjmp.h>
extern jmp_buf x;

void f()
{
    // ...
    g();
}

void g()
{
    if ( something_wrong() )
    {
        longjmp(x,2);  // throw
    }
}
```

Exceptions in C++

- ```
  try  // dangerous area
  {
    f();   // someth
  }
  catch (T1 e1) { /* handler for T1 */ }
  catch (T2 e2) { /* handler for T2 */ }
  catch (T3 e3) { /* handler for T3 */ }

  void f()
  {
    //...
    T e;
    throw e;    /* throws exception of type T */

    // or:
    throw T();  /* throws default value of T */
  }
  ```
  ```

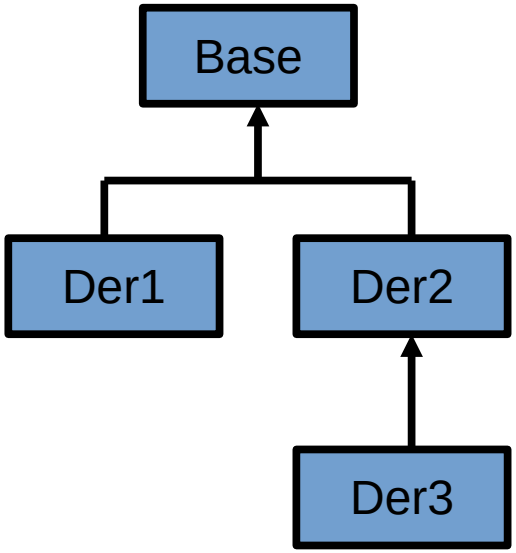
Which handler?

A handler of type H catches the exception of type E if

- H and E is the same type
- H is unambiguous base type of E
- H and E are pointers or references and some of the above stands

Exception hierarchies

- Resolved run-time
- Not overloading!




```
class Base { … };
class Der1 : public Base { … };
class Der2 : public Base { … };
class Der3 : public Der2 { … };

try
{
  f();
  // …
}
catch (Der3 &e1)  { /* handler for Der3 */ } /* the most derived handler first */
catch (Der2 &e2)  { /* handler for Der2 */ }
catch (Der1 &e3)  { /* handler for Der1 */ }
catch (Base &e3)  { /* handler for Base */ } /* the base handler last */

void f()
{
  if ( ... )
    throw Der3();  /* throw the most derived type */

}
```

Exception hierarchies 2

- Multiple inheritance is possible
- And sometimes even useful

```
class net_error { ... };
class file_error { ... };

class nfs_error : public net_error, public file_error { ... };

void f()
{
    try
    {
        …
    }
    catch( nfs_error nfs ) { ... }
    catch( file_error fe ) { ... }
    catch( net_error ne )  { ... }
}
```

Re-throw

- Re-throw works only inside a catch block
- Re-throws the original object, not the (possible sliced) one

```
class Base { … };
class Der1 : public Base { … };

void g()
{
  throw Der1;  // throw derived exception Der1
}

void f()
try  // function block itself can be try block
{
  g();
}
catch ( Base b )  // catch Base by value: copied, since C++11 can be moved
{
  must_do_at_exception(b);
  throw;  // re-throw original exception Der1
}
catch ( ... )  // catch all
{
  must_do_at_exception();
  throw;  // re-throw original exception
}
```

Exception hierarchies 3

```
#include <stdexcept>
struct matrixError // perhaps member in class Matrix
{
    matrixError( std::string r) : reason(r) { }
    std::string reason;
    virtual ~matrixError() { }
};
struct indexError : public matrixError, public std::out_of_range
{
    indexError( int i, const char *r="Bad index") :
matrixError(r),out_of_range(r),index(i)
    {
        std::ostringstream os;
        os << index;
        reason += ", index = ";
        reason += os.str();
    }
    const char *what() const noexcept override
    {
        return reason.c_str();
    }
    virtual ~indexError() { }
    int index;
};
struct rowIndexError : public indexError
{
    rowIndexError(int i) : indexError( i, "Bad row index") { }
};
struct colIndexError : public indexError
{
    colIndexError(int i) : indexError( i, "Bad col index") { }
};
```

- Catch by:
  - std::logic_error
  - indexError
  - Etc...

std exception hierarchy

```
class exception {};                                  // in <exception>

class bad_exception : public exception {};           // calls unexpected()
class bad_weak_ptr : public exception {};            // C++11 weak_ptr -> shared_ptr
class bad_function_call : public exception {}; // C++11 function::operator()
class bad_typeid : public exception {};              // typeid(0)
class bad_cast   : public exception {};              // dynamic_cast
  class bad_any_cast : public bad_cast {};           // C++17
class bad_variant_access : exception {}              // C++17
class bad_optional_access : exception {}             // C++17
class bad_alloc  : public exception {};              // new  <new>
  class bad_array_new_length : bad_alloc {}          // C++11, new T[-1]

class logic_error : public exception {};
  class domain_error : public logic_error {}; // domain error,std::sqrt(-1)
  class invalid_argument : public logic_error {}; // bitset char != 0 or 1
  class length_error : public logic_error {}; // length str.resize(-1)
  class out_of_range : public logic_error {}; // bad index in container or string
  class future_error : public logic_error {}; // C++11: promise abandons the shared state

class runtime_error : public exception {};
  class range_error     : public runtime_error {}; // floating point ovf or unf
  class overflow_error  : public runtime_error {}; // int overflow INT_MAX+1
  class underflow_error : public runtime_error {}; // int underflow INT_MIN-1
  class system_error    : public runtime_error {}; // e.g. std::thread constr.
    class ios_base::failure : public system_error {}; // C++11
    class filesystem::filesystem_error : public system_error {}; // C++17
    class nonexistent_local_time : public system_error // C++20
    class ambigous_local_time : public system_error    // C++20
    class format_error : public system_error           // C++20
```

Exception specification before C++11

```
class E1;
class E2;

void f() throw(E1)  // throws only E1 or subclasses
{
   ...
   throw E1();     // throws exception of type E1
   ...
   throw E2();     // calls unexpected() which calls terminate()
}



// same as:

void f()
try {
   ...
}
catch(E1)  { throw; }
catch(...) { std::unexpected(); }
```

- Unexpected before C++17 != unexpected since C++23
- Deprecated since C++11, removed since C++17

Exception specification before C++11

```
class E1;
class E2;

void f() throw(E1, std::bad_exception)  // throws only E1 or subclasses
{
   ...
   throw E1();     // throws exception of type E1
   ...
   throw E2();     // calls unexpected() which throws bad_exception()
}

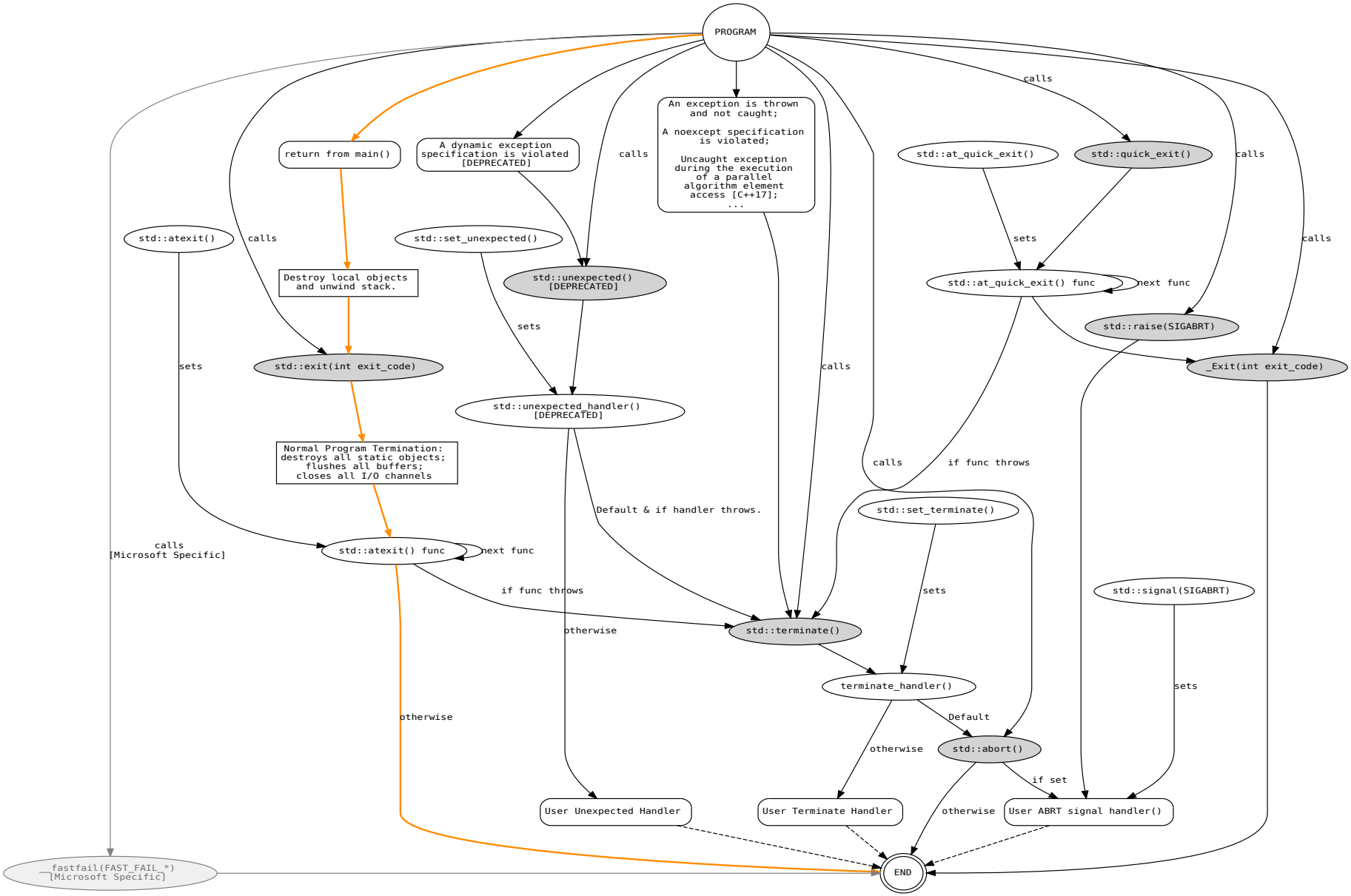
typedef void (*terminate_handler)();
terminate_handler set_terminate(terminate_handler);
terminate_handler get_terminate();   // C++11

// until C++17
typedef void (*unexpected_handler)();
unexpected_handler set_unexpected(unexpected_handler);
unexpected_handler get_unexpected(); // C++11
```

- `void f() throw()   // does not throw, can be optimized`

- Deprecated since C++11, removed since C++17

Program termination



https://github.com/adishavit/Terminators/blob/master/README.md

Noexcept specifier in C++11

- Since C++11
  - Part of function type since C++17
  - But not part of of function signature (so no overload on noexcept)
- C++ functions are either non-throwing or potentially throwing
- C++ cannot execute full compile time check on possible exceptions
  - Partially due to possible call of non C++ functions
- Destructors, deallocation functions are non-throwing
- Implicitly declared or defaulted default-, copy- and move constructors, copy, move oper.
  - Unless base class or called operations throw
- Noexcept is important for optimizations and program safety

```
void f() noexcept(expr) { }
void f() noexcept(true) { }
void f() noexcept       { }  // noexcept(true)
```

Noexcept operator in C++11

- **bool noexcept(expr);**
- Can be used inside function template noexcept specifier
- Compile-time check: does not evaluate *expr* (like **sizeof**)
- False if
  - Expr throws
  - Expr has dynamic_cast or typeid
  - Has function which is no noexcept(true) and not constexpr
- Otherwise **true**

```
template <typename T>
void f() noexcept ( noexcept( T::g() ) )
{
    T::g();
}
```

Destructors

Destructors must not throw!

- Exception thrown during exception triggers **std::terminate()**
- Since C++11 every destructor is implicit **noexcept**
- It is possible to declare the destructor as **noexcept(false)**
- If exception is thrown during stack unwinding: **std::terminate** is called instead
- But the real situation is more complex:

https://akrzemi1.wordpress.com/2011/09/21/destructors-that-throw/

Exception safety

```
class T1 { ... };
class T2 { ... };

template <typename T1, typename T2>
void f( T1*, T2*);

void g()
{
    f( new T1(), new T2() );
    // ...
}
```

Scenario1 (before C++17)

Allocates memory for T1
Allocates memory for T2
Constructs T1
Constructs T2
Calls f

Scenario2

Allocates memory for T1
Constructs T1
Allocates memory for T2
Constructs T2
Calls f

Exception safety

- Usually solved by rearranging the source and use sequence points
- Since C++17, parameter evaluation order changed

undefined -> unspecified

```
class T1 { ... };
class T2 { ... };

template <typename T1, typename T2>
void f( std::unique_ptr<T1>, std::unique_ptr<T1>);



void g()
{
   std::unique_ptr<T1> ptr1(new T1());
   std::unique_ptr<T2> ptr2(new T2());

   f( ptr1, ptr2 );
   // ...
}
```

Exception safety in STL

- **Basic guarantee:** no memory leak or other resource issue
- **Strong guarantee:** the operation is atomic:
  it either succeeds or has no effect

  e.g. push_back() for vector, insert() for assoc. cont.
- **Nothrow guarantee:** the operation does not throw

  e.g. pop_back() for vector, erase() for assoc. cont., swap()

Strong guarantee

```
template <class T>
class Vec
{
  // ...
private:
  size_t cap;
  size_t sz;
  T     *v;
};

Vec<T>& Vec<T>::operator=( const Vec& rhs)
{
  if ( this != &rhs )
  {
    delete[] v;
    cap = sz = rhs.sz;
    v = new T[cap];
    for (size_t i = 0; i < cap; ++i)
      v[i] = rhs.v[i];
  }
  return *this;
}
```

Strong guarantee

```
template <class T>
class Vec
{
  // ...
private:
  size_t cap;
  size_t sz;
  T     *v;
};

Vec<T>& Vec<T>::operator=( const Vec& rhs)
{
  if ( this != &rhs )
  {
    delete[] v;
    cap = sz = rhs.sz;
    v = new T[cap];
    for (size_t i = 0; i < cap; ++i)
      v[i] = rhs.v[i];
  }
  return *this;
}
```

```
  if ( this != &rhs )
  {
    Vec tmp(rhs);
    cap = sz = tmp.sz;
    std::swap( v, tmp.v);
  }
  // tmp deleted here with the old buffer

  return *this;
}
```

Exception safety in STL

	vector	deque	list	map
clear()	nothrow(copy)	nothrow(copy)	nothrow	nothrow
erase()	nothrow(copy)	nothrow(copy)	nothrow	nothrow
insert() one	strong(copy)	strong(copy)	strong	strong
insert() more	strong(copy)	strong(copy)	strong	strong
merge()			nothrow(comp)	
push_back()	strong	strong	strong	
push_front()		strong	strong	
pop_back()	nothrow	nothrow	nothrow	
pop_front()		nothrow	nothrow	
remove()			nothrow(comp)	
remove_if()			nothrow(pred)	
reverse()			nothrow	
splice()			nothrow	
swap()	nothrow	nothrow	nothrow	nothrow(cp,co)
unique()			nothrow(comp)	

Some new features in C++11

- **class exception_ptr** smart pointer type,default constructible, copyable, == if null or points to the same
- **make_exception_ptr(E e)** creates an exception_ptr pointing to the exception object **e**.
- **current_exception()** null ptr if called outside of exception handling or it returns an exception_ptr pointing to the current exception
- **rethrow_exception(std::exception_ptr p)** rethrow exception **p**
- **class nested_exception** polymorphic mixin class capture and store current exception
  has **rethrow_nested() const** member function
- **throw_with_nested(T&& t)**
  throw_if_nested(const E& e)

exception_ptr

```
#include <iostream>
#include <string>
#include <exception>
#include <stdexcept>

void handle_eptr(std::exception_ptr eptr) // passing by value is ok
{
    try
    {
        if (eptr != std::exception_ptr())
        {
            std::rethrow_exception(eptr);
        }
    }
    catch(const std::exception& e)
    {
        std::cout << "Caught exception \"" << e.what() << "\"\n";
    }
}

int main()
{
    std::exception_ptr eptr;
    try
    {
        std::string().at(1); // this generates an std::out_of_range
    }
    catch(...)
    {
        eptr = std::current_exception(); // capture
    }
    handle_eptr(eptr);
} // destructor for std::out_of_range called here, when the eptr is destructed

// output: Caught exception "basic_string::at"
```

Nesting exceptions

```
#include <iostream>
#include <stdexcept>
#include <exception>
#include <string>
#include <fstream>

void print_exception(const std::exception& e, int level =  0) // prints the string of an exception.
{                 I                                            // if nested, recurses
    std::cerr << std::string(level, ' ') << "exception: " << e.what() << '\n';
    try {
        std::rethrow_if_nested(e);
    } catch(const std::exception& e) {
        print_exception(e, level+1);
    } catch(...) {}
}
void open_file(const std::string& s)    // catches an exception and wraps it in a nested exception
{
    try {
        std::ifstream file(s);
        file.exceptions(std::ios_base::failbit);
    } catch(...) {
        std::throw_with_nested( std::runtime_error("Couldn't open " + s) );
    }
}
void run() // sample function that catches an exception and wraps it in a nested exception
{
    try {
        open_file("nonexistent.file");
    } catch(...) {
        std::throw_with_nested( std::runtime_error("run() failed") );
    }
}
int main() // runs the sample function above and prints the caught exception
try {                                                // exception: run() failed
      run();                                         //  exception: Couldn't open nonexistent file
    } catch(const std::exception& e) {               //   exception: basic_ios::clear
        print_exception(e);

}
```

Optional (C++17)

- Maybe monad implementation
- Replaces return types like **std::pair<T,bool>**
- Optional contains value
  - Initialized/assigned with value of T
  - Initialized/assigned with **optional<T>** which contains value
- Optional does not contain value
  - Default initialized or initialized with value of **std::nullopt_t**
  - Initialized/assigned with **optional<T>** which does not contain value
- If **optional<T>** contains a value, than it is allocated as T
  - Not a pointer based heap storage
- Convertible to **bool**: true if contains value
- No optional reference

std::optional

```
std::optional<int> convert( const std::string& s)
try
{
    return std::stoi(s);  // C++11
}
catch (std::invalid_argument e)  // s is not an integer
{
    return {};   // std::optional<int>{std::nullopt}
}
catch (std::out_of_range e)   // result cannot be represented in int
{
    return {};   // std::optional<int>{std::nullopt};
}

int main()
{
    int i = convert("42").value_or(-1);
}
```

Use of optional

```
void f(bool b1)
{
  std::optional<int> opt1;  // default constr: std::nullopt
  std::cout << opt1.value_or(-1) << '\n';   // -1
  try
  {
    std::cout << opt1.value() << '\n';  // throw std::bad_optional_access
  }
  catch( std::bad_optional_access& e)
  {
    std::cerr << e.what() << '\n';
  }
  opt1 = b1 ? std::optional<int>(42) : std::nullopt;  // 42

  std::cout << opt1.value_or(-1) << '\n';  // 42
  if ( opt1 )  // true
  {
    std::cout << opt1.value() << '\n';  // 42
    *opt1 = 2;   // access contained data, also ->  exists
    int i = opt1.value();
    std::cout << i << '\n';   // 2
  }
}
```

```
-1
bad optional access
42
42
2
```

Use of pointers

```
void f(bool b1)
{
  std::optional<std::string> opt2;  // std::nullopt
  *opt2 = "Hello";   // undefined behavior if std::nullopt

  std::cout << *opt2 << '\n';
  std::cout << std::boolalpha << opt2.has_value() << '\n';   // false

  std::cout << opt2.value_or("no value") << '\n';   // “no value”
  std::string s = *std::move(opt2);

  std::cout << s << ", " << opt2->size() << '\n';
}
```

```
Hello
false
no value
Hello, 0
```

Expected (C++23)

Always holds either value_type or unexpected_type

https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0323r12.html

https://youtu.be/PH4WBuE1BHI (Andrei Alexandrescu CppCon 2018)

```
#include <expected>  // since C++23

template <class T, class E>    // T can be void, but T must not be unexpected<>
class expected {
  // types
  using value_type = T;
  using error_type = E;
  using unexpected_type = std::unexpected<E>;
  template <class U> using rebind = expected<U, error_type>;

  // accessors
  bool has_value()
  operator bool()
  operator void()   // if T == void
  T*   operator->() // undefined behavior if not expected
  T&   operator*()  // undefined behavior if not expected
  void operator*()  // if T == void, undefined behavior if not expected
  T&    value()     // may throw std::bad_expected_value<E>
  E&    error()     // undefined behavior if expected
  T value_or(U def)
};
```

Constant magic

- Constants in C++
- Const correctness
- Pointers to const
- Const member functions
- Const and mutable members
- STL const safety
- Constexpr

Design goals of C++

- Type safety
- Resource safety
- Performance control
- Predictability
- Readability
- Learnability

Mapping semantic issues to syntax

```
#include <stdio.h>
int main()
{
    int fahr;
    for ( fahr = -100; fahr <= 400; fahr += 25 )
    {
        printf( "Fahr = %d,\tCels = %d\n", fahr, 5/9*(fahr-32) );
    }
    return 0;
}
```

Mapping semantic issues to syntax

```
#include <stdio.h>
int main()
{
    int fahr;
    for ( fahr = -100; fahr <= 400; fahr += 25 )
    {
        printf( "Fahr = %d,\tCels = %d\n", fahr, 5/9*(fahr-32) );
    }
    return 0;
}

$ ./a.out
Fahr = -100, Cels = 0
Fahr = -75,  Cels = 0
Fahr = -50,  Cels = 0
Fahr = -25,  Cels = 0
Fahr = 0,    Cels = 0
Fahr = 25,   Cels = 0
```

Mapping semantic issues to syntax

```
#include <stdio.h>
int main()
{
    int fahr;
    for ( fahr = -100; fahr <= 400; fahr += 25 )
    {
        printf( "Fahr = %d,\tCels = %d\n", fahr, 5/9*(fahr-32) );
    }
    return 0;
}

$ ./a.out
Fahr = -100, Cels = 0
Fahr = -75,  Cels = 0
Fahr = -50,  Cels = 0
Fahr = -25,  Cels = 0
Fahr = 0,    Cels = 0
Fahr = 25,   Cels = 0
```

Mapping semantic issues to syntax

```
#include <stdio.h>
int main()
{
    int fahr;
    for ( fahr = -100; fahr <= 400; fahr += 25 )
    {
        printf( "Fahr = %d,\tCels = %d\n", fahr, 5./9.*(fahr-32) );
    }
    return 0;
}
```

Mapping semantic issues to syntax

```
#include <stdio.h>
int main()
{
    int fahr;
    for ( fahr = -100; fahr <= 400; fahr += 25 )
    {
        printf( "Fahr = %d,\tCels = %d\n", fahr, 5./9.*(fahr-32) );
    }
    return 0;
}
```

```
$ gcc fahr2cels

2c.c: In function ‘main’:

2c.c:7:38: warning: format ‘%d’ expects argument of type ‘int’, but
argument 3 has type ‘double’ [-Wformat=]
```

Mapping semantic issues to syntax

```
#include <stdio.h>
int main()
{
    int fahr;
    for ( fahr = -100; fahr <= 400; fahr += 25 )
    {
        printf( "Fahr = %d,\tCels = %d\n", fahr, 5./9.*(fahr-32) );
    }
    return 0;
}

$ ./a.out
Fahr = -100,Cels = 913552376
Fahr = -75,Cels = -722576928
Fahr = -50,Cels = -722576928
Fahr = -25,Cels = -722576928
Fahr = 0,   Cels = -722576928
Fahr = 25,  Cels = -722576928
```

Mapping semantic issues to syntax

```
#include <stdio.h>
int main()
{
    int fahr;
    for ( fahr = -100; fahr <= 400; fahr += 25 )
    {
        printf( "Fahr = %d,\tCels = %d\n", fahr, 5./9.*(fahr-32) );
    }
    return 0;
}

$ ./a.out
Fahr = -100,Cels = 913552376
Fahr = -75,Cels = -722576928
Fahr = -50,Cels = -722576928
Fahr = -25,Cels = -722576928
Fahr = 0,   Cels = -722576928
Fahr = 25,  Cels = -722576928
```

Mapping semantic issues to syntax

```
#include <iostream>
int main()
{
    for ( int fahr = -100; fahr <= 400; fahr += 25 )
    {
        std::cout << "Fahr = " << fahr << ", Cels = "
                  << 5./9.*(fahr – 32) << '\n';
    }
    return 0;
}
```

```
$ ./a.out
Fahr = -100,Cels = -73.333333
Fahr = -75,Cels = -59.444444
Fahr = -50,Cels = -45.555556
Fahr = -25,Cels = -31.666667
Fahr = 0,  Cels = -17.777778
Fahr = 25, Cels = -3.888889
```

Mapping semantic issues to syntax

```
/* C language I/O */

#include <stdio.h>
int main()
{
   FILE *fp = fopen( "input.txt", "r");

   . . .

   fprintf( fp, "%s\n", "Hello input!");
   return 0;   /* not strictly necessary since C99 */
}
```

- Runtime error!

Mapping semantic issues to syntax

```
/* C++ language I/O */

#include <iostream>
#include <fstream>
int main()
{
   std::ifstream f;

   . . .

   f << "Hello input!" << std::endl;
   return 0;
}
```

- Compile-time error!

Side note: Schwartz error

```
while ( cin >> i )  // 1st attempt: cin.operator int() Jerry Schwarz

std::istream& tempc = cin.operator>>(i);
int tempi = tempc.operator int();
while ( tempi )


while ( cin << i )

int tempi1 = cin.operator int();
int tempi2 = tempi1 << i;
while ( tempi2)


operator bool() is not better: bool -> int promotion
```

Side note: safe bool conversion

```
while ( cin >> i )  // 2nd attempt: cin.operator void*() Jerry Schwarz

std::istream& tempc = cin.operator>>(i);
void* tempptr = tempc.operator void*();
while ( tempi )     // contextually converted to bool


while ( cin << i )

void* tempi1 = cin.operator void*();  // no operator<<(void*,int)
delete std::cin;            // delete ( operator void*(std::cin) )


// C++11 solution: explicit specifier
explicit operator bool() const; // not to use in implicit conversion
                                // works in if, while, etc...

// C++20
explicit (expr) operator bool() const; // explicit if expr is true
```

Const literals

```
char t1[] = {'H','e','l','l','o','\0'};
char t2[] = "Hello";
char t3[] = "Hello";

/* const */ char *s1 = "Hello"; // s1 points to 'H'
/* const */ char *s2 = "Hello"; // s2 points to the same place

// assignment to array elements ok:
*t1 = 'x'; *t2 = 'q'; *t3 = 'y';

// modifying string literal: can be segmentation error:
*s1 = 'w'; *s2 = 'z';
```

Preprocessor macro names

```
#define LOWER  -100
#define UPPER   400
#define STEP     40

int main()
{
  for( int fahr = LOWER; fahr <= UPPER; fahr += STEP )
  {
    std::cout << "fahr = " << std::setw(4) << fahr
          << ", cels = " << std::fixed << std::setw(7)
          << std::setprecision(2) << 5./9. * (fahr-32)
          << std::endl;
  }
  return 0;
}
```

Named constants

- ```
  int f(int i) { return i; }
  int main()
  {
    const int c1 = 1;    // initialized compile time
    const int c2 = 2;    // initialized compile time
    const int c3 = f(3); // f() is not constexpr
    int t1[c1];
    int t2[c2];
    ~~int t3[c3];~~ // VLA (Variadic Length Aray) is C99 not C++
    switch(i)
    {
    case c1: std::cout << "c1"; break;
    case c2: std::cout << "c2"; break;
    // case label c3 does not reduce to an integer constant
    ~~case c3:~~ std::cout << "c3"; break;
    }
    return 0;
  }
  ```
  ```

When named const needs memory?

```
int f(int i) { return i; }

int main()
{
  const int   c1 = 1;    // initialized compile time
  const int   c2 = 2;    // initialized compile time
  const int   c3 = f(3); // f() is not constexpr
  const int *ptr = &c2;
  return 0;
}
```

- Named pointers require storage when
  - Initialized at run-time (or)
  - Address is used

Be careful with optimization!

```
int main()
{
    const int ci = 10;
    int *ip = const_cast<int*>(&ci);
    ++*ip;
    cout << ci << " " << *ip << endl;
    return 0;
}
```

```
$ ./a.out
```

Be careful with optimization!

```
int main()
{
    const int ci = 10;
    int *ip = const_cast<int*>(&ci); // undefined behavior
    ++*ip;
    cout << ci << " " << *ip << endl;
    return 0;
}
```

```
$ ./a.out
10 11
```

Be careful with optimization!

```
int main()
{
    volatile const int ci = 10;
    int *ip = const_cast<int*>(&ci); // undefined behavior
    ++*ip;
    cout << ci << " " << *ip << endl;
    return 0;
}
```

```
$ ./a.out
11 11
```

Const correctness

```
#include <iostream>

int my_strlen(char *s)
{
    char *p = s;
    while ( ! (*p = 0) ) ++p;
    return p - s;
}

// This program likely cause run-time error
int main()
{
    char t[] = "Hello";
    std::cout << my_strlen(t) << std::endl;
    return 0;
}
```

Const correctness

```
#include <iostream>

int my_strlen(char *s)
{
    char *p = s;
    while ( ! (*p = 0) ) ++p; // FATAL ERROR
    return p - s;
}

// This program likely cause run-time error
int main()
{
    char t[] = "Hello";
    std::cout << my_strlen(t) << std::endl;
    return 0;
}
```

Const correctness

```
#include <iostream>

int my_strlen(const char *s)
{
    const char *p = s;
    while ( ! (*p = 0) ) ++p; // Compile-time error!
    return p - s;
}


int main()
{
    char t[] = "Hello";
    std::cout << my_strlen(t) << std::endl;
    return 0;
}
```

Const correctness

```
#include <iostream>

int my_strlen(const char *s)
{
    const char *p = s;
    while ( ! (*p == 0) ) ++p; // FIX
    return p - s;
}

// this program is fine
int main()
{
    char t[] = "Hello";
    std::cout << my_strlen(t) << std::endl;
    return 0;
}
```

Side note on defensive programming

```
#include <iostream>

int my_strlen(char *s)
{
    char *p = s;
    while ( ! (0 = *p) ) ++p; // compile-time error!
    return p – s;             // “Yoda condition”
}

// This program likely cause run-time error
int main()
{
    char t[] = "Hello";
    std::cout << my_strlen(t) << std::endl;
    return 0;
}
```

Const correctness and pointers

```
      int i = 4;   // not constant, can be modified:
          i = 5;

const int ci = 6;   // const, must be initialized
          ci = 7;   // syntax error, cannot be modified

     int *ip;
          ip = &i;
         *ip = 5;   // ok
```

Const correctness and pointers

```
      int i = 4;   // not constant, can be modified:
          i = 5;

const int ci = 6;   // const, must be initialized
          ci = 7;   // syntax error, cannot be modified

     int *ip;
          ip = &i;
         *ip = 5;   // ok

         if ( input )
           ip = &ci; // ??

         *ip = 7;   // can I do this?
```

Pointer to const

```
        int i = 4;   // not constant, can be modified:
            i = 5;

const int ci = 6;   // const, must be initialized
          ci = 7;   // syntax error, cannot be modified



const int *cip = &ci; // ok
          *cip = 7;   // syntax error
       int *ip = cip; // syntax error, C++ keeps const

           cip = ip;  // ok, constness gained
          *cip = 5;   // syntax error,
                      // wherever cip points to
```

Pointer constant

```
        int i = 4;   // not constant, can be modified:
            i = 5;

const int ci = 6;   // const, must be initialized
          ci = 7;   // syntax error, cannot be modified


int * const ipc = &i; // ipc is const, must initialize
            *ipc = 5;  // OK, *ipc points is NOT a const


int * const ipc2 = &ci; // syntax error,
                        // ipc is NOT a pointer to const

const int * const cccp = &ci; // const pointer to const
```

const – pointer cheat sheet

- **const** keyword left to * means pointer to **const**
  - Can point to **const** variable
  - Can be changed
  - Pointed element is handled as constant

```
const int ci = 10;
const int * cip = &ci;
int const * pic = &ci;
```

- **const** keyword right to * means pointer is constant
  - Must be initialized
  - Can not be changed
  - Can modify pointed element

```
int i;
int * const ipc = &i;
```

Const X** = X**

```
int main()
{
  int   i   = 42;
  int  *ip  = &i;
  int **ipp = &ip;





}
```

Const X** = X**

```
int main()
{
  int   i   = 42;
  int  *ip  = &i;
  int **ipp = &ip;

  const int  *cip  = ip;
  const int **cipp = ipp;



}
```

Const X** = X**

```
int main()
{
  int   i   = 42;
  int  *ip  = &i;
  int **ipp = &ip;

  const int  *cip  = ip;
  const int **cipp = ipp;



}
```

Const X** = X**

```
int main()
{
  int   i   = 42;
  int  *ip  = &i;
  int **ipp = &ip;

  const int  *cip  = ip;
  const int **cipp = ipp;

        int *const *pcip  = ipp;

}
```

Const X** = X**

```
int main()
{
  int   i   = 42;
  int  *ip  = &i;
  int **ipp = &ip;

  const int  *cip  = ip;
  const int **cipp = ipp;

        int *const *pcip  = ipp;
  const int *const *cpcip = ipp;
}
```

Const X** = X**

```
int main()
{
  int   i   = 42;
  int  *ip  = &i;
  int **ipp = &ip;

  const int  *cip  = ip;
  const int **cipp = ipp;

        int *const *pcip  = ipp;
  const int *const *cpcip = ipp;
}
```

Const X** = X**

```
int main()
{
  const int ci = 42;

  int *ip;
  const int **cipp = &ip;  // if this would work


  *cipp = &ci;
  *ip = 43;
}
```

Const X** = X**

```
int main()
{
  const int ci = 42;

  int *ip;
  const int **cipp = &ip;  // must not allow


  *cipp = &ci;
  *ip = 43;
}
```

Base** = Derived**

https://isocpp.org/wiki/faq/proper-inheritance#derivedptrptr-to-baseptrptr

```
class Vehicle {
public:
  virtual ~Vehicle() { }
  virtual void startEngine() = 0;
};

class Car : public Vehicle {
public:
  virtual void startEngine();
  virtual void openGasCap();
};

class NuclearSubmarine : public Vehicle {
public:
  virtual void startEngine();
  virtual void fireNuclearMissile();
};

int main()
{
  Car   car;
  Car*  carPtr = &car;
  Car** carPtrPtr = &carPtr;
  Vehicle** vehiclePtrPtr = carPtrPtr;  // This is an error in C++

  NuclearSubmarine  sub;
  NuclearSubmarine* subPtr = &sub;
  *vehiclePtrPtr = subPtr;  // carPtr to point to Submarine
  carPtr->openGasCap();     // This might call fireNuclearMissile()!
}
```

User-defined types

```
class Date
{
public:
    Date( int year, int month = 1, int day = 1);
    // …
    int getYear();
    int getMonth();
    int getDay();
    void set(int y, int m, int d);
    // …
private:
    int year;
    int month;
    int day;
};
const Date my_birthday(1963,11,11); // const can be initialized
      Date curr_date(2015,7,10);

  curr_date = my_birthday; // ok, const can be read
my_birthday = curr_date;   // compile-time error: write const
```

User-defined types

```
class Date
{
public:
    Date( int year, int month = 1, int day = 1);
    // …
    int getYear();
    int getMonth();
    int getDay();
    void set(int y, int m, int d);
    // …
private:
    int year;
    int month;
    int day;
};
const Date my_birthday(1963,11,11);
      Date curr_date(2015,7,10);

int x = my_birthday.getYear(); // can I read ??
my_birthday.set(2015,7,10);    // can I write ??
```

User-defined types

```
class Date
{
public:
    Date( int year, int month = 1, int day = 1);
    // …
    int getYear() const;
    int getMonth() const ;
    int getDay() const;
    void set(int y, int m, int d);
    // …
private:
    int year;
    int month;
    int day;
};
const Date my_birthday(1963,11,11);
      Date curr_date(2015,7,10);

int x = my_birthday.getYear(); // works
my_birthday.set(2015,7,10);    // compile error
```

User-defined types

```
class Date
{
public:
  Date( int year, int month = 1, int day = 1);
  // …
  int getYear() const;          -> _ZNK4Date7getYearEv(const Date* this);
  int getMonth() const;
  int getDay() const;
  void set(int y, int m, int d);-> _ZN4Date3setEiii(Date *this,int y,int m,int d)
  // …
private:
  int year;
  int month;
  int day;
};
const Date my_birthday(1963,11,11);
      Date curr_date(2015,7,10);


int x = my_birthday.getYear(); // ok
my_birthday.set(2015,7,10);    // compile-time error!
```

Overloading on const

```
template <typename T, ... >
class vector
{
public:
    T&       operator[](size_t i);
    const T& operator[](size_t i) const;
    // ...
};
int main()
{
          std::vector<int>   iv;
    const std::vector<int>  civ;
    // ...
    iv[i] = 42;        // non-const
    int i = iv[5];
    int j = civ[5]     // const
    // ...
}
```

Const members

```
class Msg
{
public:
    Msg(const char *t);
    int getId() const { return id; }
private:
    const int id;
    std::string txt;
};

Msg m1("first"), m2("second");
m1.getId() != m2.getId();

MSg::Msg(const char *t)
{
    txt = t;
    id  = getNewId();  // syntax error, id is const
}

//initialization list works
MSg::Msg(const char *t) : id(getNextId()),txt(t) { }
```

Const members

```
class Msg
{
public:
    Msg(const char *t);
    int getId() const { return id; }
private:
    const int id = getNewId(); // since C++11
    std::string txt;
};
```

Mutable members

```
struct Point
{
    void getXY(int& x, int& y) const;
    double xcoord;
    double ycoord;
    mutable int read_cnt;
};

void f()
{
  const Point a;
  ++a.read_cnt;  // ok, Point::read_cnt is mutable
}

void Point::getXY(int& x, int& y) const
{
  // …
  ++read_cnt; // ok, Point::read_cnt is mutable
};
```

Mutexes are usually mutables

```
#include <mutex>
```

- ```
  struct Point
  {
  public:
    void getXY(int& x, int& y) const;
    // …
  private:
    double  xcoord;
    double  ycoord;
    mutable std::mutex m;
  };

  void getXY(int& x, int& y) const // atomic read of point
  {
    std::lock_guard< std::mutex > guard(m); // locking of m
    x = xcoord;
    y = ycoord;
  } // unlocking m
  ```
  ```

Static const

```
// x.h

class X
{
  static const int  c1 = 7;    // ok,scalar
  static       int  i2 = 8;    // error: not const
  const        int  c3 = 9;    // C++11: ok
  static const int  c4 = f(2); // error: non-const init.
  static const float f = 3.14; // ok,scalar
};


// x.cpp

const int X::c1; // initializer is here xor in-class
```

STL is const safe

```
template <typename It, typename T>
It find( It begin, It end, const T& t)
{
    while (begin != end) {
        if ( *begin == t )
            return begin;
        ++begin;
    }
    return end;
}

const char t[] = { 1, 2, 3, 4, 5 };
const char *p = std::find( t, t+sizeof(t), 3)
if ( p != t+sizeof(t) )
{
    std::cout << *p; // ok to read
    // syntax error: *p = 6;
}
const std::vector<int>  v(t, t+sizeof(t));
std::vector<int>::const_iterator i = std::find( v.begin(), v.end(), 3);
if ( v.end() != i )
{
     std::cout << *i; // ok to read
    // syntax error: *i = 6;
}
```

STL is const safe

```
//  C++11

std::vector<int>  v1(4,5);
auto i = std::find( v1.begin(), v1.end(), 3);
// i is vector<int>::iterator

const std::vector<int>  v2(4,5);
auto j = std::find( v2.begin(), v2.end(), 3);
// j is vector<int>::const_iterator

auto k = std::find( v1.cbegin(), v1.cend(), 3);
// k is vector<int>::const_iterator
```

STL is const safe

```
//  C++11

std::vector<int>  v1(4,5);
auto i = std::find( std::begin(v1), std::end(v1), 3);
// i is vector::iterator

const std::vector<int>  v2(4,5);
auto j = std::find( std::begin(v2), std::end(v2), 3);
// j is vector::const_iterator

auto k = std::find( std::cbegin(v1), std::cend(v1), 3);
// k is vector::const_iterator
```

Constexpr objects in C++11

- Const objects having value known at translation time.
- translation time = compilation time + linking time
- They may have placed to ROM
- Immediately constructed or assigned
- Must contain only literal values, constexpr variables and functions
- The constructor used must be constexpr constructor

Constexpr in C++11

```
enum Flags { good=0, fail=1, bad=2, eof=4 };

constexpr int operator|(Flags f1, Flags f2)
{
  return Flags(int(f1)|int(f2));
}

void f(Flags x)
{
  switch (x)
  {
  case bad:          /* ... */ break;
  case eof:          /* ... */ break;
  case bad|eof:      /* ... */ break;
  default:           /* ... */ break;
  }
}
```

Constexpr in C++11

```
include <cstdio>

size_t constexpr length(const char* str)
{
    return *str ? 1 + length(str + 1) : 0;
}

void f()
{
    printf("%d %d", length("abcd"), length("abcdefgh"));
}
```

- Pattern matching + recursion == Turing complete

Constexpr functions in C++11/14

- Can produce constexpr values when called with compile-time constants.
- Otherwise can run with non-constexpr parameters
- Must not be virtual
- Return type must be literal type
- Parameters must be literal type
- Since C++14 they can be more than a return statement
  - if / else / switch
  - for / ranged-for / while / do-while

Constexpr in C++14

```
#include <iostream>

constexpr int strlen(const char *s)
{
    const char *p = s;
    while ( '\0' != *p )  ++p;
    return p-s;
}
int main()
{
    std::cout << strlen("Hello") << std::endl;
    return 0;
}

constexpr int pow( int base, int exp) noexcept
{
    auto result = 1;
    for (int i = 0; i < exp; ++i) result *= base;
    return result;
}
```

Constexpr in C++14

```
class Point
{
public:
    constexpr Point(double xVal=0, double yVal=0) : x(xVal),y(yVal) noexcept {}
    constexpr double xValue() const noexcept { return x; }
    constexpr double yValue() const noexcept { return y; }

    // can be constexpr since C++14
    constexpr void setX(double newX) noexcept { x = newX; }
    constexpr void setY(double newY) noexcept { y = newY; }
private:
    double x, y;
};

constexpr Point p1(42.0, -33.33); // fine, constexpr ctor during compilation
constexpr Point p2(25.0, 33.3);   // also fine

constexpr Point midpoint(const Point& p1, const Point& p2) noexcept
{
    return { (p1.xValue() + p2.xValue()) / 2,   // call constexpr
             (p1.yValue() + p2.yValue()) / 2 }; // member funcs
}

constexpr auto mid = midpoint(p1, p2); // init constexpr object with
                                       // result of constexpr
```

Constexpr lambda in C++17

- Constexpr lambda (+ template lambda)
- Closure objects are literal types (as long as captured members are literal types)

```
template <typename I>
constexpr auto adder(I i) {
  //use a lambda in constexpr context
  return [i](auto j){ return i + j; };
}

//constexpr closure object
constexpr auto add5  = adder(5);
constexpr auto add15 = adder(15);

template <unsigned N>
class X{};

int foo()
{
  //use in a constant expression
  X<add5(22)> x27;
  int t25[add15(10)];
}
```

Constexpr in C++20

- Union (also needed for constexpr string)
- Try and catch (throw not allowed, so catch works only runtime)
- dynamic_cast and typeid (since we have virtual functions)
- Constexpr allocation
  - Transient: deallocated before evaluation completes
  - Non-transient: not (yet)
- Virtual calls in constexpr, constexpr virtual funtions/overrides
- Library: constexpr vector and string

Constexpr in C++20

```
// C++20

constexpr auto naiveSum(unsigned int n)
{
    auto p = new int[n];  // transient allocation C++20

    // iota is constexpr in C++20
    std::iota(p, p+n, 1);

    // accumulate is constexpr in C++20
    auto tmp = std::accumulate(p, p+n, 0);

    delete [] p;    // compiler detects delete/delete[] issues
    return tmp;
}
```

Consteval in C++20

- Always evaluated in compile-time
- Immediate function: always evaluated in compiler time
  - Constexpr can be evaluated in run-time

```
consteval int sqr(int n)
{
  return n*n;
}
constexpr int r = sqr(100);  // Okay.
int x = 100;
int r2 = sqr(x);  // Error: Call does not produce a constant.


if ( std::is_constant_evaluated() ) {  } else {  }   // C++20

if  consteval {   } else {     }      // C++23, the same as above
if !consteval {   } else {     }
```

Consteval in C++2c?

- Consteval variables
- Immediate variables: replacement for MACROs
  - Never seen by the compiler back-end / code generation

```
consteval int RN = 42;
int LN = 42;

int f(int const&);

int r1 = f(LN);  // Direct reference binding.
int r2 = f(RN);  // Not a direct binding (temporary created).
```

Constinit in C++20

- A variable to be constant initialized but can be mutable
  - Solves the problem of Static Initialization Order Fiasco
  - Local static: no need for mutex on initialization
- Should be applied for static storage duration
- Requires constant initializer

```
// Foo.h
struct Foo {
  constinit static int x;
};
// Foo.cpp
int Foo::x = 42;
int Foo::x = f(); // error

void g()
{
  ++Foo::x;  // ok
}
```

Static IF in C++17

- Evaluate conditional expression in compile time
- The compiler eliminates the false branch(es)
- The rest of the code should be syntactically correct (compiler should parse)
- … but the compiler does not compile it

```
template <class T>
auto foo(T t) {
  if constexpr(std::is_same_v<T, X>)   {
    return t.a_function_that_exists_only_for_X();
  }
  else   {
    std::cout << "Not T\n";
    return;        // returns void if T is not int
  }
}
void bar() {
  X x;
  auto i = foo(x); // calls x.a_function_that_exists_only_for_X();
  foo(23);         // would be compile error for non-static if
}
```

Static IF in C++17

- Before C++17: overloading, SFINAE, enable_if
- Since C++17 we can write common code

```
template <typename T>
auto get_value(T t)
{
    if constexpr (std::is_pointer_v<T>)
        return *t;
    else
        return t;
}

template <class Head, class... Tail>
void print(Head const& head, Tail const&... tail)
{
    std::cout << head;
    if constexpr(sizeof...(tail) > 0)
    {
        std::cout << ", ";
        print(tail...);
    }
}
```

Fibonacci with static IF

```
// before C++17
template<int  N>
constexpr int fibonacci() {return fibonacci<N-1>() + fibonacci<N-2>(); }

template<>
constexpr int fibonacci<1>() { return 1; }

template<>
constexpr int fibonacci<0>() { return 0; }

// since C++17
template<int N>
constexpr int fibonacci()
{
    if constexpr (N >= 2)
        return fibonacci<N-1>() + fibonacci<N-2>();
    else
        return N;
}

int main()
{
  constexpr int fib12 = fibonacci<12>();  // 144
}
```

More on constants

- C++Now 2019 Daveed Vandevoorde “C++ constants”

  https://www.youtube.com/watch?v=m9tcmTjGeho
- C++Now 2017 Ben Deane & Jason Turner: “constexpr ALL the things!”

  https://www.youtube.com/watch?v=HMB9oXFobJc
- C++Now 2021 David Sankel: “Don’t constexpr ALL the things!”

  https://www.youtube.com/watch?v=NNU6cbG96M4
- C++Weekly Jason Turner: “C++23 if consteval”

  https://www.youtube.com/watch?v=AtdlMB_n2pI

Constructors/Destructors/Operators

- Constructor types
- Constructor as conversion operator
- How to define operators
- Initialization list
- Destructors / array destruction
- Cloning
- Delegated constructor
- Using constructor
- Universal constructor
- Construction and exceptions

Constructor/Destructor

- In OO languages constructors to initialize objects
- Other style: using factory objects
- Constructors are also
  - Conversion operators
  - Create temporaries
- Destructors are to free resources
  - Resource: much more than memory
  - e.g. unlock, fclose, ...

Default-, value- and zero-initialization

```
T global;     // zero-initialization,
              // then default-initialization

void foo() {
  T i;        // default-initialization
  T j{};      // value-initialization (C++11)
  T k = T(); // value-initialization
  T l = T{}; // value-initialization (C++11)
  T m();      // function-declaration
  new T;      // default-initialization
  new T();    // value-initialization
  new T{};    // value-initialization (C++11)
}

struct A { T t; A() : t() {} }; // t is value-initialized

struct B { T t; B() : t{} {} }; // t is value-initialized,C++11

struct C { T t; C()       {} }; // t is default-initialized
```

Default-, value- and zero-initialization

- Default initialization
  - If T is a class: default constructor is called
  - If an array, each element is default initialized
  - Otherwise: no initialization, results indeterminate value
- Value initialization
  - If T is a class: default initialized
    - (after zero initialized if T's default constructor is not user defined / deleted)
  - If an array, each element is value initialized
  - Otherwise: zero initialized
- Zero initialization
  - Applied to static and thread local variables before any other initialization
  - If T scalar (number, pointer, enum) set to 0
  - If Z is a class, all subobjects are zero initialized

A constructor quiz

```
#include <iostream>

struct foo {
    foo() = default;
    int a;
};

struct bar {
    bar();
    int b;
};

bar::bar() = default;

int main() {
    foo a{};
    bar b{};
    std::cout << a.a << ' ' << b.b;
}
```

A constructor quiz: answers

```
#include <iostream>

struct foo {
    foo() = default;  // foo() is not user-provided
    int a;
};

struct bar {
    bar();            // bar() is user-provided
    int b;
};

bar::bar() = default; // bar() is still user-provided

int main() {
    foo a{};
    bar b{};
    std::cout << a.a << ' ' << b.b;
}
```

User provided constructors

```
struct foo {
    foo() = default;  // foo() is not user-provided
};

struct bar {
    bar();            // bar() is user-provided
};
bar::bar() = default; // bar() is still user-provided

//ill-formed, const with no user-provided constructor
const int my_int;

//well-formed, const but has a user-provided constructor
const std::string my_string;

//ill-formed, const with no user-provided constructor
const foo my_foo;

//well-formed, const but has a user-provided constructor
const bar my_bar;
```

A constructor quiz: answers

```
#include <iostream>

struct foo {
    foo() = default;  // foo() is not user-provided
    int a;
};

struct bar {
    bar();            // bar() is user-provided
    int b;
};

bar::bar() = default; // bar() is still user-provided

int main() {
    foo a{};
    bar b{};
    std::cout << a.a << ' ' << b.b;
}
```

A constructor quiz: answers

```
#include <iostream>

struct foo {
    foo() = default;  // foo() is not user-provided
    int a;
};

struct bar {
    bar();            // bar() is user-provided
    int b;
};

bar::bar() = default; // bar() is user-provided

int main() {
    foo a{};    // zero initialized then value initialized
    bar b{};    // value initialized
    std::cout << a.a << ' ' << b.b;
}
```

Default-, value- and zero-initialization

- Default initialization
  - If T is a class: default constructor is called
  - If an array, each element is default initialized
  - Otherwise: no initialization, results indeterminate value
- Value initialization
  - If T is a class: default initialized
    - (after zero initialized if T's default constructor is not user defined / deleted)
  - If an array, each element is value initialized
  - Otherwise: zero initialized
- Zero initialization
  - Applied to static and thread local variables before any other initialization
  - If T scalar (number, pointer, enum) set to 0
  - If Z is a class, all subobjects are zero initialized

A constructor quiz: answers

```
#include <iostream>

struct foo {
    foo() = default;  // foo() is not user-provided
    int a;
};

struct bar {
    bar();            // bar() is user-provided
    int b;
};

bar::bar() = default; // bar() is user-provided

int main() {
    foo a{};    // zero initialized then value initialized
    bar b{};    // value initialized
    std::cout << a.a << ' ' << b.b;   // 0
}
```

A constructor quiz: answers

```
#include <iostream>

struct foo {
    foo() = default;  // foo() is not user-provided
    int a;
};

struct bar {
    bar();            // bar() is user-provided
    int b;
};

bar::bar() = default; // bar() is user-provided

int main() {
    foo a{};    // zero initialized then value initialized
    bar b{};    // value initialized
    std::cout << a.a << ' ' << b.b;   // 0 undefined
}
```

Quote from the standard

N4140 (essentially C++14) [dcl.fct.def.default]/5:

Explicitly-defaulted functions and implicitly-declared functions are collectively called defaulted functions, and the implementation shall provide implicit definitions for them, which might mean defining them as deleted. A function is user-provided if it is user-declared and not explicitly defaulted or deleted on its first declaration. A user-provided explicitly-defaulted function (i.e., explicitly defaulted after its first declaration) is defined at the point where it is explicitly defaulted; if such a function is implicitly defined as deleted, the program is ill-formed.

Some more initializations :)

```
int i;          // default
int i{};        // value
static int i;   // zero
static int i = constexpr_fun();  // constant

int i{1}, int j(42);   // direct
int i = 42, j = i;     // copy
int i = {42};          // copy-list
int i{42};             // direct-list

int iarr[3] = {0,1};   // aggregate iarr[2] == 0
int iarr[3] = {.1=2};  // aggregate
const int &iref = 42;  // reference

// static: zero- or const initialization
// dynamic: non static
// unordered: dynamic init. of class template static member
// ordered: dynamic init. Of other non-locals with static life
// implicit: default or value
```

https://i.imgur.com/3wlxtI0.mp4

More to read on initialization

[accu]
https://accu.org/index.php/journals/2379#FN01

[cppcore]
https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Res-always

[cppref1] value-initialization
value-initialization http://en.cppreference.com/w/cpp/language/value_initialization

[cppref2] default-initialization
default-initialization http://en.cppreference.com/w/cpp/language/default_initialization

[cppref3] zero-initialization
zero-initialization http://en.cppreference.com/w/cpp/language/zero_initialization

A simple date class

```
class date { ... };

main()
{
    date exam{2019, 12, 15}; // constructor with 3 ints
    date semester{2022,2};   // constructor with 2 ints

    if ( exam < date{2020} ) { ... } // 2020.1.1
    else if ( exam < date{} ){ ... } // today

    date exam2{"2019.12.15"}; // constructor with string

}
```

- How many constructors should we define?

A simple date class

```
class date
{
public:

  date();      // default constructor
  date( int y, int m=1, int d=1 ); // initilizer constructor
  explicit date( const char *s );  // explicite constructor
  // …
private:
  // …
  int year;
  int month;
  int day;
};
```

Side note: Schwartz error

```
while ( cin >> i )  // 1st attempt: cin.operator int() Jerry Schwarz

std::istream& tempc = cin.operator>>(i);
int tempi = tempc.operator int();
while ( tempi )


while ( cin << i )

int tempi1 = cin.operator int();
int tempi2 = tempi1 << i;
while ( tempi2 )


operator bool() is not better: bool -> int promotion
```

Side note: safe bool conversion

```
while ( cin >> i )  // 2nd attempt: cin.operator void*() Jerry Schwarz

std::istream& tempc = cin.operator>>(i);
void* tempptr = tempc.operator void*();
while ( tempi )     // contextually converted to bool


while ( cin << i )

void* tempi1 = cin.operator void*();  // no operator<<(void*,int)
delete std::cin;            // delete ( operator void*(std::cin) )


// C++11 solution: explicit specifier
explicit operator bool() const; // not to use in implicit conversion
                                // works in if, while, etc...

// C++20
explicit (expr) operator bool() const; // explicit if expr is true
```

How to define operators?

- a + b

  a.**operator**+(b)

  operator+(a,b)

- a = b, a[b], a(b1,b2,...), a-> only member

  a.**operator**= (b)

  a.**operator**[ ] (b)

  a.**operator**( ) (b1, b2, …)

  a.**operator**->()

Where to define operators?

- Theory says: data and operations on it have strong binding
  - Member operators
- Some operators can't be members

```
std::ostream& operator<<(std::ostream&,const X&)
```

- Sometimes members creates unwanted dependencies

```
std::getline(std::basic_istream&,std::basic_string&)
```

- Sometime operators should be symmetric

Symmetry

```
class date
{
public:
    bool operator<(const date& rhs);
};

int main()
{
    date today;   // current date from OS


    if ( today < date(2016) )  // works

    if ( today < 2016 )        // works



};
```

Symmetry

```
class date
{
public:
    bool operator<(const date& rhs);
};

int main()
{
    date today;   // current date from OS

    if ( today < date(2016) )  // works

    if ( today < 2016 )        // works

    if ( 2016 < today )        // does not work
                               // if operator< is member
};
```

Symmetry

```
class date
{
public:
};
bool operator<(const date& lhs, const date& rhs);

int main()
{
    date today;   // current date from OS

    if ( today < date(2016) )  // works

    if ( today < 2016 )        // works

    if ( 2016 < today )        // works
                               // if operator< is gobal
};
```

Inline functions

```
// date.h
class date
{
public:
    // …
private:
    // …
    int year;
    int month;
    int day;
};

       bool operator< ( date d1, date d2 );
inline bool operator==( date d1, date d2 ) { ... }
inline bool operator!=( date d1, date d2 ) { ... }
inline bool operator<=( date d1, date d2 ) { ... }
inline bool operator>=( date d1, date d2 ) { ... }
inline bool operator> ( date d1, date d2 ) { ... }
```

Creation of subobjects

```
// date.h
class date
{
public:
    date(int y, int m, int d);
private:
    // …
    int year;
    int month;
    int day;
};

date::date(int y, int m, int d) : day(d),month(m),year(y)
{ . . . }

// rearranged to declaration order:
date::date(int y, int m, int d) : year(y),month(m),day(d)
{ . . . }
```

Creation of subobjects C++11

```
class Simple
{
public:
    Simple( ) { cout << a << b << s << '\n'; }


private:
    int a{1};
    int b{2};
    std::string s{"string"};
};

int main()
{
    Simple obj;         // ?
    return 0;
}
```

Creation of subobjects C++11

- ```
  class Simple
  {
  public:
      Simple( ) { cout << a << b << s << '\n'; }


  private:
      int a{1};
      int b{2};
      std::string s{"string"};
  };

  int main()
  {
      Simple obj;         // 12string
      return 0;
  }
  ```
  ```

Creation of subobjects C++11

```
class Simple
{
public:
    Simple( ) { cout << a << b << s << '\n'; }
    Simple(int aa, int bb) : a(aa), b(bb)
                { cout << a << b << s << '\n'; }
private:
    int a{1};
    int b{2};
    std::string s{"string"};
};

int main()
{
    Simple obj{5,6};   // ?
    return 0;
}
```

Creation of subobjects C++11

```
class Simple
{
public:
    Simple( ) { cout << a << b << s << '\n'; }
    Simple(int aa, int bb) : a(aa), b(bb)
                { cout << a << b << s << '\n'; }
private:
    int a{1};
    int b{2};
    std::string s{"string"};
};

int main()
{
    Simple obj{5,6};   // 56string
    return 0;
}
```

Creation of subobjects C++11

```
class Simple
{
public:
    Simple( ) { cout << a << b << s << '\n'; }
    Simple(int aa, int bb) : a(b), b(bb)
                { cout << a << b << s << '\n'; }
private:
    int a{1};
    int b{2};
    std::string s{"string"};
};

int main()
{
    Simple obj{5,6};   // ?
    return 0;
}
```

Creation of subobjects C++11

```
class Simple
{
public:
    Simple( ) { cout << a << b << s << '\n'; }
    Simple(int aa, int bb) : a(b), b(bb)
                { cout << a << b << s << '\n'; }
private:
    int a{1};
    int b{2};
    std::string s{"string"};
};

int main()
{
    Simple obj{5,6};   // 06string  a is undefined!
    return 0;
}
```

Destructors

- Should be virtual?
  - If used in polymorphic way
  - If there is multiple inheritance

```
class Base
{
public:
    virtual ~Base();
};
class Der : public Base
{
public:
    virtual ~Der();
};
```

```
void f()
{
    Base *bp = new Der;
    // …
    delete bp;
}
```

Arrays are not polymorphic!

```
• struct Base
  {
      Base() { cout << "Base" << " "; }
      virtual ~Base() { cout << "~Base" << endl; }
      int i;
  };
  struct Der : public Base
  {
      Der() { cout << "Der" << endl; }
      virtual ~Der() { cout << "~Der" << " "; }
      int it[10];
  };
  int main()
  {
      Base *bp = new Der;
      Base *bq = new Der[5];

      delete    bp;   // fine: virtual destructor called
      delete [] bq;
  }
```

Arrays are not polymorphic!

```
• struct Base
  {
      Base() { cout << "Base" << " "; }
      virtual ~Base() { cout << "~Base" << endl; }
      int i;
  };
  struct Der : public Base
  {
      Der() { cout << "Der" << endl; }
      virtual ~Der() { cout << "~Der" << " "; }
      int it[10];
  };
  int main()
  {
      Base *bp = new Der;
      Base *bq = new Der[5];

      delete    bp;   // fine: virtual destructor called
      delete [] bq;   // this line causes runtime error
  }
```

Arrays are not polymorphic!

```
struct Base
{
    Base() { cout << "Base" << " "; }
    virtual ~Base() { cout << "~Base" << endl; }
    int i;
};
struct Der : public Base
{
    Der() { cout << "Der" << endl; }
    virtual ~Der() { cout << "~Der" << " "; }
    int it[10]; // sizeof(Base) != sizeof(Der)
};
int main()
{
    Base *bp = new Der;
    Base *bq = new Der[5];

    delete    bp;   // fine: virtual destructor called
    delete [] bq;   // this line causes runtime error
}
```

Cloning – “Virtual” constructors

- Constructors are not virtual
- But sometimes we need “virtual” behavior

```
std::vector<Base*> source;
std::vector<Base*> target;

source.push_back( new Derived1() );
source.push_back( new Derived2() );
source.push_back( new Derived3() );

// should create new instances of the
// corresponding Derived classes and
// place them to target
deep_copy( target, source );
```

Wrong approach

```
deep_copy( std::vector<Base*> &target,
           const std::vector<Base*> &source )
{
  for( auto i = source.begin(); i!=source.end(); ++i )
  {
    target.push_back( new Base(**i) ); // creates Base
  }
}
```

Wrong approach 2

```
deep_copy( std::vector<Base*> &target,
           const std::vector<Base*> &source )
{
  for( auto i = source.begin(); i!=source.end(); ++i )
  {
    if ( Derived1 *dp = dynamic_cast<Derived1*>(*i) )
      target.push_back( new Derived1(*dp) );
    else if ( Derived2 *dp = dynamic_cast<Derived2*>(*i) )
      target.push_back( new Derived2(*dp) );
    else if ( Derived3 *dp = dynamic_cast<Derived3*>(*i) )
      target.push_back( new Derived3(*dp) );
  }
}
```

Cloning

```
class Base
{
public:
  virtual Base* clone() const = 0;
};

class Derived : public Base
{
public:
  virtual Derived* clone() const { return new Derived(*this); }
};

deep_copy(std::vector<Base*> &target, const std::vector<Base*> &source)
{
  for( auto i = source.begin(); i != source.end(); ++i )
  {
    target.push_back( (*i)->clone() ); // inserts Derived()
  }
}
```

Assignment operator

```
class MyClass
{
public:
  MyClass& operator=(const MyClass& rhs);
  // ...
private:
  std::vector<int> v;
  // resources...
};

MyClass& MyClass::operator=(const MyClass& rhs)
{
  if ( &rhs != this )
  {
    // release old resources
    // allocate new resources
    // copy resources from rhs to new
    v = rhs.v;   // release+allocate+copy
  }
  return *this;
}
```

Assignment operator

```
class MyClass
{
public:
  MyClass& operator=(const MyClass& rhs);
  // ...
private:
  std::vector<int> v;
  // resources...
};

MyClass& MyClass::operator=(const MyClass& rhs)
{
  if ( &rhs != this )
  {
    MyClass temp(rhs);  // allocate+copy
    v.swap(temp.v);     // do not throw
  }                     // release old resource
  return *this;
}
```

Assignment operator

```
class MyClass
{
public:
  MyClass& operator=(const MyClass& rhs);
  // ...
private:
  std::vector<int> v;
  // resources...
};

MyClass& MyClass::operator=(MyClass rhs)  // allocate+copy or move!
{
  if ( &rhs != this )
  {
    // MyClass temp(rhs);
    v.swap(rhs.v);      // do not throw
  }                     // release old resource
  return *this;
}
```

Virtual assignment operator?

```
#include <iostream>

struct Base
{
  virtual Base&
  operator=(const Base& rhs)
  {
    std::cout<<"Base::operator="<<'\n';
    a = rhs.a;
    return *this;
  }
  int a;
};

struct Derived : public Base
{
  virtual Derived&
  operator=(const Base& rhs) override
  {
    std::cout<<"Derived::operator="<<'\n';
    a = 42;
    b = 43;
    return *this;
  }
  int b;
};
```

```
int main()
{
  Derived d1;
  Derived d2;
  d1.a = 1; d1.b = 2;
  d2.a = 3; d2.b = 4;

  std::cout<<" d1 == "<<d1.a
                << ","<<d1.b<<'\n';
  std::cout<<" d2 == "<<d2.a
                << ","<<d2.b<<'\n';
  d1 = d2;
  std::cout<<" d1 == "<<d1.a
                << ","<<d1.b<<'\n';
}
```

Virtual assignment operator?

```
#include <iostream>

struct Base
{
  virtual Base&
  operator=(const Base& rhs)
  {
    std::cout<<"Base::operator="<<'\n';
    a = rhs.a;
    return *this;
  }
  int a;
};

struct Derived : public Base
{
  virtual Derived&
  operator=(const Base& rhs) override
  {
    std::cout<<"Derived::operator="<<'\n';
    a = 42;
    b = 43
    return *this;
  }
  int b;
};
```

```
int main()
{
  Derived d1;
  Derived d2;
  d1.a = 1; d1.b = 2;
  d2.a = 3; d2.b = 4;

  std::cout<<" d1 == "<<d1.a
                << ","<<d1.b<<'\n';
  std::cout<<" d2 == "<<d2.a
                << ","<<d2.b<<'\n';
  d1 = d2;
  std::cout<<" d1 == "<<d1.a
                << ","<<d1.b<<'\n';
}
```

```
$ ./a.out
d1 == 1,2
d2 == 3,4
Base::operator=
d1 == 3,4
```

Default and deleted constructors C++11

```
class X
{
private:
    X(const X&);                // pre-C++11
    X& operator=(const X&); // pre-C++11
};

class X : private boost::noncopyable
{
    // ...
};

class X
{
    X(const X&) = delete;                // C++11
    X& operator=(const X&) = delete; // C++11
};

class X
{
    X(const X&) = default;                // C++11
    X& operator=(const X&) = default; // C++11
};
```

Delegated constructors C++11

```
// C++98
class X
{
   int a;
   validate(int x) { if (0<x && x<=max) a=x; else throw bad_X(x); }
public:
   X(int x) { validate(x); }
   X() { validate(42); }
   X(string s) { int x = boost::lexical_cast<int>(s); validate(x); }
   // ...
};

// C++11
class X
{
   int a;
public:
   X(int x) { if (0<x && x<=max) a=x; else throw bad_X(x); }
   X() : X{42} { }
   X(string s) : X{boost::lexical_cast<int>(s)} { }
   // ...
};
```

Use constructors C++11

```
class Base
{
public:
    void f(double);   // overloaded member function
    void f(int);      // overloaded member function

    Base(double);     // overloaded constructors
    Base(int);        // overloaded constructors
    // ...
};

class Derived : public Base
{
public:
    using Base::f;    // lift Base's f into Derived scope – works in C++98
    void f(char);     // provide a new f
    void f(int);      // prefer this f to Base::f(int)

    using Base::Base; // lift Base constructors Derived scope – C++11 only
    Derived(char);    // provide a new constructor
    Derived(int);     // prefer this constructor to Base::Base(int)
    // ...
};
```

Use constructors C++11

```
#include <iostream>

struct Base
{
  Base(int i)      { std::cout << "Base(int) " << i << std::endl; }
  Base(double d)   { std::cout << "Base(double) " << d << std::endl; }

  void f(int i)    { std::cout << "Base::f(int) " << i << std::endl; }
  void f(double d) { std::cout << "Base::f(double) " << d << std::endl; }
};
struct Der : Base
{
  using Base::Base;
  Der(char c) : Base(c)   { std::cout << "Der(char) " << c << std::endl; }
  Der(double d) : Base(d) { std::cout << "Der(double) " << d << std::endl; }

  using Base::f;
  void f(char c)          { std::cout << "Der::f(char) " << c << std::endl; }
};
int main()
{
  Der x1('x'), x2(1), x3(3.14);
  x1.f('x');
  x1.f(1);
  x1.f(3.14);
}
```

```
Base(int) 120
Der(char) x
Base(int) 1
Base(double) 3.14
Der(double) 3.14
Der::f(char) x
Base::f(int) 1
Base::f(double) 3.14
```

Use constructors: strong types

```
#include <iostream>

struct Date
{
  int year, month, day;   // no constructor: aggregate type
};

struct BornDate  : Date { }; // still aggregate type

struct DeathDate : Date { }; // still aggregate type


void print( BornDate born, DeathDate death );


int main()
{
   const BornDate  born  = { 1823, 1,  1 }; // aggregate initialization
   const DeathDate death = { 1849, 7, 31 }; // aggregate initialization

   print( born, death );
// print( death, born );   // compile error

}
```

Uniform (braced) initialization C++11

```
double t[] = { 1, 2, 3.456, 99.99 };
vector<double> v = { 1, 2, 3.456, 99.99 };

int x1(0);     //
int x2 = 0;    // same as x2(0)
int x3{0};     // C++11, but do not narrowing
int x4 = {0}; // C++11, usually the same as x3{0}

std::atomic<int>  a1{0};  // ok
std::atomic<int>  a2(0);  // ok
std::atomic<int>  a3 = 0; // syntax error!  Non-copyable

class X
{
private:
  int x{0};   // ok, C++11
  int y = 0; // ok, C++11
  int z(0);  // syntax error!
};

C c2;       // Ok
C c1();     // "Most vexing parse”
C c3{};     // Ok
```

Initializer list

```
vector<double> v = { 1, 2, 3.456, 99.99 };

list<pair<string,string>> languages = {
                                        {"Nygaard","Simula"},
                                        {"Richards","BCPL"},
                                        {"Ritchie","C"}
};
map<vector<string>,vector<int>> years = {
  { {"Maurice","Vincent","Wilkes"},{1913, 1945, 1951, 1967, 2000} },
  { {"Martin","Ritchards"}, {1982, 2003, 2007} },
  { {"David","John","Wheeler"}, {1927, 1947, 1951, 2004} }
};

auto x1 = 5;     // deduced type is int
auto x2(5);      // deduced type is int
auto x3{ 5 };    // deduced type is int since C++17
auto x4{ 5, 6 };// error since C++17
auto x5 = { 5 };// std::initializer_list<int>

enum class E;
E e { 42 };      // ok, since C++17
```

Class template argument deduction

- Possible traps

```
#include <iostream>
#include <vector>

int main()
{
  std::vector v1{1,2};
  std::vector v2(v1.begin(), v1.end());
  std::vector v3{v1.begin(), v1.end()};

  std::cout << v1[0] << v2[0] << ‘\n’; //<< v3[0] << '\n';
  return 0;
}
```

Class template argument deduction

- Possible traps

```
#include <iostream>
#include <vector>

int main()
{
  std::vector v1{1,2};
  std::vector v2(v1.begin(), v1.end());
  std::vector v3{v1.begin(), v1.end()};

  std::cout << v1[0] << v2[0] << v3[0] << '\n';
  return 0;
}
```

```
ctad1.cpp:11:31: error: invalid operands to binary expression ('std::basic_ostream<char,
std::char_traits<char> >::__ostream_type' (aka 'basic_ostream<char,
std::char_traits<char> >') and
'__gnu_cxx::__alloc_traits<std::allocator<__gnu_cxx::__normal_iterator<int *,
std::vector<int, std::allocator<int> > > >, __gnu_cxx::__normal_iterator<int *,
std::vector<int, std::allocator<int> > > >::value_type' (aka
'__gnu_cxx::__normal_iterator<int *, std::vector<int, std::allocator<int> > >'))
  std::cout << v1[0] << v2[0] << v3[0] << '\n';
```

Class template argument deduction

- Possible traps

```
#include <iostream>
#include <vector>

int main()
{
  std::vector v1{1,2};
  std::vector v2(v1.begin(), v1.end());
  std::vector v3{v1.begin(), v1.end()};

  std::cout << v1[0] << v2[0] << ‘\n’; //<< v3[0] << '\n';
  return 0;
}

// vector<T>::vector<T>(std::initializer_list<T>);
// <Iter> vector<T>::vector<T>(Iter b, Iter e);
//       T := Iter::value_type
```

Initializer list

```
vector<double> v{ 1, 2, 3.456, 99.99 }; // constructor
v = { 4., 6. };                         // operator=
f( { 4., 6., 7. } );                    // void f( std::vector<double> )
for ( auto i : { 1, 2, 3, 4, 5} )  ...    // range for


const T noname[N];   // underlying temporary array
                     // may allocated in read-only area
                     // each element is copy initialized
                     // but no narrowing conversion
                     // life-time extension due to initializer_list

class initializer_list<T> // lightweight proxy
{
public:
  // usual copy operations do shallow copy of initializer_list objects
  size_type size() const;
  constexpr const T* begin() const;
  constexpr const T* end() const;
private:
  T *begin;
  T *end;    // or size_t n;
};
```

Constructors and exceptions

```
class C
{
public:
    C(int i, int j)
    try
        : x(i), z(j)    // x(i) or z(j) may throw exception
    { }
    catch( ... )
    {
        // we get here if either x(i) or z(j) throws exception
        // if x(i) throws, then z uninitialized
        // if z(j) throws, then ~X::X() has already executed

        // what to do here ??
    }
private:
    X x;
    Z z;
};
```

Constructors and exceptions

```
class C
{
public:
    C(int i, int j)
    try
        : x(i), z(j)    // x(i) or z(j) may throw exception
    { }
    catch( ... )
    {
        // do something
        if ( ... )
            throw OtherException();
        // otherwise
        //    constructor will re-throw original exception
    }
private:
    X x;
    Z z;
};
```

Constructors and exceptions

```
class C
{
public:
    C(int i, int j)
    try
        : x(i), z(j)    // x(i) or z(j) may throw exception
    { }
    catch( ... )
    {
        // do something
        if ( ... )
            throw OtherException();
        // otherwise
        //    constructor will re-throw original exception
    }
private:
    X x;
    Z z;
};
```

Destructors must not throw!

Complex initialization

```
void f(int param, bool flag)
{
    const int ci1 = 10 * param + 3;

    const int ci2 = flag ? 2 * param : 2 + param;

    const int ci3 =  ?    /* very complex initialization */




};
```

Complex initialization

```
void f(int param, bool flag)
{
    const int ci1 = 10 * param + 3;

    const int ci2 = flag ? 2 * param : 2 + param;

    const int ci3 =  f(); /* very complex initialization */




};

int f(int param, bool flag /*, many extra params (by reference) */)
{
    int value;
    /* very complex initialization of value */
    return value;
}
```

Complex initialization

```
void f(int param, bool flag)
{
    const int ci1 = 10 * param + 3;

    const int ci2 = flag ? 2 * param : 2 + param;

    const int ci3 =  [&]{ /* access to the environment */

       int value;
       /* very complex initialization of value */
       return value;
    }(); /* immediately executed lambda */
};
```

std::launder

```
struct X { const int n;   };
union  U { X x, float f; };

void f()
{
    U u{{ 42 }};      // aggregate initialization, u.x.n == 42
    X *xp = new (&u.x) X { 99 };     // placement new


    std::cout << u.x.n << '\n';      // 42 ??
};
```

https://timsong-cpp.github.io/cppwp/n4659/basic.life#8

If, after the lifetime of an object has ended and before the storage which the object occupied is reused or released, a new object is created at the storage location which the original object occupied, a pointer that pointed to the original object, a reference that referred to the original object, or the name of the original object will automatically refer to the new object and, once the lifetime of the new object has started, can be used to manipulate the new object, if:

(8.1) the storage for the new object exactly overlays the storage location which the original object occupied, and
(8.2) the new object is of the same type as the original object (ignoring the top-level cv-qualifiers), and
(8.3) the type of the original object is not const-qualified, and, if a class type, does not contain any non-static data member whose type is const-qualified or a reference type, and
(8.4) the original object was a most derived object of type T and the new object is a most derived object of type T (that is, they are not base class subobjects).

std::launder

```
struct X { const int n;   };
union  U { X x, float f; };

void f()
{
    U u{{ 42 }};      // aggregate initialization, u.x.n == 42
    X *xp = new (&u.x) X { 99 };     // placement new

    auto ptr = std::launder(&u.x.n); // optimization firewall
    std::cout << u.x.n << '\n';      // 99
};
```

https://timsong-cpp.github.io/cppwp/n4659/basic.life#8

If, after the lifetime of an object has ended and before the storage which the object occupied is reused or released, a new object is created at the storage location which the original object occupied, a pointer that pointed to the original object, a reference that referred to the original object, or the name of the original object will automatically refer to the new object and, once the lifetime of the new object has started, can be used to manipulate the new object, if:

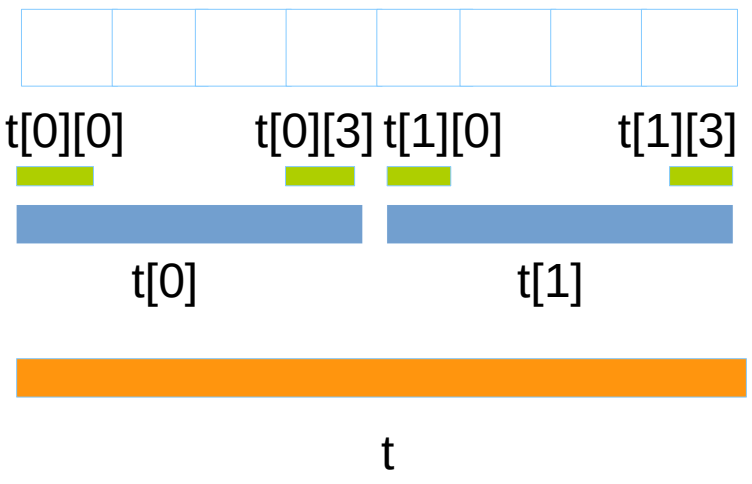
(8.1) the storage for the new object exactly overlays the storage location which the original object occupied, and
(8.2) the new object is of the same type as the original object (ignoring the top-level cv-qualifiers), and
(8.3) the type of the original object is not const-qualified, and, if a class type, does not contain any non-static data member whose type is const-qualified or a reference type, and
(8.4) the original object was a most derived object of type T and the new object is a most derived object of type T (that is, they are not base class subobjects).

Move semantics

- Pointers and arrays
- References
- Value vs move semantics
- Right value references
- Move semantics in C++11
- Perfect forwarding
- Traps and pitfalls

Arrays

- An array is a strictly continuous memory area.
- Arrays do not know their size, but: sizeof(t) / sizeof(t[0])
- Array names could be converted to pointer value to the first element.
- No multidimensional arrays. But there are arrays of arrays.
- No operations on arrays, only on array elements.




```
int t[2][4];

assert(sizeof(t) == 8*sizeof(int));
assert(sizeof(t[0] == 4*sizeof(int));
assert(sizeof(t[0][0])==sizeof(int));

t[0][1] = t[1][1];
// t[0] = t[1];  syntax error
```

Array decay

- Array to pointer conversion (the first of the “standard conversions”)
- Reference binding can avoid decay

```
void  aDecay( int *ap)      {std::cout << sizeof(ap) << "\n";}
void  pDecay( int (*p)[6]) {std::cout << sizeof(p)  << "\n";}
void noDecay( int (&a)[6]) {std::cout << sizeof(a)  << "\n";}
template <typename T, int N>
void tNoDecay(T (&a)[N]) { std::cout << sizeof(T) << " " << N
                                << " " <<sizeof(a) << "\n"; }
void f()
{
  int t[6] = { 0,1,2,3,4,5 };
  std::cout << sizeof(t) << "\n";  // 24
  aDecay(t);                       //  8
  pDecay(&t);                      //  8
  noDecay(t);                      // 24
  tNoDecay(t);                     // 4 6 24
}
```

Pointers

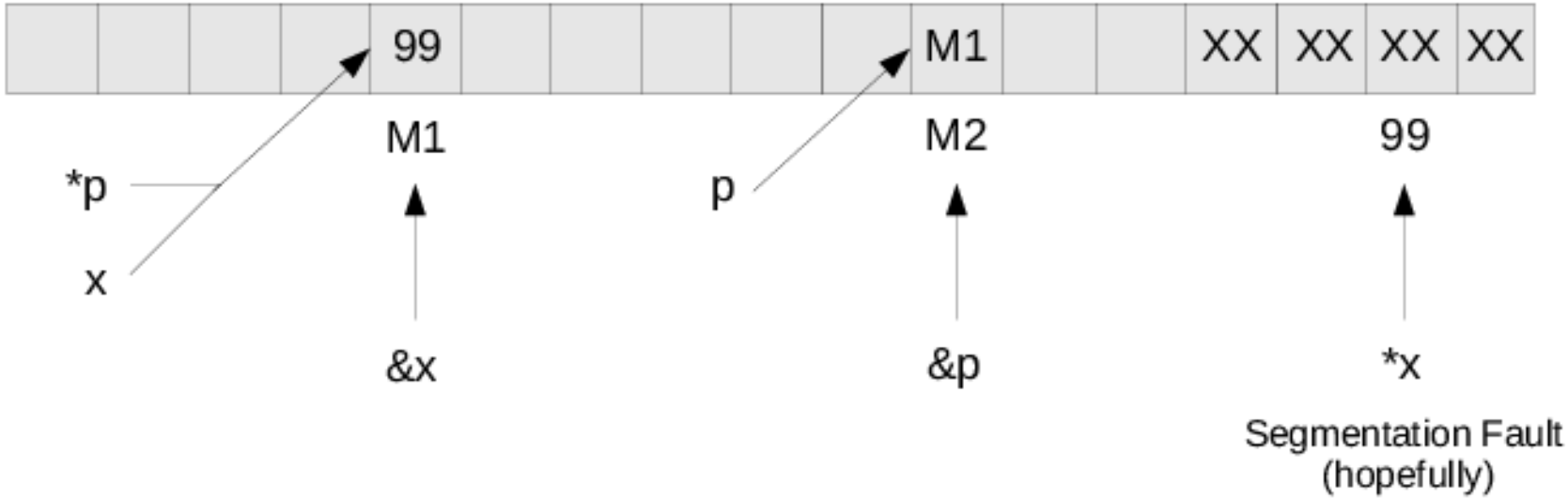
Pointers in C

C Code

```
int x;
int *p;

x = 99;    //holds a value
p = &x;    //holds an address of a value
```

Memory



Pointers

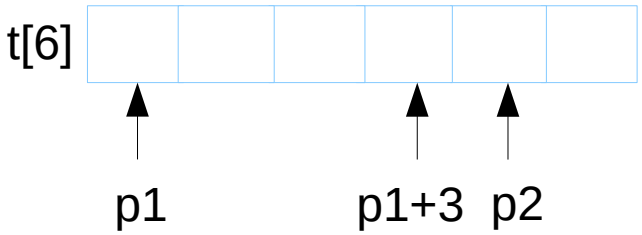
- Pointer is a value that refers to a(n abstract) memory location
- Pointers can refer to ANY valid memory locations (unlike e.g. PASCAL)
- **nullptr** (NULL) is a universal null-pointer value
- Non-null pointers are considered as TRUE value

```
char *ptr = (char *) malloc(1024);

if ( ptr )
{
   // ptr != NULL here
}
```

Pointer arithmetics

- Integers can be added and subtracted from pointers
- Pointers pointing to the same array can be subtracted
- Pointer arithmetics depends on the pointed type!
- No pointer arithmetics on void *




```
int t[6];
int *p1 = &t[0];
int *p2 = &t[4];

assert( &t[3] == p1+3 );
assert( p2 - p1 == 4 );
assert( p1 + 4 == p2 );
```

```
  p = &t[0];
  p = t;
  p + k == &t[k]
*(p + k) == t[k]
```

Pointers and arrays

- Pointers and array names can be used similarly in expressions

```
int t[10];
int i = 3;

int *p = t;       // t is used as pointer to the first element

p[i] = *(t + i); // p used as array, t is used as pointer
```

But pointers ARE NOT EQUIVALENT TO arrays !!!

Pointers and arrays

```c
// s1.c
#include <stdio.h>

int t[] = {1,2,3};

void f( int *par)
{
  printf("%d",par[1]);
  printf("%d",t[1]);
}
int main()
{
  int *p = t;
  printf("%d",t[1]);
  printf("%d",p[1]);
  f(t);
  g(t);
}
```

```c
// s2.c
#include <stdio.h>

extern int *t;

void g( int *par)
{
  printf("%d",par[1]);
  printf("%d",t[1]);
  // the program crashes here
}
```

Pointers and arrays

```c
// s1.c
#include <stdio.h>

int t[] = {1,2,3};

void f( int *par)
{
  printf("%d",par[1]);
  printf("%d",t[1]);
}
int main()
{
  int *p = t;
  printf("%d",t[1]);
  printf("%d",p[1]);
  f(t);
  g(t);
}
```

```c
// s2.c
#include <stdio.h>

extern int *t;

void g( int *par)
{
  printf("%d",par[1]);
  printf("%d",t[1]);
  // the program crashes here
}
```

Pointers and arrays

COMPILER EXPLORER

Add... More

Support diversity in C++ with #include <C++>

C++ source #1

C++

```cpp
int t[] = { 1, 2, 3};

int main()
{
  int *p = t;
  int x = t[1];
  int y = p[1];
}
```

x86-64 gcc 9.2 (Editor #1, Compiler #1) C++

x86-64 gcc 9.2

Compiler options...

11010 ./a.out .LX0: lib.f: .text // \s+ Inte

```asm
t:
        .long   1
        .long   2
        .long   3
main:
        push    rbp
        mov     rbp, rsp
        mov     QWORD PTR [rbp-8], OFFSET FLAT:t
        mov     eax, DWORD PTR t[rip+4]
        mov     DWORD PTR [rbp-12], eax
        mov     rax, QWORD PTR [rbp-8]
        mov     eax, DWORD PTR [rax+4]
        mov     DWORD PTR [rbp-16], eax
        mov     eax, 0
        pop     rbp
        ret
```

Member pointers

- Data Member pointer: Referencing to an offset inside a class
- Member function pointer: Referencing to a (possible virtual) member function of a class
- Works with 2 components: **this + mptr**

```
Type Class::*dmptr;
Type (Class::*fmptr)(P1 par1, P2 par2, …);

Class obj;
Class *ptr = &obj;

obj.*dmptr = …;
ptr->*dmptr = …;

(obj.*fmptr)(par1,par2);
(ptr->*fmptr)(par1,par2);
```

Member pointers

```
#include <iostream>

class Date
{
public:
  void set (int y, int m, int d);
  int  getYear() const  { return _year; }
  int  getMonth() const { return _month; }
  int  getDay() const   { return _day; }

  void print(std::ostream& os) const;
  void hu();
  void us();
private:
  int _year;
  int _month;
  int _day;

  int Date::*p1;
  int Date::*p2;
  int Date::*p3;
  char sep;
};
```

Member pointers

```
void Date::hu()
{
  sep = '.';
  p1 = &Date::_year;
  p2 = &Date::_month;
  p3 = &Date::_day;
}
void Date::us()
{
  sep = '/';
  p1 = &Date::_month;
  p2 = &Date::_day;
  p3 = &Date::_year;
}
int main()
{
  Date d;
  d.set(2017,4,20);
  d.hu();
  std::cout << d << std::endl;
  d.us();
  std::cout << d << std::endl;
}
```

```
void Date::set(int y, int m, int d)
{
  _year = y;
  _month = m;
  _day = d;
}

void Date::print(std::ostream& os) const
{
  os << this->*p1 << sep << this->*p2
     << sep << this->*p3;
}

std::ostream& operator<<(
         std::ostream& os, const Date& d)
{
  d.print(os);
  return os;
}
```

```
2017.4.20
4/20/2017
```

Member pointers

```
  int (Date::*g1)() const;
  int (Date::*g2)() const;
  int (Date::*g3)() const;
};

void Date::hu()
{
  sep = '.';
  g1 = &Date::getYear;
  g2 = &Date::getMonth;
  g3 = &Date::getDay;
}
void Date::us()
{
  sep = '/';
  g1 = &Date::getYear;
  g2 = &Date::getMonth;
  g3 = &Date::getDay;
}
int main()
{
  Date d;
  d.set(2017,4,20);
  d.hu();
  std::cout << d << std::endl;
  d.us();
  std::cout << d << std::endl;
}
```

```
void Date::set(int y, int m, int d)
{
  _year = y;
  _month = m;
  _day = d;
}

void Date::print(std::ostream& os) const
{
  os << (this->*g1)() << sep
     << (this->*g2)() << sep
     << (this->*p3)();
}

std::ostream& operator<<(
         std::ostream& os, const Date& d)
{
  d.print(os);
  return os;
}
```

```
2017.4.20
4/20/2017
```

Nullptr

- nullptr is a new literal since C++11 of type std::nullptr_t
- Helps to overload between pointers and integer
- Automatic conversion from null pointer of any type and from NULL

```
void f(int*); // 1
void f(int);  // 2

f(0);         // calls 2
f(nullptr);   // calls 1
```

Reference

- In modern languages definitions hide two important but orthogonal concepts:
  - Allocate memory for a variable
  - Bind a name with special scope to it
- In most languages this is inseparable
- In C++ we can separate the two steps

```
void f()
{
  int i;                  // allocate memory, bind i as name
  int &i_ref = i;         // binds a new name
  int *iptr = new int;    // allocate memory, no binded name
  int &k = *iptr;         // binds a new name to unnamed int area
  delete iptr;            // memory invalidated name k still lives
  k = 5;                  // compiles, later run-time error
}                         // k goes out of scope
```

Pointer vs reference

- Pointers have extremal element: nullptr
  - nullptr means: pointing to no valid memory
- References always should refer to valid memory
  - Use exception, if something fails

```
if ( Derived *dp = dynamic_cast<Derived*>(bp) )
{
    // use dp as Derived*
}

try
{
  Derived &dr = dynamic_cast<Derived&>(*bp); // may throw
  // use dr as Derived&
}
catch(bad_cast &e) { . . . }
```

Parameter passing

- Parameter passing in C++ follows initialization semantics
  - Value initialization copies the object
  - Reference initialization just set up an alias

```
void f1( int x, int y) { ... }
void f2( int &xr, int &yr) { ... }

int i = 5, j = 6;

f1( i, j);   int x = i;  // creates local x and copies i to x
             int y = j;  // creates local y and copies j to y

f2( i, j);   int &xr = i; // binds xr name to i outside
             int &yr = j; // binds yr name to j outside
```

Swap before move semantics

```
void swap( int &x, int &y)
{
  int tmp = x;
  x = y;
  y = tmp;
}

int main()
{
  int i = 5;
  int j = 6;

  swap( i, j);
  assert(i==6 && j==5);
}
```

Reference binding

- Non-const (left) reference binds only to left value
- Const reference binds to right values too

```
int i = 5, j = 6; double d = 7.0;

swap(i,j);      // ok
swap(i,7);      // error: could not bind reference to 7
int &ir1 = 7;   // error: could not bind reference to 7

swap(i,d);              // error: int(d) creates non-left value
int &ir2 = int(3.14); // error: int(3.14) creates non-left value

const int &ir3 = 7;          // ok, lifetime extension
const int &ir4 = int(3.14); // ok, lifetime extension
```

Returning with reference

- By default C++ functions return with copy
- Returning reference just binds the function result to an object

```
int f1()
{
  int i = 5;
  return i;   // ok, copies i before evaporating
}

int &f2()
{
  int i = 5;
  return i;   // oops, binds to evaporating i
}
```

Usage example

```
class date
{
public:
  date& setYear(int y)  { _year = y;  return *this; }
  date& setMonth(int m) { _month = m; return *this; }
  date& setDay(int d)   { _day = d;   return *this; }

  date& operator++() { ++_day; return *this; }
  date  operator++(int) // should return temporary
                { date curr(*this); ++_day; return curr; }
private:
  int _year;
  int _month;
  int _day;
};

void f()
{
  date d;
  ++d.setYear(2011).setMonth(11).setDay(11); // still left value
}
```

Usage example

```
template <typename T>
class matrix
{
public:
      T& operator()(int i, int j)       { return v[i*cols+j]; }
const T& operator()(int i, int j) const { return v[i*cols+j]; }
    matrix& operator+=( const matrix& other)
    {
        for (int i = 0; i < cols*rows; ++i)
            v[i] += other.v[i];
        return *this;
    }
private:
    // ...
    T* v;
};
template <typename T> matrix<T>  // returns value
operator+(const matrix<T>& left, const matrix<T>& right)
{
    matrix<T> result(left);  // copy constructor
    result += right;
    return result;
}
```

Left vs right value

- Assignment in earlier languages work the following way:
  <variable> = <expression>, like x = a+5;

- In C/C++ however it can be:
  <expression> = <expression>, like *++ptr = *++qtr;

- But not all expressions are valid, like a+5 = x;

  An **lvalue** is an expression that refers to a memory location and allows us to take the address of that memory location via the & operator. An **rvalue** is an expression that is not an lvalue

- A rigorous definition of lvalue and rvalue:
  https://accu.org/journals/overload/12/61/kilpelainen_227/

Value categories

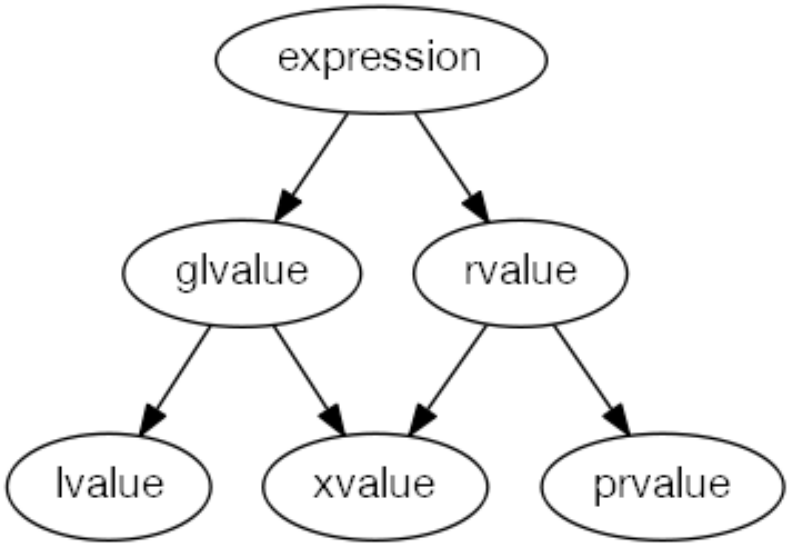
— An lvalue (so called, historically, because lvalues could appear on the left-hand side of an assignment expression) designates a function or an object. [Example: If E is an expression of pointer type, then *E is an lvalue expression referring to the object or function to which E points. As another example, the result of calling a function whose return type is an lvalue reference is an lvalue. —end example]

— An xvalue (an “eXpiring” value) also refers to an object, usually near the end of its lifetime (so that its resources may be moved, for example). An xvalue is the result of certain kinds of expressions involving rvalue references (8.3.2). [Example: The result of calling a function whose return type is an rvalue reference is an xvalue. —end example]

— A glvalue (“generalized” lvalue) is an lvalue or an xvalue.

— An rvalue (so called, historically, because rvalues could appear on the right-hand side of an assignment expressions) is an xvalue, a temporary object (12.2) or subobject thereof, or a value that is not associated with an object.

— A prvalue (“pure” rvalue) is an rvalue that is not an xvalue. [Example: The result of calling a function whose return type is not a reference is a prvalue. The value of a literal such as 12, 7.3e5, or true is also a prvalue. —end example]



Left value vs. right value

```
int  i = 42;
int &j = i;
int *p = &i;

 i = 99;
 j = 88;
*p = 77;

int *fp() { return &i; } // returns pointer to i: lvalue
int &fr() { return i; }  // returns reference to i: lvalue

*fp() = 66;  // i = 66
 fr() = 55;  // i = 55

// rvalues:

int f() { int k = i; return k; } // returns rvalue

  i = f();  // ok
  p = &f(); // bad: can't take address of rvalue
f() = i;    // bad: can't use rvalue on left-hand-side
```

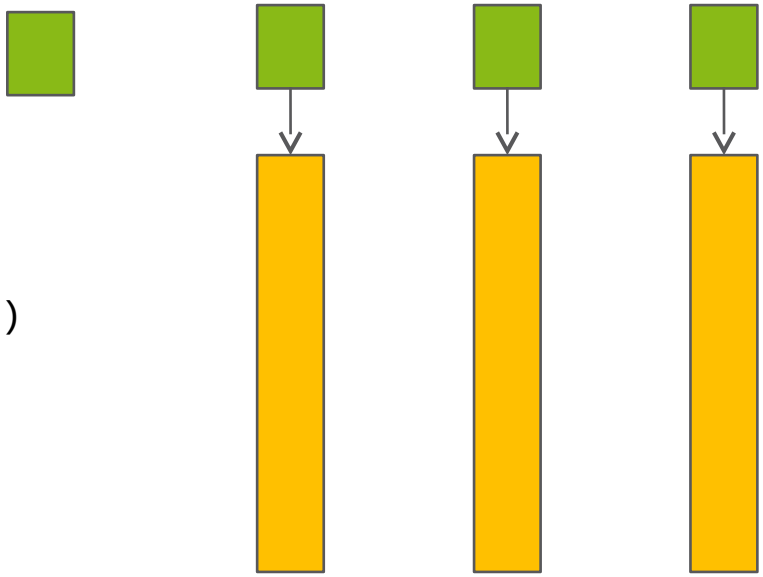
Value semantics

- C++ has value semantics
  - Clear separation of memory areas
  - Significant performance loss when copying large objects
  - This can lead to improper use of (smart) pointers

Value semantics

```
class Array
{
public:
   Array (const Array&);
   Array& operator=(const Array&);
   ~ Array ();
private:
   double  *val;
};
Array operator+(const Array& left,const Array& right)
{
   Array res = left;
   res += right;
   return res;
}

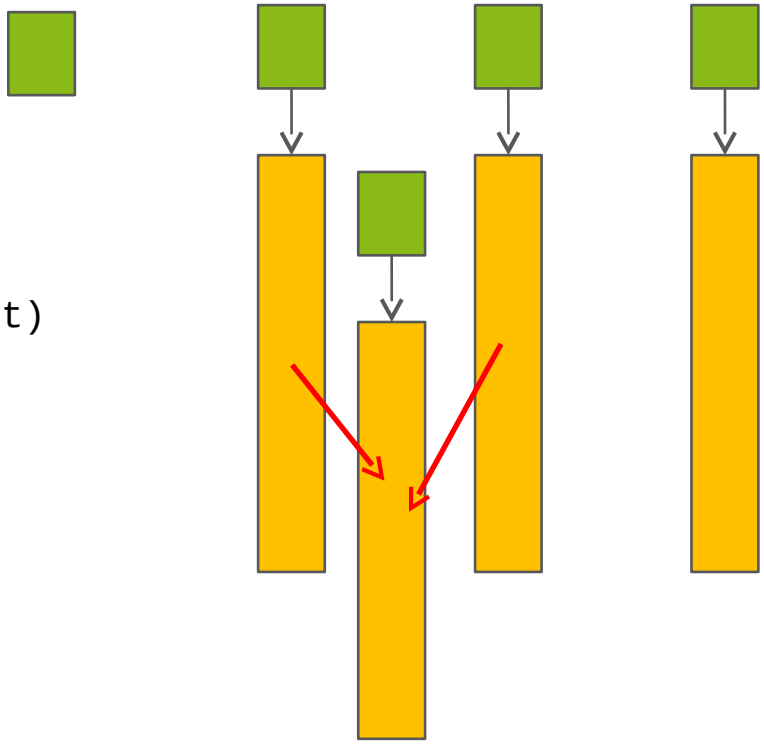
void f()
{
   Array b, c, d;
   …
   Array a = b + c + d;
}
```



Value semantics

```
class Array
{
public:
   Array (const Array&);
   Array& operator=(const Array&);
   ~ Array ();
private:
   double  *val;
};
Array operator+(const Array& left, const Array& right)
{
   Array res = left;
   res += right;
   return res;
}

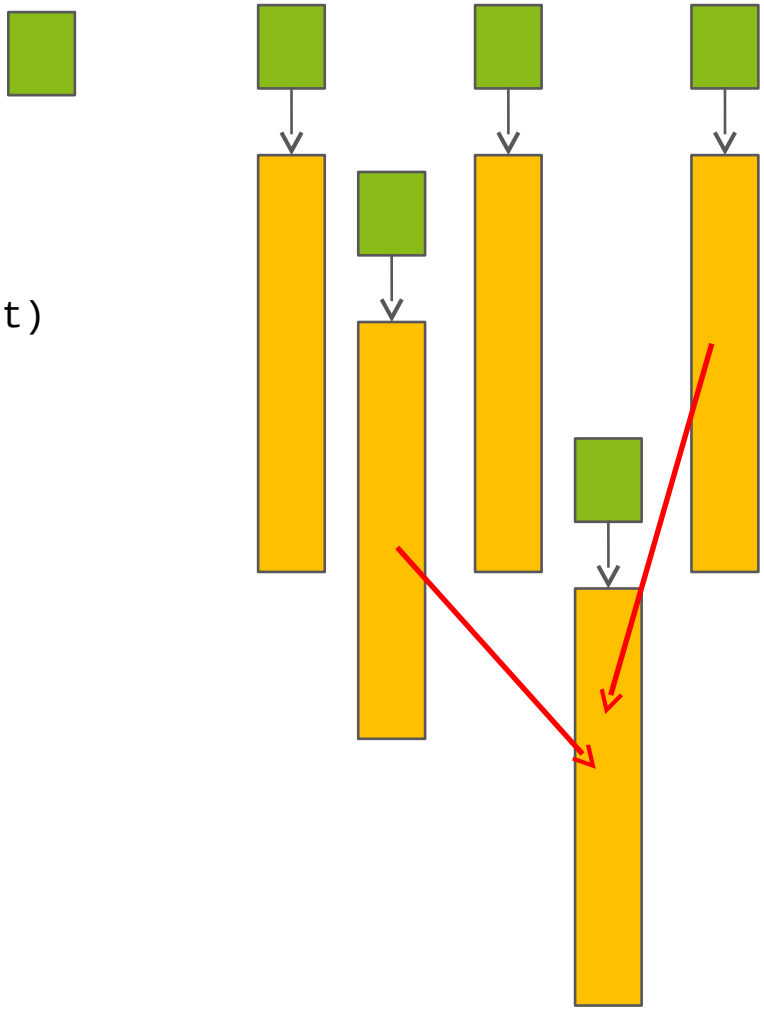
void f()
{
   Array b, c, d;
   …
   Array a = b + c + d;
}
```



Value semantics

```
class Array
{
public:
   Array (const Array&);
   Array& operator=(const Array&);
   ~ Array ();
private:
   double  *val;
};
Array operator+(const Array& left, const Array& right)
{
   Array res = left;
   res += right;
   return res;
}

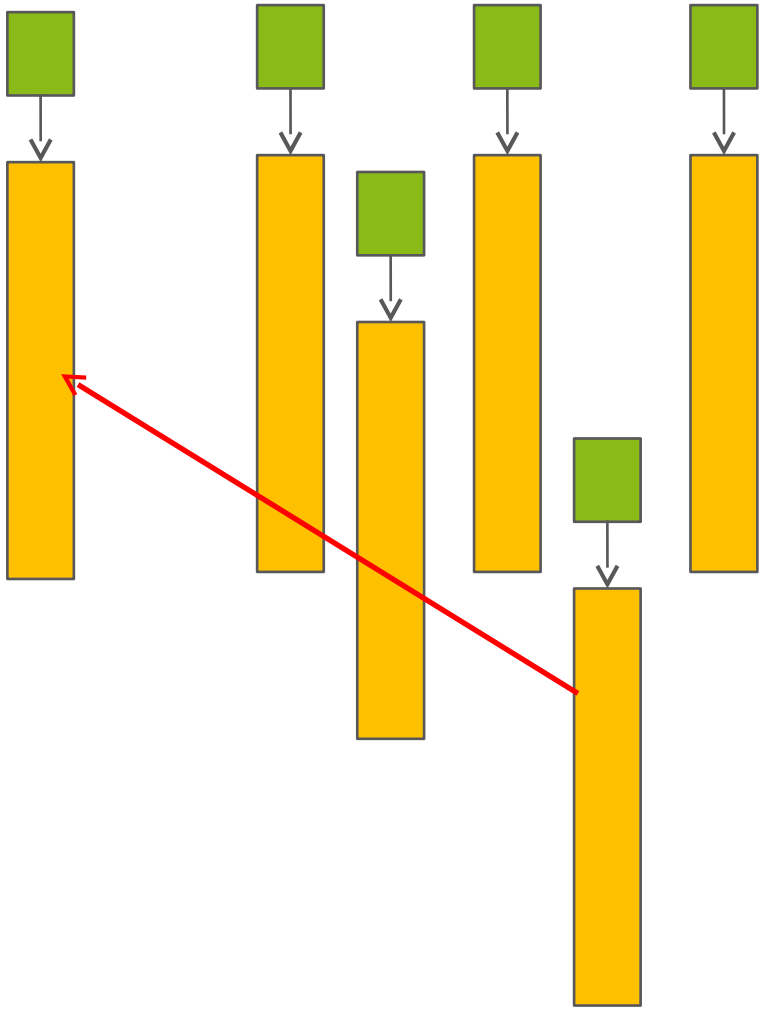
void f()
{
   Array b, c, d;
   …
   Array a = b + c + d;
}
```



Value semantics

```
class Array
{
public:
   Array (const Array&);
   Array& operator=(const Array&);
   ~ Array ();
private:
   double  *val;
};
Array operator+(const Array& left, const Array& righ
{
   Array res = left;
   res += right;
   return res;
}

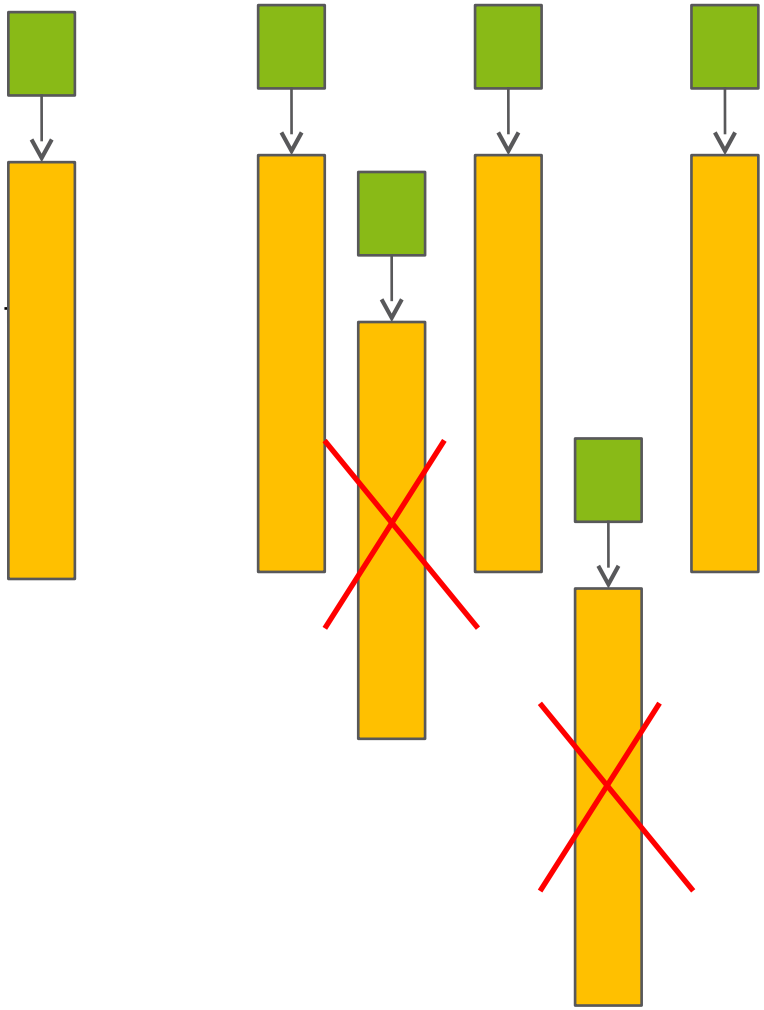
void f()
{
   Array b, c, d;
   …
   Array a = b + c + d;
}
```



Value semantics

```
class Array
{
public:
   Array (const Array&);
   Array& operator=(const Array&);
   ~ Array ();
private:
   double  *val;
};
Array operator+(const Array& left, const Array& righ
{
   Array res = left;
   res += right;
   return res;
}

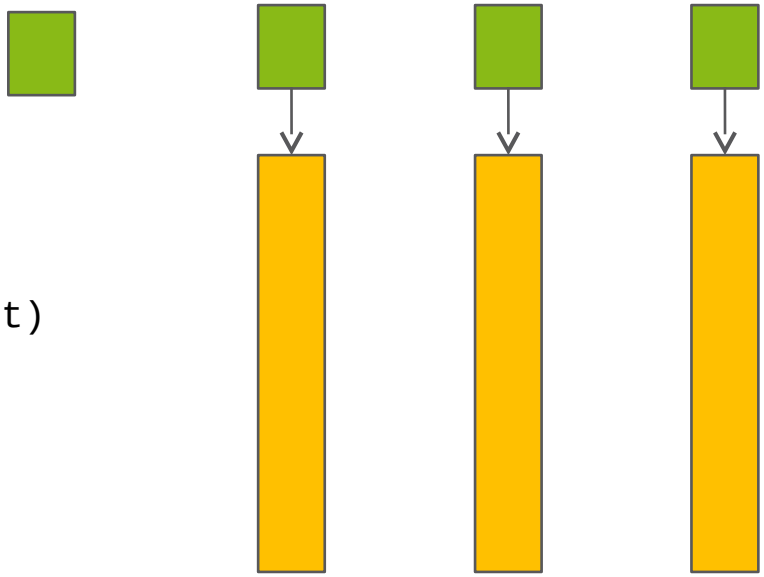
void f()
{
   Array b, c, d;
   …
   Array a = b + c + d;
}
```



Move semantics

```
class Array
{
public:
   Array (const Array&);
   Array& operator=(const Array&);
   ~ Array ();
private:
   double  *val;
};
Array operator+(const Array& left, const Array& right)
{
   Array res = left;
   res += right;
   return res;
}

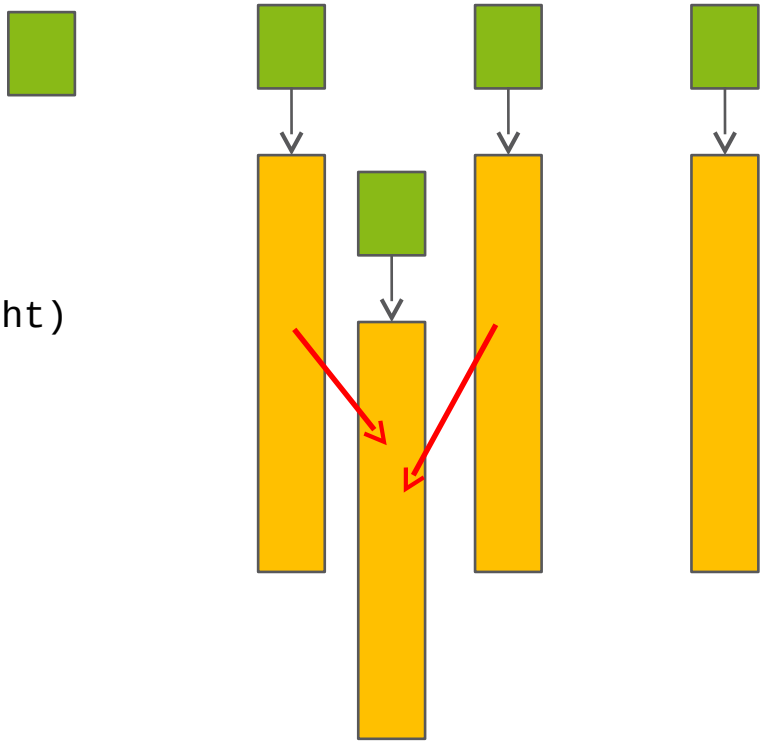
void f()
{
   Array b, c, d;
   …
   Array a = b + c + d;
}
```



Move semantics

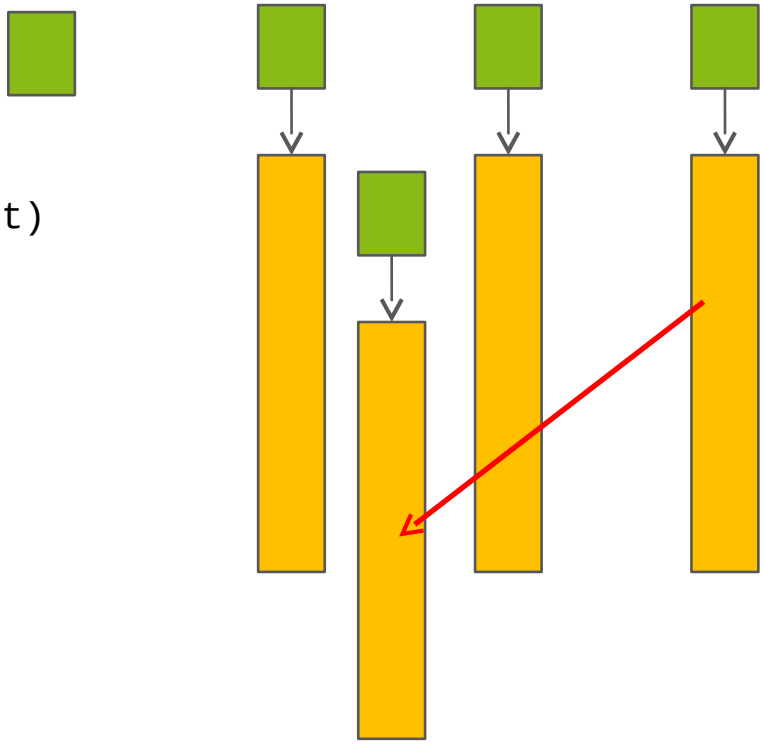
```
class Array
{
public:
   Array (const Array&);
   Array& operator=(const Array&);
   ~ Array ();
private:
   double  *val;
};
Array operator+(const Array& left, const  Array& right)
{
   Array res = left;
   res += right;
   return res;
}

void f()
{
   Array b, c, d;
   …
   Array a = b + c + d;
}
```



Move semantics

```
class Array
{
public:
   Array (const Array&);
   Array& operator=(const Array&);
   …
};
Array operator+(const Array& left, const Array& right)
{
   Array res = left;
   res += right;
   return res;
}
Array operator+(Array&& left, const Array& right)
{
   left += right;
   return left;
}
void f()
{
   Array b, c, d;
   …
   Array a = b + c + d;
}
```

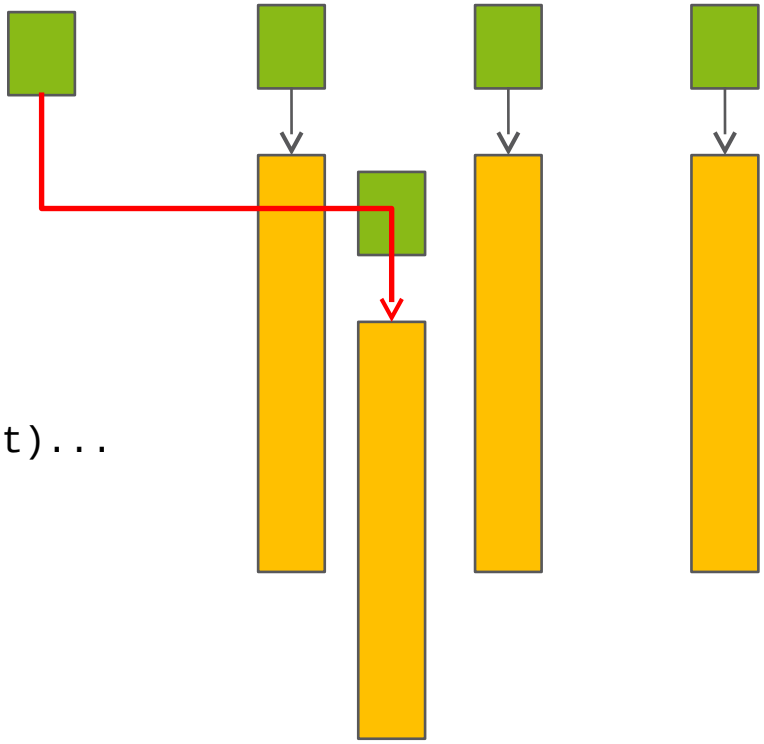


Move semantics

```
class Array
{
public:
   Array (const Array&);
   Array (Array&&);
   Array& operator=(const Array&);
   Array& operator=(Array&&);
   ~ Array ();
private:
   double  *val;
};

Array operator+(const Array& left, const Array& right)...
Array operator+(Array&& left, const Array& right)...


void f()
{
   Array b, c, d;
   …
   Array a = b + c + d;
}
```



Right value reference

- For overloading, we need a new type
  - Reference type for performance reasons
  - Overload resolution should prefer this new type on rvalue objects

```
void f(X& arg_)       // lvalue reference parameter
void f(X&& arg_)      // rvalue reference parameter
void f(const X& arg_) // const lvalue reference parameter

X x;
X g();

f(x);    // lvalue argument --> f(X&)
f(g());  // rvalue argument --> f(X&&)
```

Move semantics

- Move semantics
  - Instead of copying steal the resources
  - Leave the other object in a destructible state
  - Rule of three becomes rule of five
  - All standard library components were extended

- Reverse compatibility
  - If we implement the old-style member functions with lvalue reference but do not implement the rvalue reference overloading versions
    we keep the old behaviour -> gradually move to move semantics.
  - If we implement only rvalue operations we cannot call these on lvalues -> no default copy ctor or **operator**= will be generated.

- Serious performance gain
  - Except some rare RVO situations

Special member functions

```
class X
{
public:
  X(const X& rhs);
  X(X&& rhs);

  X& operator=(const X& rhs);    // = default  or = delete
  X& operator=(X&& rhs);
private:
  // ...
};
X& X::operator=(const X& rhs)
{
  // free old resources than allocate and copy resource from rhs
  return *this;
}
X& X::operator=(X&& rhs)  // draft version, will be revised
{
  // free old resources than move resource from rhs
  // leave rhs in a valid, destructable state
  return *this;
}
```

Generation of special member functions

1. The two **copy operations** (copy constructor and copy assignment) **are independent.** Declaring copy constructor does nor prevent compiler to generate copy assignment (and vice versa). (same as in C++98)

2. **Move operations are not independent**. Declare either prevents the compiler to generate the other.

3. If any of the **copy operation is declared**, then **none of the move** operation will be generated.

4. If any of the **move operation is declared**, then **none of the copy** operation will be generated. This is the opposite rule of (3).

5. If a **destructor is declared**, than **none of the move** operation will be generated. Copy operations are still generated for reverse compatibility with C++98.

6. **Default constructor** generated only no constructor is declared (same as in C++98)

Move operations

- For reverse compatibility, move operations are generated only when
  - No copy operations are declared
  - No move operations are declared
  - No destructor is declared
- Function templates do not considered here
  - Templated copy constructor, assignment does not prevent move operation generations
  - Same rule since C++98 with copy operations
- Play safe!

  … it is easy...
- Really?

A simple program

```
#include <iostream>
#include <vector>

struct S
{
    S() { a = ++cnt; }
    int a;
    static int cnt;
};
int S::cnt = 0;

int main()
{
    std::vector<S>  sv(5);
    sv.push_back(S());

    for (std::size_t i = 0; i< sv.size(); ++i)
        std::cout << sv[i].a << " ";
    std::cout << std::endl;
}
```

First amendment to the C++ standard

"The committee shall make no rule that prevents C++ programmers from shooting themselves in the foot."

quoted by Thomas Becker

http://thbecker.net/articles/rvalue_references/section_04.html

std::move

```
struct S
{
    S() { a = ++cnt; std::cout << "S() ";  }
    S(const S& rhs)    { a = rhs.a; std::cout << "copyCtr "; }
    S(S&& rhs)         { a = rhs.a; std::cout << "moveCtr "; }
    S& operator=(const S& rhs) { a = rhs.a; std::cout << "copy= "; return *this; }
    S& operator=(S&& rhs)      { a = rhs.a; std::cout << "move= "; return *this; }
    int a;
    static int cnt;
};
int S::cnt = 0;

template<class T>
void swap(T& a, T& b)
{
    T tmp(a);
    a = b;
    b = tmp;
}
int main()
{
    S a, b;
    swap( a, b);
}
```

std::move

```
struct S
{
    S() { a = ++cnt; std::cout << "S() ";  }
    S(const S& rhs)    { a = rhs.a; std::cout << "copyCtr "; }
    S(S&& rhs)         { a = rhs.a; std::cout << "moveCtr "; }
    S& operator=(const S& rhs) { a = rhs.a; std::cout << "copy= "; return *this; }
    S& operator=(S&& rhs)      { a = rhs.a; std::cout << "move= "; return *this; }
    int a;
    static int cnt;
};
int S::cnt = 0;

template<class T>
void swap(T& a, T& b)
{
    T tmp(a);
    a = b;
    b = tmp;
}
int main()
{
    S a, b;
    swap( a, b);
}
```

```
$ ./a.out
S() S() copyCtr copy= copy=
```

std::move

```
struct S
{
    S() { a = ++cnt; std::cout << "S() ";  }
    S(const S& rhs)    { a = rhs.a; std::cout << "copyCtr "; }
    S(S&& rhs)         { a = rhs.a; std::cout << "moveCtr "; }
    S& operator=(const S& rhs) { a = rhs.a; std::cout << "copy= "; return *this; }
    S& operator=(S&& rhs)      { a = rhs.a; std::cout << "move= "; return *this; }
    int a;
    static int cnt;
};
int S::cnt = 0;

template<class T>
void swap(T& a, T& b)
{
    T tmp(a);
    a = b;
    b = tmp;
}
int main()
{
    S a, b;
    swap( a, b);
}
```

```
$ ./a.out
S() S() copyCtr copy= copy=
```

If it has a name: LVALUE

std::move

```
struct S
{
    S() { a = ++cnt; std::cout << "S() ";  }
    S(const S& rhs)    { a = rhs.a; std::cout << "copyCtr "; }
    S(S&& rhs)         { a = rhs.a; std::cout << "moveCtr "; }
    S& operator=(const S& rhs) { a = rhs.a; std::cout << "copy= "; return *this; }
    S& operator=(S&& rhs)      { a = rhs.a; std::cout << "move= "; return *this; }
    int a;
    static int cnt;
};
int S::cnt = 0;

template<class T>
void swap(T& a, T& b)
{
    T tmp(std::move(a));
    a = std::move(b);
    b = std::move(tmp);
}
int main()
{
    S a, b;
    swap( a, b);
}
```

std::move

```
struct S
{
    S() { a = ++cnt; std::cout << "S() ";  }
    S(const S& rhs)    { a = rhs.a; std::cout << "copyCtr "; }
    S(S&& rhs)         { a = rhs.a; std::cout << "moveCtr "; }
    S& operator=(const S& rhs) { a = rhs.a; std::cout << "copy= "; return *this; }
    S& operator=(S&& rhs)      { a = rhs.a; std::cout << "move= "; return *this; }
    int a;
    static int cnt;
};
int S::cnt = 0;

template<class T>
void swap(T& a, T& b)
{
    T tmp(std::move(a));
    a = std::move(b);
    b = std::move(tmp);
}
int main()
{
    S a, b;
    swap( a, b);
}
```

```
$ ./a.out
S() S() moveCtr move= move=
```

std::move(x)

- Right value reference cast
- Usually has positive effect of performance
  - Many standard lib function utilize right-value references
- Sometimes we have to use it
  - Movable non-copyable classes
  - std::unique_ptr, std::fstream, std::thread
- Might be dangerous
  - A variable with name left with unspecified value

Rvalue and constness

```
#include <iostream>

struct MyString {
    MyString() { std::cerr << "defCtor" << std::endl; }
    MyString(const MyString& rhs) { std::cerr << "copyCtor" << std::endl; }
    MyString(MyString&& rhs) { std::cerr << "moveCtor" << std::endl; }
};
class MoveConstr {
public:
    MoveConstr( MyString s) : value(std::move(s)) { }
private:
    MyString value;
};
int main() {
    MoveConstr mc(MyString());
    return 0;
}
```

Most vexing parse

```
MoveConstr mc(MyString())
```

- Function declaration:
  - mc returning MoveConstr type
  - One parameter: a pointer to function returns MyString and has no parameter

```
$ clang++ -std=c++11 -Wvexing-parse m1.cpp
m1.cpp:21:16: warning: parentheses were disambiguated as a function declaration
      [-Wvexing-parse]
  MoveConstr mc(MyString());
               ^~~~~~~~~~~~
m1.cpp:21:17: note: add a pair of parentheses to declare a variable
  MoveConstr mc(MyString());
                ^
                (         )
1 warning generated.
```

Rvalue and constness

```
#include <iostream>

struct MyString {
    MyString() { std::cerr << "defCtor" << std::endl; }
    MyString(const MyString& rhs) { std::cerr << "copyCtor" << std::endl; }
    MyString(MyString&& rhs) { std::cerr << "moveCtor" << std::endl; }
};
class MoveConstr {
public:
    MoveConstr( MyString s) : value(std::move(s)) { }
private:
    MyString value;
};
int main() {
    MoveConstr mc{ MyString() };     // MoveConstr mc((MyString()));
    return 0;
}
$ ./a.out
defCtor
moveCtor
```

Rvalue and constness

```
#include <iostream>

struct MyString {
    MyString() { std::cerr << "defCtor" << std::endl; }
    MyString(const MyString& rhs) { std::cerr << "copyCtor" << std::endl; }
    MyString(MyString&& rhs) { std::cerr << "moveCtor" << std::endl; }
};
class MoveConstr {
public:
    MoveConstr( const MyString s) : value(std::move(s)) { }
private:
    MyString value;
};
int main() {
    MoveConstr mc{ MyString() };
    return 0;
}
$ ./a.out
defCtor
copyCtor
```

Inheritance

```
class Base
{
public:
    Base(const Base& rhs);    // non-move semantics
    Base(Base&& rhs);            // move semantics
};
class Derived : public Base
{
    Derived(const Derived& rhs);  // non-move semantics
    Derived(Derived&& rhs);          // move semantics
};
Derived(Derived const & rhs) : Base(rhs)  // non-move semantics
{
// copy derived specific…
}
Derived(Derived&& rhs) : Base(rhs)           // move semantics
{
// move derived specific…
}
```

Inheritance

```
class Base
{
public:
    Base(const Base& rhs);    // non-move semantics
    Base(Base&& rhs);            // move semantics
};
class Derived : public Base
{
    Derived(const Derived& rhs);  // non-move semantics
    Derived(Derived&& rhs);          // move semantics
};
Derived(Derived const & rhs) : Base(rhs)  // non-move semantics
{
// copy derived specific…
}
Derived(Derived&& rhs) : Base(rhs)           // wrong!
{
// move derived specific…
}
```

Inheritance

```
class Base
{
public:
    Base(const Base& rhs);    // non-move semantics
    Base(Base&& rhs);            // move semantics
};
class Derived : public Base
{
    Derived(const Derived& rhs);  // non-move semantics
    Derived(Derived&& rhs);          // move semantics
};
Derived(Derived const & rhs) : Base(rhs)  // non-move semantics
{
// copy derived specific…
}
Derived(Derived&& rhs) : Base(std::move(rhs))        // good, calls Base(Base&& rhs)
{
// move derived specific…
}
```

Vector

```
#include <iostream>
#include <vector>

struct S
{
    S() { a = ++cnt; std::cout << "S() ";  }
    S(const S& rhs)    { a = rhs.a; std::cout << "copyCtr ";  }
    S(S&& rhs)         { a = rhs.a; std::cout << "moveCtr "; }
    S& operator=(const S& rhs) { a = rhs.a; std::cout << "copy= "; return *this; }
    S& operator=(S&& rhs)      { a = rhs.a; std::cout << "move= "; return *this; }
    int a;
    static int cnt;
};
int S::cnt = 0;

int main()
{
    std::vector<S>  sv(5);
    sv.push_back(S());

    for (std::size_t i = 0; i < sv.size(); ++i)
        std::cout << sv[i].a << " ";
    std::cout << std::endl;
}
```

Vector

```
#include <iostream>
#include <vector>

struct S
{
    S() { a = ++cnt; std::cout << "S() ";  }
    S(const S& rhs)    { a = rhs.a; std::cout << "copyCtr ";  }
    S(S&& rhs)         { a = rhs.a; std::cout << "moveCtr "; }
    S& operator=(const S& rhs) { a = rhs.a; std::cout << "copy= "; return *this; }
    S& operator=(S&& rhs)      { a = rhs.a; std::cout << "move= "; return *this; }
    int a;
    static int cnt;
};
int S::cnt = 0;

int main()
{
    std::vector<S>  sv(5);
    sv.push_back(S());

    for (std::size_t i = 0; i < sv.size(); ++i)
        std::cout << sv[i].a << " ";
    std::cout << std::endl;
}
```

```
$ g++ -std=c++11  vec.cpp &&  ./a.out
S() S() S() S() S() S() moveCtr
copyCtr copyCtr copyCtr copyCtr copyCtr
1 2 3 4 5 6
```

Vector

```
#include <iostream>
#include <vector>

struct S
{
    S() { a = ++cnt; std::cout << "S() ";  }
    S(const S& rhs)    { a = rhs.a; std::cout << "copyCtr ";  }
    S(S&& rhs)         { a = rhs.a; std::cout << "moveCtr "; }
    S& operator=(const S& rhs) { a = rhs.a; std::cout << "copy= "; return *this; }
    S& operator=(S&& rhs)      { a = rhs.a; std::cout << "move= "; return *this; }
    int a;
    static int cnt;
};
int S::cnt = 0;

int main()
{
    std::vector<S>  sv(5);
    sv push_back(S());

    for (std::size_t i = 0; i < sv.size(); ++i)
        std::cout << sv[i].a << " ";
    std::cout << std::endl;
}
```

```
$ g++ -std=c++11  vec.cpp &&  ./a.out
S() S() S() S() S() S() moveCtr
copyCtr copyCtr copyCtr copyCtr copyCtr
1 2 3 4 5 6
```

Vector

```
#include <iostream>
#include <vector>

struct S
{
    S() { a = ++cnt; std::cout << "S() ";  }
    S(const S& rhs)    { a = rhs.a; std::cout << "copyCtr ";  }
    S(S&& rhs)   noexcept    { a = rhs.a; std::cout << "moveCtr "; }
    S& operator=(const S& rhs) { a = rhs.a; std::cout << "copy= "; return *this; }
    S& operator=(S&& rhs)      { a = rhs.a; std::cout << "move= "; return *this; }
    int a;
    static int cnt;
};
int S::cnt = 0;

int main()
{
    std::vector<S>  sv(5);
    sv.push_back(S());

    for (std::size_t i = 0; i < sv.size(); ++i)
        std::cout << sv[i].a << " ";
    std::cout << std::endl;
}
```

Vector

```
#include <iostream>
#include <vector>

struct S
{
    S() { a = ++cnt; std::cout << "S() ";  }
    S(const S& rhs)    { a = rhs.a; std::cout << "copyCtr ";  }
    S(S&& rhs)   noexcept    { a = rhs.a; std::cout << "moveCtr "; }
    S& operator=(const S& rhs) { a = rhs.a; std::cout << "copy= "; return *this; }
    S& operator=(S&& rhs)      { a = rhs.a; std::cout << "move= "; return *this; }
    int a;
    static int cnt;
};
int S::cnt = 0;

int main()
{
    std::vector<S>  sv(5);
    sv.push_back(S());

    for (std::size_t i = 0; i < sv.size(); ++i)
        std::cout << sv[i].a << " ";
    std::cout << std::endl;
}
```

```
$ g++ -std=c++11  vec.cpp &&  ./a.out
S() S() S() S() S() S()
moveCtr moveCtr moveCtr moveCtr moveCtr moveCtr
1 2 3 4 5 6
```

std::move(b,e,b2)

```
#include <iostream>
#include <vector>
#include <list>
#include <algorithm>

struct S
{
    S() { a = ++cnt; std::cout << "S() ";  }
    S(const S& rhs) { a = rhs.a; std::cout << "copyCtr ";  }
    S(S&& rhs)   noexcept    { a = rhs.a; std::cout << "moveCtr "; }
    S& operator=(const S& rhs) { a = rhs.a; std::cout << "copy= "; return *this; }
    S& operator=(S&& rhs)      { a = rhs.a; std::cout << "move= "; return *this; }
    int a ;
    static int cnt;
};
int S::cnt = 0;

int main()
{
    std::list<S> sl = { S(), S(), S(), S(), S() };
    std::vector<S> sv(5);
    std::move( sl.begin(), sl.end(), sv.begin());

    for (const S& s : sv)
        std::cout << s.a << " ";
     std::cout << std::endl;
}
```

std::move(b,e,b2)

```
#include <iostream>
#include <vector>
#include <list>
#include <algorithm>

struct S
{
    S() { a = ++cnt; std::cout << "S() ";  }
    S(const S& rhs) { a = rhs.a; std::cout << "copyCtr ";  }
    S(S&& rhs)   noexcept    { a = rhs.a; std::cout << "moveCtr "; }
    S& operator=(const S& rhs) { a = rhs.a; std::cout << "copy= "; return *this; }
    S& operator=(S&& rhs)      { a = rhs.a; std::cout << "move= "; return *this; }
    int a ;
    static int cnt;
};
int S::cnt = 0;

int main()
{
    std::list<S> sl = { S(), S(), S(), S(), S() };
    std::vector<S> sv(5);
    std::move( sl.begin(), sl.end(), sv.begin());

    for (const S& s : sv)
        std::cout << s.a << " ";
     std::cout << std::endl;
}
```

```
$ g++ -std=c++11  &&  ./a.out
S() S() S() S() S() copyCtr
copyCtr copyCtr copyCtr copyCtr
S() S() S() S() S()
move= move= move= move= move=
1 2 3 4 5
```

std::move(b,e,b2)

```
#include <iostream>
#include <vector>
#include <set>
#include <algorithm>

struct S
{
    S() { a = ++cnt; std::cout << "S() ";  }
    S(const S& rhs) { a = rhs.a; std::cout << "copyCtr ";  }
    S(S&& rhs)   noexcept    { a = rhs.a; std::cout << "moveCtr "; }
    S& operator=(const S& rhs) { a = rhs.a; std::cout << "copy= "; return *this; }
    S& operator=(S&& rhs)      { a = rhs.a; std::cout << "move= "; return *this; }
    int a ;
    static int cnt;
};
int S::cnt = 0;
bool operator<(const S& x, const S& y) { return x.a < y.a; }
int main()
{
    std::set<S> sl = { S(), S(), S(), S(), S() };
    std::vector<S> sv(5);
    std::move( sl.begin(), sl.end(), sv.begin());

    for (const S& s : sv)
        std::cout << s.a << " ";
     std::cout << std::endl;
}
```

std::move(b,e,b2)

```
#include <iostream>
#include <vector>
#include <set>
#include <algorithm>

struct S
{
    S() { a = ++cnt; std::cout << "S() ";  }
    S(const S& rhs) { a = rhs.a; std::cout << "copyCtr ";  }
    S(S&& rhs)   noexcept    { a = rhs.a; std::cout << "moveCtr "; }
    S& operator=(const S& rhs) { a = rhs.a; std::cout << "copy= "; return *this; }
    S& operator=(S&& rhs)      { a = rhs.a; std::cout << "move= "; return *this; }
    int a ;
    static int cnt;
};
int S::cnt = 0;
bool operator<(const S& x, const S& y) { return x.a < y.a; }
int main()
{
    std::set<S> sl = { S(), S(), S(), S(), S() };
    std::vector<S> sv(5);
    std::move( sl.begin(), sl.end(), sv.begin());

    for (const S& s : sv)
        std::cout << s.a << " ";
     std::cout << std::endl;
}
```

```
$ g++ -std=c++11  &&  ./a.out
S() S() S() S() S() copyCtr
copyCtr copyCtr copyCtr copyCtr
S() S() S() S() S()
copy= copy= copy= copy= copy=
1 2 3 4 5
```

Overloading on right value

```
T &      value(optional<T> &      par_);
T &&     value(optional<T> &&     par_);
T const& value(optional<T> const& par_);
```

But in object-oriented programming, sometimes we want to overload on the **this** parameter too.

```
template <typename T>
class optional
{
  // ...
  T&       value() &;
  T&&      value() &&;
  T const& value() const&;
};
```

RVO

```
std::vector<double>  fill();

int main()
{
    std::vector<double> vd = fill();
    // ...
}

std::vector<double>  fill()
{
   std::vector<double> local;
   // fill the elements
   return local;      // return or move?
}
```

David Abrahams wrote an article on this:
https://www.scribd.com/document/316704010/Want-Speed-Pass-by-Value
(originally at: http://cpp-next.com/archive/2009/08/want-speed-pass-by-value/)

Forwarding/Universal reference

Forwarding/Universal reference is used in case of type deduction

```
class X;
Void f(X&& param)        // rvalue reference
X&& var1 = X();          // rvalue reference
auto&& var2 = var1;      // NOT rvalue reference: universal reference
template <typename T>
void f(std::vector<T>&& param); // rvalue reference (1)
template <typename T>
void f(T&& param);        // NOT rvalue reference: universal reference (2)
template <typename T>
void f(const T&& param); // rvalue reference

X var;
f(var);                  // lvalue passed: param type is: X& (2)
f(std::move(var));       // rvalue passed: param type is: X&& (2)
std::vector<int> v;
f(v);                    // syntax error: can't bind lvalue to rvalue  (1)
```

Universal reference

Universal reference is used in case of type deduction

```
template <class T, class Allocator = allocator<T>>
class vector
{
public:
  void push_back(T&& x);  // rvalue reference, no type deduction here
  // ...
};

template <class T, class Allocator = allocator<T>>
class vector
{
public:
  template <class... Args>
  void emplace_back(Args&&... args);    // universal reference, type deduction
  // ...
};
```

Perfect forwarding

```
template<typename T, typename Arg>
shared_ptr<T> factory(Arg arg)
{
  return shared_ptr<T>(new T(arg));    // call T(arg) by value. Bad!
}

// A half-good solution is passing arg by reference:
template<typename T, typename Arg>
shared_ptr<T> factory(Arg& arg)
{
  return shared_ptr<T>(new T(arg));
}

But this does not work for rvalue parameters:

factory<X>(f());        // error if f() returns by value
factory<X>(42);         // error
```

Perfect forwarding

```
// If f() called on "lvalue of A"  T --> A&     argument type --> A&
// If f() called on "rvalue of A"  T --> A      argument type --> A

template<typename T, typename Arg>
shared_ptr<T> factory(Arg&& arg)
{
  return shared_ptr<T>(new T(std::forward<Arg>(arg)));
}
template<class S>
S&& forward(typename remove_reference<S>::type& a) noexcept
{
  return static_cast<S&&>(a);
}

// Reference collapsing:

A& &   --> A&
A& &&  --> A&
A&& &  --> A&
A&& && --> A&&
```

Perfect forwarding

```
shared_ptr<A> factory(X&& arg)
{
  return shared_ptr<A>(new A(std::forward<X>(arg)));
}
X&& forward(X& a) noexcept  // std::forward keeps move semantic
{
  return static_cast<X&&>(a);
}

template <typename T>           // C++11
typename remove_reference<T>::type&&
std::move(T&& a) noexcept
{
  typedef typename remove_reference<T>::type&& RvalRef;
  return static_cast<RvalRef>(a);
}
template <typename T>           // C++14
decltype(auto) std::move(T&& a)
{
  using ReturnType = remove_reference_t<T>&&;
  return static_cast<ReturnType>(a);
}
```

Lambda functions

- Terminology
- How it is compiled
- Capture by value and reference
- Mutable lambdas
- Use of this
- Init capture and generalized lambdas in C++14
- Constexpr lambda and capture *this and C++17

Functor

```
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

struct PrinterFunctor
{
  void operator()(int n) const { cout << n << " "; }
};

int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    for_each(v.begin(), v.end(), PrinterFunctor());
    cout << endl;
    return 0;
}

$ g++ l.cpp
$ ./a.out
0 1 2 3 4 5 6 7 8 9
```

Lambda

```
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    for_each(v.begin(), v.end(), [](int n) { cout << n << " "; });
    cout << endl;
    return 0;
}

$ g++ l.cpp
$ ./a.out
0 1 2 3 4 5 6 7 8 9
```

Lambdas are mapped to functors

```
int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    for_each(v.begin(), v.end(), [](int n) { cout << n << " "; } );
    cout << endl;
    return 0;
}
```

Lambdas are mapped to functors

```
// [](int n) { cout << n << " "; }


struct LambdaFunctor
{
    void operator() (int n) const { cout << n << " "; }
};

int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    for_each(v.begin(), v.end(), LambdaFunctor() );
    cout << endl;
    return 0;
}
```

Lambda

```
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    for_each(v.begin(), v.end(), [](int n) { cout << n << " "; });
    cout << endl;
    return 0;
}
```

Lambda introducer with opt. capture

Lambda parameter declaration

Optional return type
in form: -> type

```
$ g++ l.cpp
$ ./a.out
0 1 2 3 4 5 6 7 8 9
```

Lambda terminology

- Lambda expression

  [ ] (int n) { }

- Closure object
  - Runtime object created from lambda expression
  - Copy constructable (but not copy assignable)
  - Can be stored in std::function
  - May hold captured variables
- Closure class
  - The type of the closure object
  - Deleted default constructor and copy assignment operator

Explicit return type

```
int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    deque<double> dd;
    deque<int>    di;

    transform(v.begin(), v.end(), front_inserter(dd),
                                  [](int n) -> double { return n / 2.0; } );

    transform(v.begin(), v.end(), back_inserter(di),
                                  [](int n) -> int    { return n / 2.0; } );

    for_each(dd.begin(), dd.end(), [](double n) { cout << n << " "; });
    cout << endl;
    for_each(di.begin(), di.end(), [](double n) { cout << n << " "; });
    cout << endl;

    return 0;
}
```

```
4.5 4 3.5 3 2.5 2 1.5 1 0.5 0
0 0 1 1 2 2 3 3 4 4
```

Can contain multiple statements

```
#include <algorithm>
#include <iostream>
#include <ostream>
#include <vector>

using namespace std;

int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    for_each(v.begin(), v.end(), [](int n) {
            cout << n ;
            if ( n % 2 )
                cout << ":odd ";
            else
                cout << ":even ";
                        });
    cout << endl;
    return 0;
}
0:even 1:odd 2:even 3:odd 4:even 5:odd 6:even 7:odd 8:even 9:odd
```

Capture

```
int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    int x = 0;
    int y = 0;

    cin >> x >> y;  // read 3 6

    v.erase( remove_if(v.begin(),v.end(),[x,y](int n) { return x < n && n < y; }),
             v.end()
            );

    for_each(v.begin(), v.end(), [](int n) { cout << n << " "; });

    cout << endl;
    return 0;
}
```

Capture by value

```
3 6
0 1 2 3 6 7 8 9
```

Capture by value

```
[x,y](int n) { return x < n && n < y; }

struct LambdaFunctor
{
public:
    LambdaFunctor(int a, int b) : m_a(a), m_b(b) { }
    bool operator()(int n) const { return m_a < n && n < m_b; }
private:
    int m_a;
    int m_b;
};

// ...

 v.erase( remove_if(v.begin(),v.end(),LambdaFunctor(x,y)), v.end());
```

copy

The x and y parameters are copied and being stored in the function object. We cannot modify the captured values because the **operator**() in functor is **const**. It is a **"real"** copy, therefore the modification of x and y is not reflected inside the lambda.

Capture by reference

```
int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    int   x = 0;
    int   y = 0;
    int sum = 0;

    cin >> x >> y;  // read 2 7

    for_each(v.begin(),v.end(),[&sum,x,y](int n) { if (x < n && n < y) sum += n; });

    cout << "sum = " << sum << endl;

    return 0;
}
```

```
2 7
sum = 18
```

Mutable lambda

```
int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    int   x = 0;
    int   y = 0;
    int sum = 0;

    cin >> x >> y;  // read 2 7

    for_each(v.begin(),v.end(),[&sum,x,y](int n) { if (x-- < n && n < y++) sum += n; });

    cout << "sum = " << sum << endl;

    return 0;
}

$ g++ -std=c++11 -Wall -pedantic lambda4.cpp
lambda4.cpp: In lambda function:
lambda4.cpp:18:62: error: decrement of read-only variable ‘x’
     std::for_each(v.begin(),v.end(),[&sum,x,y](int n) { if (x--<n && n<y++) sum+=n; });
                                                                  ^~
lambda4.cpp:18:73: error: increment of read-only variable ‘y’
 std::for_each(v.begin(),v.end(),[&sum,x,y](int n) { if (x--<n && n<y++) sum+=n; });
                                                                      ^~
```

Mutable lambda

```
int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    int   x = 0;
    int   y = 0;
    int sum = 0;

    cin >> x >> y;  // read 2 7

    for_each(v.begin(),v.end(),[&sum,x,y](int n) mutable { if (x-- < n && n < y++)
                                                                    sum += n; });
    cout << "sum = " << sum << endl;
    cout << "x = " << x << endl;
    cout << "y = " << y << endl;
    return 0;
}
```

```
2 7
sum = 44
x = 2
y = 7
```

Globals are not captured

```
int   x = 0;
int   y = 0;

int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    int sum = 0;

    cin >> x >> y;  // read 2 7

    for_each(v.begin(),v.end(),[&sum,x,y](int n) mutable { if (x-- < n && n < y++)
                                                                    sum += n; });
    cout << "sum = " << sum << endl;
    cout << "x = " << x << endl;
    cout << "y = " << y << endl;
    return 0;
}

$ g++ -std=c++11 -Wall -pedantic lambda4.cpp
lambda5.cpp: In function main:
lambda5.cpp:19:43: warning: capture of variable ‘x’ with non-automatic storage duration
lambda5.cpp:19:45: warning: capture of variable ‘y’ with non-automatic storage duration
```

Globals are not captured

```
int   x = 0;
int   y = 0;

int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    int sum = 0;

    cin >> x >> y;  // read 2 7

    for_each(v.begin(),v.end(),[&sum](int n) mutable { if (x-- < n && n < y++)
                                                                    sum += n; });
    cout << "sum = " << sum << endl;
    cout << "x = " << x << endl;
    cout << "y = " << y << endl;
    return 0;
}

2 7
sum = 44
x = -8
y = 15
```

Capturing summary

- No capture

  [ ]

- By value

  [x,y] [=]

- By reference

  [&x, &y] [&]

- Mixed:

  [=, &x, &y] [&, x, y]

- Only automatic lifetime (local) variables could be captured
- Constness is preserved on capture
- Global variables, static members, heap storage can be used if visible but they are not captured

Capturing *this*

```
struct X
{
    int s;
    vector<int> v;
    void print() const
    {
        for_each(v.begin(), v.end(), [](int n) { cout << n*s << " "; });
    }
};

int main()
{
    X x;
    x.s = 2;
    for(int i = 0; i < 10; ++i)
        x.v.push_back(i);

    x.print();
    return 0;
}
```

```
$ g++ l10.cpp
l10.cpp: In lambda function:
l10.cpp:15:60: error: ‘this’ was not captured for this lambda function
```

Capturing *this*

```
struct X
{
    int s;
    vector<int> v;
    void print() const
    {
        for_each(v.begin(), v.end(), [this](int n) { cout << n*s << " "; });
    }
};

int main()
{
    X x;
    x.s = 2;
    for(int i = 0; i < 10; ++i)
        x.v.push_back(i);

    x.print();
    return 0;
}
```

```
0 2 4 6 8 10 12 14 16 18
```

Capturing *this*

```
struct X
{
    int s;
    vector<int> v;
    void print() const
    {
        int s = 9;
        for_each(v.begin(), v.end(), [this,s](int n) { cout << n*s << " "
                                                        << this->s << " "; });
    }
};

int main()
{
    X x;
    x.s = 2;
    for(int i = 0; i < 10; ++i)
        x.v.push_back(i);

    x.print();
    return 0;
}
```

```
0 2 9 2 18 2 27 2 36 2 45 2 54 2 63 2 72 2 81 2
```

Capturing *this*

- The **this** not captured by default
- The **this** is always captured by value
- [=] implicitly captures **this**
- Since C++17 ***this** can be captured (by value)
- Capturing **this** can be dangerous
  - Storing a non-smart pointer
  - Lifetime may already finished when lambda function is called

Capturing *this*

```
std::function<void (int)> f;  // global

struct X
{
  X(int i) : ii(i) {}
  int ii;
  void addLambda()
  {
    f = [=](int n) { if (n == ii) cout << n;  // [=] captures this if needs
                     else         cout << ii; // this->ii, indicates capturing this
                   };
  }
};

int main()
{
  {
    std::unique_ptr<X> up = std::make_unique<X>(4);
    up->addLambda();
    f(4);
  }  // obect pointed by up destroyed here

  f(4);    // Likely aborts! The captured this points to already dead object
  return 0;
}
```

Copying lambda

```
int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    int   x = 0;
    int   y = 0;
    int sum = 0;
    auto f = [=,&sum](int n) mutable { if( x-- < n && n < y++ ) sum += n; };

    cin >> x >> y;  // read 2 7


    for_each(v.begin(),v.end(),f);

    cout << "sum = " << sum << endl;
    cout << "x = " << x << endl;
    cout << "y = " << y << endl;
    return 0;
}

2 7
sum = 0
x = 2
y = 7
```

Copying lambda

```
int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    int   x = 0;
    int   y = 0;
    int sum = 0;
    // auto f = [=,&sum](int n) mutable { if( x-- < n && n < y++ ) sum += n; };

    cin >> x >> y;  // read 2 7
    auto f = [=,&sum](int n) mutable { if( x-- < n && n < y++ ) sum += n; };

    for_each(v.begin(),v.end(),f);

    cout << "sum = " << sum << endl;
    cout << "x = " << x << endl;
    cout << "y = " << y << endl;
    return 0;
}
```

```
2 7
sum = 44
x = 2
y = 7
```

Copying lambda

```
int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    int   x = 0;
    int   y = 0;
    int sum = 0;
    // auto f = [=,&sum](int n) mutable { if( x-- < n && n < y++ ) sum += n; };

    cin >> x >> y;  // read 2 7
    auto f = [=,&sum](int n) mutable { if( x-- < n && n < y++ ) sum += n; };

    for_each(v.begin(),v.end(),f);
    for_each(v.begin(),v.end(),f);  // 2nd time

    cout << "sum = " << sum << endl;
    cout << "x = " << x << endl;
    cout << "y = " << y << endl;
    return 0;
}

2 7
sum = 88
x = 2
y = 7
```

Copying lambda

```
int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    int   x = 0;
    int   y = 0;
    int sum = 0;

    cin >> x >> y;  // read 2 7
    auto f = [=,&sum](int n) mutable { if( x-- < n && n < y++ ) sum += n; }; // copy constr

    for_each(v.begin(),v.end(),f);  // copy constr





    cout << "sum = " << sum << endl;
    cout << "x = " << x << endl;
    cout << "y = " << y << endl;
    return 0;
}
```

```
2 7
sum = 44
x = 2
y = 7
```

Copying lambda

```
int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    int   x = 0;
    int   y = 0;
    int sum = 0;

    cin >> x >> y;  // read 2 7
    auto f = [=,&sum](int n) mutable { if( x-- < n && n < y++ ) sum += n; }; // copy constr

    for_each(v.begin(),v.end(),f);  // copy constr

    // f = [=,&sum](int n) mutable { if( x-- < n && n < y++ ) sum += n; };  op= deleted





    cout << "sum = " << sum << endl;
    cout << "x = " << x << endl;
    cout << "y = " << y << endl;
    return 0;
}

2 7
sum = 44
x = 2
y = 7
```

Copying lambda

```
int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    int   x = 0;
    int   y = 0;
    int sum = 0;

    cin >> x >> y;  // read 2 7
    auto f = [=,&sum](int n) mutable { if( x-- < n && n < y++ ) sum += n; }; // copy constr

    for_each(v.begin(),v.end(),f);  // copy constr

    // f = [=,&sum](int n) mutable { if( x-- < n && n < y++ ) sum += n; };  op= deleted

    const auto& f2 = [=,&sum](int n) mutable { if( x-- < n && n < y++ ) sum += n; }; // ref

    for_each(v.begin(),v.end(),f2);  // copy constr

    cout << "sum = " << sum << endl;
    cout << "x = " << x << endl;
    cout << "y = " << y << endl;
    return 0;
}
```

```
2 7
sum = 88
x = 2
y = 7
```

Nullary lambdas

```
int main()
{
    vector<int> v;
    int i = 0;

    generate_n(back_inserter(v), 10, [&] { return i++; } );

    for_each(v.begin(), v.end(), [](int n) { cout << n << " "; });
    return 0;
}
```

```
0 1 2 3 4 5 6 7 8 9
```

Conversion to function pointer

```
int main()
{
    vector<int> v;
    for(int i = 0; i < 10; ++i)
        v.push_back(i);

    deque<double> dd;
    deque<int>    di;

    double (*fp1)(int) = [](int n) -> double  { return n / 2.0; };   // no caption
    int    (*fp2)(int) = [](int n) -> int     { return n / 2.0; };   // no caption
    void   (*fp3)(double) = [](double n) { std::cout << n << " "; }; // no caption

    std::transform(v.begin(), v.end(), front_inserter(dd), fp1);
    std::transform(v.begin(), v.end(), back_inserter(di),  fp2);

    for_each(dd.begin(), dd.end(), fp3);
    cout << endl;
    for_each(di.begin(), di.end(), fp3);
    cout << endl;

    return 0;
}
```

```
4.5 4 3.5 3 2.5 2 1.5 1 0.5 0
0 0 1 1 2 2 3 3 4 4
```

IIFE – Immediately Invoked Function Expression

```
/* const */ int i = some_default_value; // can't do it const
                                        // since value depends
if(someConditionIstrue)                 // on some condition.
{
    // Do some operations and calculate the value of i;
    i = // some calculated value;
}
int x = i; // use i

// But unfortunately in this case there is no way to guarantee
// it is used as a constant, so now if some one comes and does
i = 10; // This is valid
==========================================================

const int i = [&]{

    int i = some_default_value;

    if(someConditionIstrue)
    {
        // Do some operations and calculate the value of i;
        i = // some calculated value;
    }
    return i;

} (); // note: () invokes the lambda!
```

Generalized lambdas in C++14

```
auto L = [](const auto& x, auto& y){ return x + y; };
```

means:

```
struct /* anonymous */
{
    template <typename T, typename U>
    auto operator()(const T& x, U& y) const // N3386 Return type deduction
    {
        return x + y;
    }
} L;
```

Generalized lambdas in C++14

```
int main()
{
    auto my_lambda = [](auto a, auto b) { return a < b; };

    float af = 1.5, bf = 2.0;
    int ai = 3, bi = 1;
    std::string as = “Hello”, bs = “World”;

    std::cout << "Float: "   << my_lambda(af, bf) << std::endl;
    std::cout << "Integer: " << my_lambda(ai, bi) << std::endl;
    std::cout << "String: "  << my_lambda(as, bs) << std::endl;

    return 0;
}
```

Init capture in C++14

```
auto up = std::make_unique<X>();

auto func = [up = std::move(up)] { return up->f(); }
```

up is a member inside the lambda

outer up is captured

here we use the member up inside the lambda

Or:

```
auto func = [up = std::make_unique<X>()] { return up->f(); }
```

Init capture example

```
#include <algorithm>
#include <functional>
#include <iostream>
#include <vector>

int main()
{
  std::vector<int> v;
  int i = 0;

  auto f = [cnt = 0](int n) mutable { std::cout << ++cnt << ":" << n << " "; };

  std::generate_n( std::back_inserter(v), 10, [&] { return i++; } );

  std::for_each( v.begin(), v.end(), f);
  std::for_each( v.begin(), v.end(), f);
  return 0;
}
```

```
1:0 2:1 3:2 4:3 5:4 6:5 7:6 8:7 9:8 10:9 1:0 2:1 3:2 4:3 5:4 6:5 7:6 8:7 9:8 10:9
```

Constexpr lambda in C++17

```
#include <iostream>

int main()
{
    constexpr auto multi = [](int a, int b){ return a * b; };

    static_assert(multi(3,7) == 21, "3x7 == 21");
    static_assert(multi(4,5) == 15, "4x5 != 15");

    return 0;
}
```

Constexpr lambda in C++17

```
template<typename Range, typename Func, typename T>
constexpr T SimpleAccumulate(const Range& range, Func func, T init)
{
    for (auto &&elem: range)
    {
        init += func(elem);
    }
    return init;
}
int main()
{
    constexpr int t[] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };

    constexpr int x = 2;
    constexpr int y = 7;

    constexpr auto f = [x,y](int n) { return (x < n && n < y) ? 0 : n; };
    constexpr int sum = SimpleAccumulate( t, f, 0);


    static_assert( 27 == sum );

    std::cout << "sum = " << SimpleAccumulate( t, f, 0) << '\n';
    std::cout << "x = " << x << '\n';
    std::cout << "y = " << y << '\n';
    return 0;
}
```

Constexpr lambda in C++17

```
template<typename Range, typename Func, typename T>
constexpr T SimpleAccumulate(const Range& range, Func func, T init)
{
    for (auto &&elem: range)
    {
        init += func(elem);
    }
    return init;
}
int main()
{
    constexpr int t[] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };

    constexpr int x = 2;
    constexpr int y = 7;

    // constexpr auto f = [x,y](int n) { return (x < n && n < y) ? 0 : n; };
    constexpr int sum = SimpleAccumulate( t,
                            [x,y](int n) { return (x < n && n < y) ? 0 : n; };, 0);

    static_assert( 27 == sum );

    std::cout << "sum = " << SimpleAccumulate( t, f, 0) << '\n';
    std::cout << "x = " << x << '\n';
    std::cout << "y = " << y << '\n';
    return 0;
}
```

Capture *this in C++17

```
struct my_struct
{
  int x;
  int y;
  void value();
};

void my_struct::value()
{
  [=, this](){};     // C++17 error: = captures this by default, ok since C++20
  [=, *this](){};    // captures my_struct by value since C++17
  [this, *this](){}; // ok since C++20: repeating this in capture
}
```

C++20

- Allow [=,this]
- Pack expression in init capture [...args = std::move(args)]
- Capture for structured bindings
- Template lambdas (with concepts)
- Default constructible and assignable lambdas (if no state)
- Lambdas in unevaluated context (pl. sizeof)

Advanced memory handling

- Storage classes in C++
- The new and delete operators
- Overloading new and delete
- New and delete expressions
- Objects with restricted storage classes
- The RAII idiom

Storage classes in C++

- String literals are read-only objects
- Automatic (local) variables
- Global (namespace) variables with static lifetime
- Local static variables
- Dynamic memory with new/delete or malloc/free
- Temporaries
- Arrays
- Subobjects (non-static class members)

Temporaries

- Created when evaluating an expression
- Guaranteed to live until the full expression is evaluated

```
void f( string &s1, string &s2, string &s3)
{
  const char *cs = (s1+s2).c_str();
  cout << cs;     // Bad!!

   if ( strlen(cs = (s2+s3).c_str()) < 8 && cs[0] == 'a' ) // Ok
        cout << cs;     // Bad!!
}
void f( string &s1, string &s2, string &s3)
{
    cout << s1 + s2;            // lifetime extension:
    const string &s = s2 + s3; // binding to name keeps temporary
                                // alive until name goes out of scope
    if ( s.length() < 8 && s[0] == 'a' )
        cout << s;  // Ok
}
// s2+s3 destroyed here: when the "s" reference goes out of scope
```

Lifetime extension

- When a (const) reference is set to a temporary, the temporary will live until the reference goes out of scope

```
struct mystring : std::string {
  mystring(const std::string& s) : std::string(s) {};
  ~mystring() { std::cerr<<"lifetime end: "<< c_str() << '\n'; }
};
mystring operator+( const mystring& a,const mystring& b){
  std::string aa(a), bb(b);
  std::string res(aa+bb);
  return mystring(res);
}
int main() {
  {
    mystring first("first");
    mystring second("second");
    mystring third("third");
    mystring fourth("fourth");
    mystring fifth("fifth");

    const mystring& ref1 = first + second;
    static const mystring& ref2 = first + third;
    static const mystring& ref3 = std::max(fourth,fifth);
    std::cerr << "end of block" << '\n';
  }
  std::cerr << "end of main" << '\n';
}
```

Lifetime extension

- When a (const) reference is set to a temporary, the temporary will live until the reference goes out of scope

```
struct mystring : std::string {
  mystring(const std::string& s) : std::string(s) {};
  ~mystring() { std::cerr<<"lifetime end: "<< c_str() << '\n'; }
};
mystring operator+( const mystring& a,const mystring& b){
  std::string aa(a), bb(b);
  std::string res(aa+bb);
  return mystring(res);
}
int main() {
  {
    mystring first("first");
    mystring second("second");
    mystring third("third");
    mystring fourth("fourth");
    mystring fifth("fifth");

    const mystring& ref1 = first + second;                // ok
    static const mystring& ref2 = first + third;          // ok
    static const mystring& ref3 = std::max(fourth,fifth); // wrong!!!
    std::cerr << "end of block" << '\n';
  }
  std::cerr << "end of main" << '\n';
}
```

```
end of block
lifetime end: firstsecond
lifetime end: fifth
lifetime end: fourth
lifetime end: third
lifetime end: second
lifetime end: first
end of main
lifetime end: firstthird
```

Lifetime extension

- When a (const) reference is set to a temporary, the temporary will live until the reference goes out of scope

```
template <class T>
constexpr const T& max(const T& a, const T& b);
```

```
int main() {
  {
    mystring first("first");
    mystring second("second");
    mystring third("third");
    mystring fourth("fourth");
    mystring fifth("fifth");

    const mystring& ref1 = first + second;                // ok
    static const mystring& ref2 = first + third;          // ok
    static const mystring& ref3 = std::max(fourth,fifth); // wrong!!!
    std::cerr << "end of block" << '\n';
  }
  std::cerr << "end of main" << '\n';
}
```

```
end of block
lifetime end: firstsecond
lifetime end: fifth
lifetime end: fourth
lifetime end: third
lifetime end: second
lifetime end: first
end of main
lifetime end: firstthird
```

Lifetime extension

- When a (const) reference is set to a temporary, or member of a temporary the temporary will live until the reference goes out of scope

```
struct mystring : std::string {
  mystring(const std::string& s) : std::string(s) {};
  ~mystring() { std::cerr<<"lifetime end: "<< c_str() << '\n'; }
  int ifield;
};
mystring operator+( const mystring& a,const mystring& b){
  std::string aa(a), bb(b);
  std::string res(aa+bb);
  return mystring(res);
}
int main() {
  {
    mystring first("first");
    mystring fourth("fourth");
    mystring fifth("fifth");

    static const int &ref4 = (first + fourth).ifield;
    static const char *const &ref5 = (first + fifth).c_str();
    std::cerr << "end of block" << '\n';
  }
  std::cerr << "end of main" << '\n';
}
```

Lifetime extension

- When a (const) reference is set to a temporary, or member of a temporary the temporary will live until the reference goes out of scope

```
struct mystring : std::string {
  mystring(const std::string& s) : std::string(s) {};
  ~mystring() { std::cerr<<"lifetime end: "<< c_str() << '\n'; }
  int ifield;
};
mystring operator+( const mystring& a,const mystring& b){
  std::string aa(a), bb(b);
  std::string res(aa+bb);
  return mystring(res);
}
int main() {
  {
    mystring first("first");
    mystring fourth("fourth");
    mystring fifth("fifth");

    static const int &ref4 = (first + fourth).ifield;          // ok
    static const char *const &ref5 = (first + fifth).c_str(); // wrong!!!
    std::cerr << "end of block" << '\n';
  }
  std::cerr << "end of main" << '\n';
}
```

```
end of block
lifetime end: firstfifth
lifetime end: fifth
lifetime end: fourth
lifetime end: first
end of main
lifetime end: firstfourth
```

New and delete

- New and delete expressions
- New and delete operators

```
void f()
{
  try
  {
    int *ip1 = new int;     // new expression (uninitialized)
    int *ip2 = new int(42); // new expression (initialized)
    int *ap1 = new int[10]; // array new expression (uninitialized)
    int *ap2 = new int[]{1,2,3}; // array new expression
    auto ip3 = new auto(1); // creates int *ip3 and one int in heap

    // new operator
    int *ptr = static_cast<int *>(::operator new(sizeof(int)));

    ::operator delete(ptr);  // delete operator
    delete ip;               // delete expression
    delete [] ap;          // array delete expression
  }
  catch(std::bad_alloc e) { ... }
}
```

New expression

New expression do the following 3 steps when called as new X{}

- Allocate memory for X (usually calling operator new(sizeof(X)))
- Calls the constructor of X passing parameters if exists
- Converts pointer to the new object from void* to X* and returns

- But, what if
  - Operator new throws bad_alloc?
  - The constructor throws something?

```
X *ptr;

try
{
   ptr = new X(par1, par2);
}
catch( ... )
{
    // handle exceptions. Memory leak when constructor throws???
}
```

New and delete operators

- New expr. guaranties no leak if the constructor throws exception,

```
X *ptr;
try
{

      ptr = new     X(par1, par2); // allocate and run the constructor





}
catch( ... )
{
    // handle exceptions. No memory leak
}
```

New and delete operators

- New expr. guaranties no leak if the constructor throws exception

```
X *ptr;
try
{
   void *vp = operator new( sizeof(X) );  // allocate memory
   try {
      ptr = new(vp) X(par1, par2);  // run the constructor @vp
   }
   catch( ... ) {                   // catch all exceptions
      operator delete(vp);          // delete memory
      throw;                        // rethrow exxception
   }
   ptr = reinterpret_cast<X*>(vp);  // set pointer
}
catch( ... )
{
    // handle exceptions. No memory leak
}
```

New and delete operators

- May throw std::bad_alloc exception
- Returns void*

```
namespace std
{
  class bad_alloc : public exception { /* ... */ };
}

void* operator new(size_t);    // operator new() may throw bad_alloc
void  operator delete(void *)  // operator delete() never throws
```

New and delete operators

- May throw std::bad_alloc exception
- Returns void*

```
namespace std
{
  class bad_alloc : public exception { /* ... */ };
}

void* operator new(size_t);    // operator new() may throw bad_alloc
void  operator delete(void *) noexcept;  // delete() since C++11
```

Nothrow version

- In some environments we want to avoid exceptions
- Returns nullptr if allocation is unsuccessful

```
// indicator for allocation that doesn't throw exceptions
struct nothrow_t {};
extern const nothrow_t nothrow;

// what to do, when error occurs on allocation
typedef void (*new_handler)();
new_handler set_new_handler(new_handler new_p) throw();

// nothrow version
void* operator new(size_t, const nothrow_t&);
void  operator delete(void*, const nothrow_t&) noexcept;

void* operator new[](size_t, const nothrow_t&);
void  operator delete[](void*, const nothrow_t&) noexcept;
```

Placement new

- Never allocate / deallocate memory

```
// placement new and delete
void* operator new(size_t, void* p)   { return p; }
void  operator delete(void* p, void*) noexcept { }

void* operator new[](size_t, void* p)   { return p; }
void  operator delete[](void* p, void*) noexcept { }

#include <new>
void f()
{
  char *cp = new char[sizeof(C)];
  for( long long i = 0; i < 10000000; ++i)
  {
     C *dp = new(cp) C(i);
     // ...
     dp->~C();  // explicit call of destructor
  }
  delete [] cp;
  return 0;
}
```

Overloading new and delete

- New and delete operators can be overloaded at two level
  - Class level (automatically static member functions)
  - Namespace level

```
struct node
{
          node( int v)  { val = v; left = rigth = 0; }
    void  print() const { cout << val << " "; }

    /* static */ void *operator new( size_t sz) throw (bad_alloc);
    /* static */ void  operator delete(void *p) noexcept;

    int   val;
    node *left;
    node *right;
};
```

Overloading new and delete

```
// member new and delete as static member
void *node::operator new( size_t sz) throw (bad_alloc)
{
    return ::operator new(sz);
}
void node::operator delete(void *p) noexcept
{
    ::operator delete(p);
}
// global new and delete
void *operator new( size_t sz) throw (bad_alloc)
{
    return std::malloc(sz);
}
void operator delete( void *p) noexcept
{
    std::free(p);
}
```

Extra parameters for new

- One can define new and delete with extra parameters
  - Only new expression can pass extra parameters

```
struct node
{
          node( int v);
    void  print() const;

    static void *operator new( size_t sz, int i);
    static void  operator delete(void *p, int i) noexcept;

    int   val;
    node *left;
    node *right;
};
void f()
{
    int arena = 3;
    node *r = new(arena) node(i);
}
```

Alignment

```
#include <cassert> // example from https://www.bfilipek.com/2019/08/newnew-align.html
#include <cstdint>
#include <iostream>
#include <malloc.h>
#include <new>

class alignas(32) Vec3d {
    double x, y, z;
};

int main() {
    std::cout << "sizeof(Vec3d) is " << sizeof(Vec3d) << '\n';
    std::cout << "alignof(Vec3d) is " << alignof(Vec3d) << '\n';

    auto Vec = Vec3d{};
    auto pVec = new Vec3d[10];

    if(reinterpret_cast<uintptr_t>(&Vec) % alignof(Vec3d) == 0)
        std::cout << "Vec is aligned to alignof(Vec3d)!\n";
    else
        std::cout << "Vec is not aligned to alignof(Vec3d)!\n";

    if(reinterpret_cast<uintptr_t>(pVec) % alignof(Vec3d) == 0)
        std::cout << "pVec is aligned to alignof(Vec3d)!\n";
    else
        std::cout << "pVec is not aligned to alignof(Vec3d)!\n";

    delete[] pVec;
}
```

Alignment

```
// Before C++17

sizeof(Vec3d) is 32
alignof(Vec3d) is 32
Vec is aligned to alignof(Vec3d)!
pVec is not aligned to alignof(Vec3d)!
```

Alignment

```
// Before C++17

sizeof(Vec3d) is 32
alignof(Vec3d) is 32
Vec is aligned to alignof(Vec3d)!
pVec is not aligned to alignof(Vec3d)!


// Since C++17

sizeof(Vec3d) is 32
alignof(Vec3d) is 32
Vec is aligned to alignof(Vec3d)!
pVec is aligned to alignof(Vec3d)!
```

Alignment

```
// Before C++17

sizeof(Vec3d) is 32
alignof(Vec3d) is 32
Vec is aligned to alignof(Vec3d)!
pVec is not aligned to alignof(Vec3d)!


// Since C++17

sizeof(Vec3d) is 32
alignof(Vec3d) is 32
Vec is aligned to alignof(Vec3d)!
pVec is aligned to alignof(Vec3d)!


#include <cstdlib>

void *malloc( std::size_t size);  // align to std::max_align_t
// Since C++17 over-aligning:
void *alligned_alloc( std::size_t allignment, std::size_t size);
```

Operator new in C++17

- 14 global new() operator functions
- 8 class specific new() operators
- Corresponding delete() operators
- Overloading on std::align_val_t

```
enum class align_val_t : std::size_t {};

void *operator new( std::size_t sz, std::align_val_t al);
void  operator delete( void *ptr, std::align_val_t al) noexcept;

void f();
{
  auto ip   = new int{};   //   uses usual default alignment == 16
  auto pVec = new Vec3d{}; // uses class specified alignment == 32
  auto p64i = new(std::align_val_t{64}) int{};  // alignment == 64

  delete ip;
  delete pVec;
  delete p64i;
}
```

Objects only in heap

- Sometimes restriction for storage location is useful

```
class X  // a class should be allocated only on heap
{
public:
    X() {}
    void destroy() const { delete this; }
protected:
    ~X() {}
};
class Y : public X { };
//  class Z { X xx; };  // use pointer instead!
void f()
{
    X* xp = new X;
    Y* yp = new Y;

    delete xp;       // compile error
    xp->destroy();   // ok
}
```

Objects never in heap

- Sometimes restriction for storage location is useful

```
class X  // Class should be allocated only outside of heap
{
private:
 static void *operator new( size_t);
 static void operator delete(void *) noexcept;
 // static void operator delete(void *) noexcept = delete; in C++11
};

class Y : public X { };
class Z { X xx; };  // ok!
void f()
{
    X* xp = new X;  // compile error
    X  x;           // ok
}
```

RAII

- Resource Acquisition Is Initialization
- The idea: keep a resource is expressed by object lifetime

```
// is this correct?
void f()
{
    char *cp = new char[1024];

    g(cp);
    h(cp);

    delete [] cp;
}
```

RAII

- Resource Acquisition Is Initialization
- The idea: keep a resource is expressed by object lifetime

```
// is this maintainable?
void f()
{
    char *cp = new char[1024];

    try
    {
      g(cp);
      h(cp);
      delete [] cp;
    }
    catch (...)
    {
      delete [] cp;
      throw;
    }
}
```

RAII

- Constructor allocates resource
- Destructor deallocates

```
// RAII
struct Res
{
    Res(int n) { cp = new char[n]; }
    ~Res() { delete [] cp; }
    char *getcp() const { return cp; }
};

void f()
{
    Res res(1024);

    g(res.getcp());
    h(res.getcp());
}
// resources will be free here
```

RAII

- Constructor allocates resource
- Destructor deallocates

```
// RAII
struct Res
{
    Res(int n) { cp = new char[n]; }
    ~Res() { delete [] cp; }
    char *getcp() const { return cp; }
};

void f()
{
    Res res(1024);

    g(res.getcp());
    h(res.getcp());
}
// resources will be free here
```

What about copying Res?

RAII

- Should be careful when implementing RAII
- Destructor calls only when living object goes out of scope
- Object lives only when constructor has successfully finished

```
// But be careful:
struct BadRes
{
    Res(int n) { cp = new char[n]; ... init(); ... }
    ~Res()     { delete [] cp; }
    char *cp;
    void init()
    {
        ... if (error) throw XXX;
    }
};
```

RAII

- Should be careful when implementing RAII
- Destructor calls only when living object goes out of scope
- Object lives only when constructor has successfully finished

```
// But be careful:
struct BadRes
{
    Res(int n) { cp = new char[n]; ... init(); ... }
    ~Res()     { delete [] cp; }
    char *cp;
    void init()
    {
        ... if (error) throw XXX;
    }
};
```

Possible memory leak if throws!

Typical RAII solutions

- Smart pointers for memory handling
- Guards for locking
- ifstream, ofstream objects for file-i/o
- std::containers

```
class X
{
public:
    void *non_thread_safe();
private:
    Mutex lock_;
};

void *X::non_thread_safe();
{
    Guard<Mutex> guard(lock_);
    /* critical section is here*/
}
```

RAII

- Resource Acquisition Is Initialization
- The idea: keep a resource is expressed by object lifetime

```
// is this correct?
void f()
{
    char *cp = new char[1024];

    g(cp);
    h(cp);

    delete [] cp;
}
```

RAII

- Resource Acquisition Is Initialization
- The idea: keep a resource is expressed by object lifetime

```
// is this maintainable?
void f()
{
    char *cp = new char[1024];

    try
    {
      g(cp);
      h(cp);
      delete [] cp;
    }
    catch (...)
    {
      delete [] cp;
      throw;
    }
}
```

RAII

- Constructor allocates resource
- Destructor deallocates

```
// RAII
struct Res
{
    Res(int n) { cp = new char[n]; }
    ~Res() { delete [] cp; }
    char *getcp() const { return cp; }
};

void f()
{
    Res res(1024);

    g(res.getcp());
    h(res.getcp());
}
// resources will be freed here
```

RAII

- Constructor allocates resource
- Destructor deallocates

```
// RAII
struct Res
{
    Res(int n) { cp = new char[n]; }
    ~Res() { delete [] cp; }
    char *getcp() const { return cp; }
};  // Copy? Move?

void f()
{
    Res res(1024);

    g(res.getcp());
    h(res.getcp());
}
// resources will be freed here
```

RAII

- Should be careful when implementing RAII
- Destructor calls only when living object goes out of scope
- Object lives only when constructor has successfully finished

```
// But be careful:
struct BadRes
{
    Res(int n) { cp = new char[n]; ... init(); ... }
    ~Res()     { delete [] cp; }
    char *cp;
    void init()
    {
       /* ... */  if (error) throw XXX;  /* ... */
    }
};
```

Smart pointers

- The deprecated auto pointer
- How smart pointers work?
- Unique_ptr
- Shared_ptr and weak_ptr
- Make_ functions
- Shared pointer from this
- Traps and pitfalls

How to handle ownership?

- Owner
  - Responsible releasing the resource
    - Single owner
    - Reference counting
  - Examples
    - Memory std::vector<>, std::array<>, std::shared_ptr<>, std::string
    - std::lock_guard<>
    - std::ifstream
- Observer
  - Examples
    - std::weak_ptr<>
    - std::string_view<>

Auto_ptr

- The only smart pointer in C++98/03
- Cheap, ownership-based
- Not works well with STL containers and algorithms
- Not works with arrays
- Deprecated in C++11
- Removed from C++ since C++17

Unique_ptr

- Single ownership pointer (similar to auto_ptr)
- Carefully designed to work with STL and other language features
- Movable but not copyable
- Deleter is type parameter – cannot be changed in run-time

```
#include <memory>
template< class T, class Deleter = std::default_delete<T>>
class unique_ptr;

template <class T, class Deleter>
class unique_ptr<T[],Deleter>;

void f()
{
  std::unique_ptr<MyClass>   up1(new MyClass());  // * and ->
  std::unique_ptr<MyClass[]> up2(new MyClass[n]); // []
  ...
}  // proper delete called here
```

Unique_ptr

```
#include <memory>

void f()
{
  std::unique_ptr<Foo> up(new Foo{});  // up is the only owner
  std::unique_ptr<Foo> up2(up);// compile error: can't copy unique_ptr
  std::unique_ptr<Foo> up3;    // nullptr: not owner
  const std::unique_ptr<Foo> up4(new Foo{}); // const pointer

  up3 = up;              // compile error: can't assign unique_ptr
  up3 = std::move(up);  // ownership moved to up3
  up3 = std::move(up4); // compile error: const can’t move

  std::vector<unique_ptr<int>> vi;
  vi.push_back(std::make_unique<int>(42));

} // vi destroyed:  int{42} is destroyed
  // up4 destroyed: Foo object is destructed
  // up3 destroyed: Foo object is destructed
  // up  destroyed: nop
```

Polymorphism

```
struct Base {
  virtual void f() { std::cout << "Base::f\n"; }
  ~Base()  { std::cout << "Base::~Base()\n"; }
};
struct Derived : Base {
  virtual void f() override { std::cout << "Derived::f\n"; }
  ~Derived() { std::cout << "Derived::~Derived()\n"; }
};

void g() {
  auto dp = std::make_unique<Derived>(); // std::unique_ptr<Derived>
  std::unique_ptr<Base> bp = std::move(dp);

  std::vector<std::unique_ptr<Base>> v;
  v.push_back(std::make_unique<Base>());
  v.push_back(std::make_unique<Derived>());
  for ( auto& p : v)  p->f();
};
```

Polymorphism

```
struct Base {
  virtual void f() { std::cout << "Base::f\n"; }
  ~Base()  { std::cout << "Base::~Base()\n"; }
};
struct Derived : Base {
  virtual void f() override { std::cout << "Derived::f\n"; }
  ~Derived() { std::cout << "Derived::~Derived()\n"; }
};

void g() {
  auto dp = std::make_unique<Derived>();
  std::unique_ptr<Base> bp = std::move(dp);

  std::vector<std::unique_ptr<Base>> v;
  v.push_back(std::make_unique<Base>());
  v.push_back(std::make_unique<Derived>());
  for ( auto& p : v)  p->f();
};

Base::f         // v[1] is Base
Derived::f      // v[2] is Derived
Base::~Base()   // oops v[2] is Derived  during destruction of v
Base::~Base()   // ok   v[1] is Base     during destruction of v
Base::~Base()   // oops                        delete bp
```

Polymorphism

```
struct Base {
  virtual void f() { std::cout << "Base::f\n"; }
  virtual ~Base() { std::cout << "Base::~Base()\n"; }
};
struct Derived : Base {
  virtual void f() override { std::cout << "Derived::f\n"; }
  virtual ~Derived() override { std::cout << "Derived::~Derived()\n"; }
};

void g() {
  auto dp = std::make_unique<Derived>();
  std::unique_ptr<Base> bp = std::move(dp);  // deleter is not copied

  std::vector<std::unique_ptr<Base>> v;
  v.push_back(std::make_unique<Base>());
  v.push_back(std::make_unique<Derived>());  // deleter is not copied
  for ( auto& p : v)  p->f();
};

Base::f              // v[1] is Base
Derived::f           // v[2] is Derived
Base::~Base()        // base part of v[2] Derived
Derived::~Derived() // derived part of v[2] since virtual destructor
Base::~Base()        // v[1]
Derived::~Derived() // bp points to Derived with virtual destructor
Base::~Base()        // base part of *bp
```

Polymorphism – shared_ptr

```
struct Base {
  virtual void f() { std::cout << "Base::f\n"; }
  virtual ~Base()  { std::cout << "Base::~Base()\n"; }
};
struct Derived : Base {
  virtual void f() override { std::cout << "Derived::f\n"; }
  virtual ~Derived() override { std::cout << "Derived::~Derived()\n"; }
};

void g() {
  auto dp = std::make_shared<Derived>();
  std::shared_ptr<Base> bp = std::move(dp);  // deleter is moved

  std::vector<std::shared_ptr<Base>> v;
  v.push_back(std::make_shared<Base>());
  v.push_back(std::make_shared<Derived>());  // deleter is copied
  for ( auto& p : v)  p->f();
};

Base::f              // v[1] is Base
Derived::f           // v[2] is Derived
Base::~Base()        // base part of v[2] Derived
Derived::~Derived()  // derived part of v[2] since default_deleter<Derived>
Base::~Base()        // v[1]
Derived::~Derived()  // bp has default_deleter<Derived>
Base::~Base()        // base part of *bp:
```

Default deleter empty base optimization

```
template<class _T, class _DeleterT = std::default_delete<_T>>
class unique_ptr
{
public:
  // public interface...
private:
  // using empty base class optimization to save space
  // making unique_ptr with default_delete the same size as pointer
  class _UniquePtrImpl : private _DeleterT
  {
  public:
    constexpr _UniquePtrImpl() noexcept = default;
    // some other constructors...
    deleter_type& _Deleter() noexcept  { return *this; }
    const deleter_type& _Deleter() const noexcept { return *this; }
    pointer& _Ptr() noexcept { return _MyPtr; }
    const pointer _Ptr() const noexcept { return _MyPtr; }
  private:
     pointer   _MyPtr;
  };
  _UniquePtrImpl   _MyImpl;
};
```

How inheritance implemented?

- Raw pointers: assign Derived* to Base* works
- But unique_ptr<Derived> is not inherited from unique_ptr<Base>
- The deleter is not copied – different from shared_ptr !!!

```
template<class T, class D>
class unique_ptr
{
private:
  T* ap;    // refers to the actual owned object (if any)
public:
  typedef T element_type;

  explicit unique_ptr (T* ptr = 0) : ap(ptr) { }
  unique_ptr (unique_ptr&& rhs) : ap(rhs.release()) { }
  template<class Y> unique_ptr(unique_ptr<Y>&& rhs):ap(rhs.release()){}

  unique_ptr& operator=(unique_ptr&& rhs) { ... }
  template<class Y,class D>
  unique_ptr& operator=(unique_ptr<Y,D>&& rhs) { ... }
};
```

Abstract factory pattern

```
#include <memory>

class Base { ... };
class Derived1 : public Base { ... };
class Derived2 : public Base { ... };


template <typename... Ts>
std::unique_ptr<Base>  makeBase( Ts&&... params)  { ... }

void f() // client code:
{
  auto pBase = makeBase( /* arguments */ );
}
// destroy object
```

Abstract Factory Pattern

```
auto delBase = [](Base *pBase)
               {
                   makeLogEntry(pBase);
                   delete pBase;  // delete object
               };

template <typename... Ts>
std::unique_ptr<Base, decltype(delBase)>  makeBase( Ts&&... params)
{
  std::unique_ptr<Base, decltype(delBase)> pBase(nullptr, delBase);

  if ( /* Derived1 */ )
  {
    pBase.reset(new Derived1( std::forward<Ts>(params)...) );
  }
  else if ( /* Derived2 */ )
  {
    pBase.reset(new Derived2( std::forward<Ts>(params)...) );
  }
  return pBase;
}
```

Evaluation

- Can be used in standard containers when polymorphic use needed
- The sizeof(unique_ptr<>) is sizeof(raw pointer) + deleter size
- If deault_deleter is used, then no extra size penalty
- If no deleter state (e.g. lambda with no capture): + sizeof(funptr)
- If deleter with state, the size increases
- Prefer unique_ptr when possible
- No copy of deleter :(
- No downcast operation :(

Downcast unique_ptr

```
template<typename Derived, typename Base, typename Del>
std::unique_ptr<Derived, Del>
dynamic_unique_ptr_cast( std::unique_ptr<Base, Del>&& p )
{
  if(Derived *r = dynamic_cast<Derived *>(p.get()))
  {
    p.release();
    return std::unique_ptr<Derived, Del>(r,std::move(p.get_deleter()));
  }
  return std::unique_ptr<Derived, Del>(nullptr, p.get_deleter());
}
```

shared_ptr

- Shared ownership pointer with reference counter
- Copy constructible and assignable
- No array specializations (no shared_ptr<T[ ]>)
- Deleter type parameter – copied

```
#include <memory>
template< class T, class Deleter = std::default_delete<T>>
class shared_ptr;

void f()
{
  std::shared_ptr<MyClass>   sp1(new MyClass()); // * and ->
  std::shared_ptr<MyClass>   sp2(new MyClass[n], // * and ->
              std::default_delete<MyClass[]>()); //    before C++17
  std::shared_ptr<MyClass[]> sp3(new MyClass[n]);// [] since C++17
  ...
}  // proper delete called here
```

shared_ptr array specializaton

- Array spec. calls delete[]
- Only [ ] operator, no * and ->
- No conversion from shared_ptr<Der[ ]> to shared_ptr<Base[ ]>

```
struct Base {
  virtual void f() { std::cout << "Base::f\n"; }
  ~Base()  { std::cout << "Base::~Base()\n"; }
};
struct Derived : Base {
  virtual void f() override { std::cout << "Derived::f\n"; }
  ~Derived()  { std::cout << "Derived::~Derived()\n"; }
};

int main() {
  std::shared_ptr<Derived[]> sp(new Derived[5]);
  std::shared_ptr<Base[]>    bp(sp);  // compile error
  auto p  = sp[0];
  auto d0 = *sp;   // compile error
}
```

shared_ptr

```
void f()
{
  std::shared_ptr<int> p1(new int{5});
  std::shared_ptr<int> p2 = p1; // now both own the memory.

  p1.reset(); // memory still exists, due to p2.
} // p2 out of scope: delete the memory, since no one else owns.
```

```
T* get() const noexcept;
T& operator*() const noexcept;
T* operator->() const noexcept;
T& operator[](idx) const noexcept; // returns get()[idx]
```

```
long use_count() const noexcept;
bool unique() const noexcept;
explicit operator bool() const noexcept;
```

weak_ptr

- Not owns the memory
- But part of the “sharing group”
- No direct operation to access the memory
- Can be converted to shared_ptr with lock()

```
long use_count() const noexcept;
bool expired() const noexcept;     // use_count() == 0

shared_ptr<T> lock() const noexcept;
// return expired() ? shared_ptr<T>() : shared_ptr<T>(*this)

void reset() noexcept;
```

Using lock()

```
void f()
{
    std::shared_ptr<X> ptr1 = std::make_shared<X>();
    std::shared_ptr<X> ptr2 = ptr1;

    std::weak_ptr<X>   wptr = ptr2;

    if ( auto sp = wptr1.lock() )
    {
        // use sp
    } // destructor of sp called here: release X object
    else
    {
        // expired
    }
} // destructor of X object is called here
```

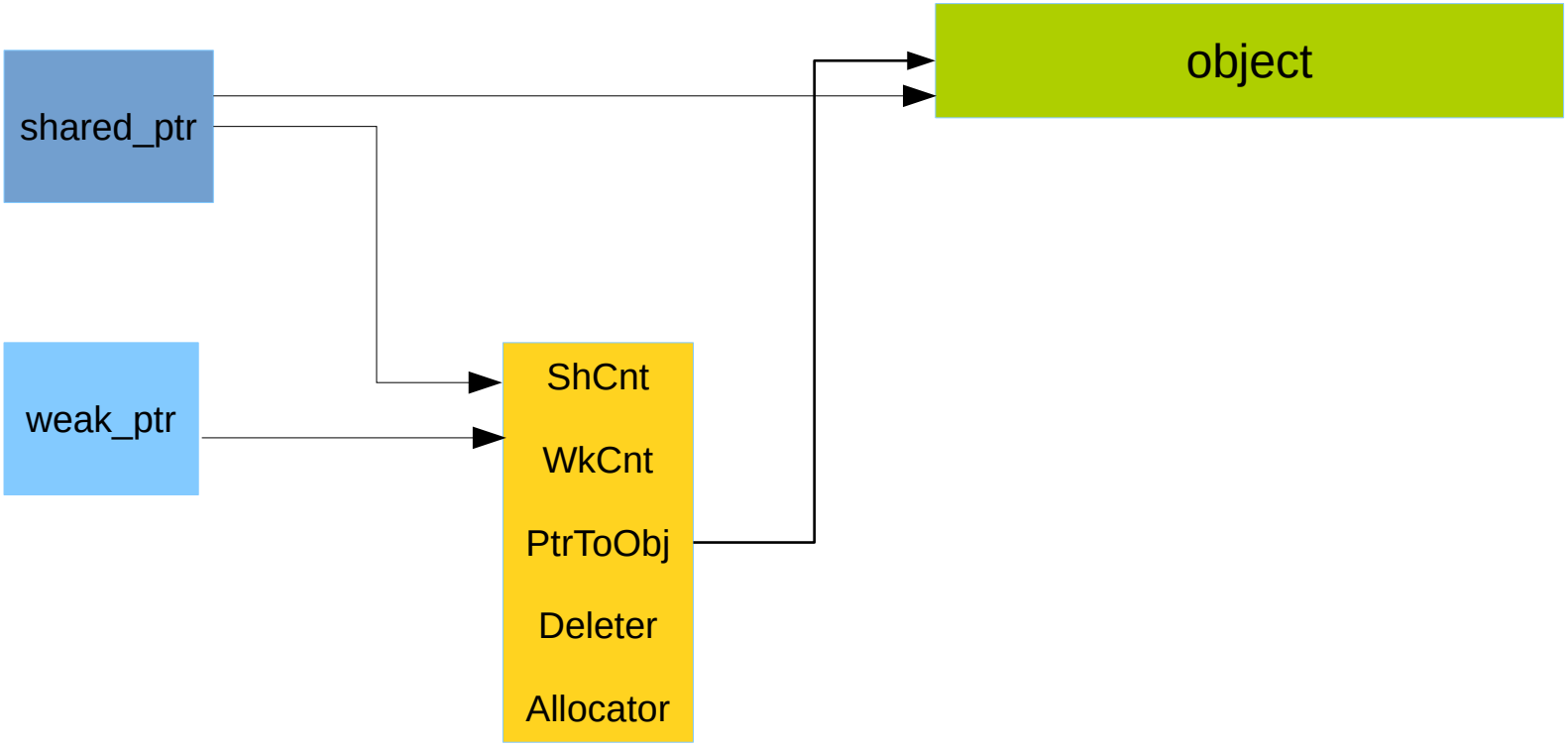
Using lock()

```
int main ()
{
  std::shared_ptr<int> sp1,sp2;
  std::weak_ptr<int>   wp;
                                          // sharing group:
                                          // ---------------
  sp1 = std::make_shared<int> (20);       // sp1
  wp = sp1;                               // sp1, wp

  sp2 = wp.lock();                        // sp1, wp, sp2
  sp1.reset();                            //      wp, sp2

  sp1 = wp.lock();                        // sp1, wp, sp2
}
```

Typical shared_ptr implementation



Enable shared from this

```
#include <memory>
#include <cassert>

class Y
{
public:

    std::shared_ptr<Y> f()
    {
        return shared_ptr<Y>(this);  // ???
    }
};

int main()
{
    std::shared_ptr<Y> p(new Y);
    std::shared_ptr<Y> q = p->f();
    assert(p == q);
    assert(!(p < q || q < p));
}
```

Enable shared from this

```
#include <memory>
#include <cassert>

class Y
{
public:

    std::shared_ptr<Y> f()   // BAD!!!
    {
        return shared_ptr<Y>(this);  // ???
    }
};

int main()
{
    std::shared_ptr<Y> p(new Y);
    std::shared_ptr<Y> q = p->f();
    assert(p == q);                // failes
    assert(!(p < q || q < p));     // failes
}
```

Enable shared from this

```
#include <memory>
#include <cassert>

class Y
{
public:
    Y() : ptr_to_me(std::shared_ptr<Y>(this)) { }
    std::shared_ptr<Y> f()
    {
        return ptr_to_me;  // ???
    }
private:
    std::shared_ptr<Y> ptr_to_me;
};

int main()
{
    std::shared_ptr<Y> p(new Y);
    std::shared_ptr<Y> q = p->f();
    assert(p == q);
    assert(!(p < q || q < p));
}
```

Enable shared from this

```
#include <memory>
#include <cassert>

class Y
{
public:
    Y() : ptr_to_me(std::shared_ptr<Y>(this)) { }
    std::shared_ptr<Y> f()  // BAD!!!
    {
        return ptr_to_me;  // ???
    }
private:
    std::shared_ptr<Y> ptr_to_me;
};

int main()
{
    std::shared_ptr<Y> p(new Y);
    std::shared_ptr<Y> q = p->f();
    assert(p == q);
    assert(!(p < q || q < p));
}
```

Enable shared from this

```
#include <memory>
#include <cassert>

class Y : public std::enable_shared_from_this<Y>
{
public:

    std::shared_ptr<Y> f()  // OK
    {
        return shared_from_this();
    }
};

int main()
{
    std::shared_ptr<Y> p(new Y);
    std::shared_ptr<Y> q = p->f();
    assert(p == q);
    assert(!(p < q || q < p)); // p and q share ownership
}
```

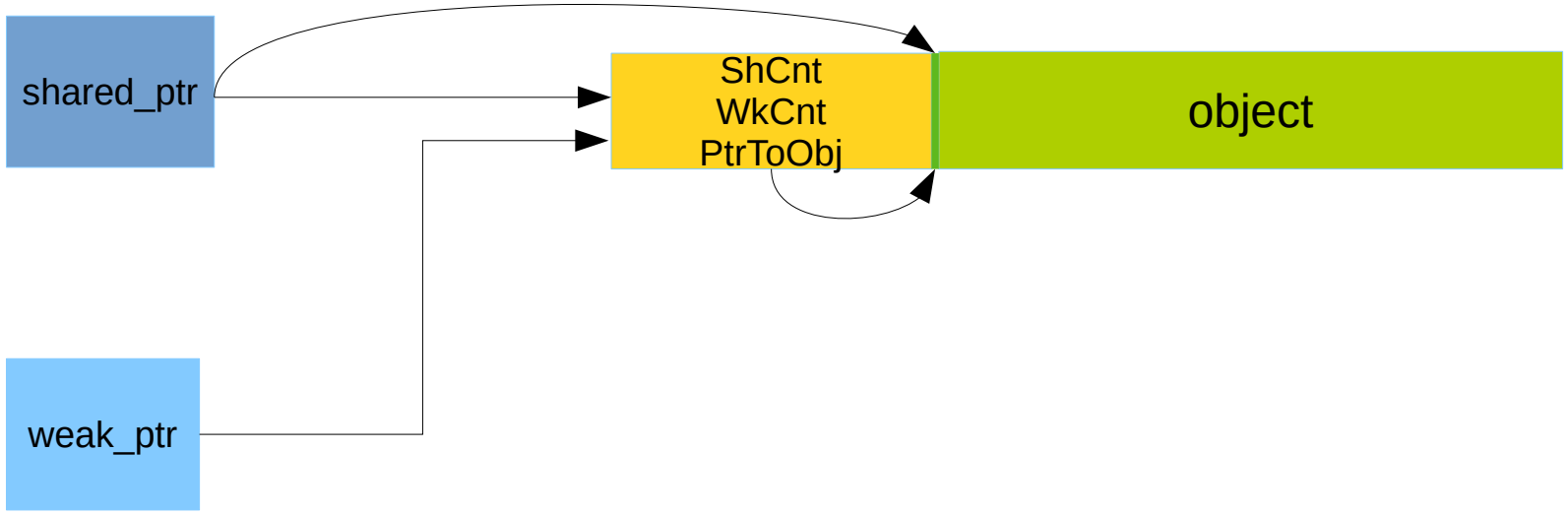
Make functions

```
// For unique_ptr
// default constructor of T
std::unique_ptr<T> v1 = std::make_unique<T>();
// constructor with params
std::unique_ptr<T> v2 = std::make_unique<T>(x,y,z);
// array of 5 elements
std::unique_ptr<T[]> v3 = std::make_unique<T[]>(5);

// similar methods for shared_ptr
```

Make functions

```
// For shared_ptr
// default constructor of T
std::shared_ptr<T> v1 = std::make_shared<T>();
// constructor with params
std::shared_ptr<T> v2 = std::make_shared<T>(x,y,z);
// array of 5 elements
std::shared_ptr<T[]> v3 = std::make_shared<T[]>(5);
```



Trap: exception safety

```
int f(); // may throw exception

// possible memory leak
std::pair<std::unique_ptr<MyClass>,int> foo()
{
  return std::make_pair(std::unique<MyClass>(new MyClass()), f());
}
```

Trap: exception safety

```
int f(); // may throw exception

// possible memory leak
std::pair<std::unique_ptr<MyClass>,int> foo()
{
  return std::make_pair(std::unique<MyClass>(new MyClass()), f());
}
```

1. Runs **new** MyClass
2. Runs f() and **throw** exception
3. std::unique_ptr constructor is not called

Trap: exception safety

```
int f(); // may throw exception

// possible memory leak
std::pair<std::unique_ptr<MyClass>,int> foo()
{
  return std::make_pair(std::unique<MyClass>(new MyClass()), f());
}
```

```
int f(); // may throw exception

// safe
std::pair<std::unique_ptr<MyClass>,int> foo()
{
  return std::make_pair(std::make_unique<MyClass>(), f());
}
```

Trap: exception safety

```
int f(); // may throw exception

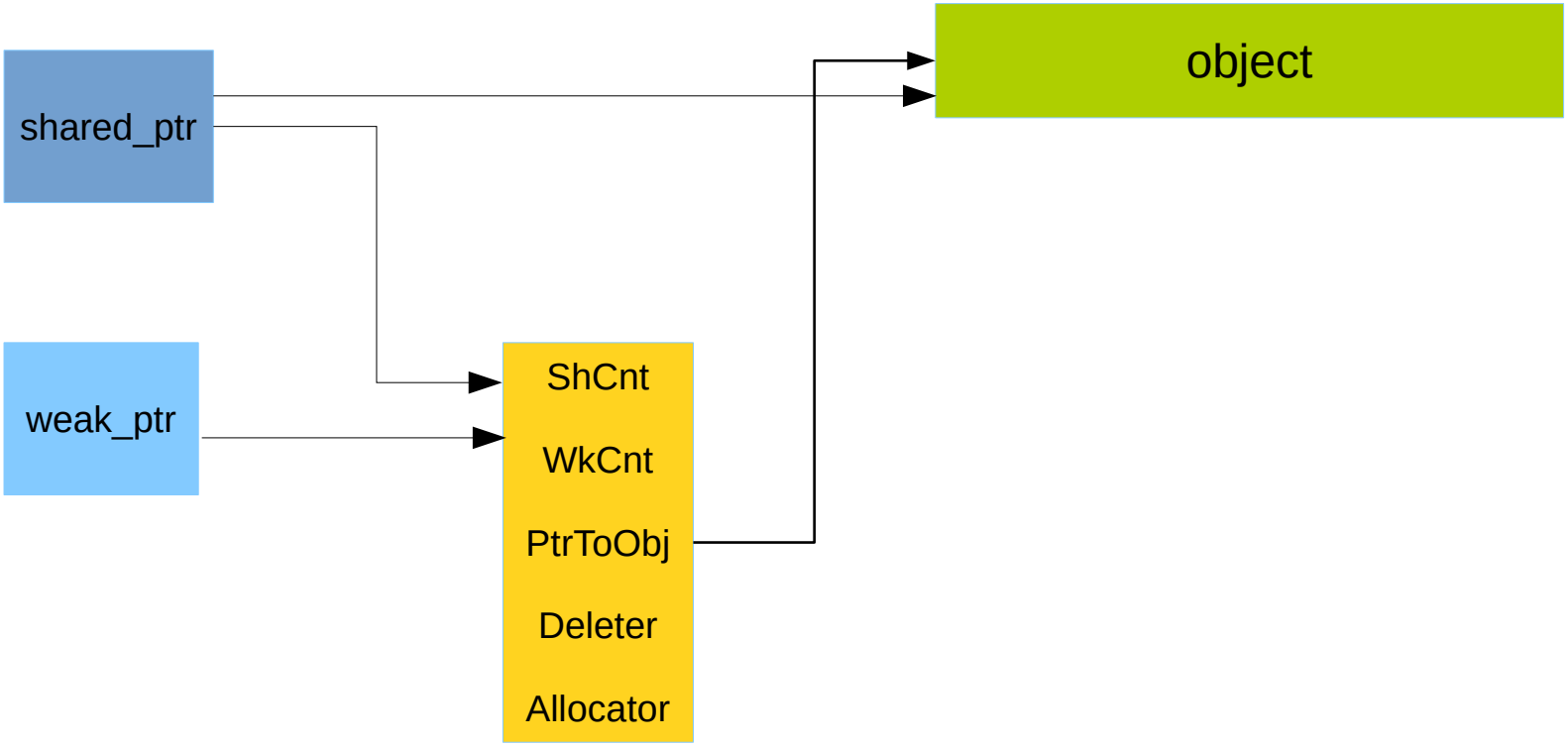
// possible memory leak
std::pair<std::unique_ptr<MyClass>,int> foo()
{
  return std::make_pair(std::unique<MyClass>(new MyClass()), f());
}
```

```
int f(); // may throw exception

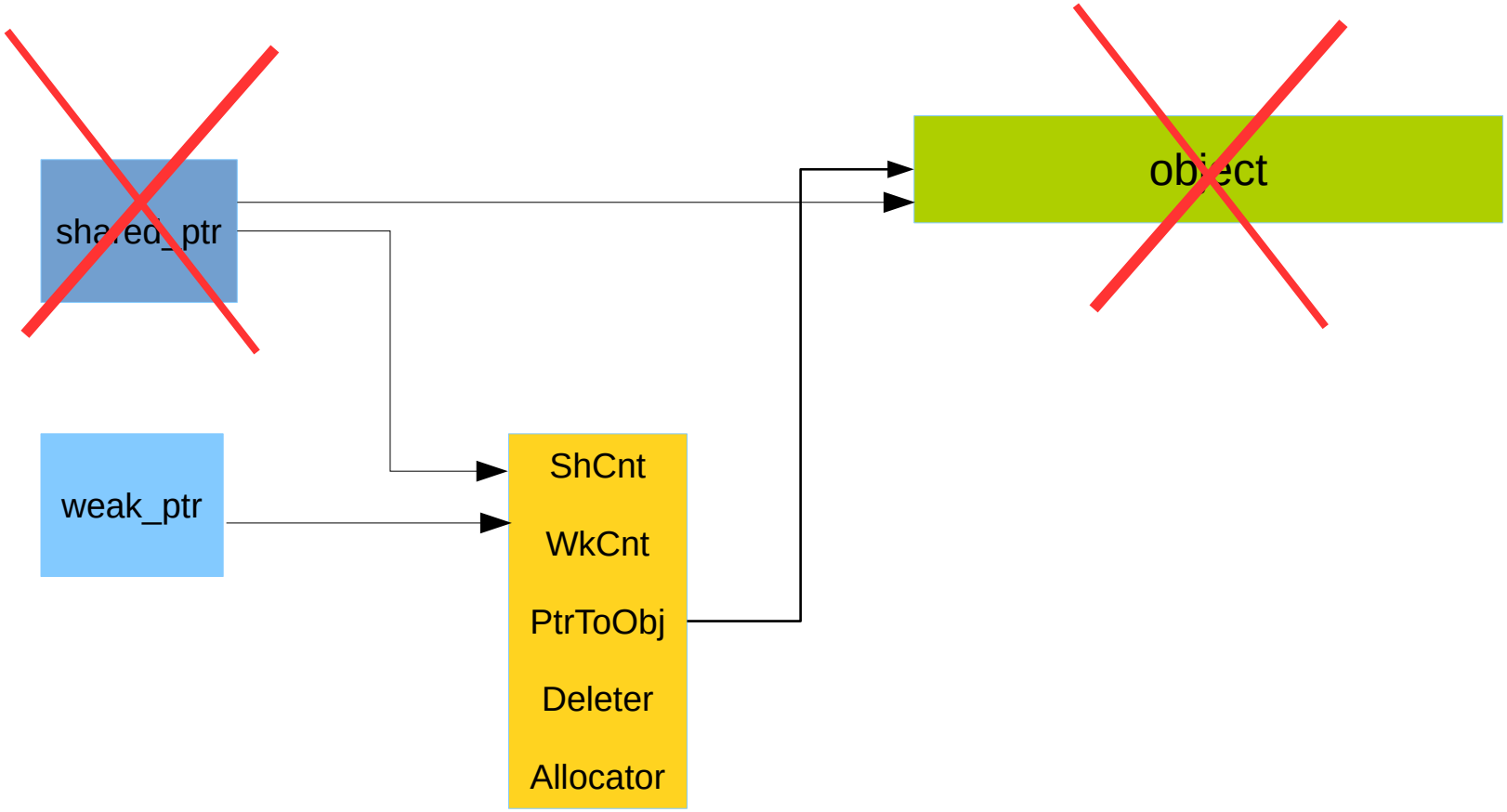
// safe
std::pair<std::unique_ptr<MyClass>,int> foo()
{
  return std::make_pair(std::make_unique<MyClass>(), f());
}
```

No news – good news!

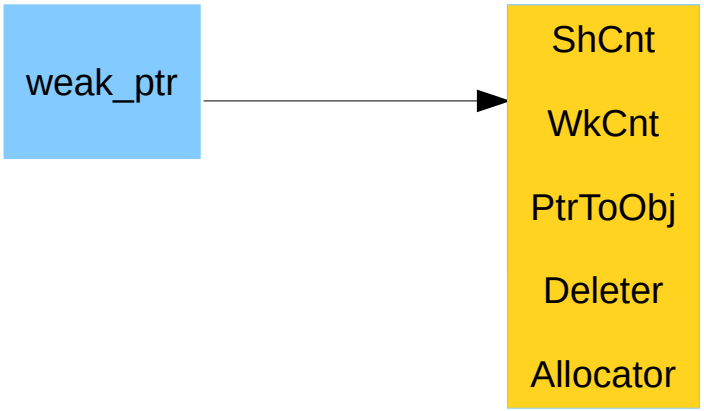
Trap: overuse of memory



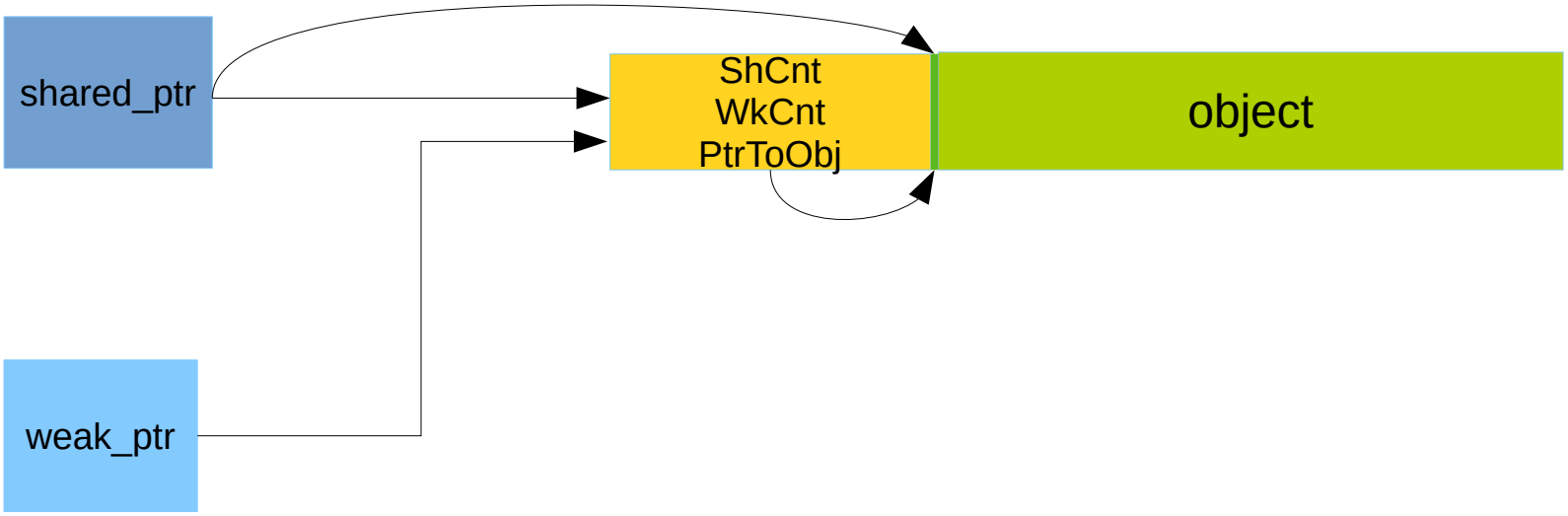
Trap: overuse of memory



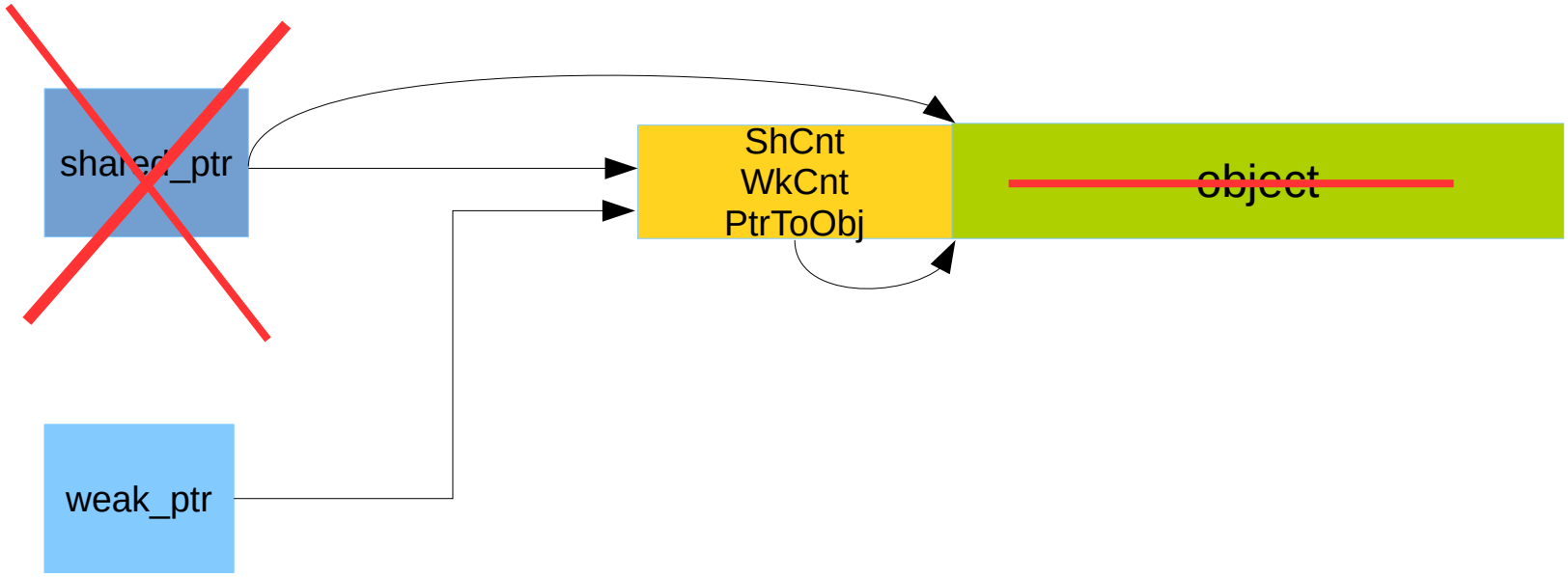
Trap: overuse of memory



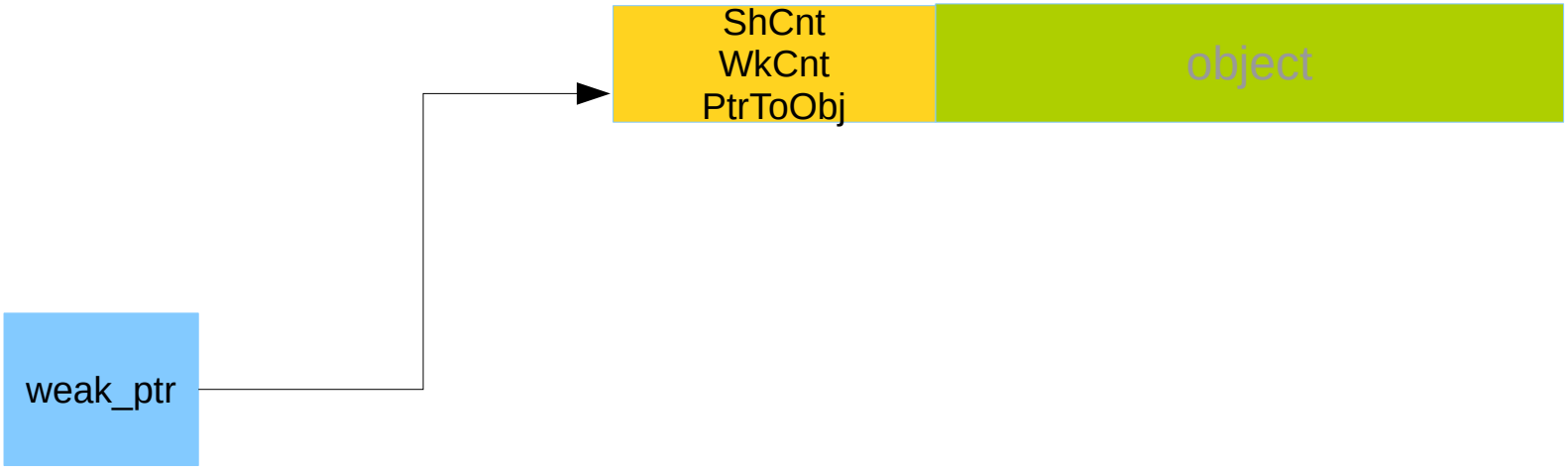
Trap: overuse of memory



Trap: overuse of memory



Trap: overuse of memory



When NOT to use make_*

- Both
  - You need custom deleter
  - You want to use braced initializer
- std::unique_ptr
  - You want custom allocator
- std::shared_ptr
  - Long living weak_ptrs
  - Class-specific new and delete
  - Potential false sharing of the object and the reference counter

Shared_ptr aliasing constructor

```
template< class Y >
shared_ptr( const shared_ptr<Y>& r, element_type* ptr ) noexcept;

// since C++20
template< class Y >
shared_ptr( shared_ptr<Y>&& r, element_type* ptr ) noexcept;
```

- Shares owneship with r
  - Reference counter is common with r
- Points to p
  - get() and - > returns p

Shared_ptr aliasing constructor

```
struct Data {
  int _i;
  Data( int i) : _i{i} { }
  virtual ~Data()  { std::cout<<"Data::~Data(): "<<_i<<"\n"; }
};
struct Wrapper {
  Data _data;
  Wrapper(int i) : _data{i} { }
  ~Wrapper()  { std::cout<<"Wrapper::~Wrapper(): "<<_data._i<<"\n"; }
};
int main()
{
  {
    Wrapper{1};
    const Data &dr = Wrapper{2}._data;
    std::cout << "end block\n";
  }
}
```

Shared_ptr aliasing constructor

```
struct Data {
  int _i;
  Data( int i) : _i{i} { }
  virtual ~Data()  { std::cout<<"Data::~Data(): "<<_i<<"\n"; }
};
struct Wrapper {
  Data _data;
  Wrapper(int i) : _data{i} { }
  ~Wrapper()  { std::cout<<"Wrapper::~Wrapper(): "<<_data._i<<"\n"; }
};
int main()
{
  {
    Wrapper{1};
    const Data &dr = Wrapper{2}._data;  // lifetime extension
    std::cout << "end block\n";
  }
}
```

```
Wrapper::~Wrapper(): 1
Data::~Data(): 1
end block
Wrapper::~Wrapper(): 2
Data::~Data(): 2
```

Shared_ptr aliasing constructor

```
struct Data {
  int _i;
  Data( int i) : _i{i} { }
  virtual ~Data()  { std::cout<<"Data::~Data(): "<<_i<<"\n"; }
};
struct Wrapper {
  Data _data;
  Wrapper(int i) : _data{i} { }
  ~Wrapper()  { std::cout<<"Wrapper::~Wrapper(): "<<_data._i<<"\n"; }
};
int main()
{
  {
    std::shared_ptr<Wrapper> wp = std::make_shared<Wrapper>(1);
    std::shared_ptr<Data>    dp(wp, &wp->_data);
    wp.reset();
    std::cout << dp->_i << '\n';
    std::cout << "end block\n";
  }
}
```

```
1
end block
Wrapper::~Wrapper(): 1
Data::~Data(): 1
```

Templates

- From macros to templates
- Parameter deduction, instantiation,specialization
- Class templates, partial specialization
- Explicit instantiation
- Two phase lookup
- Mixins
- CRTP (Curiously Recurring Template Pattern)
- Variadic templates in C++11
- Fold expressions in C++17

Templates

- Originally Stroustrup planned only Macros
- Side effects are issue with Macros: no types known
- Templates are integrated to C++ type system
- Unconstrained generics (but Concepts since C++20)
- Templates are not functions, they are skeletons
- Parameter deduction
- Instantiation
- No (non-trivial) conversion

Templates

```
template <typename T>
void swap( T& x, T& y)
{
    T temp = x;
    x = y;
    y = temp;
}
template <typename T>
T max( T a, T b)
{
    if ( a > b )
        return a;
    else
        return b;
}
void f()
{
    int i = 3, j = 4, k;
    double f = 3.14, g = 5.55, h;
    k = max(i,j);
    h = max(f,g);
    h = max(i,f);
}
```

Templates

```
template <typename T>
void swap( T& x, T& y)
{
    T temp = x;
    x = y;
    y = temp;
}
template <typename T>
T max( T a, T b)
{
    if ( a > b )
        return a;
    else
        return b;
}
void f()
{
    int i = 3, j = 4, k;
    double f = 3.14, g = 5.55, h;
    k = max(i,j); // 4
    h = max(f,g); // 5.55
    h = max(i,f); // ?
}
```

```
template <typename T>
void swap( T& x, T& y)
{
    T temp{std::move(x)};
    x = std::move(y);
    y = std::move(temp);
}
template <typename T>
constexpr const T& max(const T& a,const T& b)
{
    if ( a < b )
        return b;
    else
        return a;
}
```

Templates with more types

```
template <class T, class S>
T max( T a, S b)   // is this ok?
{
    if ( a > b )
        return a;
    else
        return b;
}

int f()
{
  int     i = 3;
  double  x = 3.14;
  double  z;

  z = max( i, x);     // z == 3.0
}
```

Templates with more types

```
template <class R, class T, class S>
R max( T a, S b)    // is this ok?
{
    if ( a > b )
        return a;
    else
        return b;
}

int f()
{
  int      i = 3;
  double   x = 3.14;
  double   z;

  z = max( i, x);     // compile error: no deduction on return type
}
```

No deduction on return type

```
template <class R, class T, class S>
R max( T a, S b, R)
{
    if ( a > b )
        return a;
    else
        return b;
}
z = max( i, x, 0.0); // works, but...

template <class R, class T, class S>
R max( T a, S b)
{
    if ( a > b )
        return a;
    else
        return b;
}
z = max<double>( i, x);              // ok, returns 3.14
k = max<long, long, int>( i, x);     // converts long(i) and int(x)
k = max<int, int, int>( i, j);       // too complex notation
```

No deduction on return type

```
template <class T, class S>
std::common_type<T,S>::value max( T a, S b) // since C++11
{
    if ( a > b )
        return a;
    else
        return b;
}


template <class T, class S>
auto max( T a, S b)          // since C++14: return type deduction
{
    if ( a > b )
        return a;
    else
        return b;
}
```

Template overloading

```
template <class R, class T, class S>    R max(T,S);
template <class T>                      T max(T,T);

template <>  // explicit (full) specialization
const char *max<const char *>( const char *s1, const char *s2)
{
    return  strcmp( s1, s2) > 0 ? s1 : s2;
}

int i = 3, j = 4, k;
double x = 3.14, z;
const char *s1 = "hello"; const char *s2 = "world";

k = max( i, j);             // max(T,T)
z = max<double>( i, x);     // max(T,S) returns 3.14
std::cout << max( s1, s2); // max(const char*,const char*) "world"
```

Template overloading

```
template <class R, class T, class S>    R max(T,S);
template <class T>                      T max(T,T);

template <>  // explicit (full) specialization
const char *max<const char *>( const char *s1, const char *s2)
{
    return  strcmp( s1, s2) > 0 ? s1 : s2;
}

int i = 3, j = 4, k;
double x = 3.14, z;
const char *s1 = "hello"; const char *s2 = "world";

k = max( i, j);             // max(T,T)
z = max<double>( i, x);     // max(T,S) returns 3.14
std::cout << max( s1, s2); // max(const char*,const char*) "world"
```

Template classes

- All member functions are templates
- Lazy instantiation
- Possibility of partial specialization
- Specialization(s) may completely different
- Default parameters are allowed

Dependent types

- Until type parameter is given, we are not sure on member
- Specialization can change
- If we mean type: typename keyword should be used

```
long ptr;
template <typename T>
class MyClass
{
   T::SubType * ptr;   // declaration or multiplication?
   //...
};
template <typename T>
class MyClass
{
   typename T::SubType * ptr;
   //...
};

typename T::const_iterator pos;
```

Dependent types

- There are a few exceptions, where **typename** is not needed
- Before C++20
  - Inheritance
  - Member initialization ids
- Since C++20
  - Using declaration
  - Data member declaration
  - Function parameters
  - Default argument of template
  - Type of casts

```
template <typename T>
// Before C++20
class MyClass : T::X  // base class
{
   int i{T::val}; // member initializ.

   // Since C++20
   using TX = T::X; // using
   T::X  member;    // data member
   void f( T::X param); // func param
};
```

Two phase lookup

- There is two phases for template parse and name lookup

```
void bar()
{
  std::cout << "::bar()" << std::endl;
}

template <typename T>
class Base
{
public:
    void bar() { std::cout << "Base::bar()" << std::endl; }
};

template <typename T>
class Derived : public Base<T>
{
public:
    void foo() { bar(); }  // compile error or calls external bar()
};
```

Two phase lookup

- There is two phases for template parse and name lookup

```
void bar()
{
  std::cout << "::bar()" << std::endl;
}

template <typename T>
class Base
{
public:
    void bar() { std::cout << "Base::bar()" << std::endl; }
};

template <typename T>
class Derived : public Base<T>
{
public:
    void foo() { this->bar(); }   // or Base::bar()
};
```

Mixins

- Class inheriting from its own template parameter
- Not to mix with other mixins (e.g. Scala)
- Reversing the inheritance relationship:
  - One can define the Derived class before Base class
  - Policy/Strategy can be injected

```
template <class Base>
class Mixin : public Base  { ... };

class RealBase { ... };
Mixin<RealBase> rbm;

class Strategy { ... };
Mixin<Strategy> mws;
```

Liskov substitutional principle

- Barbara Liskov 1977: object of subtype can replace object of base
- Mixin<Derived> is not inherited from Mixin<Base>

```
class Base { ... };
class Derived : public base { ... };

template <class T> class Mixin : public T { ... };

Base        b;
Derived     d;

Mixin<Base>     mb;
Mixin<Derived>  md;

b  = d      // OK
mb = md;    // Error!
```

Liskov substitutional principle

- Barbara Liskov 1977: object of subtype can replace object of base
- Mixin<Derived> is not inherited from Mixin<Base>

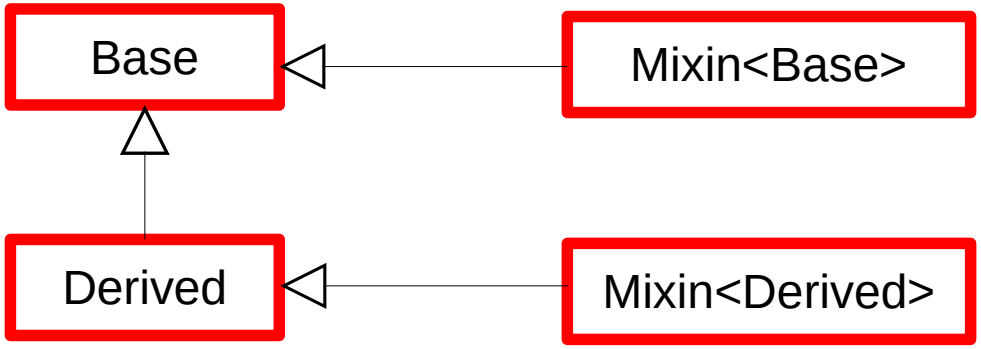
```
class Base { ... };
class Derived : public Base { ... };

template <class T> class Mixin : public T { ... };

Base          b;
Derived       d;

Mixin<Base>       mb;
Mixin<Derived>    md;

b  = d        // OK
mb = md;      // Error!
```



Liskov substitutional principle

- Barbara Liskov 1977: object of subtype can replace object of base
- Mixin<Derived> is not inherited from Mixin<Base>

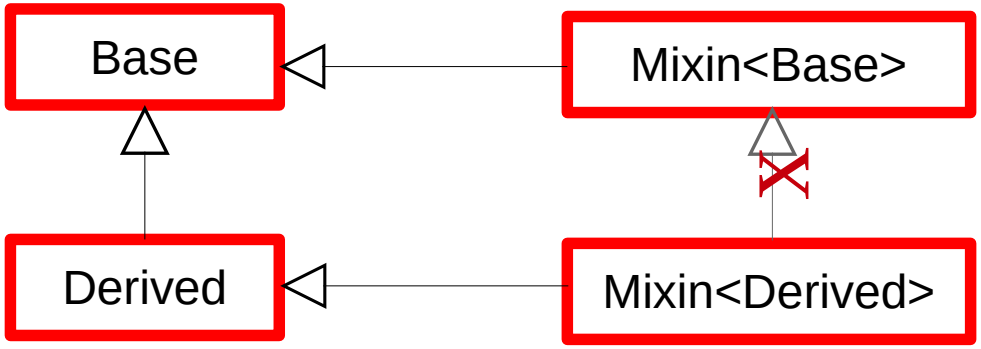
```
class Base { ... };
class Derived : public Base { ... };

template <class T> class Mixin : public T { ... };

Base          b;
Derived       d;

Mixin<Base>       mb;
Mixin<Derived>    md;

b  = d        // OK
mb = md;      // Error!
```



Constraints

- Concept checking only since C++20
- Lazy instantiation

```
template <class Sortable>
class MyMixin : public Sortable
{
public:
    void f()  // will use Sortable::sort
    {
        this->sort();  // lookup delayed due to two phase lookup
    }
};

struct X { void srot(); };  // not sortable

void client()
{
  MyMixin<X> w;  // this line still compiles
  w.f()          // syntax error only here!
}
```

Curiously Recurring Template Pattern (CRTP)

- James Coplien
- Static polymorphism

```
template <typename T>
struct Base
{
    // ...
};

struct Derived : Base<Derived>
{
    // ...
};
```

Curiously Recurring Template Pattern (CRTP)

- James Coplien 1995
- F-bounded polymorphism 1980's
- Operator generation, Enable_shared_from_this, Static polymorphism

```
template <typename T>
struct Base
{
    // ...
};

struct Derived : Base<Derived>
{
    // ...
};
```

Operator generator

```
class C
{
  // ...
};

bool operator<(const C& l, const C& r)
{
  // ...
};

inline bool operator!=(const C& l, const C& r) { return l<r || r<l; }
inline bool operator==(const C& l, const C& r) { return ! (l!=r); }
inline bool operator>=(const C& l, const C& r) { return r < l; }
// ...
```

Operator generator

```
class C
{
  // ...
};

bool operator<(const C& l, const C& r)
{
  // ...
};

inline bool operator!=(const C& l, const C& r) { return l<r || r<l; }
inline bool operator==(const C& l, const C& r) { return ! (l==r); }
inline bool operator>=(const C& l, const C& r) { return r < l; }
// ...
```

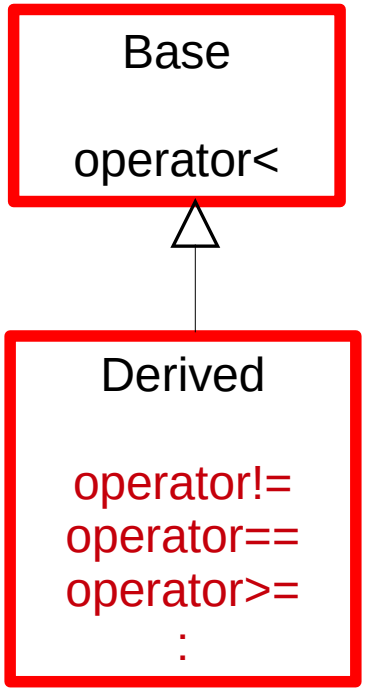
- We want automatically generate the operators from operator<

Operator generator in Derived

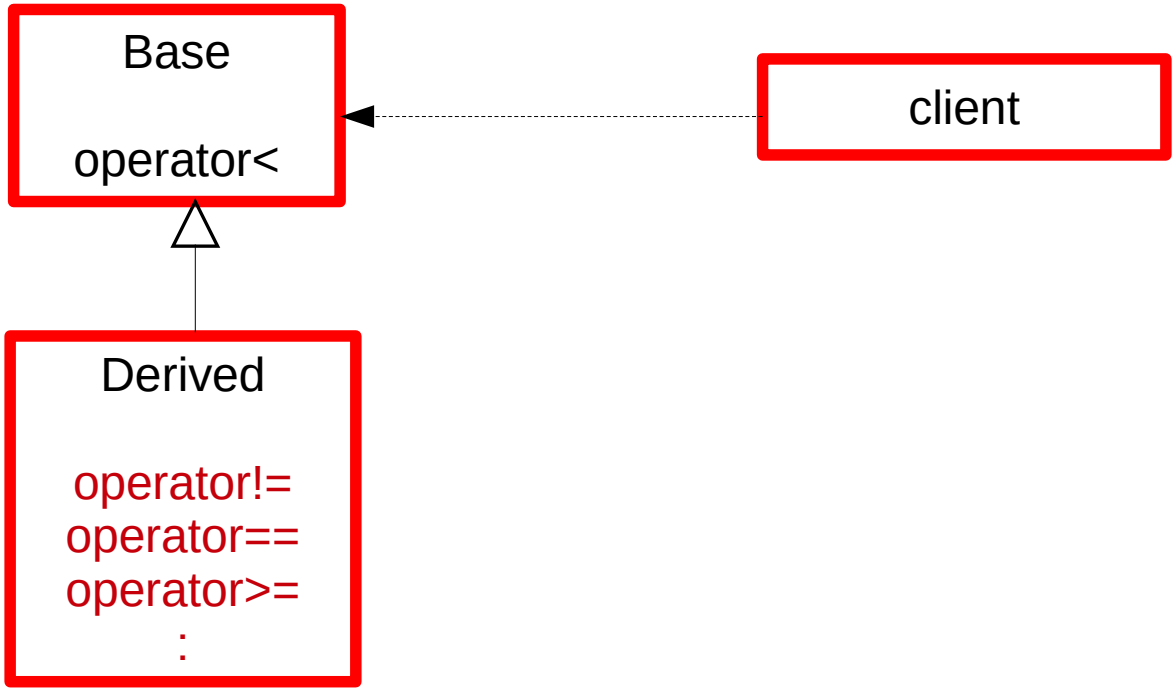
Base

operator<

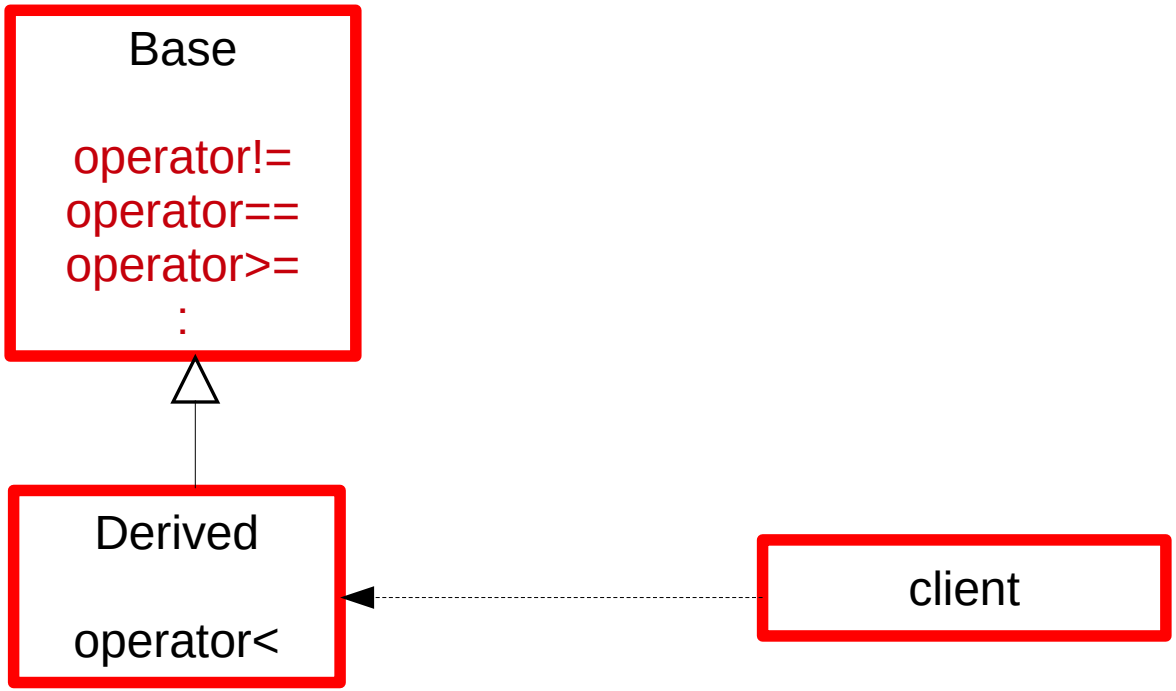
Operator generator in Derived



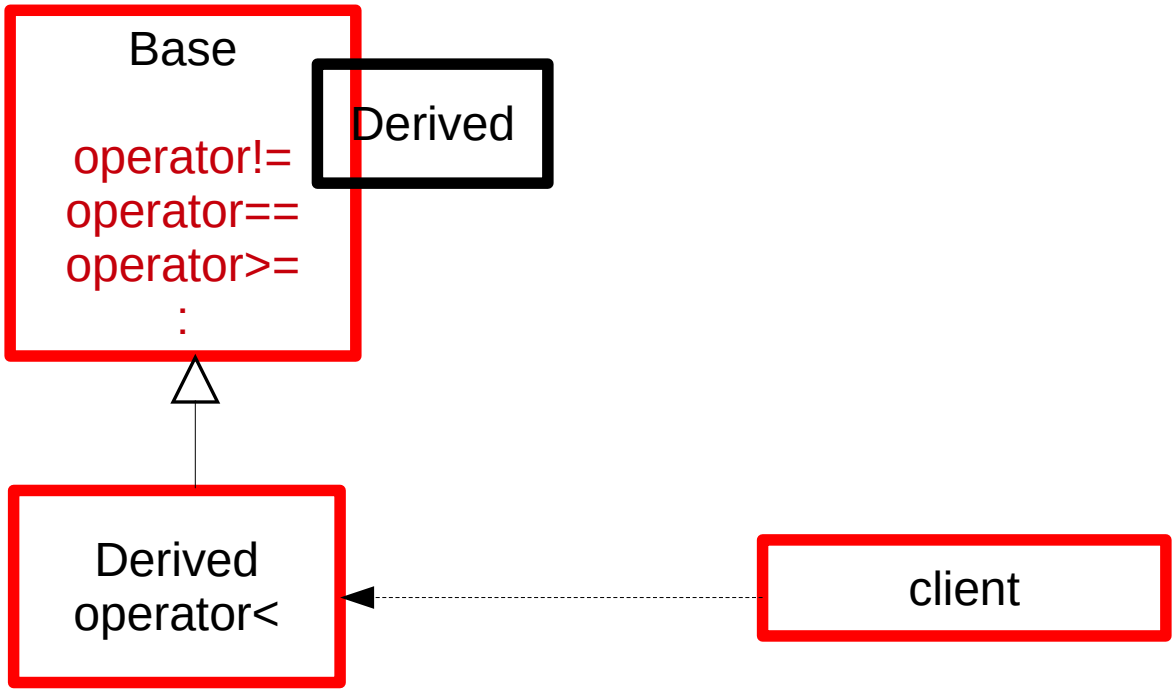
Operator generator in Derived



Operator generator in Base



Operator generator in Base



Enable shared from this

```
#include <memory>
#include <cassert>

class Y : public std::enable_shared_from_this<Y>
{
public:

    std::shared_ptr<Y> f()
    {
        return shared_from_this();
    }
};

int main()
{
    std::shared_ptr<Y> p(new Y);
    std::shared_ptr<Y> q = p->f();
    assert(p == q);
    assert(!(p < q || q < p)); // p and q must share ownership
}
```

Object counter

```
template <typename T>
struct counter {
    static int objects_created;
    static int objects_alive;
    counter() {
        ++objects_created;
        ++objects_alive;
    }
    counter(const counter&) {
        ++objects_created;
        ++objects_alive;
    }
protected:
    ~counter() // objects should never be removed through pointers of this type
    {
        --objects_alive;
    }
};
template <typename T> int counter<T>::objects_created( 0 );
template <typename T> int counter<T>::objects_alive( 0 );

class X : counter<X> { ... };
class Y : counter<Y> { ... };
```

Polymorphic chaining

```
class Printer
{
public:
  Printer(ostream& pstream) : m_stream(pstream) {}

  template <typename T>
  Printer& print(T&& t) { m_stream << t; return *this; }

  template <typename T>
  Printer& println(T&& t) { m_stream << t << endl; return *this; }
private:
  ostream& m_stream;
};
```

```
void f()
{
  Printer{myStream}.println("hello").println(500);  // works!


}
```

Polymorphic chaining

```
class Printer
{
public:
  Printer(ostream& pstream) : m_stream(pstream) {}

  template <typename T>
  Printer& print(T&& t) { m_stream << t; return *this; }

  template <typename T>
  Printer& println(T&& t) { m_stream << t << endl; return *this; }
private:
  ostream& m_stream;
};
class ColorPrinter : public Printer
{
public:
  ColorPrinter() : Printer(cout) {}
  ColorPrinter& SetConsoleColor(Color c) { /* ... */ return *this; }
};
void f()
{
  Printer{myStream}.println("hello").println(500);  // works!


}
```

Polymorphic chaining

```
class Printer
{
public:
  Printer(ostream& pstream) : m_stream(pstream) {}

  template <typename T>
  Printer& print(T&& t) { m_stream << t; return *this; }

  template <typename T>
  Printer& println(T&& t) { m_stream << t << endl; return *this; }
private:
  ostream& m_stream;
};
class ColorPrinter : public Printer
{
public:
  ColorPrinter() : Printer(cout) {}
  ColorPrinter& SetConsoleColor(Color c) { /* ... */ return *this; }
};
void f()
{
  Printer{myStream}.println("hello").println(500);  // works!
  // compile error              v here we have Printer, not ColorPrinter
  ColorPrinter().print("Hello").SetConsoleColor(Color.red).println("Printer!");
}
```

Polymorphic chaining

```
template <typename ConcretePrinter>
class Printer
{
public:
  Printer(ostream& pstream) : m_stream(pstream) {}

  template <typename T>
  ConcretePrinter& print(T&& t) { m_stream << t;
                                  return static_cast<ConcretePrinter&>(*this); }
  template <typename T>
  ConcretePrinter& println(T&& t) { m_stream << t << endl;
                                  return static_cast<ConcretePrinter&>(*this); }
 private:
  ostream& m_stream;
};
class ColorPrinter : public Printer<ColorPrinter>
{
public:
  ColorPrinter() : Printer(cout) {}
  ColorPrinter& SetConsoleColor(Color c) { /* ... */ return *this; }
};
void f()
{
  Printer{myStream}.println("hello").println(500);  // works!
  ColorPrinter().print("Hello").SetConsoleColor(Color.red).println("Printer!");
}
```

Static polymorphism

- When we separate interface and implementation
- But no run-time variation between objects

```
template <class Derived>
struct Base
{
  void interface()  {
    static_cast<Derived*>(this)->implementation();
  }
};
template <typename T>
void execute( std::vector<T*> v) {
  for( auto ptr : v ) ptr->interface();
}

struct Derived1 : Base<Derived1> {
  void implementation();
};
struct Derived2 : Base<Derived2> {
  void implementation();
};
std::vector<Base<Derived1>*> v1; /* ... */ execute(v1);
std::vector<Base<Derived2>*> v2; /* ... */ execute(v2);
```

Using (C++11)

- Typedef won't work well with templates
- Using introduce type alias

```
using myint = int;
template <class T> using ptr_t = T*;

void f(int) { }
// void f(myint) { }   syntax error: redeclaration of f(int)

// make mystring one parameter template
template <class CharT> using mystring =
    std::basic_string<CharT,std::char_traits<CharT>>;
```

Variadic templates (C++11)

- Type pack defines sequence of type parameters
- Recursive processing of pack

```
template<typename T>
T sum(T v)
{
  return v;
}
template<typename T, typename... Args>
T sum(T first, Args... args)
{
  return first + sum(args...);
}

int main()
{
  double lsum = sum(1, 2, 3.14, 8L, 7);

  std::string s1 = "x", s2 = "aa", s3 = "bb", s4 = "yy";

}
```

Variadic templates (C++11)

- Type pack defines sequence of type parameters
- Recursive processing of pack

```
template<typename T>
T sum(T v)
{
  return v;
}
template<typename T, typename... Args>//<-- template parameter pack
T sum(T first, Args... args)//<------------ function parameter pack
{
  return first + sum(args...);
}

int main()
{
  double lsum = sum(1, 2, 3.14, 8L, 7);

  std::string s1 = "x", s2 = "aa", s3 = "bb", s4 = "yy";

}
```

Parameter packs

- Pattern(Args)… is replaced by Pattern(Arg1), Pattern(Arg2),...
- Recursive processing of pack

```
template <class... Ts>
void f(Ts... args)      // ---> void f(int arg1, char arg2)
{
  g(args...);       // Pattern ==   args -----> g(  arg1,   arg2);
  g(++args...);     // Pattern == ++args -----> g(++arg1, ++arg2);
  g(1, ++args...); // Pattern == ++args -----> g(1, ++arg1, ++arg2);
  g(++args..., 9); // Pattern == ++args -----> g(++arg1, ++arg2, 9);
  g(++args)...;     // Pattern == g(++args) --> g(++arg1), g(++arg2);
  h(g(cons_cast<const Ts*>(&args))...);
                    // Pattern == g(cons_cast<const Ts*>(&args) --->
                    //     h( g(const_cast<const int*>(&arg1)),
                    //        g(const_cast<const char*>(&arg2)) );
  h(g(args...)+args...); // nested:
                // inner: h(g(arg1,arg2) + args...);
                // outer: h(g(arg1,arg2)+arg1, g(arg1,arg2)+arg2 );
}
f(1,'a');
```

Variadic templates (C++11)

- Type pack defines sequence of type parameters
- Recursive processing of pack

```
template<typename T>
T sum(T v)
{
  return v;
}
template<typename T, typename... Args>
T sum(T first, Args... args)
{
  return first + sum(args...);
}

int main()
{
  double lsum = sum(1, 2, 3.14, 8L, 7);

  std::string s1 = "x", s2 = "aa", s3 = "bb", s4 = "yy";

}
```

Variadic templates (C++11)

- Type pack defines sequence of type parameters
- Recursive processing of pack

```
template<typename T>
T sum(T v)
{
  return v;
}
template<typename T, typename... Args>
std::common_type<T,Args...>::type sum(T first, Args... args)
{
  return first + sum(args...);
}

int main()
{
  double lsum = sum(1, 2, 3.14, 8L, 7);

  std::string s1 = "x", s2 = "aa", s3 = "bb", s4 = "yy";

}
```

Class template deduction (C++17)

- The compiler can deduce template parameter(s) from
  - Declaration that specifies initialization
  - New expression
  - Function-style cast expressions

```
// examples from cppreference.com
std::pair p(2,4.5)  // C++11: std::pair<int,double>(2,4.5)
std::vector v = { 1, 2, 3, 4}; // std::vetor<int>

template <class T> struct A { A(T,T); };
auto y = new A{1,2}; // A<int>{1,2}

std::mutex mtx;
auto lck = std::lock_guard(mtx); // std::lock_guard<std::mutex>(mtx)
std::copy_n(v1,3,std::back_insert_iterator(v2)); // back_inserter(v2)
```

Class template deduction (C++17)

- Automatic and User defined deduction guideline

```
// example from cppreference.com
template <class T> struct Container
{
  Container(T t) {}
  template<class It> Container(It beg, It end);
};

template<class It>
Container(It beg, It end) ->
      Container<typename std::iterator_traits<It>::value_type>;

int main()
{
  Container c(7) // ok T=int, using automatic guide
  std::vector<double> vd = { 3.14, 4.14, 5.14 };
  auto c = Container(v.begin(), v.end()); // ok, T=double using guide
  Container d = {5,6}; // error
}
```

Class template argument deduction

- Possible traps

```
#include <iostream>
#include <vector>

int main()
{
  std::vector v1{1,2};
  std::vector v2(v1.begin(), v1.end());
  std::vector v3{v1.begin(), v1.end()};

  std::cout << v1[0] << v2[0] << ‘\n’; //<< v3[0] << '\n';
  return 0;
}

// vector<T>::vector<T>(std::initializer_list<T>);
// <Iter> vector<T>::vector<T>(Iter b, Iter e);
//       T := Iter::value_type
```

Class template argument deduction

- Possible traps
- C++17 : The Biggest Traps - Nicolai Josuttis [C++ on Sea 2019]

```
#include <iostream>
#include <vector>

void f(std::set<int> coll)
{
  std::vector v1{1,2};      // vector<int>   2 elements
  std::vector v2{1};        // vector<int>   1 elements
  std::vector v3(3,4);      // vector<int>   3 elements
  std::vector v4(8);        // error: can't deduct T
  std::vector v5(8,"");     // error: vector<const char[1]> deduced
  std::vector v6(coll.begin(), coll.end()); // vector<int> deduced
  std::vector v7{coll.begin(), coll.end()}; // vector<iter> deduced
  std::vector v8{"hello", "word"}; // vector<const char *> deduced
  std::vector v8("hello", "word"); // vector<It=const char *> deduced
}
```

Class template argument deduction

- Possible traps

```
#include <iostream>
#include <type_traits>
#include <optional>

int main()
{
  std::optional  maybe_int(1);
  std::optional  maybe_maybe_int(maybe_int);
  std::cout <<
    std::is_same_v<decltype(maybe_int), decltype(maybe_maybe_int)>
            << '\n';
  return 0;
}
```

Class template argument deduction

- Possible traps

```
#include <iostream>
#include <type_traits>
#include <optional>

int main()
{
  std::optional  maybe_int(1);
  std::optional  maybe_maybe_int(maybe_int);
  std::cout <<
    std::is_same_v<decltype(maybe_int), decltype(maybe_maybe_int)>
            << '\n';
  return 0;
}
```

```
1
```

Class template argument deduction

- Possible traps
- https://akrzemi1.wordpress.com/2018/12/09/deducing-your-intentions/

```
#include <iostream>
#include <type_traits>
#include <optional>

int main()
{
  std::optional  maybe_int(1);                  // intention: wrap
  std::optional  maybe_maybe_int(maybe_int);  // intention: copy
  std::cout <<
    std::is_same_v<decltype(maybe_int), decltype(maybe_maybe_int)>
            << '\n';
  return 0;
}

template <class T> auto __deduce(T) -> optional<T>;
template <class T> auto __deduce(const optional<T>&) -> optional<T>;
```

Mixin reloded (C++11)

- Variadic templates make us possible to define variadic set of base

```
struct A {};
struct B {};
struct C {};
struct D {};

template<class... Mixins>
class X : public Mixins...
{
public:
    X(const Mixins&... mixins) : Mixins(mixins)... { }
};

int main()
{
  A a; B b; C c; D d;

  X<A,B,C,D> xx(a,b,c,d);
}
```

std::visit

```
using var_t = std::variant<int, long, double, std::string>; // the variant to visit
template<class T> struct always_false : std::false_type {}; // helper type for the visitor

int main()
{
    std::vector<var_t> vec = {10, 15l, 1.5, "hello"};
    for(auto& v: vec)
    {
        // 1. void visitor, only called for side-effects (here, for I/O)
        std::visit([](auto&& arg){std::cout << arg;}, v);

        // 2. value-returning visitor, demonstrates the idiom of returning another variant
        var_t w = std::visit([](auto&& arg) -> var_t {return arg + arg;}, v);

        // 3. type-matching visitor: a lambda that handles each type differently
        std::cout << ". After doubling, variant holds ";
        std::visit([](auto&& arg) {
            using T = std::decay_t<decltype(arg)>;
            if constexpr (std::is_same_v<T, int>)
                std::cout << "int with value " << arg << '\n';
            else if constexpr (std::is_same_v<T, long>)
                std::cout << "long with value " << arg << '\n';
            else if constexpr (std::is_same_v<T, double>)
                std::cout << "double with value " << arg << '\n';
            else if constexpr (std::is_same_v<T, std::string>)
                std::cout << "std::string with value " << std::quoted(arg) << '\n';
            else
                static_assert(always_false<T>::value, "non-exhaustive visitor!");
        }, w);
    }
}
```

std::visit

```
using var_t = std::variant<int, long, double, std::string>; // the variant to visit

// helper type for the visitor
template<class... Ts> struct overloaded : Ts... { using Ts::operator()...; };
template<class... Ts> overloaded(Ts...) -> overloaded<Ts...>;

int main()
{
    std::vector<var_t> vec = {10, 15L, 1.5, "hello"};

    for (auto& v: vec)
    {
        // type-matching visitor: a class with 3 overloaded operator()'s
        std::visit(overloaded {
            [](auto arg) { std::cout << arg << ' '; },
            [](double arg) { std::cout << std::fixed << arg << ' '; },
            [](const std::string& arg) { std::cout << std::quoted(arg) << ' '; },
        }, v);
    }

}
```

```
10 15 1.500000 “hello”
```

Fold expressions (C++17)

- Reduces (folds) a parameter pack over a binary operator
- Syntax

(pack op …)	unary right fold	E1 op (…op(En-1 op En))
(pack op … op init)	binary right fold	E1 op (…op(En-1 op (En op i)))
(… op pack)	unary left fold	((E1 op E2) op…) op En
(init op … op pack)	binary left fold	(((i op E1) op E2) op…) op En

```
template <typename... Args>
bool all(Args... args) { return ( ... && args); }

int main()
{
    bool b = all( i1, i2, i3, i4); // = ((i1 && i2) && i3) && i4
}
```

Examples: variadic template

```
#include <sstream>
#include <iostream>
#include <vector>

template <typename T>
std::string to_string_impl(const T& t)
{
  std::stringstream ss;
  ss << t;
  return ss.str();
}
std::vector<std::string> to_string()
{
  return {};
}
template <typename P1, typename ...Param>
std::vector<std::string> to_string(const P1& p1, const Param&... params)
{

  std::vector<std::string> s;
  s.push_back(to_string_impl(p1));

  const auto remainder = to_string(params...);
  s.insert(s.end(), remainder.begin(), remainder.end());
  return s;
}
int main()
{
  const auto vec = to_string("hello", 1, 4.5);
  for (const auto& x : vec )
    std::cout << x << std::endl;
}
```

Examples: variadic template

```
#include <sstream>
#include <iostream>
#include <vector>

template <typename T>
std::string to_string_impl(const T& t)
{
  std::stringstream ss;
  ss << t;
  return ss.str();
}
std::vector<std::string> to_string()
{
  return {};
}
template <typename P1, typename ...Param>
std::vector<std::string> to_string(const P1& p1, const Param&... params)
{
  return { to_string_impl(params)... };  // std::initializer_list
  std::vector<std::string> s;
  s.push_back(to_string_impl(p1));

  const auto remainder = to_string(params...);
  s.insert(s.end(), remainder.begin(), remainder.end());
  return s;
}
int main()
{
  const auto vec = to_string("hello", 1, 4.5);
  for (const auto& x : vec )
    std::cout << x << std::endl;
}
```

Examples: variadic template

```
#include <sstream>
#include <iostream>
#include <vector>

template <typename ...Param>
std::vector<std::string> to_string(const Param&... params)
{
  const auto to_string_impl = [](const auto& t) {   // generic lambda C++14
                                    std::stringstream ss;
                                    ss << t;
                                    return ss.str();
                                };

  return { to_string_impl(params)... };  // std::initializer_list
}
int main()
{
  const auto vec = to_string("hello", 1, 4.5);
  for (const auto& x : vec )
    std::cout << x << std::endl;
}
```

Examples: fold expressions

```
#include <iostream>

template <typename ...T>
auto sum(T... t)
{
  typename std::common_type<T...>::type result{};
  std::initializer_list<int>{ (result += t, 0)... };
  return result;
}

int main()
{
  std::cout << sum(1,2,3.0,4.5) << std::endl;
}
```

Examples: fold expressions

```
#include <iostream>

template <typename ...T>
auto sum(T... t)
{
  typename std::common_type<T...>::type result{};
  std::initializer_list<int>{ (result += t, 0)... };
  return ( t + ... );  // from C++17 e.g. clang-3.8
}

int main()
{
  std::cout << sum(1,2,3.0,4.5) << std::endl;
}
```

Examples: fold expressions

```
#include <iostream>

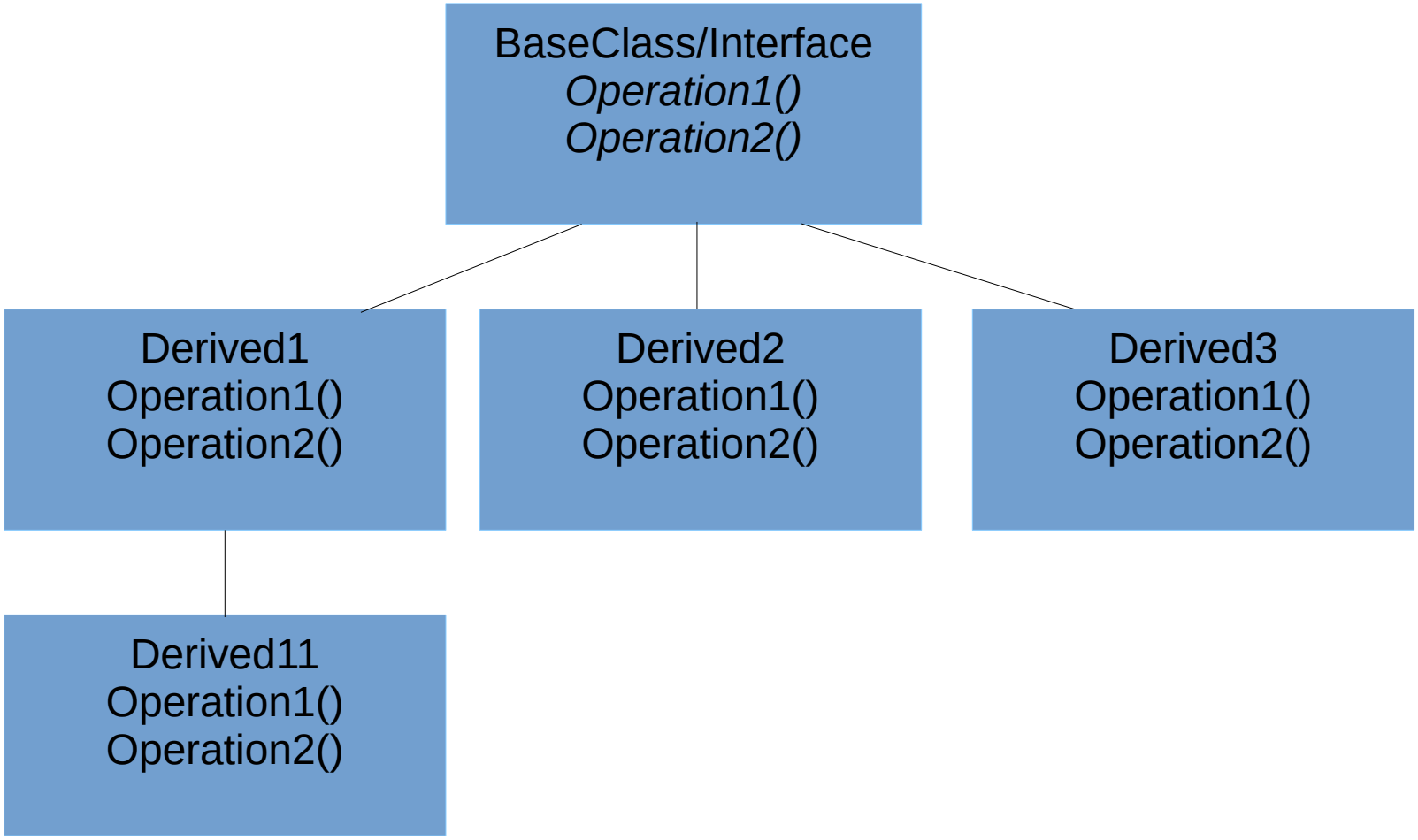
template <typename ...T>
auto sum(T... t)
{
  typename std::common_type<T...>::type result{};
  std::initializer_list<int>{ (result += t, 0)... };
  return ( t + ... );  // from C++17 e.g. clang-3.8
}
template <typename ...T>
auto avg(T... t)
{
  return ( t + ... ) / sizeof...(t); // from C++17
}

int main()
{
  std::cout << sum(1,2,3.0,4.5) << std::endl;
  std::cout << avg(1,2,3.0,4.5) << std::endl;
}
```

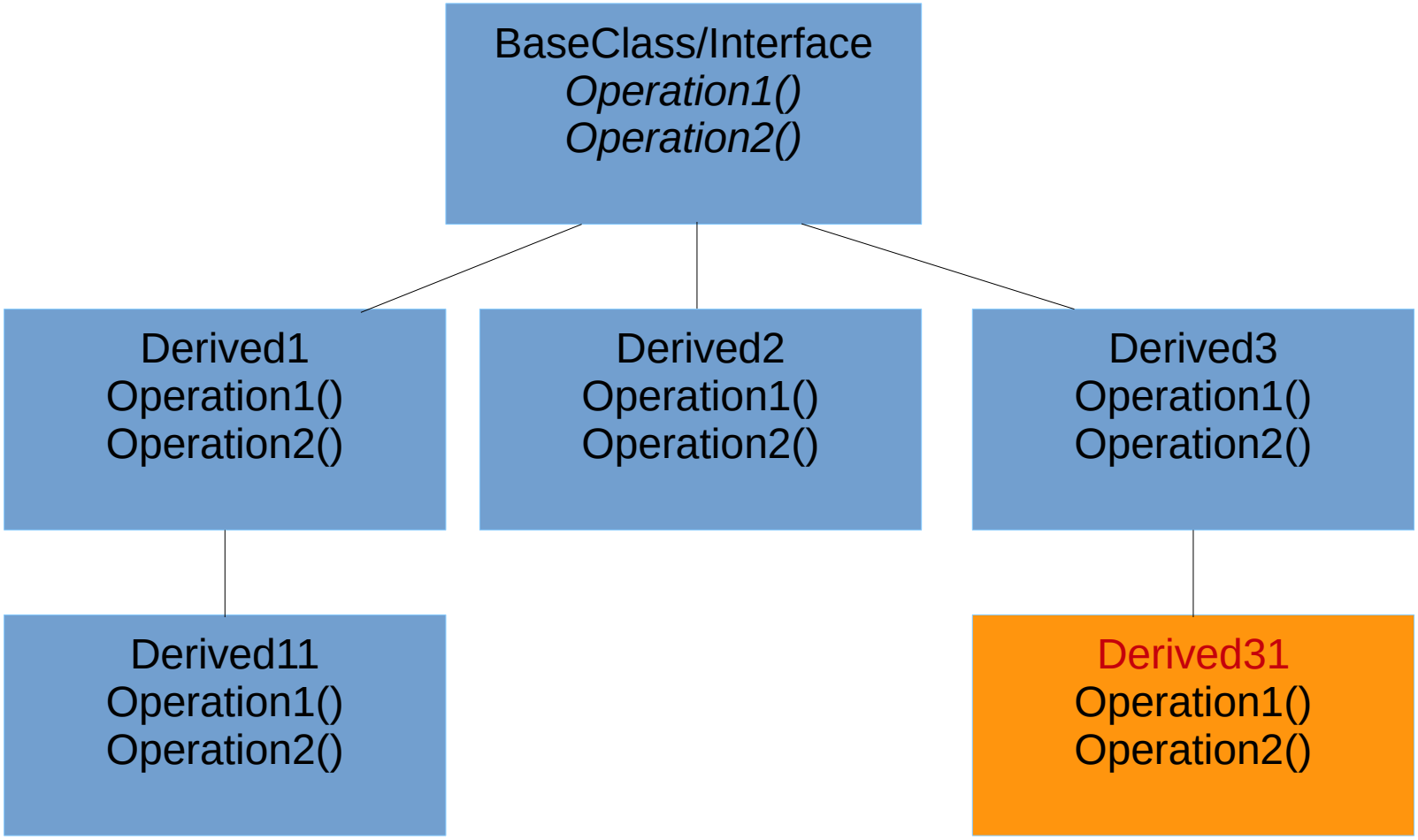
STL

- Expression problem
- Generic programming
- An example – inserters, iterator-adapters, functors
- Efficiency
- Memory consumption characteristics
- Array and forward_list in C++11
- Unordered containers in C++11

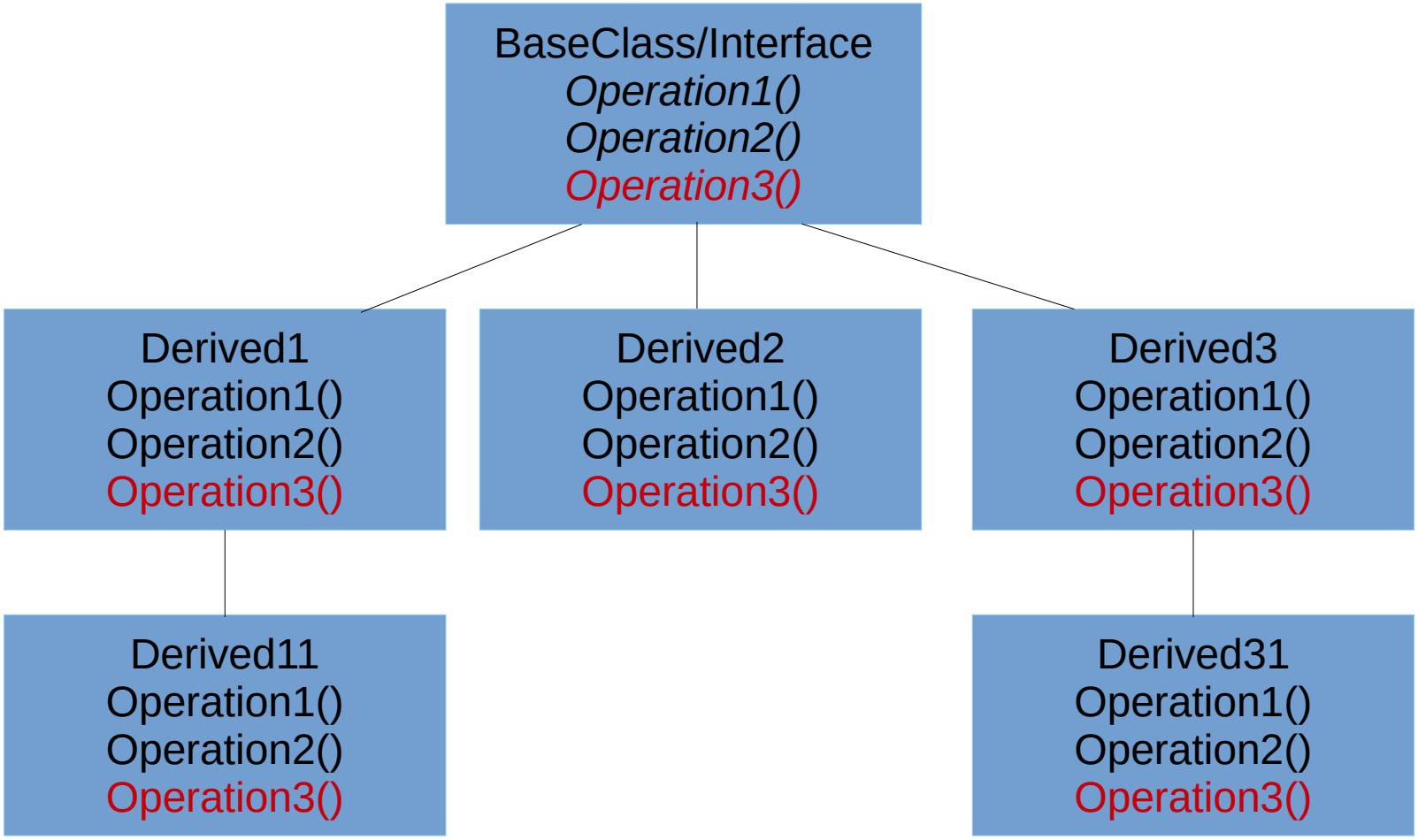
The expression problem



The expression problem



The expression problem



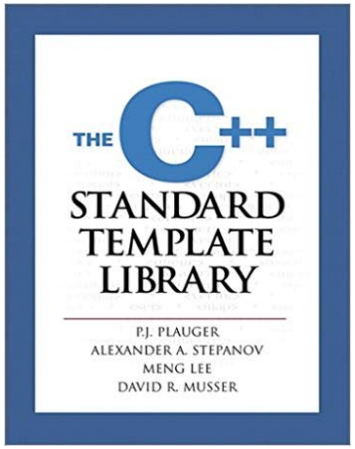
The expression problem

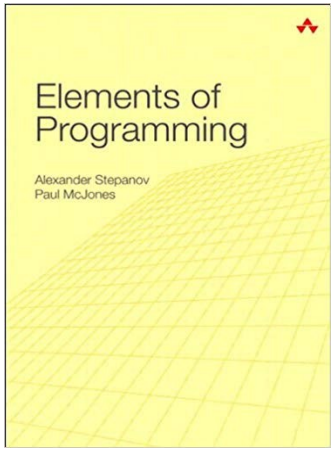
- Philip Wadler: expression problem mail, 1990
- Shriram Krishnamurthi, Matthias Felleisen, Daniel P. Friedman: "Synthesizing Object-Oriented and Functional Design to Promote Re-Use", 1998

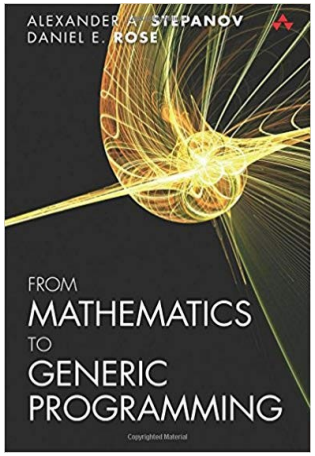
The expression problem

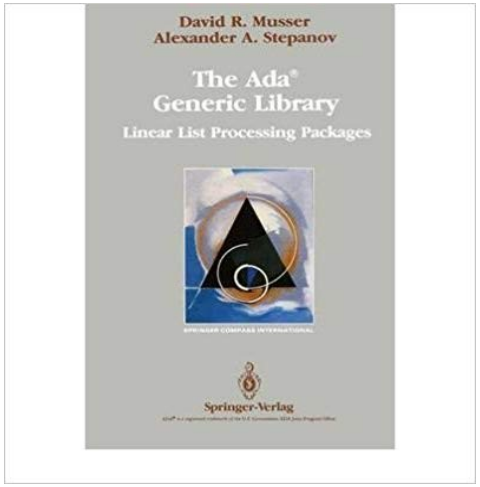
- Philip Wadler: expression problem mail, 1990
- Shriram Krishnamurthi, Matthias Felleisen, Daniel P. Friedman: "Synthesizing Object-Oriented and Functional Design to Promote Re-Use", 1998

- Aleksey Stepanov: generic programmig, 1985



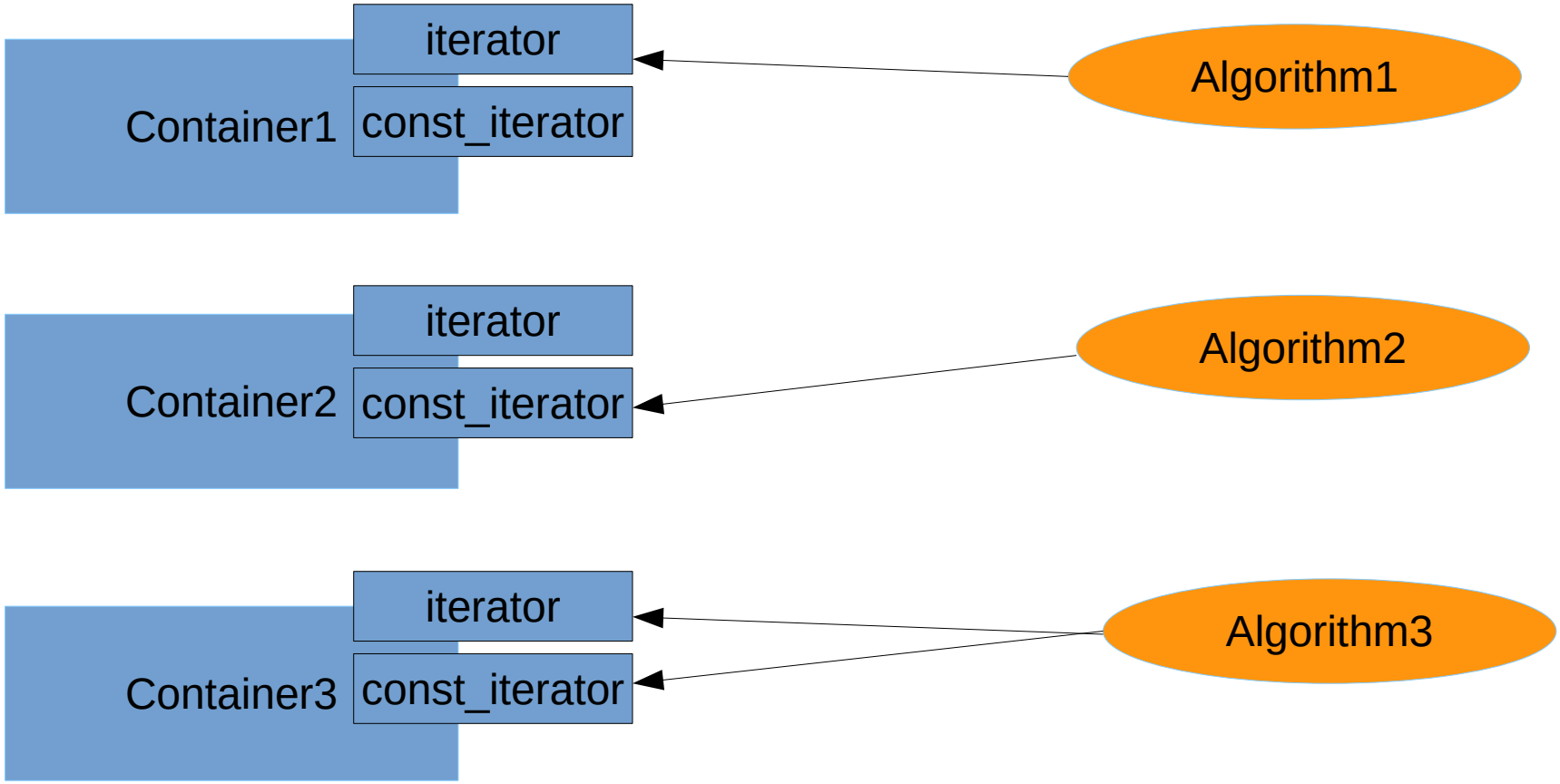




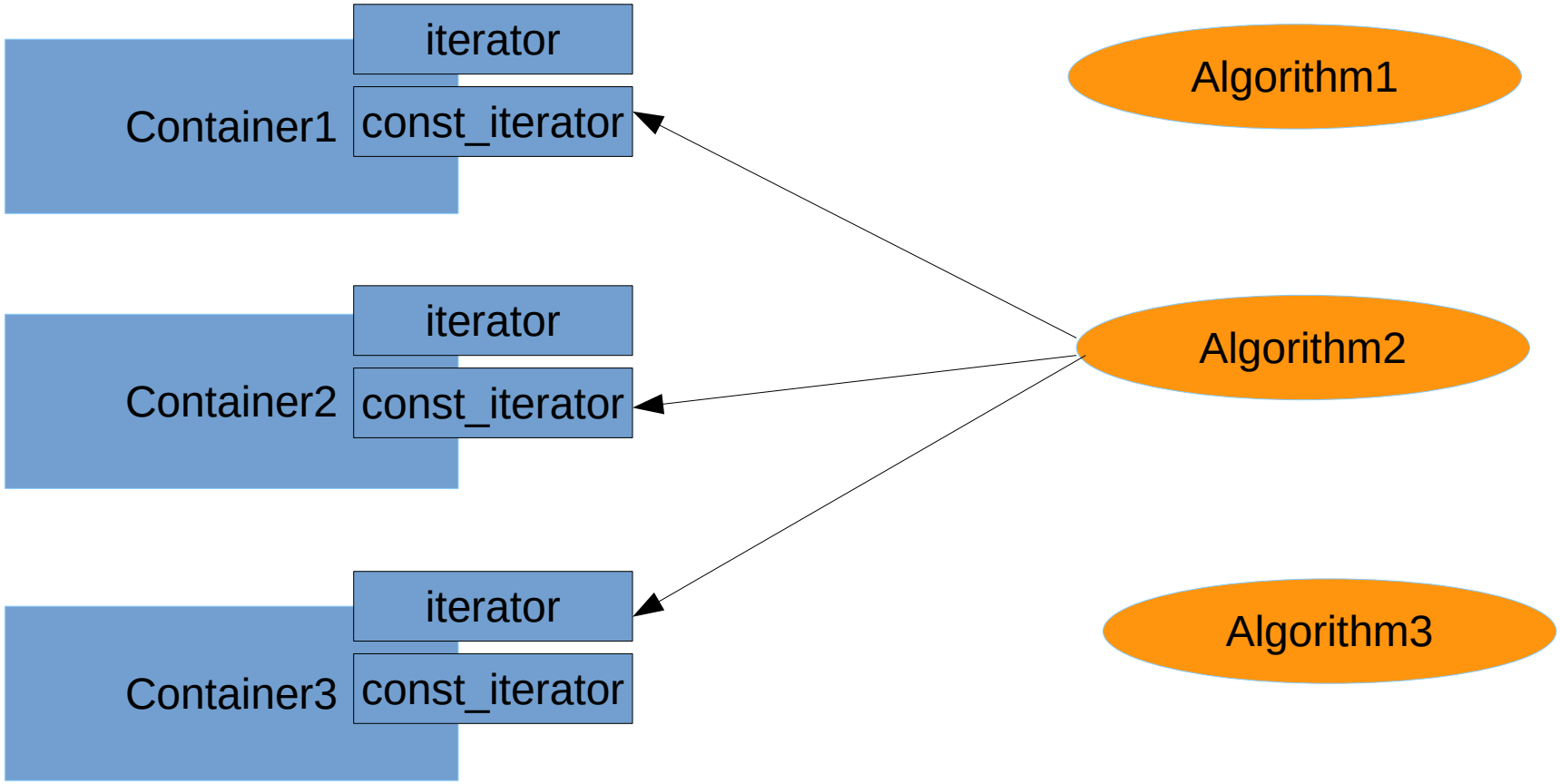


The STL solution

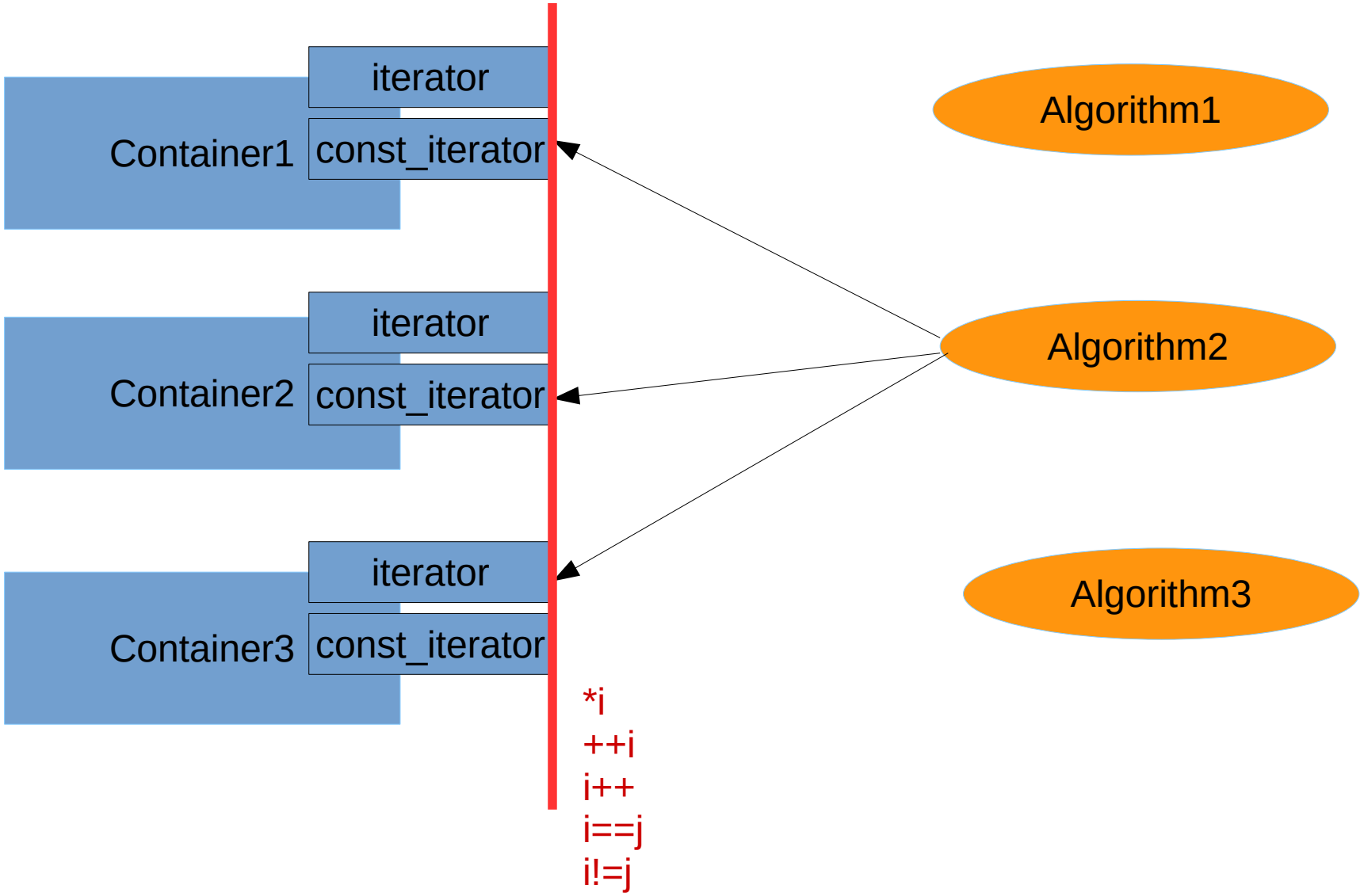
The STL solution



The STL solution



The STL solution



How to implement

```
int t[] = { 1, 3, 5, ... };

// find the first occurrence of value 55
int *pi = find( t, t+sizeof(t)/sizeof(t[0]), 55);

if ( pi )
{
  *pi = 56
}

// a very specific algorithm: works only on integer arrays
int *find( int *begin, int *end, int x)
{
  while ( begin != end )
  {
    if ( *begin == x )
    {
      return begin;
    }
    ++begin;
  }
  return nullptr;
}
```

Simple solution

```
int t[] = { 1, 3, 5, ... };

// find the first occurrence of value 55
int *pi = find( t, t+sizeof(t)/sizeof(t[0]), 55);

if ( pi )
{
  *pi = 56
}
```

Template based

```
double t[] = { 1.0, 3.14, 5.55, ... };

// find the first occurrence of a value
double *pi = find( t, t+sizeof(t)/sizeof(t[0]), 55.5);

if ( pi )
{
  *pi = 56.5
}

// Templated algorithm
template <typename T>
T *find( T *begin, T *end, const T& x)
{
  while ( begin != end )
  {
    if ( *begin == x )
    {
      return begin;
    }
    ++begin;
  }
  return nullptr;
}
```

Iterator based

```
std::list<int> li = { 1, 3, 5, ... };

// find the first occurrence of value 55
auto it = find( li.begin(), li.end(), 55);

if ( li.end() != it )
{
  *it = 56
}

// Iterator based algorithm
template <typename It, typename T>
It find( It begin, It end, const T& x)
{
  while ( begin != end )
  {
    if ( *begin == x )
    {
      return begin;
    }
    ++begin;
  }
  return end;  // not nullptr
}
```

Universal usage

```
std::list<int> li = { 1, 3, 5, ... };
std::vector<double> vd = { 1.0, 3.3, 5.5, ... };

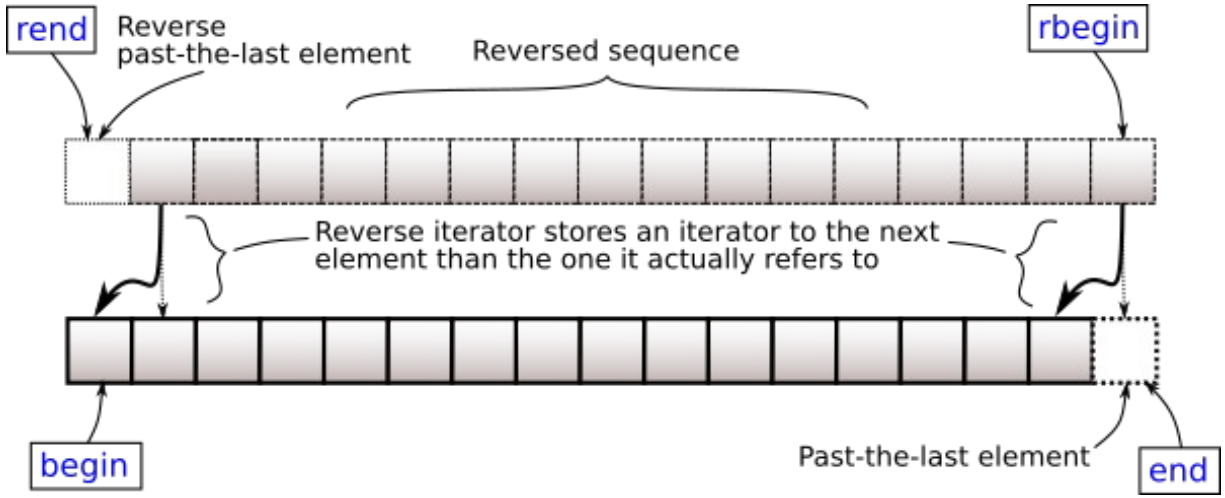
template <typename Container>
auto generic_find( const Container& c, typename Container::value_type& v)
{
  return std::find( c.begin(), c.end(), v);
}
```

- ```
  template<class C>
  typename C::value_type generic_sum(const C& c)
  {
    typename C::value_type s{};                  // value construct
    typename C::const_iterator it = c.begin();  // auto it = c.begin()

    while ( it != c.end() )
    {
      s += *it;        // requires operator+= on C::value_type
      ++it;
    }
    return s;
  }

  auto i = generic_find(li, 5);
  auto d = generic_sum(vd);
  ```
  ```

Reverse iterator




```
template<class C>
typename C::iterator find_last(C& c, const typename C::value_type& v)
{
  typename C::reverse_iterator p = c.rbegin(); // view sequence in revers
  while ( p != c.rend() )
  {
    if ( *p == v )
    {
      typename C::iterator i = p.base();
      return --i;
    }
    ++p;            // note: increment, not decrement (--)
  }
  return c.end();   // use c.end() to indicate "not found”
}
```

Iterators are const safe

```
//  C++11
std::vector<int>  v1 = { 1, 2, 3, 4, ... };
auto i = std::find( v1.begin(), v1.end(), 3);
// i is vector::iterator

const std::vector<int>  v2 = { 1, 2, 3, 4, ... };
auto j = std::find( v2.begin(), v2.end(), 3);
// j is vector::const_iterator

auto k = std::find( v1.cbegin(), v1.cend(), 3);
// k is vector::const_iterator


auto j2 = std::find( std::begin(v1), std::end(v1), 3);
// j2 is vector::iterator

auto k2 = std::find( std::cbegin(v1), std::cend(v1), 3);
// k2 is vector::const_iterator
```

Functors

```
std::list<int> li = { 1, 3, 5, ... };

// find the third occurrence of value less than 55
auto it = find_if( li.begin(), li.end(), ?);

if ( li.end() != it )
{
  *it = 56
}

// Iterator based algorithm
template <typename It, typename Pred>
It find_if( It begin, It end, Pred p)
{
  while ( begin != end )
  {
    if ( p(*begin) )
    {
      return begin;
    }
    ++begin;
  }
  return end;
}
```

Functors

```
std::list<int> li = { 1, 3, 5, ... };

// find the third occurrence of value less than 55
bool less55_3rd(int x)
{
  static int cnt = 0;
  if ( x < 55 )    ++cnt;
  return 3 == cnt;
}

// Iterator based algorithm
template <typename It, typename Pred>
It find_if( It begin, It end, Pred p)
{
  while ( begin != end )
  {
    if ( p(*begin) )  // calls less55_3rd(*begin)
    {
      return begin;
    }
    ++begin;
  }
  return end;
}
```

Functors

```
std::list<int> li = { 1, 3, 5, ... };


bool less55_3rd(int x)
{
  static int cnt = 0;
  if ( x < 55 )    ++cnt;
  return 3 == cnt;
}

// find the third occurrence of value less than 55
auto it = find_if( li.begin(), li.end(), less55_3rd);

if ( li.end() != it )
{
  *it = 56;

}
```

Functors

```
std::list<int> li = { 1, 3, 5, ... };


bool less55_3rd(int x)
{
  static int cnt = 0;
  if ( x < 55 )    ++cnt;
  return 3 == cnt;
}

// find the third occurrence of value less than 55
auto it = find_if( li.begin(), li.end(), less55_3rd);

if ( li.end() != it )
{
  *it = 56;
   it = find_if( ++it, li.end(), less55_3rd);  // works?
}
```

Functors

```
struct less55_3rd
{
  less55_3rd() : cnt(0) { }
  bool operator()(int x)
  {
    if ( x < 55 )    ++cnt;
    return 3 == cnt;
  }
private:
  int cnt;
};

template <typename It, typename Pred>
It find_if( It begin, It end, Pred p)
{
  while ( begin != end )
  {
    if ( p(*begin) )  // calls p.operator()(*begin)
    {
      return begin;
    }
    ++begin;
  }
  return end;
}
```

Functors

```
struct less55_3rd
{
  less55_3rd() : cnt(0) { }
  bool operator()(int x)
  {
    if ( x < 55 )    ++cnt;
    return 3 == cnt;
  }
private:
  int cnt;
};

// find the third occurrence of value less than 55
auto it = find_if( li.begin(), li.end(), less55_3rd{});


if ( li.end() != it )
{
  *it = 56;
   it = find_if( ++it, li.end(), less55_3rd{});  // new object, cnt = 0
}
```

Functors

```
struct less55_3rd
{
  less55_3rd() : cnt(0) { }
  bool operator()(int x)
  {
    if ( x < 55 )    ++cnt;
    return 3 == cnt;
  }
private:
  int cnt;
};

// find the third occurrence of value less than 55
auto it = find_if( li.begin(), li.end(), [](int x) { static int cnt = 0;
                                                     if(x < 55) ++cnt;
                                                     return 3 == cnt;} );
if ( li.end() != it )
{
  *it = 56;
   it = find_if( ++it, li.end(), [](int x) { ... }); // new lambda, cnt = 0
}
```

Functors

```
struct less55_3rd
{
  less55_3rd() : cnt(0) { }
  bool operator()(int x)
  {
    if ( x < 55 )    ++cnt;
    return 3 == cnt;
  }
private:
  int cnt;
};

// find the third occurrence of value less than 55
auto it = find_if( li.begin(), li.end(), [cnt=0](int x) { // since C++14
                                                          if(x < 55) ++cnt;
                                                          return 3 == cnt;} );
if ( li.end() != it )
{
  *it = 56;
   it = find_if( ++it, li.end(), [cnt=0](int x) { ... }); // new lambda
}
```

Functors

```
template <typename T>
struct less_Nth
{
  less_Nth(const T& t, int n) : value(t), nth(n), cnt(0) { }
  bool operator()(const T& x)
  {
    if ( x < value )    ++cnt;
    return nth == cnt;
  }
private:
  T   value;
  int nth;
  int cnt;
};

// find the fifth occurrence less than value 3.14
auto it = find_if( li.begin(), li.end(), less_Nth<double>{3.14,5});

if ( li.end() != it )
{
  *it = 2.178;
   it = find_if( ++it, li.end(), less_Nth<double>{99.9,7});
}
```

Example: merge two files

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

int main()   // simple merge:
{
    string s1, s2;
    ifstream f1("file1.txt");
    ifstream f2("file2.txt");

    f1 >> s1; f2 >> s2;
    while (f1 || f2)
    {
        if (f1 && ((s1 <= s2) || !f2))
        {
            cout << s1 << endl;
            f1 >> s1;
        }
        if (f2 && ((s1 >= s2) || !f1))
        {
            cout << s2 << endl;
            f2 >> s2;
        }
    }
    return 0;
}
```

http://www.caesar.elte.hu/progmat/kultura/humor/zenfothi.html

https://terebess.hu/zen/mesterek/fothi-akos.html

Example: naïve STL usage

```
#include <iostream>
#include <fstream>
#include <string>
#include <algorithm>    // merge( b1, e1, b2, e2, b3 [,opc_rend])
#include <vector>

using namespace std;
int main()
{
    ifstream if1("file1.txt");
    ifstream if2("file2.txt");

    string s;
    vector<string> v1;
    while ( if1 >> s ) v1.push_back(s); // do we have enough memory?
    vector<string> v2;
    while ( if2 >> s ) v2.push_back(s); // do we have enough memory?

    // allocate the elements for the result, do we have enough memory?
    vector<string> v3(v1.size() + v2.size());   // constructs empty strings...

    // *b3++ = *b1 < *b2 ? *b1++ : *b2++
    merge( v1.begin(),v1.end(),v2.begin(),v2.end(),v3.begin());

    for ( int i = 0; i < v3.size(); ++i)
        cout << v3[i] << endl;

    return 0;
}
```

std::merge

```
template<class InputIt1, class InputIt2, class OutputIt>
OutputIt merge(InputIt1 first1, InputIt1 last1,
               InputIt2 first2, InputIt2 last2,
               OutputIt d_first)
{
    for (; first1 != last1; ++d_first) {
        if (first2 == last2) {
            return std::copy(first1, last1, d_first);
        }
        if (*first2 < *first1) {
            *d_first = *first2;
            ++first2;
        } else {
            *d_first = *first1;
            ++first1;
        }
    }
    return std::copy(first2, last2, d_first);
}
```

std::merge

```
template<class InputIt1, class InputIt2, class OutputIt>
OutputIt merge(InputIt1 first1, InputIt1 last1,
               InputIt2 first2, InputIt2 last2,
               OutputIt d_first, Compare comp)
{
    for (; first1 != last1; ++d_first) {
        if (first2 == last2) {
            return std::copy(first1, last1, d_first);
        }
        If ( comp(*first2,*first1) ) {
            *d_first = *first2;
            ++first2;
        } else {
            *d_first = *first1;
            ++first1;
        }
    }
    return std::copy(first2, last2, d_first);
}
```

Example: inserters

```
#include <iostream>
#include <fstream>
#include <string>
#include <algorithm>
#include <vector>

using namespace std;

int main()
{
    ifstream if1("file1.txt");
    ifstream if2("file2.txt");

    string s;
    vector<string> v1;
    while ( if1 >> s ) v1.push_back(s);
    vector<string> v2;
    while ( if2 >> s ) v2.push_back(s);
    vector<string> v3;
    v3.reserve( v1.size() + v2.size() ); // alloc buffer but do not construct, size == 0

    merge(v1.begin(),v1.end(),v2.begin(),v2.end(),back_inserter(v3)); // v3.push_back(*c)

    for ( int i = 0; i < v3.size(); ++i)
        cout << v3[i] << endl;

    return 0;
}
```

Example: inserters

```
template<typename _Cont>
class back_insert_iterator : public
      iterator<output_iterator_tag,void,void,void,void>
{
  back_insert_iterator&
  operator=(const typename _Cont::value_type& __value)
  {   // same overloaded to reference and rvalue ref
    container->push_back(__value);
    return *this;
  }
  back_insert_iterator& operator*()   { return *this; }
  back_insert_iterator& operator++()  { return *this; }
  back_insert_iterator operator++(int){ return *this; }
protected:
  _Cont* container;
};
```

```
#include <iostream>
#include <fstream>
#include <string>
#include <algorithm>
#include <vector>

using namespace std;

int main()
{
    ifstream if1("file1.txt");
    ifstream if2("file2.txt");

    string s;
    vector<string> v1;
    while ( if1 >> s ) v1.push_back(s);
    vector<string> v2;
    while ( if2 >> s ) v2.push_back(s);
    vector<string> v3;
    v3.reserve( v1.size() + v2.size() ); // alloc buffer but do not construct, size == 0

    merge(v1.begin(),v1.end(),v2.begin(),v2.end(),back_inserter(v3)); // v3.push_back(*c)

    for ( int i = 0; i < v3.size(); ++i)
        cout << v3[i] << endl;

    return 0;
}
```

Example: stream iterators

```
#include <iostream>
#include <fstream>
#include <string>
#include <algorithm>
#include <iterator>     // input- and output-iterators

using namespace std;

int main()
{
    ifstream if1("file1.txt");
    ifstream if2("file2.txt");

    // istream_iterator(if1) -> if1 >> *current
    // istream_iterator() -> EOF
    // ostream_iterator(of,x) -> of << *current << x
    merge( istream_iterator<string>(if1), istream_iterator<string>(),
           istream_iterator<string>(if2), istream_iterator<string>(),
           ostream_iterator<string>(cout,"\n") );

    return 0;
}
```

Example: comparator

```
#include <iostream>
#include <fstream>
#include <string>
#include <cctype>
#include <algorithm>
#include <iterator>

struct my_less  // function object: "functor"
{
    bool operator()(const std::string& s1, const std::string& s2)  {
        std::string us1 = s1;
        std::string us2 = s2;
        transform( s1.begin(), s1.end(), us1.begin(), toupper);  // TODO: use <locale>
        transform( s2.begin(), s2.end(), us2.begin(), toupper);
        return us1 < us2;
    }
};

int main()
{
    ifstream if1("file1.txt");
    ifstream if2("file2.txt");

    merge( istream_iterator<string>(if1), istream_iterator<string>(),
           istream_iterator<string>(if2), istream_iterator<string>(),
           ostream_iterator<string>(cout,"\n"), my_less() );
    return 0;
}
```

Example: template comparator

```
template <typename T>
class distr
{
public:
    distr(int l, int r, bool fl = true) : left_(l), right_(r), from_left_(fl), cnt_(0) { }
    // formal reasons: "compare" has two parameters of type T
    bool operator()( const T&, const T&)   {
        bool ret = from_left_;   // from_left_ is “smaller” currently
        const int max = from_left_ ? left_ : right_;
        if ( ++cnt_ == max )
        {
            cnt_ = 0;
            from_left_ = ! from_left_;
        }
        return ret;
    }
private:
    const int left_;  // read left_ element from left
    const int right_; // read right_ element from right
    int from_left_;   // start from left
    int cnt_;
};

   // ...
   istream_iterator<string>(if2), istream_iterator<string>(),
   ostream_iterator<string>(cout,"\n"), distr<std::string>(left,right) );
   // ...
```

Vector vs Associative containers

```
#include <iostream>
#include <string>
#include <algorithm>
#include <set>
using namespace std;

int main()   /* print unique sorted elems  */
{
    set<string> coll( istream_iterator<string>(cin), istream_iterator<string>());
    copy( coll.begin(), coll.end(), ostream_iterator<string>(cout, "\n"));
}
```

```
#include <iostream>
#include <string>
#include <algorithm>
#include <vector>
using namespace std;

int main()   /* print unique sorted elems  */
{
    vector<string> coll( istream_iterator<string>(cin), istream_iterator<string>());
    sort (coll.begin(), coll.end());   // sort elements
    unique_copy (coll.begin(), coll.end(), ostream_iterator<string>(cout, "\n"));
}
```

Vector vs Associative containers

```
#include <iostream>
#include <string>
#include <algorithm>
#include <set>
using namespace std;

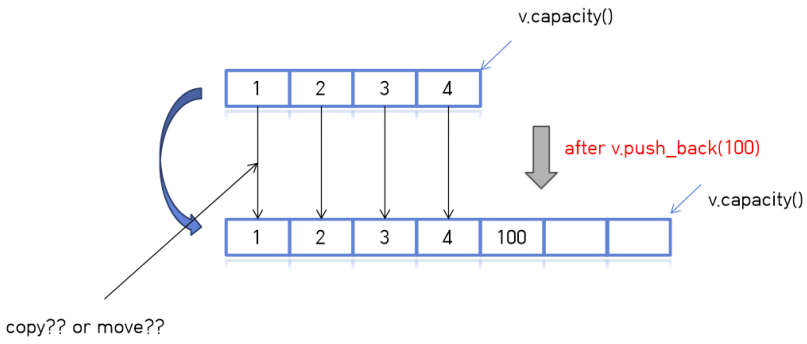
int main()   /* print unique sorted elems  */
{
    set<string> coll( istream_iterator<string>(cin), istream_iterator<string>());
    copy( coll.begin(), coll.end(), ostream_iterator<string>(cout, "\n"));
}
```

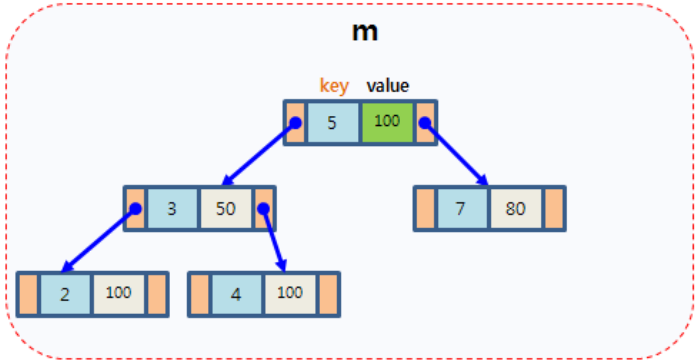
```
#include <iostream>
#include <string>
#include <algorithm>
#include <vector>
using namespace std;

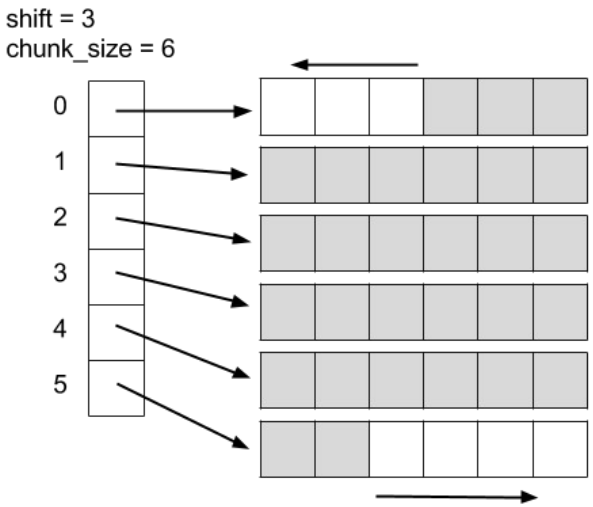
int main()   /* print unique sorted elems  */
{
    vector<string> coll( istream_iterator<string>(cin), istream_iterator<string>());
    sort (coll.begin(), coll.end());   // sort elements
    unique_copy (coll.begin(), coll.end(), ostream_iterator<string>(cout, "\n"));
}
```

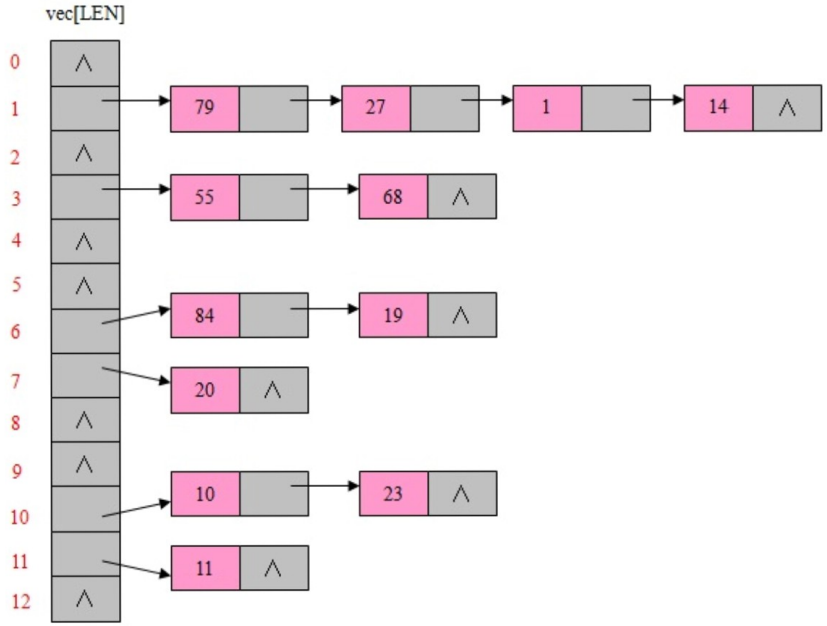
```
//  150.000 string:  vector solution is better with 10%
//       + reserve:                                 15%
// multiset + copy:                                 40%
```

Typical container implementations

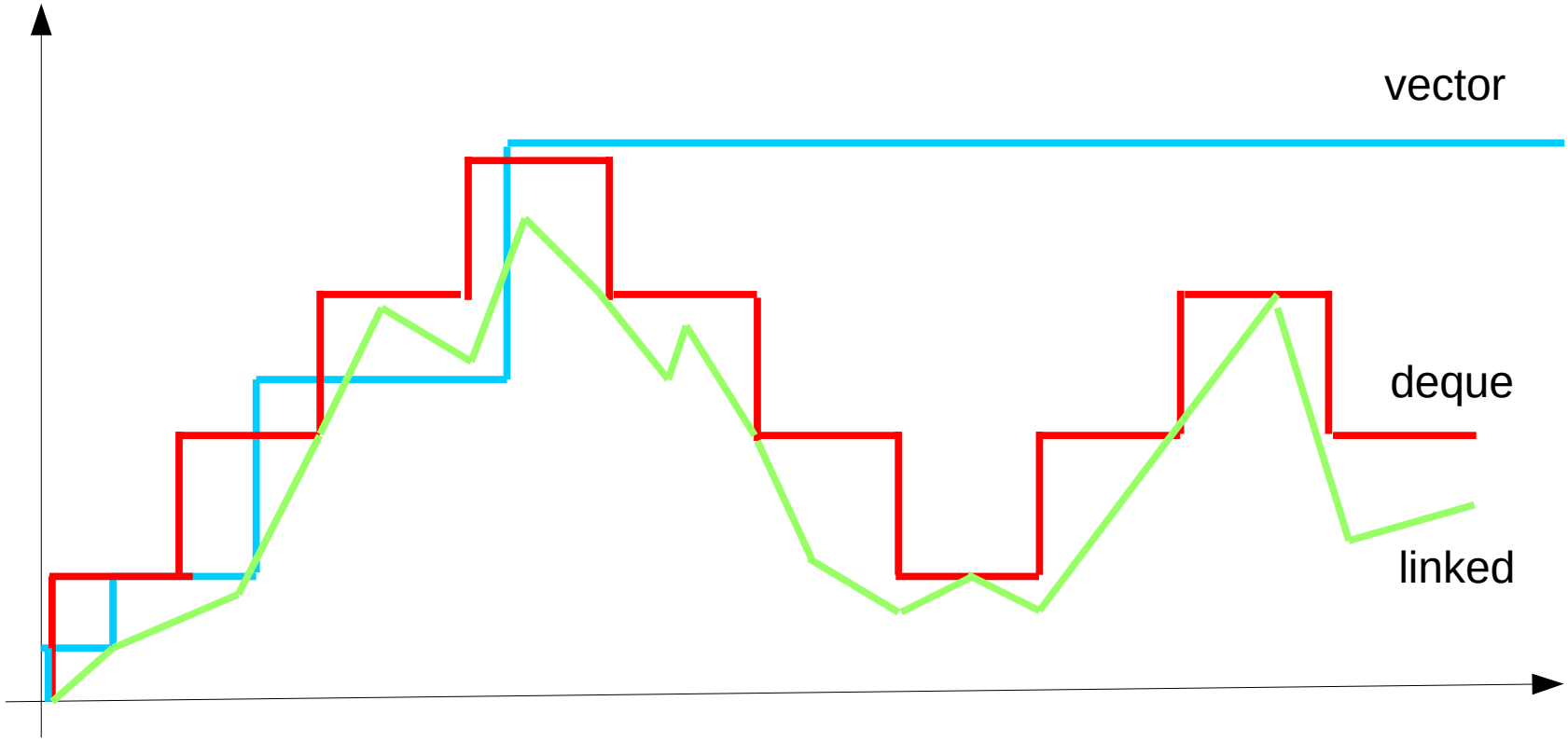








Memory consumption



Vector size and capacity

```
int main()
{
    std::vector<int> v;
    std::cout << "Default-constructed capacity is " << v.capacity() << '\n';

    v.resize(100);
    std::cout << "Capacity of a 100-element vector is " << v.capacity() << '\n';

    v.clear();
    std::cout << "Capacity after clear() is " << v.capacity() << '\n';

    // std::vector<int>(v).swap(v); // C++98

    v.shrink_to_fit(); // C++11
    std::cout << "Capacity after shrink_to_fit() is " << v.capacity() << '\n';
}
```

```
Default-constructed capacity is 0
Capacity of a 100-element vector is 100
Capacity after clear() is 100
Capacity after shrink_to_fit() is 0
```

Iterator invalidation

Category	Container	After **insertion**, are...		After **erasure**, are...		Conditionally
		iterators valid?	**references** valid?	**iterators** valid?	**references** valid?	
Sequence containers	`array`	N/A		N/A		
	`vector`	No		N/A		Insertion changed capacity
		Yes		Yes		Before modified element(s) (for insertion only if capacity didn't change)
		No		No		At or after modified element(s)
	`deque`	No	Yes	Yes, except erased element(s)		Modified first or last element
			No	No		Modified middle only
	`list`	Yes		Yes, except erased element(s)		
	`forward_list`	Yes		Yes, except erased element(s)		
Associative containers	`set` `multiset` `map` `multimap`	Yes		Yes, except erased element(s)		
Unordered associative containers	`unordered_set` `unordered_multiset` `unordered_map` `unordered_multimap`	No	Yes	N/A		Insertion caused rehash
		Yes		Yes, except erased element(s)		No rehash

From: https://en.cppreference.com/w/cpp/container

Std::array

```
#include <string>
#include <iterator>
#include <iostream>
#include <algorithm>
#include <array>

int main()
{
    // construction uses aggregate initialization
    std::array<int, 3> a1{ {1, 2, 3} }; // double-braces required in C++11 (not in C++14)
    std::array<int, 3> a2 = {1, 2, 3};  // never required after =
    std::array<std::string, 2> a3 = { std::string("a"), "b" };

    // container operations are supported
    std::sort(a1.begin(), a1.end());
    std::reverse_copy(a2.begin(), a2.end(), std::ostream_iterator<int>(std::cout, " "));

    std::cout << '\n';

    // ranged for loop is supported
    for(const auto& s: a3)
        std::cout << s << ' ';

    std::array  a1{"foo"};          // C++17 CTAD: std::array<const char*,1>{"foo"}
    auto a2 = std::to_array{"foo"}; // std::array<char,4>{'f','o','o','\n'};
}
```

Std::forward_list

```
template <typename T, typename Alloc = allocator<T> >
class forward_list
{
public:
    void clear();
    iterator insert_after(const_iterator pos, const T& value); // +move, +interval
    iterator emplace_after(const_iterator pos, Args&&... args);
    iterator erase_after(const_iterator pos); // +interval
    void push_front(const T& value);          // +move
    void emplace_front( Args&&... args );
    void pop_front();
    void resise(size_type count); void resise(size_type count, const T& value);
    void swap(forward_list& other);

    iterator before_begin(), begin(), end();
    bool empty();

    void merge(forward_list&& other, Compare comp);
    void splice_after(const_iterator pos, forward_list& other);
    void remove(const T& value);
    void remove_if(UnaryPredicate p);

    void reverse();
    void unique(BinaryPredicate p);
    sort(Compare comp);

     // no reverse iteration
     // no back() or push_back()
     // no size()
};
```

Hash-based containers

- Unordered_map
- Unordered_set
- Unordered_multimap
- Unordered_multiset

std::unordered_map

```
#include <unordered_map>
#include <string>

using namespace std;

int main()
{
    unordered_map<string, string> hashtable;
    hashtable.emplace("www.zolix.hu", "212.92.23.158");
    hashtable.insert(make_pair("www.elte.hu", "212.92.23.159"));

    cout << "IP Address: " << hashtable["www.zolix.com"] << endl;

    for (auto &obj : hashtable)
    {
        cout << obj.first << ": " << obj.second << endl;
    }

    // returns std::unordered_map<std::string,double>::const_iterator
    auto it = hashtable.find("www.elte.com");
    if (hashtable.end() != it)   // hashtable.count("www.elte.com") > 0
    {
        cout << it->first << ": " << it->second << endl;
    }
    return 0;
}
```

std::unordered_map

```
#include <unordered_map>
#include <string>

class MyClass
{
    std::string name;
    int         age;
public:
    Bool operator==(const MyClass& rhs) { ... }   // should be reflexive
    // ...
};
class MyClassHash
{
public:
    size_t operator()(const MyClass& m) const
    {
        return std::hash<std::string>()(m.name) ^ hash<int>()(m.age);
    }
};
int main()
{
    MyClass jim(...), joe(...);
    unordered_map<MyClass, double> salary;
    hashtable.emplace( jim, 20000);
    hashtable.emplace( joe, 22000);
    // ...
}
```

Load factor

- Average insert/find is constant
- Worst-case: linear in container size
- Iterators are invalidated only on rehash
- Load factor == size() / bucket_count() // default 1.0
- Control of buckets:
  - size_type bucket_count() const; // #of buckets
  - float max_load_factor() const; // get max load factor
  - void max_load_factor(float z); // set max load factor
  - size_type bucket_size (size_type n) const; // #of bucket n
  - size_type bucket(const key_t& key) const; // where key goes?
  - void rehash(size_type n); // sets #of buckets

std::unordered_map

```
// unordered_map::bucket_count
#include <iostream>
#include <string>
#include <unordered_map>

int main ()
{
  std::unordered_map<std::string,std::string> mymap = {
       {"house","maison"},{"apple","pomme"},{"tree","arbre"},
       {"book","livre"},{"door","porte"},{"grapefruit","pamplemousse"}
  };
  unsigned n = mymap.bucket_count();
  std::cout << "mymap has " << n << " buckets.\n";

  for (unsigned i=0; i<n; ++i) {
    std::cout << "bucket #" << i << " contains: ";
    for (auto it = mymap.begin(i); it!=mymap.end(i); ++it)
      std::cout << "[" << it->first << ":" << it->second << "] ";
    std::cout << "\n";
  }
}
mymap has 7 buckets.
bucket #0 contains: [book:livre] [house:maison]
bucket #1 contains:
bucket #2 contains:
bucket #3 contains: [grapefruit:pamplemousse] [tree:arbre]
bucket #4 contains:
bucket #5 contains: [apple:pomme]
bucket #6 contains: [door:porte]
```

Load factor

```
// unordered_map::max_load_factor
int main ()
{
  std::unordered_map<std::string,std::string> mymap = {
    {"Au","gold"},{"Ag","Silver"},{"Cu","Copper"},{"Pt","Platinum"}
  };

  std::cout << "current max_load_factor: " << mymap.max_load_factor() << std::endl;
  std::cout << "current size: " << mymap.size() << std::endl;
  std::cout << "current bucket_count: " << mymap.bucket_count() << std::endl;
  std::cout << "current load_factor: " << mymap.load_factor() << std::endl;

  float z = mymap.max_load_factor();
  mymap.max_load_factor ( z / 2.0 );

  std::cout << "new max_load_factor: " << mymap.max_load_factor() << std::endl;
  std::cout << "new size: " << mymap.size() << std::endl;
  std::cout << "new bucket_count: " << mymap.bucket_count() << std::endl;
  std::cout << "new load_factor: " << mymap.load_factor() << std::endl;
  return 0;
}
current max_load_factor: 1
current size: 4
current bucket_count: 5
current load_factor: 0.8
new max_load_factor: 0.5
new size: 4
new bucket_count: 11
new load_factor: 0.363636
```

Std::unordered_multi...

```
// unordered_multiset::equal_range

#include <iostream>
#include <string>
#include <unordered_set>

int main ()
{
  std::unordered_multiset<std::string> myums =
                        {"cow","pig","pig","chicken","pig","chicken"};

  auto myrange = myums.equal_range("pig");

  std::cout << "These pigs were found:";

  while ( myrange.first != myrange.second ) {
      std::cout << " " << *myrange.first++;
  }
  std::cout << std::endl;

  return 0;
}
```

Example: word counter

```
#include <cctype>
#include <iterator>
#include <iostream>
#include <vector>
#include <algorithm>



int main()
{
  std::vector<char> v{};
  std::cin >> std::noskipws;
  std::copy( std::istream_iterator<char>{std::cin}, std::istream_iterator<char>{},
                                                                  std::back_inserter(v));
  int  cnt = 0;
  char prev = '\n';  // the imaginary char on -1 position is a white space.

  for ( char curr : v )
  {
    if ( std::isspace(prev) && !std::isspace(curr) )  // new word starts
      ++cnt;
    prev = curr;
  }
  std::cout << cnt << '\n';
}
```

Example: word counter

```
#include <cctype>
#include <iterator>
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>


int main()
{
  std::vector<char> v{};
  std::cin >> std::noskipws;
  std::copy( std::istream_iterator<char>{std::cin}, std::istream_iterator<char>{},
                                                                  std::back_inserter(v));
  int cnt = 0;
  if ( ! v.empty() )
  {
    cnt = std::transform_reduce(
       std::begin(v), std::end(v)-1, std::begin(v)+1,!std::isspace(v[0])?1:0, std::plus{},
       [](char curr, char next){ return std::isspace(curr) && !std::isspace(next); } );

    std::cout << cnt << '\n';
}
```

Example: word counter

```
#include <cctype>
#include <iterator>
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
#include <execution>

int main()
{
  std::vector<char> v{};
  std::cin >> std::noskipws;
  std::copy( std::istream_iterator<char>{std::cin}, std::istream_iterator<char>{},
                                                                std::back_inserter(v));
  int cnt = 0;
  if ( ! v.empty() )
  {
    cnt = std::transform_reduce( std::execution::par,
       std::begin(v), std::end(v)-1, std::begin(v)+1,!std::isspace(v[0])?1:0, std::plus{},
       [](char curr, char next){ return std::isspace(curr) && !std::isspace(next); } );

    std::cout << cnt << '\n';
}
```

par_algorithms (C++17)

- Based on Intel's Threading Building Blocks (TBB)
- Extends STL algorithms with execution policy
  - std::execution::seq Sequential execution
  - std::execution::par Parallel execution
  - std::execution::par_unseq Parallel SIMD execution
  - std::execution::unseq Sequential SIMD execution
- These policies are permissions not obligations. Implementation may choose what can be parallelized
- Minimal requirement: forward iterator
- The programmer's task to ensure that element access functions will not cause dead lock or data race
- In case of paralellization and vectorization access must not use any blocking synchronization

Parallel STL

```
// Example from Stroustrup

template<class T, class V>
struct Accum     // simple accumulator function object
{
    T* b;
    T* e;
    V val;
    Accum(T* bb, T* ee, const V& vv) : b{bb}, e{ee}, val{vv} {}
    V operator() () { return std::accumulate(b,e,val); }
};

double comp(vector<double>& v)   // spawn many tasks if v is large enough
{
    if (v.size()<10000) return std::accumulate(v.begin(),v.end(),0.0);

    auto f0 {async(Accum{&v[0],&v[v.size()/4],0.0})};
    auto f1 {async(Accum{&v[v.size()/4],&v[v.size()/2],0.0})};
    auto f2 {async(Accum{&v[v.size()/2],&v[v.size()*3/4],0.0})};
    auto f3 {async(Accum{&v[v.size()*3/4],&v[v.size()],0.0})};

    return f0.get()+f1.get()+f2.get()+f3.get();
}
```

Parallel STL

```
// Example from cppreference

template<class T, class V>
struct Accum     // simple accumulator function object
{
    T* b;
    T* e;
    V val;
    Accum(T* bb, T* ee, const V& vv) : b{bb}, e{ee}, val{vv} {}
    V operator() () { return std::accumulate(b,e,val); }
};

double comp(vector<double>& v)
{
    // non-deterministic if binary_op is not associative or not commutative
    double res = std::reduce(std::execution::par, v.begin(), v.end(), 0.0);
    return res;
}
```

Vectorization

```
std::vector<int> v {1,2, ... };

int sum { std::accumulate(v.begin(), v.end(), 0) };


int sum = 0;
for ( size_t i = 0; i < v.size(); ++i)
{
    sum += v[i];
}



int sum = 0;
for ( size_t i = 0; i < v.size() / 4; i+=4)
{
    sum += v[i] + v[i+1] + v[i+2] + v[i+3];   // most CPU supports this
}
// handle if (v.size()/4) is not 0
```

accumulate() vs reduce()

```
#include <iostream>
#include <vector>

int main()
{
  std::vector<long long> v1;
  for ( int i = 0; i < 10; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4}); // creates 50 elements
  }

  long long sum = 0;
  for ( std::size_t i = 0; i < v1.size(); ++i) // summa x^2  x in [0..49]
  {
    sum += v1[i]*v1[i];
  }

  std::cout << sum << '\n';

  return 0;
}
```

```
$ ./a.out
300
```

accumulate() vs reduce()

```
#include <iostream>
#include <numeric>
#include <vector>


std::vector<long long> v1;
auto sqrsum = [] (auto s, auto val) { return s + val * val; };

int main()
{
  for ( int i = 0; i < 10; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4});  // creates 50 elements
  }

  auto sum1 = std::accumulate(v1.begin(), v1.end(), 0LL, sqrsum);  // classical STL



  std::cout << sum1 << '\n';
  return 0;
}


$ ./a.out
300
```

accumulate() vs reduce()

```
#include <iostream>
#include <numeric>
#include <vector>


std::vector<long long> v1;
auto sqrsum = [] (auto s, auto val) { return s + val * val; };

int main()
{
  for ( int i = 0; i < 10; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4});  // creates 50 elements
  }
  // accumulate is guaranteed left associative
  auto sum1 = std::accumulate(v1.begin(), v1.end(), 0LL, sqrsum);  // classical STL



  std::cout << sum1 << '\n';
  return 0;
}


$ ./a.out
300
```

accumulate() vs reduce()

```
#include <iostream>
#include <numeric>
#include <vector>
#include <execution>

std::vector<long long> v1;
auto sqrsum = [] (auto s, auto val) { return s + val * val; };

int main()
{
  for ( int i = 0; i < 10; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4});
  }
  // accumulate is guaranteed left associative
  auto sum1 = std::accumulate(v1.begin(), v1.end(), 0LL, sqrsum);
  // reduce can work parallel
  auto sum2 = std::reduce(std::execution::par, v1.begin(), v1.end(), 0LL, sqrsum);

  std::cout << sum1 << “, “ << sum2 << '\n';
  return 0;
}
```

```
$ ./a.out
300, 300
```

accumulate() vs reduce()

```
#include <iostream>
#include <numeric>
#include <vector>
#include <execution>

std::vector<long long> v1;
auto sqrsum = [] (auto s, auto val) { return s + val * val; };

int main()
{
  for ( int i = 0; i < 1000; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4});
  }
  // accumulate is guaranteed left associative
  auto sum1 = std::accumulate(v1.begin(), v1.end(), 0LL, sqrsum);
  // reduce can work parallel
  auto sum2 = std::reduce(std::execution::par, v1.begin(), v1.end(), 0LL, sqrsum);

  std::cout << sum1 << “, “ << sum2 << '\n';
  return 0;
}
```

```
$ ./a.out
30000, 30000
```

accumulate() vs reduce()

```
#include <iostream>
#include <numeric>
#include <vector>
#include <execution>

std::vector<long long> v1;
auto sqrsum = [] (auto s, auto val) { return s + val * val; };

int main()
{
  for ( int i = 0; i < 1000000; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4});
  }
  // accumulate is guaranteed left associative
  auto sum1 = std::accumulate(v1.begin(), v1.end(), 0LL, sqrsum);
  // reduce can work parallel
  auto sum2 = std::reduce(std::execution::par, v1.begin(), v1.end(), 0LL, sqrsum);

  std::cout << sum1 << “, “ << sum2 << '\n';
  return 0;
}
```

```
$ ./a.out
30000000, 59820950156796
```

accumulate() vs reduce()

```
#include <iostream>
#include <numeric>
#include <vector>
#include <execution>

std::vector<long long> v1;
auto sqrsum = [] (auto s, auto val) { return s + val * val; };  // not commutative

int main()
{
  for ( int i = 0; i < 1000000; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4});
  }
  // accumulate is guaranteed left associative
  auto sum1 = std::accumulate(v1.begin(), v1.end(), 0LL, sqrsum);
  // reduce can work parallel
  auto sum2 = std::reduce(std::execution::par, v1.begin(), v1.end(), 0LL, sqrsum);

  std::cout << sum1 << “, “ << sum2 << '\n';
  return 0;
}
```

```
$ ./a.out
30000000, 59820950156796
```

accumulate() vs reduce()

```
#include <iostream>
#include <numeric>
#include <vector>
#include <execution>
#include <functional>
std::vector<long long> v1;
auto sqrsum = [] (auto s, auto val) { return s + val * val; };

int main()
{
  for ( int i = 0; i < 1000000; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4});
  }
  // accumulate is guaranteed left associative
  auto sum1 = std::accumulate(v1.begin(), v1.end(), 0LL, sqrsum);
  auto sum2 = std::transform_reduce(std::execution::par,  // map-reduce
          v1.begin(), v1.end(), 0LL,
          std::plus<>(),
          [](auto v) { return v*v; });
  std::cout << sum1 << “, “ << sum2 << '\n';
  return 0;
}

$ ./a.out
30000000, 30000000
```

Template metaprograms

- First template metaprogram: Erwin Unruh 1994
  - Printed prime numbers as compiler error messages
- Later proved to be a Turing-complete sub language of C++
- Today many use cases exist
  - Expression templates
  - DSL embedding (boost::xpressive)
  - Generators (boost::spirit)
  - Compile-time adaptation (std::enable_if)
- Many more...

Factorial

- Compile time recursion
- Specialization to stop recursion

```
template <int N>
struct Factorial
{
    enum { value = N * Factorial<N-1>::value };
};
template <>
struct Factorial<1>
{
    enum { value = 1 };
};

// example use
int main()
{
    const int fact5 = Factorial<5>::value;
    std::cout << fact5 << endl;
    return 0;
}
```

Factorial

- Compile time recursion
- Specialization to stop recursion

```
template <int N>
struct Factorial
{
    enum { value = N * Factorial<N-1>::value };
};
template <>
struct Factorial<1>
{
    enum { value = 1 };
};

// example use
int main()
{
    const int fact5 = Factorial<5>::value;
    std::cout << fact5 << endl;
    return 0;
}
```

Meta-control structures

```
template <bool condition, class Then, class Else>
struct IF
{
    typedef Then RET;
};

template <class Then, class Else>
struct IF<false, Then, Else>
{
    typedef Else RET;
};

// example use
template <typename T, typename S>
IF< sizeof(T)<sizeof(S), S, T>::RET max(T t, S s)
{
    if ( t < s)
        return s;
    else
        return t;
}
```

Template metafunction as parameter

```
template <int n, template<int> class F>
struct Accumulate
{
    enum { RET = Accumulate<n-1,F>::RET + F<n>::RET };
};

template <template<int> class F>
struct Accumulate<0,F>
{
    enum { RET = F<0>::RET };
};

template <int n>
struct Square
{
    enum { RET = n*n };
};

cout << Accumulate<3,Square>::RET << endl;
```

Why use it?

```
template <unsigned long N>
struct binary
{
    static unsigned const value = binary<N/10>::value * 2 + N % 10;
};

template <>
struct binary<0>
{
    static unsigned const value = 0;
};

int main()
{
    const unsigned int di = 12;
    const unsigned int oi = 014;
    const unsigned int hi = 0xc;

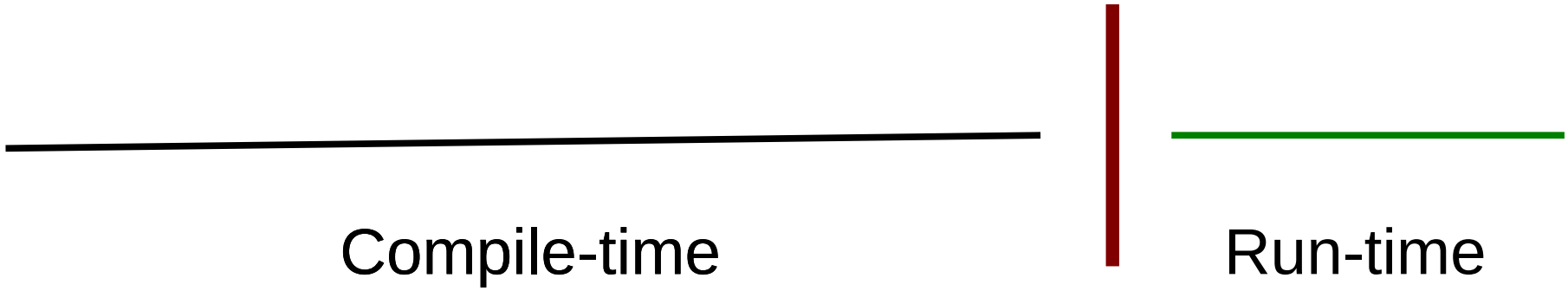
    const unsigned int bi0 = binary_value("1101");  // run-time
    const unsigned int bi1 = binary<1100>::value;   // compile-time
}
```

Motivation

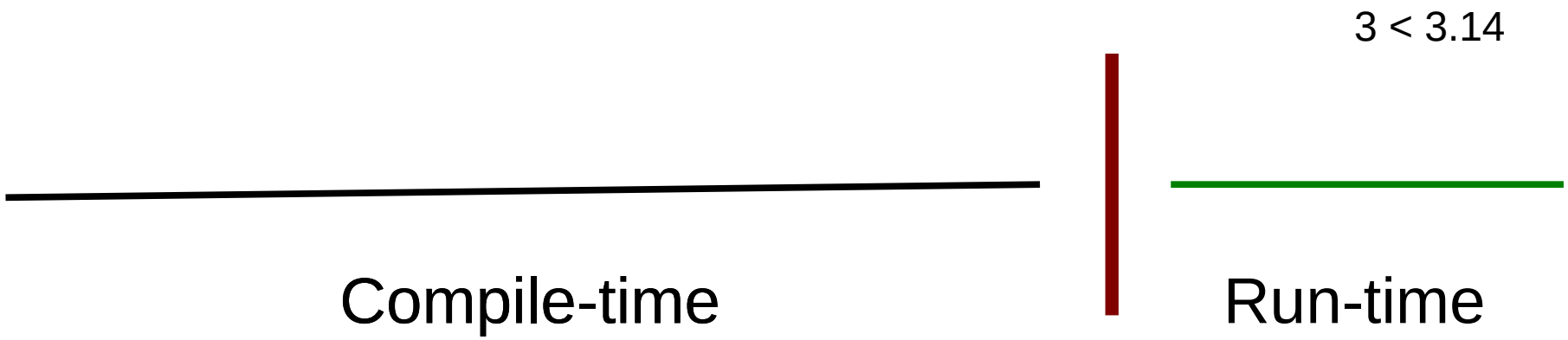
```
template <class T, class S>
? max( T a, S b)  // How to define the return value?
{
    if ( a > b )
        return a;
    else
        return b;
}

int main()
{
    short is = 3; long il = 2; double d = 3.14;
    cout << max( il, is);
    cout << max( is, d);
}
```

Compile-time vs. Run-time



Compile-time vs. Run-time



Compile-time vs. Run-time

-1.0 < 0

3 > 2L

3 < 3.14

Compile-time

Run-time

Compile-time vs. Run-time

int

long

std::string

double

-1.0 < 0

3 > 2L

3 < 3.14

Compile-time

Run-time

Motivation

```
template <class T, class S>
? max( T a, S b)  // How to define the return value?
{
    if ( a > b )
        return a;
    else
        return b;
}

int main()
{
    short is = 3; long il = 2; double d = 3.14;
    cout << max( il, is); // long is ''better'' than short
    cout << max( is, d);  // double is ''better'' than short
}
```

Compile-time vs. Run-time

int

long

std::string

double

-1.0 < 0

3 > 2L

3 < 3.14

Template design time

Compile-time
Template Instantiation

Run-time

Compile-time vs. Run-time

T

S

int

long

std::string

double

-1.0 < 0

3 > 2L

3 < 3.14

Template design time

Compile-time
Template Instantiation

Run-time

Compile-time vs. Run-time

T

S

sizeof(T) < sizeof(S)

int

long

std::string

double

-1.0 < 0

3 > 2L

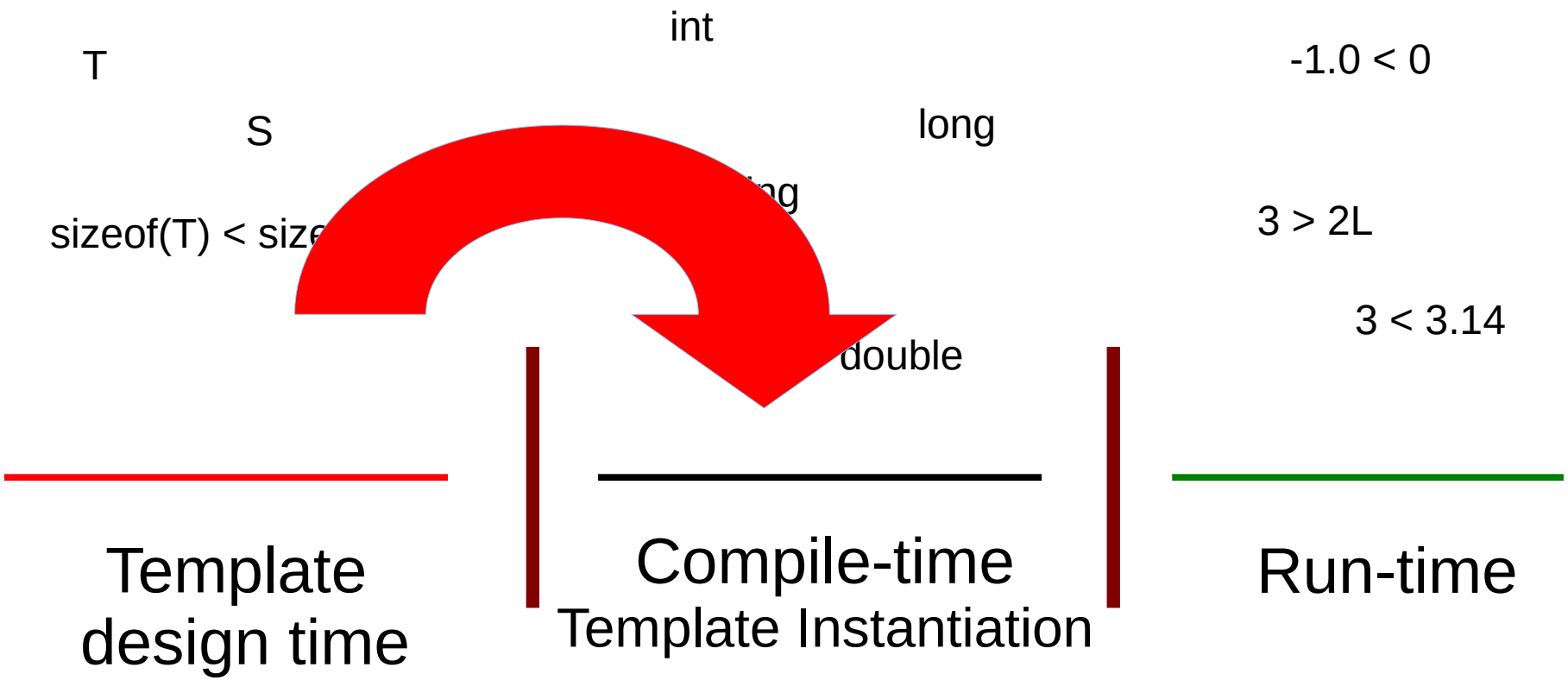
3 < 3.14

Template design time

Compile-time
Template Instantiation

Run-time

Compile-time vs. Run-time



Motivation

```
template <class T, class S>
? max( T a, S b)  // How to define the return value?
{
    if ( a > b )
        return a;
    else
        return b;
}

int main()
{
    short is = 3; long il = 2; double d = 3.14;
    cout << max( il, is); // long is ''better'' than short
    cout << max( is, d);  // double is ''better'' than short
}
```

Motivation

```
template <class T, class S>
IF< sizeof(T)<sizeof(S), S, T>::RET max( T a, S b)
{
    if ( a > b )
        return a;
    else
        return b;
}

int main()
{
    short is = 3; long il = 2; double d = 3.14;
    cout << max( il, is); // long is ''better'' than short
    cout << max( is, d);  // double is ''better'' than short
}
```

(de)Motivation

```
template <class T, class S>
auto max( T a, S b) -> decltype(a+b) // C++11
{
    if ( a > b )
        return a;
    else
        return b;
}

int main()
{
    short is = 3; long il = 2; double d = 3.14;
    cout << max( il, is);  // -> long
    cout << max( is, d);   // -> double
}
```

(de)Motivation

```
template <class T, class S>
auto max( T a, S b) -> decltype(1==1?a:b) // C++11
{
    if ( a > b )
        return a;
    else
        return b;
}

int main()
{
    short is = 3; long il = 2; double d = 3.14;
    cout << max( il, is);  // -> long
    cout << max( is, d);   // -> double
}
```

(de)Motivation

```
template <class T, class S>
typename std::common_type<T,S>::value max( T a, S b) // C++11
{
    if ( a > b )
        return a;
    else
        return b;
}

int main()
{
    short is = 3; long il = 2; double d = 3.14;
    cout << max( il, is);  // -> long
    cout << max( is, d);   // -> double
}
```

(de)Motivation

```
template <class T, class S>
auto max( T a, S b)                    // C++14
{
    if ( a > b )
        return a;
    else
        return b;
}

int main()
{
    short is = 3; long il = 2; double d = 3.14;
    cout << max( il, is);  // -> long
    cout << max( is, d);   // -> double
}
```

Use case example

```
template <class T>
class matrix
{
public:
    matrix( int i, int j );
    matrix( const matrix &other);
    ~matrix();
    matrix operator=( const matrix &other);
private:
    int  x;
    int  y;
    T   *v;
    void copy( const matrix &other);
    void check( int i, int j) const throw(indexError);
};
```

Specialization for POD types

```
template <class T>
void matrix<T>::copy( const matrix &other)
{
    x = other.x;
    y = other.y;
    v = new T[x*y];
    for ( int i = 0; i < x*y; ++i )
       v[i] = other.v[i];
}

// specialization for POD types
template <>
void matrix<long>::copy( const matrix &other)
{
    x = other.x;
    y = other.y;
    v = new long[x*y];
    memcpy( v, other.v, sizeof(long)*x*y);
}

template <>
void matrix<int>::copy( const matrix &other) ...
```

Trait

```
template <typename T> struct copy_trait
{
    static void copy( T* to, const T* from, int n)  {
        for( int i = 0; i < n; ++i ) to[i] = from[i];
    }
};
template <> struct copy_trait<long>
{
    static void copy( long* to, const long* from, int n) {
        memcpy( to, from, n*sizeof(long));
    }
};
template <class T, class Cpy = copy_trait<T> >
class matrix {  ... }

template <class T, class Cpy>
void matrix<T,Cpy>::copy( const matrix &other) {
    x = other.x;
    y = other.y;
    v = new T[x*y];
    Cpy::copy( v, other.v, x*y);
}
```

Policy

```
template <typename T, bool B> struct copy_trait
{
    static void copy( T* to, const T* from, int n) {
        for( int i = 0; i < n; ++i )
            to[i] = from[i];
    }
};
template <typename T> struct copy_trait<T, true>
{
    static void copy( T* to, const T* from, int n)    {
        memcpy( to, from, n*sizeof(T));
    }
};

template <typename T> struct is_pod {  enum { value = false }; };

template <> struct is_pod<long>     {  enum { value = true }; };

template <class T, class Cpy = copy_trait<T,is_pod<T>::value> >
class matrix  { ... }
```

Typelist

```
class NullType {};

// We now can construct a null-terminated list of typenames:
typedef Typelist< char,
           Typelist<signed char,
             Typelist<unsigned char, NullType>
           >
        > Charlist;

// For the easy maintenance, precompiler macros are defined
// to create Typelists:

#define TYPELIST_1(x)           Typelist< x, NullType>
#define TYPELIST_2(x, y)        Typelist< x, TYPELIST_1(y)>
#define TYPELIST_3(x, y, z)     Typelist< x, TYPELIST_2(y,z)>
#define TYPELIST_4(x, y, z, w)  Typelist< x, TYPELIST_3(y,z,w)>

// usage example
typedef TYPELIST_3(char, signed char, unsigned char)  Charlist;
```

Typelist operations

```
//  Length
template <class TList> struct Length;

template <>
struct Length<NullType>
{
    enum { value = 0 };
};

template <class T, class U>
struct Length <Typelist<T,U> >
{
    enum { value = 1 + Length<U>::value };
};

static const int len = Length<Charlist>::value;
```

Typelist operations

```
// IndexOf
template <class TList, class T> struct IndexOf;

template <class T>
struct IndexOf< NullType, T>
{
    enum { value = -1 };
};
template <class T>
struct IndexOf< Typelist<T, Tail>, T>
{
    enum { value = 0 };
};
template <class T, class Tail>
struct IndexOf< Typelist<Head, Tail>, T>
{
private:
    enum { temp = IndexOf<Tail, T>::value };
public:
    enum { value = (temp == -1 ? -1 : 1+temp) };
};

static const int IndexOf<Charlist, int>::value;            // -1
static const int IndexOf<Charlist, char>::value;           //  0
static const int IndexOf<Charlist, unsigned char>::value;  //  2
```

Matrix revisited

```
typedef TYPELIST_4(char, signed char, unsigned char, int)  Pod_types;

template <typename T> struct is_pod
{
    enum { value = IndexOf<Pod_types,T>::value != -1 };
};
template <typename T, bool B> struct copy_trait
{
    static void copy( T* to, const T* from, int n)   {
        for( int i = 0; i < n; ++i )
            to[i] = from[i];
    }
};
template <typename T> struct copy_trait<T, true>
{
    static void copy( T* to, const T* from, int n)   {
        memcpy( to, from, n*sizeof(T));
    }
};
template <class T, class Cpy = copy_trait<T,is_pod<T>::value> >
class matrix { ... }
```

SFINAE

```
template <class T, class U>  // is convertible from T to U
struct Conversion
{
    typedef char Small;
    class Big { char dummy[2]; };
    static Small Test(U);
    static Big   Test(...);
    static T     MakeT();
    static const bool exists = sizeof(Test(MakeT()))==sizeof(Small);
    static const bool sameType = false;
};
template <class T>
struct Conversion<T,T>
{
  static const bool exists = 1;
  static const bool sameType = 1;
};
int main()
{
  std::cout << std::boolalpha
     << Conversion<double, int>::exists << '\n'
     << Conversion<char, char*>::exists << '\n'
     << Conversion<std::vector<int>,std::list<int>>::exists << '\n';
}
```

SFINAE

```
template <class T, class U>  // is convertible from T to U
struct Conversion
{
    using Small = char;
    Using Big   = struct { char dummy[2]; };
    static Small Test(U);
    static Big   Test(...);
    static const bool exists =  /* declval is in <utility> header */
                    sizeof(Test(std::declval<T>()))==sizeof(Small);
    static const bool sameType = false;
};
template <class T>
struct Conversion<T,T>
{
  static const bool exists = 1;
  static const bool sameType = 1;
};
int main()
{
  std::cout << std::boolalpha
     << Conversion<double, int>::exists << '\n'
     << Conversion<char, char*>::exists << '\n'
     << Conversion<std::vector<int>,std::list<int>>::exists << '\n';
}
```

type_traits

```
template <class T, class U>  // is convertible from T to U
struct Conversion
{
    using Small = char;
    Using Big   = struct { char dummy[2]; };
    static Small Test(U);
    static Big   Test(...);
    static const bool exists =  /* declval is in <utility> header */
                    sizeof(Test(std::declval<T>()))==sizeof(Small);
    static const bool sameType = false;
};
#include <type_traits>
// is_convertible
// is_assignable
// is_pod
// ...
int main()
{
  std::cout << std::boolalpha
     << std::is_convertible<double, int>::value << '\n'
     << std::is_convertible<char, char*>::value << '\n'
     << std::is_convertible<std::vector<int>,std::list<int>>::value
     << '\n';
}
```

- **Run-time**
- **Compile-time**

- **Run-time**
- **Compile-time**

- Functions
- Values, literals
- Data structures
- If/else
- Loop
- Assignment
- May depend on input

- **Run-time**

- Functions
- Values, literals
- Data structures
- If/else
- Loop
- Assignment
- May depend on input

- **Compile-time**

- Metafunctions (type)
- Const, enum,constexpr
- Typelist (type)
- Pattern matching
- Recursion
- Ref. Transparency
- Deterministic

- **Run-time**
- **Compile-time**

- Imperative
- Object-oriented
- Generative
- (some) Functional

- **Run-time**

- Imperative
- Object-oriented
- Generative
- (some) Functional

- **Compile-time**

- Pure Functional

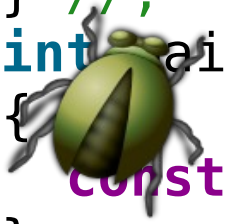
The usual factorial program ...

```
template <int N>
struct Factorial
{
  enum { value = Factorial<N-1>::value * N };
};
template <>
struct Factorial<0>
{
  enum { value = 1 };
};
int main()
{
  const int fact5 = Factorial<5>::value;
}
```

Bugs!!! ...

The java programmer ...

```
template <int N>
struct Factorial
{
  enum { value = Factorial<N-1>::value * N };
};
template <>
struct Factorial<0>
{
  enum { value = 1 };
} //;
int main()
{
  const int fact5 = Factorial<5>::value;
}
```



The java programmer ...

```
template <int N>
struct Factorial
{
  enum { value = Fact
};
template <>
struct Factorial<0>
{
  enum { value = 1 };
} //;
int main()
{
  const int fact5 = Factorial<5>::value;
}
```

```
$ clang++ fact.cpp
fact.cpp:14:2: error: expected ';' after class
}
 ^
 ;
1 error generated.
```

The vim user ...

```
template <int N>
struct Factorial
{
  enum { value = Factorial<N-1>::value * N };
};
template <>
struct Factorial<0>
{
  enum { ivalue = 1 };
};
int main()
{
  const int fact5 = Factorial<5>::value;
}
```

The vim user ...

```
template <int N>
struct Factorial
{
  enum { value = Fact
};
template <>
struct Factorial<0>
{
  enum { ivalue = 1 }
};
int main()
{
  const int fact5 = F
}
```

```
$ clang++ fact.cpp
fact.cpp:5:34: error: no member named 'value' in 'Factorial<0>'
  enum { value = Factorial<N-1>::value * N };
                 ~~~~~~~~~~~~~~~~~~^
fact.cpp:5:18: note: in instantiation of template class 'Factorial<1>'
      requested here
  enum { value = Factorial<N-1>::value * N };
                 ^
fact.cpp:5:18: note: in instantiation of template class 'Factorial<2>'
      requested here
  enum { value = Factorial<N-1>::value * N };
                 ^
fact.cpp:5:18: note: in instantiation of template class 'Factorial<3>'
      requested here
  enum { value = Factorial<N-1>::value * N };
                 ^
fact.cpp:5:18: note: in instantiation of template class 'Factorial<4>'
      requested here
  enum { value = Factorial<N-1>::value * N };
                 ^
fact.cpp:16:21: note: in instantiation of template class 'Factorial<5>'
      requested here
  const int fact5 = Factorial<5>::value;
                    ^
1 error generated.
```

The negative approach ...

```
template <int N>
struct Factorial
{
  enum { value = Factorial<N-1>::value * N };
};
template <>
struct Factorial<0>
{
  enum { value = 1 };
};
int main()
{
  const int fact5 = Factorial<-5>::value;
}
```

The negative approach ...

```
template <int N>
struct Factorial
{
  enum { value = Fact
};
template <>
struct Factorial<0>
{
  enum { value = 1 };
};
int main()
{
  const int fact5 = F
}
```

```
$ clang++ fact4.cpp
fact4.cpp:6:18: fatal error: recursive template instantiation exceeded
maximum
    depth of 512
  enum { value = Factorial<N-1>::value * N };
                 ^
fact4.cpp:6:18: note: in instantiation of template class 'Factorial<-517>'
    requested here
  enum { value = Factorial<N-1>::value * N };

Fact4.cpp:6:18: note: (skipping 503 contexts in backtrace; use
    -ftemplate-backtrace-limit=0 to see all)

fact4.cpp:18:21: note: in instantiation of template class 'Factorial<-5>'
    requested here
  const int fact5 = Factorial<-5>::value;
                    ^
fact4.cpp:6:18: note: use -ftemplate-depth=N to increase recursive
template
    instantiation depth
  enum { value = Factorial<N-1>::value * N };
                 ^
1 error generated.
```

The greedy ...

```
template <int N>
struct Factorial
{
  enum { value = Fact
};
template <>
struct Factorial<0>
{
  enum { value = 1 };
};
int main()
{
  const int fact5 = F
}
```

```
$ clang++ -ftemplate-depth=10000 fact4.cpp
```

The greedy ...

```
template <int N>
struct Factorial
{
  enum { value = Fact
};
template <>
struct Factorial<0>
{
  enum { value = 1 };
};
int main()
{
  const int fact5 = F
}
```

```
$ clang++ -ftemplate-depth=10000 fact4.cpp
clang: error: unable to execute command: Segmentation fault
clang: error: clang frontend command failed due to signal (use -v to
see invocation)
clang version 3.2 (branches/release_32 180710)
Target: x86_64-unknown-linux-gnu
Thread model: posix
clang: note: diagnostic msg: PLEASE submit a bug report to
http://llvm.org/bugs/ and include the crash backtrace, preprocessed
source, and associated run script.
clang: note: diagnostic msg:
********************

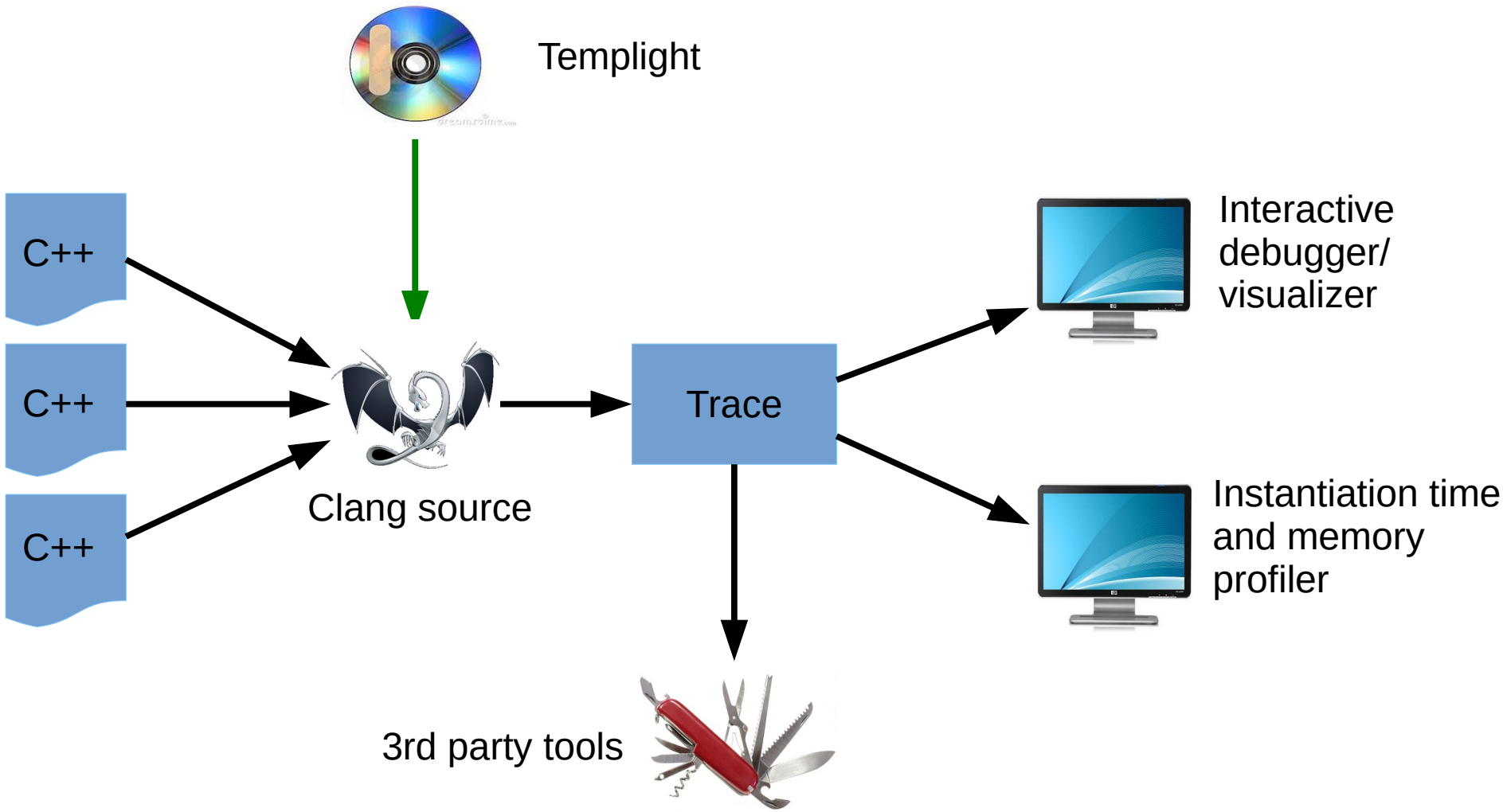
PLEASE ATTACH THE FOLLOWING FILES TO THE BUG REPORT:
Preprocessed source(s) and associated run script(s) are located at:
clang: note: diagnostic msg: /tmp/fact4-iy6zKp.cpp
clang: note: diagnostic msg: /tmp/fact4-iy6zKp.sh
clang: note: diagnostic msg:

********************
```

Templight

- Based on LLVM/Clang compiler infrastructure
- Instrument source code
  - Inject templates for begin-end of every instantiation
  - Generate warnings by specific protocol
  - Collect and present warnings
- Modify the compiler
  - First as a patch for Clang++
  - Since Clang++ 7.0 part of the main branch
  - Detect memoization

Templight


Templight
C++
C++
C++
Clang source
Trace
Interactive debugger/ visualizer
Instantiation time and memory profiler
3rd party tools

How to use

```
$ clang++-7 --cc1 -templight-dump fib.cpp

---
name:            'fact<5>'
kind:            TemplateInstantiation
event:           Begin
orig:            'fact.cpp:2:8'
poi:             'fact.cpp:15:10'
---
name:            'fact<5>'
kind:            TemplateInstantiation
event:           End
orig:            'fact.cpp:2:8'
poi:             'fact.cpp:15:10'
---
name:            'fact<5>'
kind:            TemplateInstantiation
event:           Begin
orig:            'fact.cpp:2:8'
poi:             'fact.cpp:15:10'
---
name:            'fact<4>'
kind:            TemplateInstantiation
event:           Begin
orig:            'fact.cpp:2:8'
poi:             'fact.cpp:4:32'
```

Templar

File Help

Breakpoint Filter Reset

```
1
2 template <int N>
3 struct Fib
4 {
5   static const int value = Fib<N-2>::value +
  Fib<N-1>::value;
6 };
7
8 template<>
9 struct Fib<0>
10 {
11   static const int value = 0;
12 };
13
14 template<>
15 struct Fib<1>
16 {
17   static const int value = 1;
18 };
19
```

Event type:

Kind:

Name:

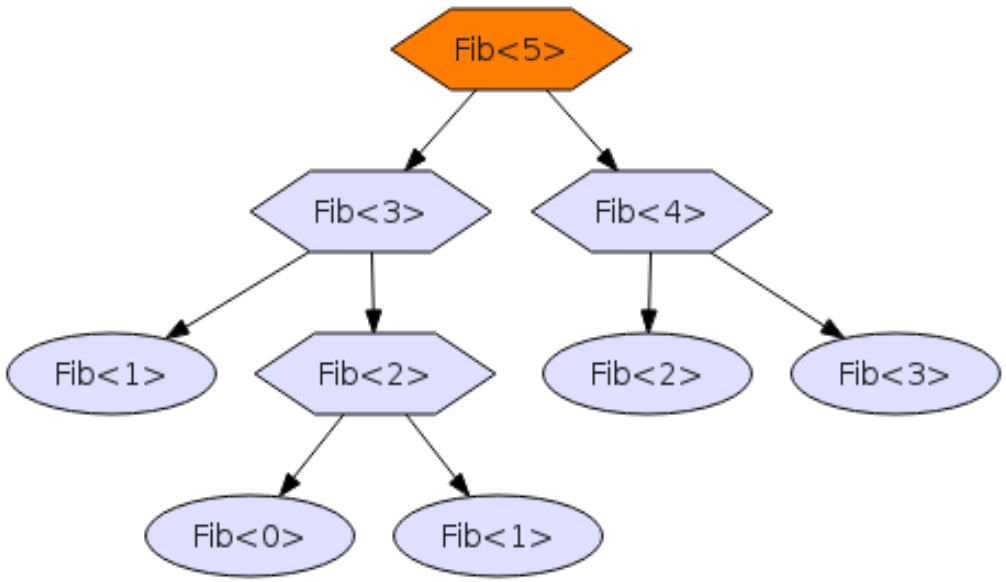
File position:

Templar

File Help

Breakpoint Filter Reset

```
10 {
11   static const int value = 0;
12 };
13
14 template<>
15 struct Fib<1>
16 {
17   static const int value = 1;
18 };
19
20 int main()
21 {
22   int fib5 = Fib<5>::value;
23 }
24
25
```



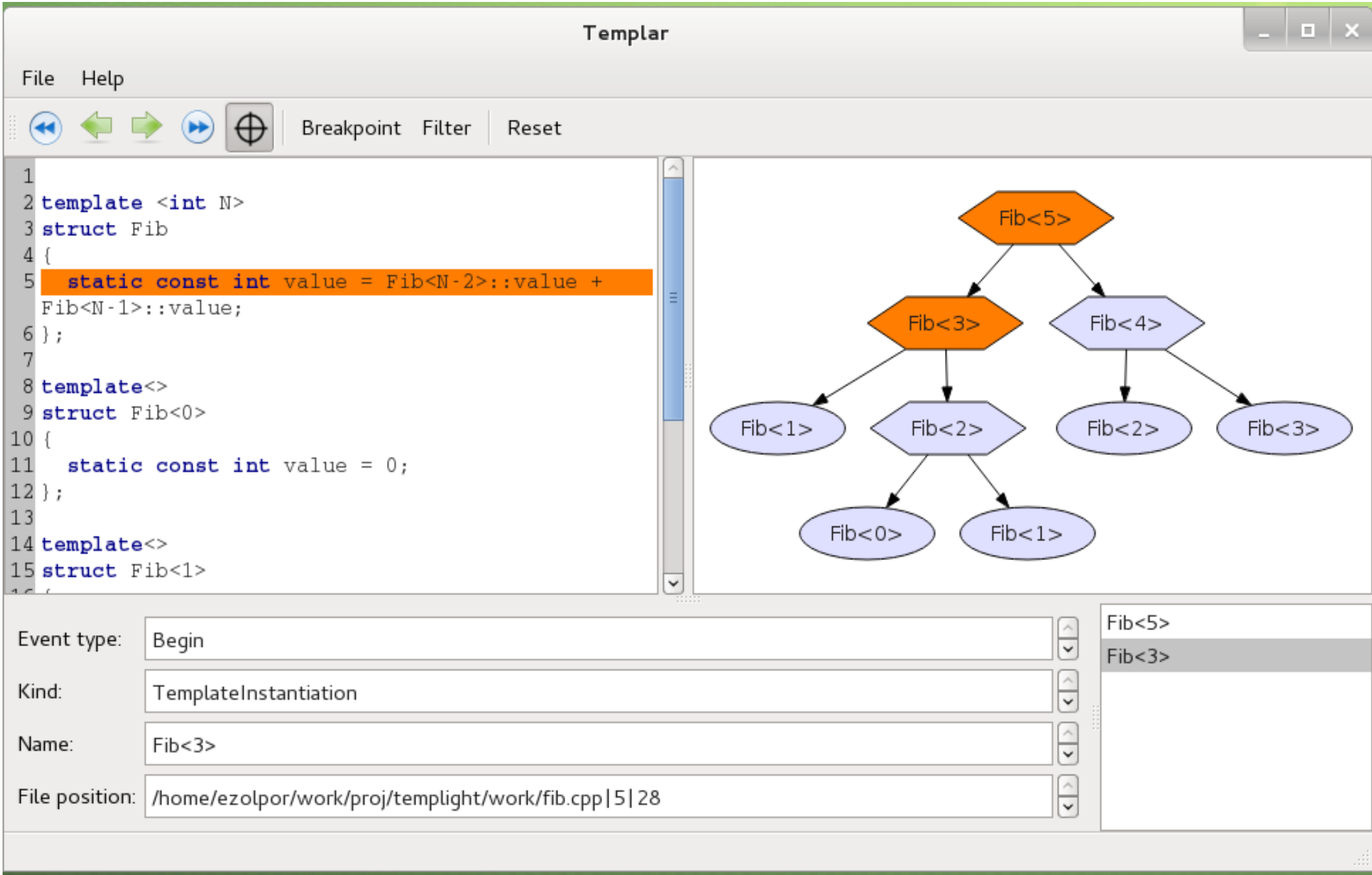
Event type: Begin

Kind: TemplateInstantiation

Name: Fib<5>

File position: /home/ezolpor/work/proj/templight/work/fib.cpp|22|14

Fib<5>

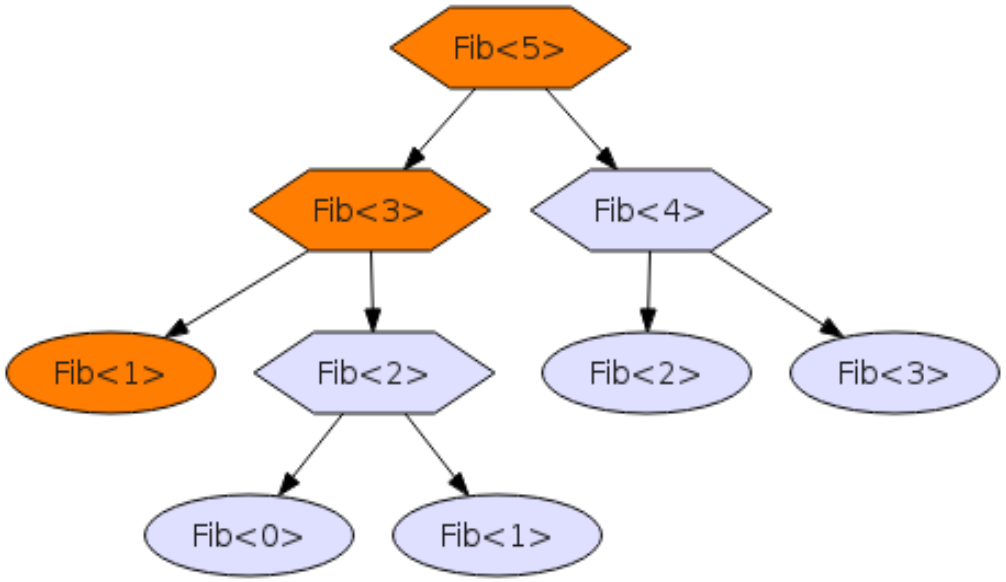

Templar
File
Help
Breakpoint
Filter
Reset
template <int N>
struct Fib
{
static const int value = Fib<N-2>::value +
Fib<N-1>::value;
};
template<>
struct Fib<0>
{
static const int value = 0;
};
template<>
struct Fib<1>
Fib<5>
Fib<3>
Fib<4>
Fib<1>
Fib<2>
Fib<2>
Fib<3>
Fib<0>
Fib<1>
Event type:
Begin
Kind:
TemplateInstantiation
Name:
Fib<3>
File position:
/home/ezolpor/work/proj/templight/work/fib.cpp|5|28
Fib<5>
Fib<3>

Templar

File Help

Breakpoint Filter Reset

```
1
2 template <int N>
3 struct Fib
4 {
5   static const int value = Fib<N-2>::value +
  Fib<N-1>::value;
6 };
7
8 template<>
9 struct Fib<0>
10 {
11   static const int value = 0;
12 };
13
14 template<>
15 struct Fib<1>
```



Event type: Begin

Kind: Memoization

Name: Fib<1>

File position: /home/ezolpor/work/proj/templight/work/fib.cpp|5|28

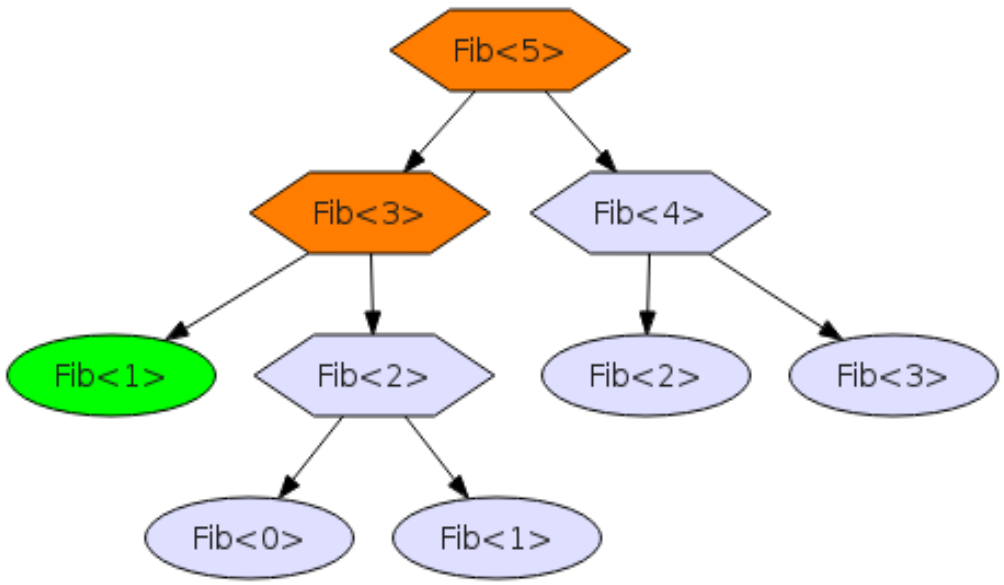
Fib<5>
Fib<3>
Fib<1>

Templar

File Help

Breakpoint Filter Reset

```
1
2 template <int N>
3 struct Fib
4 {
5   static const int value = Fib<N-2>::value +
  Fib<N-1>::value;
6 };
7
8 template<>
9 struct Fib<0>
10 {
11   static const int value = 0;
12 };
13
14 template<>
15 struct Fib<1>
```



Event type: End

Kind: Memoization

Name: Fib<1>

File position: /home/ezolpor/work/proj/templight/work/fib.cpp|5|28

Fib<5>

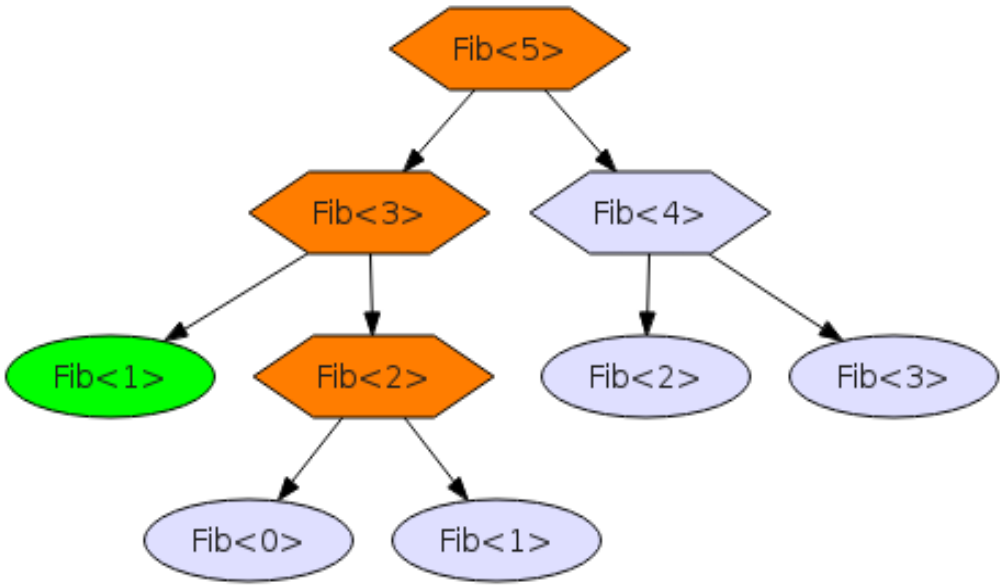
Fib<3>

Templar

File Help

Breakpoint Filter Reset

```
1
2 template <int N>
3 struct Fib
4 {
5   static const int value = Fib<N-2>::value +
  Fib<N-1>::value;
6 };
7
8 template<>
9 struct Fib<0>
10 {
11   static const int value = 0;
12 };
13
14 template<>
15 struct Fib<1>
```



Event type: Begin

Kind: TemplateInstantiation

Name: Fib<2>

File position: /home/ezolpor/work/proj/templight/work/fib.cpp|5|46

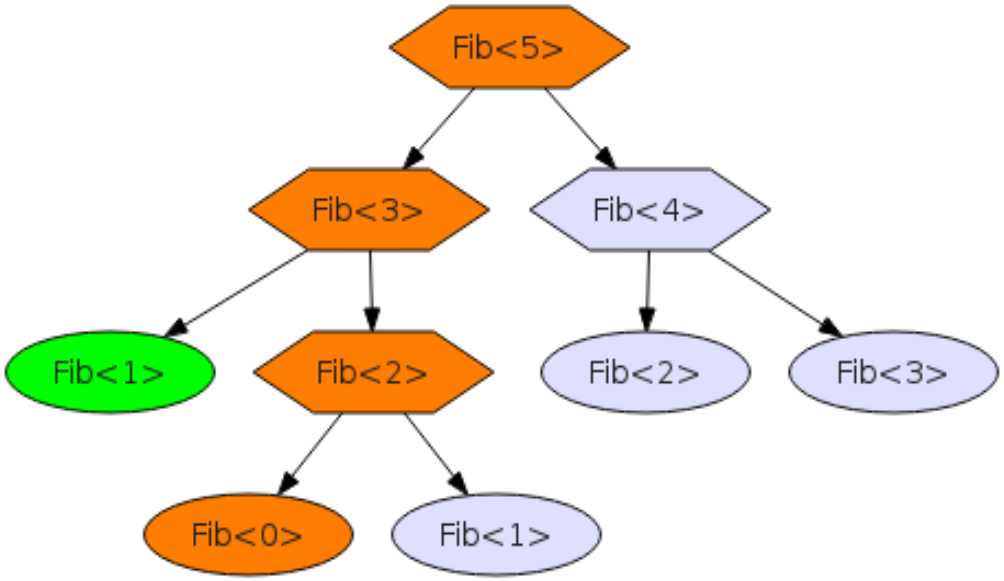
Fib<5>
Fib<3>
Fib<2>

Templar

File Help

Breakpoint Filter Reset

```
1
2 template <int N>
3 struct Fib
4 {
5   static const int value = Fib<N-2>::value +
  Fib<N-1>::value;
6 };
7
8 template<>
9 struct Fib<0>
10 {
11   static const int value = 0;
12 };
13
14 template<>
15 struct Fib<1>
```



Event type: Begin

Kind: Memoization

Name: Fib<0>

File position: /home/ezolpor/work/proj/templight/work/fib.cpp|5|28

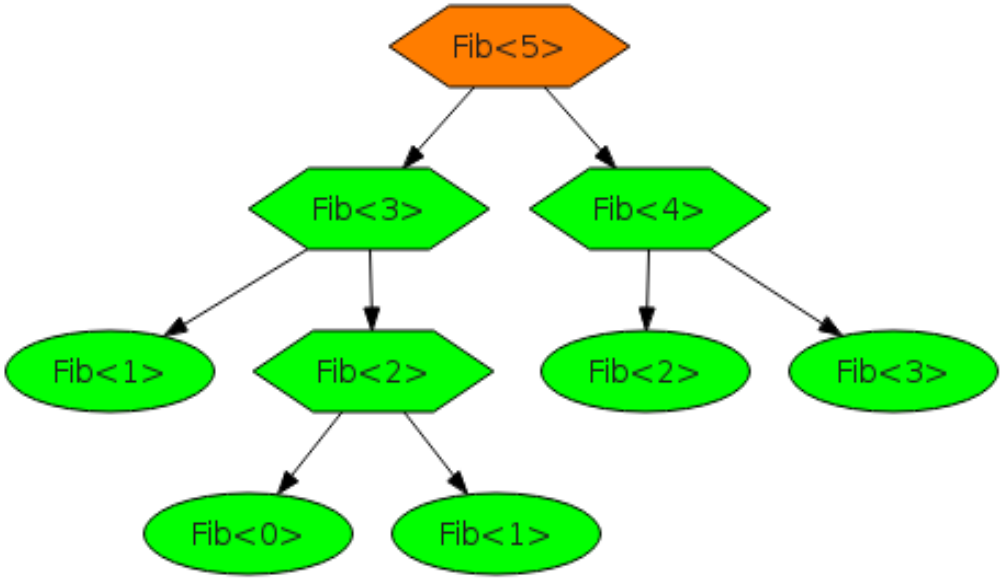
Fib<5>
Fib<3>
Fib<2>
Fib<0>

Templar

File Help

Breakpoint Filter Reset

```
1
2 template <int N>
3 struct Fib
4 {
5   static const int value = Fib<N-2>::value +
  Fib<N-1>::value;
6 };
7
8 template<>
9 struct Fib<0>
10 {
11   static const int value = 0;
12 };
13
14 template<>
15 struct Fib<1>
```



Event type: End

Kind: TemplateInstantiation

Name: Fib<4>

File position: /home/ezolpor/work/proj/templight/work/fib.cpp|5|46

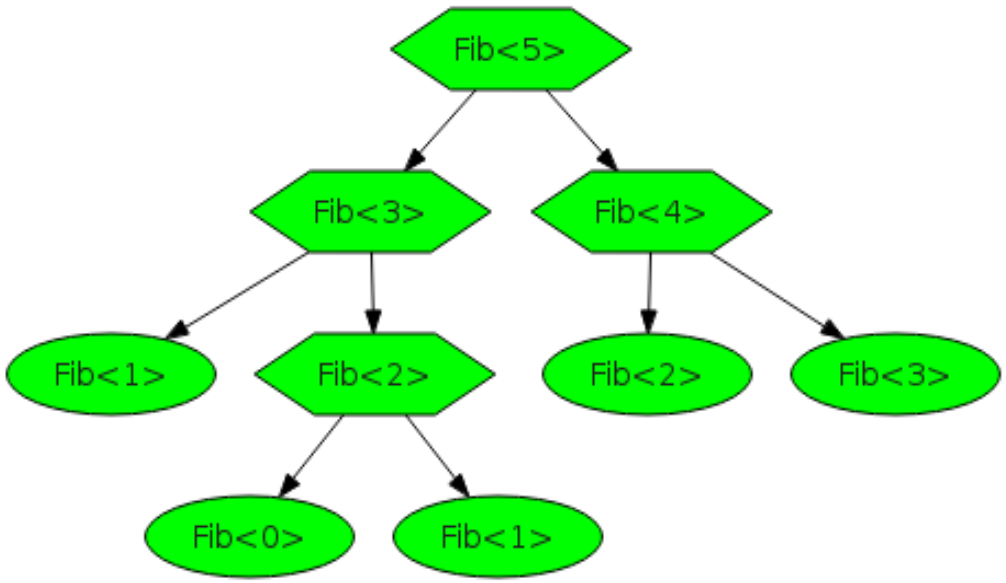
Fib<5>

Templar

File Help

Breakpoint Filter Reset

```
10 {
11   static const int value = 0;
12 };
13
14 template<>
15 struct Fib<1>
16 {
17   static const int value = 1;
18 };
19
20 int main()
21 {
22   int fib5 = Fib<5>::value;
23 }
24
25
```



Event type: End

Kind: TemplateInstantiation

Name: Fib<5>

File position: /home/ezolpor/work/proj/templight/work/fib.cpp|22|14

Templar

File Help

Breakpoint Filter Reset

```
1
2 template <int N>
3 struct Fib
4 {
5   static const int value = Fib<N-2>::value +
  Fib<N-1>::value;
6 };
7
8 template<>
9 struct Fib<0>
10 {
11   static const int value = 0;
12 };
13
14 template<>
15 struct Fib<1>
16 {
17   static const int value = 1;
18 };
19
```

Event type:

Kind:

Name:

File position:

Templar

File Help

Breakpoint Filter Reset

```
1
2 template <int N>
3 struct Fib
4 {
5   static const int value = Fib<N
  Fib<N-1>::value;
6 };
7
8 template<>
9 struct Fib<0>
10 {
11   static const int value = 0;
12 };
13
14 template<>
15 struct Fib<1>
16 {
17   static const int value = 1;
18 };
19
```

Dialog

Add

Delete

Delete All

Add Item

Enter Regexp:

Fib<1>

Cancel OK

Done

Event type:

Kind:

Name:

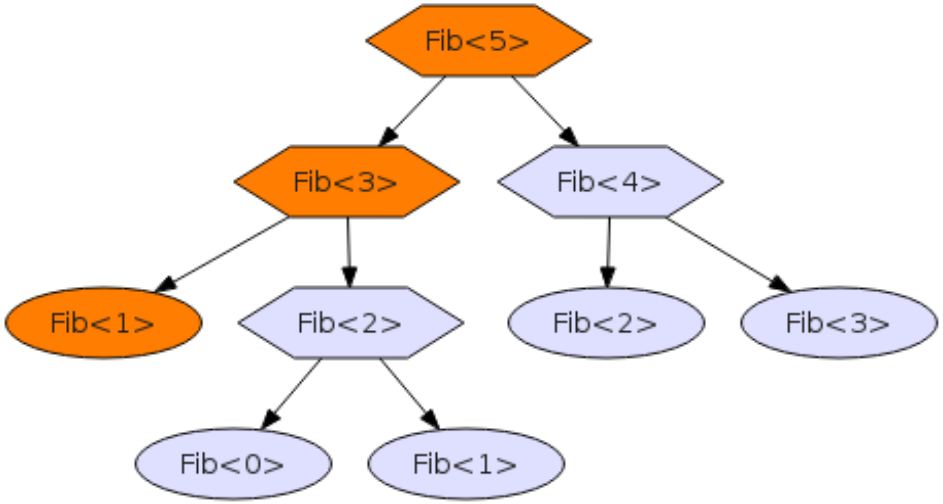
File position:

Templar

File Help

Breakpoint Filter Reset

```
1
2 template <int N>
3 struct Fib
4 {
5   static const int value = Fib<N-2>::value +
  Fib<N-1>::value;
6 };
7
8 template<>
9 struct Fib<0>
10 {
11   static const int value = 0;
12 };
13
14 template<>
15 struct Fib<1>
16 {
17   static const int value = 1;
18 };
19
```



Event type: Begin

Kind: Memoization

Name: Fib<1>

File position: /home/ezolpor/work/proj/templight/work/fib.cpp|5|28

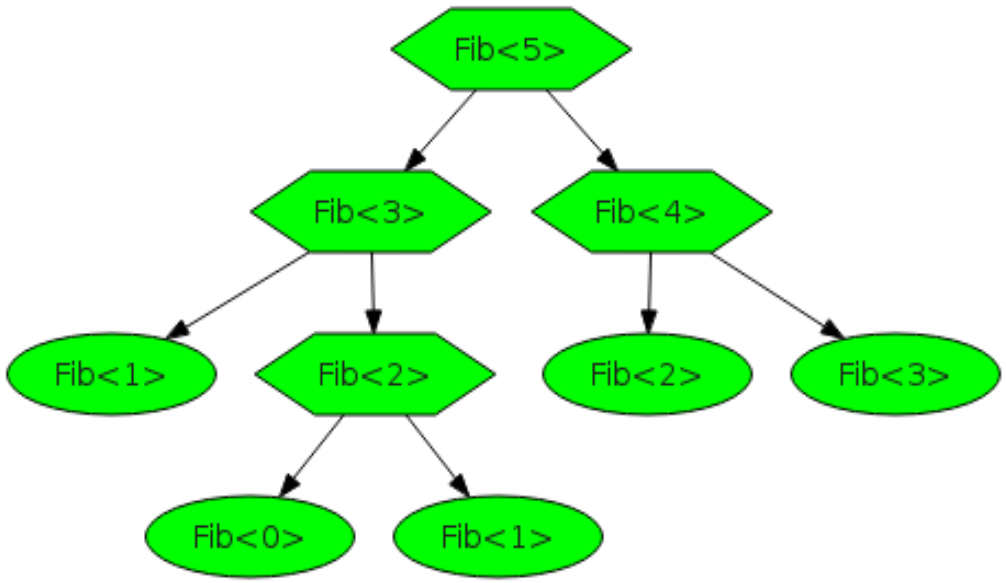
Fib<5>
Fib<3>
Fib<1>

Templar

File Help

Breakpoint Filter Reset

```
10 {
11   static const int value = 0;
12 };
13
14 template<>
15 struct Fib<1>
16 {
17   static const int value = 1;
18 };
19
20 int main()
21 {
22   int fib5 = Fib<5>::value;
23 }
24
25
```



Event type: End

Kind: TemplateInstantiation

Name: Fib<5>

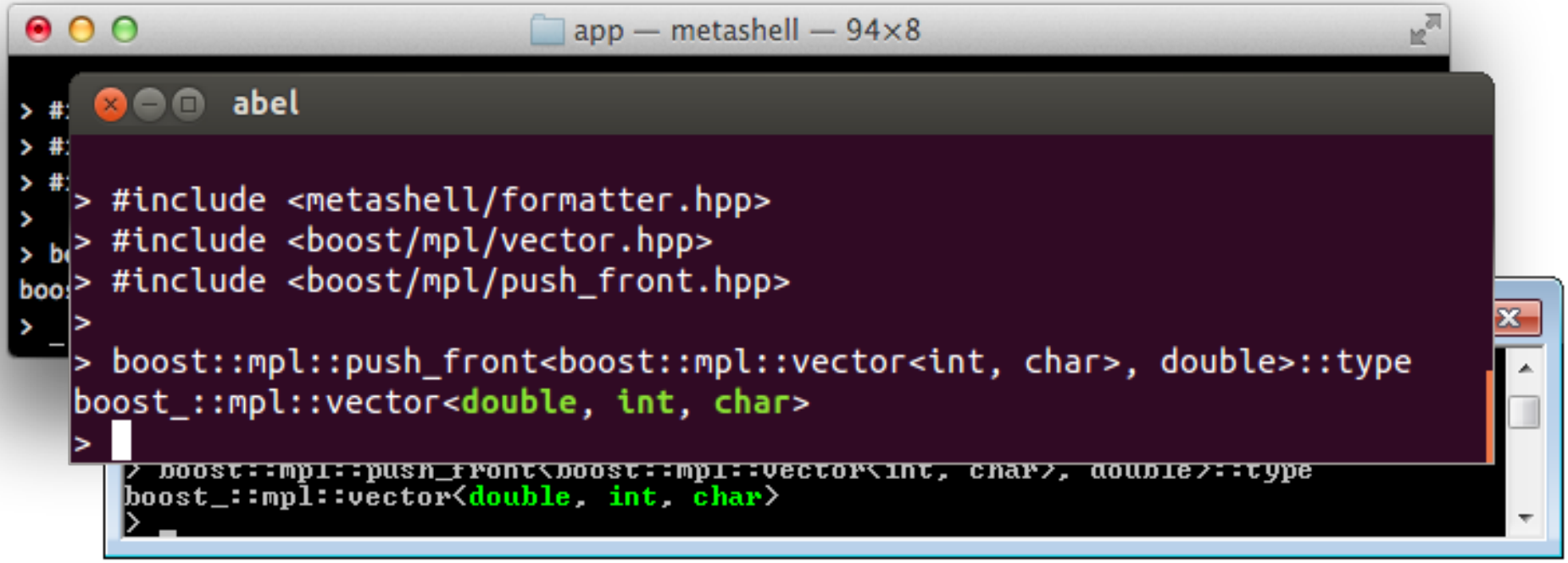
File position: /home/ezolpor/work/proj/templight/work/fib.cpp|22|14

Related work

- Debugging
  - Static assert/Concept check (Siek-Lumsdaine, McNamara-Smaragdakis, Alexandrescu, others...)
  - Warning generation (many attempt)
  - Metashell http://metashell.org/
  - Instrumentation
- Profiling
  - Measuring full compilation (Gurtovoy-Abrahams)
  - Measuring warning appearance (Watanabe)
- Visualize
  - Source execution
  - Instantiation graph

Metashell – interactive TMP REPL

- Ábel Sinkovics and András Kucsma
- Metashell https://github.com/sabel83/metashell



Other C++11/14/17 features

- C++11
  - Auto, decltype
  - Range for
  - Enum class
  - Initializer list
  - Default and delete functions
  - User defined literals
- C++17
  - Structured binding
  - Nested namespaces
  - Compile-time static if
  - Attributes

Auto, decltype

```
void f(T&& t)
{
    auto a = t;                // type deduction
    const auto& car = t;       // type deduction + modifiers
    auto&& pf = t;             // universal reference
    auto            i = 1;     // int
    decltype(auto) j = i;      // int      decltype(i)
    decltype(auto) k = (i);    // int&     decltype((i))
}
void g()
{
    auto i = 1 , d = 3.14; // error, deduced types must match
    auto i = 1 , *p = &i;  // ok, deduced type is int
}
template <typename F, typename... Args> // perfect forwarding requires decltype(auto)
decltype(auto) forwarding( F fun, Args&&... args)
{
    return fun(std::forward<Args>(args)... );
}
template <auto n>              // since C++17
auto f() -> std::pair<decltype(n), decltype(n)>  // auto can't deduce from { init-list }
{
    return { n, n};
}
```

Range for

```
void f(const vector<double>& v)
{
  for (auto x : v) cout << x << '\n';
  for (auto& x : v) ++x;  // using reference allows us to change value
}

// You can read that as "for all x in v" going through starting with
// v.begin() and iterating to v.end(). Another example:

for (const auto x : { 1,2,3,5,8,13,21,34 }) cout << x << '\n';

// The begin() (and end()) can be a member to be called v.begin()
// or a free-standing function to be called begin(v).
```

Enum classes (C++11)

```
enum Alert { green, yellow, election, red }; // traditional enum

enum class Color { red, blue };   // scoped and strongly typed enum
                                  // no export of enumerator names into enclosing scope
                                  // no implicit conversion to int
enum class TrafficLight { red, yellow, green };

Alert a = 7;                // error (as ever in C++)
Alert a1 = static_cast<Alert>(i); // ok, if i in 0..3
Color c = 7;                // error: no int->Color conversion
Color c1 = Color::blue;     // ok
int i2 = red;               // ok: Alert->int conversion
int i3 = Alert::red;        // error in C++98; ok in C++0x
int i4 = blue;              // error: blue not in scope
int i5 = Color::blue;       // error: not Color->int conversion
int i6 = static_cast<int>(Color::blue)  // ok, 1

enum class Color : char { red, blue, }; // compact representation, C++11 allows extra ,

enum struct TrafficLight { red, yellow, green };// by default, the underlying type is int
enum E { E1 = 1, E2 = 2, Ebig = 0xFFFFFFF0U }; // how big is an E?

enum class EE : unsigned long { EE1 = 1, EE2 = 2, EEbig = 0xFFFFFFF0U }; // we are specific

enum class Color_code : char;     // (forward) declaration
void foobar(Color_code* p);       // use of forward declaration

enum class Color_code : char { red, yellow, green, blue }; // definition
```

Initializer list

- Lightweight proxy accessing a temporary const T[ ] maybe in read only memory
- Created when brace-list: list-initialize, assignment, function call, auto, range for
- Initializer_list applies shallow copy

```
vector<double> v = { 1, 2, 3.456, 99.99 };
list<pair<string,string>> languages = {
                                        {"Nygaard","Simula"},
                                        {"Richards","BCPL"},
                                        {"Ritchie","C"}
};
map<vector<string>,vector<int>> years = {
  { {"Maurice","Vincent","Wilkes"},{1913, 1945, 1951, 1967, 2000} },
  { {"Martin","Ritchards"}, {1982, 2003, 2007} },
  { {"David","John","Wheeler"}, {1927, 1947, 1951, 2004} }
};

auto x1 = 5;     // deduced type is int
auto x2(5);      // deduced type is int
auto x3{ 5 };    // deduced type is int since C++17
auto x4 = { 5 };// deduced type is std::initializer_list
```

Default and delete functions

```
struct X
{
  // ...
  X& operator=(const X&) = delete;      // disallow copying
  X(const X&) = delete;  // delete must be at the first declaration
};

// Conversely, we can also say explicitly that
// we want to default copy behavior:
struct Y
{
  // ...
  Y& operator=(const Y&&) = default;     // default move semantics
  Y(const Y&&) = default;
};

const char *func()
{
  // static const char __fun__ = “func”;
  return __fun__;
}
```

User defined literals

```
void f()
{
    int i = 12, j = 014, k = 0xC, l = 0b1100; // literals
    complex<double> cd = 2 + 3i;   // looking for operator"" I (also if, il)
    auto dur = 3h+5min+25s+567ms+765us+10ns;  // chrono duration
    "Hello"s        // basic_string, since C++14
    "Hello"sv       // string_view, since C++17
}

// identifiers not starting with underscore are reserved for std::
constexpr complex<double> operator "" i(long double d)  // imaginary literal
{
    return {0,d}; // complex is a literal type
}

long double  operator "" _w(long double);
std::string  operator "" _w(const char16_t*, size_t);
unsigned int operator "" _w(const char*);

void g()
{
  1.2_w     // operator"" _w(1.2)
  u"one"_w  // operator"" _w(u"one", 3)
  12_w      // operator"" _w("12")
}
```

Using (C++11)

- Typedef won't work well with templates
- Using introduce type alias

```
using myint = int;
template <class T> using ptr_t = T*;

void f(int) { }
// void f(myint) { }   syntax error: redeclaration of f(int)

// make mystring one parameter template
template <class CharT> using mystring =
    std::basic_string<CharT,std::char_traits<CharT>>;
```

Structured bindings (C++17)

```
#include <tuple>  // C++11 tuple

auto get()        // C++14 return type deduction
{
    return std::make_tuple("hello",42);
}

int f1()
{
    auto t = get();   // t is a tuple
    std::cout << std::get<1>(t);  // 42
}

int f2()
{
    std::string s;
    int         i;

    std::tie(s,i) = get();        // C++11
    std::cout << s << i;
}
```

Structured bindings

- Structured binding helps

```
#include <iostream>
#include <vector>

int main()
{
  std::vector v = { 1, 2, 3 };

  if (auto [s,it] = std::pair{ v.size(),v.begin() }; s > 0 && s < *it) {
    std::cerr << "s < c" << '\n';
  }
  else {
    std::cerr << "s == c" << '\n';
  }
  return 0;
}
```

Structured bindings

```
#include <tuple>  // C++11 tuple

auto get()        // C++14 return type deduction
{
    return std::make_tuple("hello",42);
}

int f1()
{
    auto t = get();   // t is a tuple
    std::cout << std::get<1>(t);  // 42
}

int f2()
{
    std::string s;
    int         i;

    std::tie(s,i) = get();        // C++11
    std::cout << s << i;
}
```

Structured bindings

- Binds the specified names to subobjects or members
  - Array-like
  - Tuple-like
  - Member

```
#include <iostream>

int main()
{
  int arr[] = { 1, 2 };
  auto [x,y] = arr;

  std::cout << x << y << arr[0] << arr[1] << '\n';
  return 0;
}
```

```
1212
```

Structured bindings

- Array like

```
#include <iostream>

int main()
{
  int arr[] = { 1, 2 };
  auto [x,y] = arr;
  --x;
  std::cout << x << y << arr[0] << arr[1] << '\n';
  return 0;
}
```

```
0212
```

Structured bindings

- Introduces tmp is a ‘uniquely named’ variable
- If no reference initialization

```
#include <iostream>

int main()
{
  int arr[] = { 1, 2 };
  auto [x,y] = arr;  // creates int tmp[2];int &x=tmp[1];int &y=tmp[2]
  --x;
  std::cout << x << y << arr[0] << arr[1] << ‘\n’;
  return 0;
}
```

```
0212
```

Structured bindings

- If reference initialization happens, names bind to the initialization object

```
#include <iostream>

int main()
{
  int arr[] = { 1, 2 };
  auto& [x,y] = arr;  // creates int &x=arr[1]; int &y=arr[2]
  --x;
  std::cout << x << y << arr[0] << arr[1] << '\n';
  return 0;
}
```

Structured bindings

- If reference initialization happens, names bind to the initialization object

```
#include <iostream>

int main()
{
  int arr[] = { 1, 2 };
  auto& [x,y] = arr;  // creates int &x=arr[1]; int &y=arr[2]
  --x;
  std::cout << x << y << arr[0] << arr[1] << '\n';
  return 0;
}
```

```
0202
```

Structured bindings

- Works with std::array too

```
#include <array>
#include <iostream>

int main()
{
  std::array<int,2> arr = { 1, 2 };
  auto& [x,y] = arr;
  --x;
  std::cout << x << y << arr[0] << arr[1] << '\n';
  return 0;
}

0202
```

Structured bindings

- Tuple-like
  - Std::tuple_size<Tmp>::value should be the number of ids

```
#include <iostream>
#include <tuple>

int main()
{
  int    a = 1;
  double b = 3.14;
  long   c = 2L;

  std::tuple<int, double&, long&&> tpl{a,b,std::move(c)};
  auto  [x,y,z] = tpl;

  std::cout << x << "," << y << "," << z << '\n';
}
```

Structured bindings

- Tuple-like
  - Std::tuple_size<Tmp>::value should be the number of ids

```
#include <iostream>
#include <tuple>

int main()
{
  int    a = 1;
  double b = 3.14;
  long   c = 2L;

  std::tuple<int, double&, long&&> tpl{a,b,std::move(c)};
  auto  [x,y,z] = tpl;

  std::cout << x << "," << y << "," << z << '\n';
}
```

error: tuple is not copyable

Structured bindings

- Tuple-like
  - Std::tuple_size<Tmp>::value should be the number of ids

```
#include <iostream>
#include <tuple>

int main()
{
  int    a = 1;
  double b = 3.14;
  long   c = 2L;

  std::tuple<int, double&, long&&> tpl{a,b,std::move(c)};
  auto& [x,y,z] = tpl;

  std::cout << x << "," << y << "," << z << '\n';
}
```

```
1,3.14,2
```

Structured bindings

- Tuple-like
  - Std::tuple_size<Tmp>::value should be the number of ids

```
#include <iostream>
#include <tuple>

int main()
{
  int    a = 1;
  double b = 3.14;
  long   c = 2L;

  std::tuple<int, double&, long&&> tpl{a,b,std::move(c)};
  auto& [x,y,z] = tpl;

  std::cout << x << "," << y << "," << z << '\n';
}

1,3.14,2
```

Structured bindings

- Members
  - May not have anonymous union
  - Number of ids should the same as non-static members

```
#include <iostream>

struct C
{
  int    a = 1;

  double b = 3.14;
};

int main()
{
  C obj;
  auto [x,y] = obj;

  std::cout << x << "," << y << '\n';
}
```

```
1,3.14
```

Structured bindings

- Members
  - May not have anonymous union
  - Number of ids should the same as non-static members

```
#include <iostream>

struct C
{
  int    a = 1;
  static const int xx = 99;   // static members are not considered
  double b = 3.14;
};

int main()
{
  C obj;
  auto [x,y] = obj;

  std::cout << x << "," << y << '\n';
}
```

```
1,3.14
```

Structured bindings

```
#include <utility>
#include <map>

auto devide_reminder(int a, int b)
{
  return std::pair{ a/b, a%b };
}

void f()
{
  auto [fraction, reminder] = divide_reminder(16,3);
  std::cout << “16/3 == ” << fraction << “, reminder is “ << reminder;

  std::map<std::string, std::string> phone_book {
     {“abel”, “+36 30 123 4567”},
     {“bela”, “+36 30 234 5678”}
  };

  for ( const auto &[person, phone] : phone_book)   {
        std::cout << “person ” << person << “, phone “ << phone << '\n';
  }
}
```

Nested namespace

```
#include <iostream>

namespace A {
  namespace B {

   struct C { void f(); };
  }
}

void A::B::C::f() { std::cerr << "f()\n"; }

int main()
{
  A::B::C c;
  c.f();
  return 0;
}
```

Nested namespace

```
#include <iostream>

namespace A::B {


   struct C { void f(); };

}

void A::B::C::f() { std::cerr << "f()\n"; }

int main()
{
  A::B::C c;
  c.f();
  return 0;
}
```

Attributes (C++11)

- Extra information helping the compiler or static analysis tools or others
  - Like OpenMP
- Earlier non standard, compiler dependent
- Compiler ignores unknown attributes (since C++17)

```
#pragma once

void fatal() __attribute__ ((noreturn));

struct S
{
  char t[3];
} __attribute__ ((aligned (8)));

#if COMPILING_DLL
  #define DLLEXPORT __declspec(dllexport)
#else
  #define DLLEXPORT __declspec(dllimport)
#endif
```

Attributes

- From C++11
- Almost everything can be annotated
  - Type
  - Function
  - Enum
  - ...

```
[[attr]]
[[namespace::attr]]

// C++11
[[noreturn]] void terminate() noexcept;
[[carries_dependency]]

// C++14
[[depricated]]
[[depricated(“reason”)]]
```

Attributes in C++17

```
switch(c)
{
case 'a':
          f();
         [[fallthrough]];
case 'A':
         g();
}

[[nodiscard]] int foo();
void bar()
{
  foo(); // warning, return value of [[nodiscard]] function discarded
}
[[nodiscard]] struct NoDiscard { ... };
NoDiscard f();
void bar()
{
  f(); // warning, return value of [[nodiscard]] type discarded
}
void f()
{
  [[maybe_unused]] int y = 42; // Do not warn on unused y
}
```

Filesystem (C++17)

- Path object
- Directory_entry
- Directory iterators
- Support functions
  - Info from path
  - File manipulation: copy, move, symlink, …
  - Space, filesize, last write time, ...
  - Permissions

    ...

Examples

```
#include <filesystem>
#include <iostream>

namespace fs = std::filesystem;

int main()
{
  fs::path pathToShow("/home/gsd/ftp/file/file.cpp");

  std::cout << "exists() = " << fs::exists(pathToShow) << "\n"
            << "root_name() = " << pathToShow.root_name() << "\n"
            << "root_path() = " << pathToShow.root_path() << "\n"
            << "relative_path() = " << pathToShow.relative_path() << "\n"
            << "parent_path() = " << pathToShow.parent_path() << "\n"
            << "filename() = " << pathToShow.filename() << "\n"
            << "stem() = " << pathToShow.stem() << "\n"
            << "extension() = " << pathToShow.extension() << "\n";
}

$ g++ -std=c++17
```

Examples

```
#include <experimental/filesystem>
#include <iostream>

namespace fs = std::experimental::filesystem;

int main()
{
  fs::path pathToShow("/home/gsd/ftp/file/file.cpp");

  std::cout << "exists() = " << fs::exists(pathToShow) << "\n"
            << "root_name() = " << pathToShow.root_name() << "\n"
            << "root_path() = " << pathToShow.root_path() << "\n"
            << "relative_path() = " << pathToShow.relative_path() << "\n"
            << "parent_path() = " << pathToShow.parent_path() << "\n"
            << "filename() = " << pathToShow.filename() << "\n"
            << "stem() = " << pathToShow.stem() << "\n"
            << "extension() = " << pathToShow.extension() << "\n";
}

$ g++ -std=c++17 -lstdc++fs
$ ./a.out
exists() = 1
root_name() = ""
root_path() = "/"
relative_path() = "home/gsd/ftp/file/file.cpp"
parent_path() = "/home/gsd/ftp/file"
filename() = "file.cpp"
stem() = "file"
extension() = ".cpp"
```

Create filename

```
#include <experimental/filesystem>
#include <iostream>

namespace fs = std::experimental::filesystem;

int main( int argc, char *argv[])
{
  fs::path pathToShow("/home/gsd/ftp/file/file.cpp");

  fs::path myPath{ argc > 1 ? argv[1] : fs::current_path() };

  myPath /= "subdir1";
  myPath /= "subdir2";
  myPath /= "file";
  myPath += ".cpp";

  std::cout << "myPath = "      << myPath << "\n";
  std::cout << "filename() = " << myPath.filename() << "\n";
}

$ g++ -std=c++17 -lstdc++fs
$ ./a.out
myPath = "/home/gsd/ftp/file/subdir1/subdir2/file.cpp"
filename() = "file.cpp"
```

Recursive directory listing (1)

```
#include  <string>
#include <iostream>
#include <sstream>
#include <experimental/filesystem>

using namespace std;
namespace fs = std::experimental::filesystem;

std::uintmax_t ComputeFileSize(const fs::path& pathToCheck)
{
  if (fs::exists(pathToCheck) && fs::is_regular_file(pathToCheck))
  {
    auto err = std::error_code{};
    auto filesize = fs::file_size(pathToCheck, err);
    if (filesize != static_cast<uintmax_t>(-1))
      return filesize;
  }
  return static_cast<uintmax_t>(-1);
}

void DisplayFileInfo(const std::experimental::filesystem::v1::directory_entry & entry,
                     std::string &lead, std::experimental::filesystem::v1::path &filename)
{
  time_t cftime = chrono::system_clock::to_time_t(fs::last_write_time(entry));
  cout << lead << " " << filename << ", " << ComputeFileSize(entry)
       << ", time: " << std::asctime(std::localtime(&cftime));
}
```

Recursive directory listing (2)

```
void DisplayDirTree(const fs::path& pathToShow, int level)
{
    if (fs::exists(pathToShow) && fs::is_directory(pathToShow))
    {
        auto lead = std::string(level * 3, ' ');
        for (const auto& entry : fs::directory_iterator(pathToShow))
        {
            auto filename = entry.path().filename();
            if (fs::is_directory(entry.status()))
            {
                cout << lead << "[+] " << filename << "\n";
                DisplayDirTree(entry, level + 1);
                cout << "\n";
            }
            else if (fs::is_regular_file(entry.status()))
                DisplayFileInfo(entry, lead, filename);
            else
                cout << lead << " [?]" << filename << "\n";
        }
    }
}

int main(int argc, char* argv[])
{
  const fs::path pathToShow{ argc >= 2 ? argv[1] : fs::current_path() };
  DisplayDirTree(pathToShow, 0);
}
```

Any (C++17)

- Type-safe container for a single value of any type
- Namespace function any_cast provides access
- type() returns type_info (or typeid(void))
- Small object optimization is possible
  - But only when move constructor is nothrow

Type safe access of Any

- any_cast checks the content run-time
- May throw std::bad_any_cast

```
std::any a = 3.14;
std::cout << a.type().name() << ' ' << std::any_cast<double>(a) << '\n';
a = 42;
std::cout << a.type().name() << ' ' << std::any_cast<int>(a) << '\n';
std::cout << sizeof(a) << ' ' << sizeof(int) << '\n';

try
{
  std::cout << std::any_cast<double>(a) << '\n';
}
catch( std::bad_any_cast& e)
{
  std::cerr << e.what() << '\n';
}
```

```
d: 3.14
i: 42
i: bad_any_cast
```

Works with user types

```
struct MyStruct
{
  int i;
  double d;

};
std::ostream& operator<<(std::ostream& os, const MyStruct& s)
{
  os << "[ " << s.i << ", " << s.d << " ]";
  return os;
}
void f()
try
{
  MyStruct s{1, 3.14};
  a = s;
  std::cout << a.type().name() << ": " << std::any_cast<MyStruct>(a) << '\n';
  std::cout << sizeof(a) << " " << sizeof(s) << '\n';
}
catch (std::bad_any_cast& e)
{
   std::cout << e.what() << '\n';
}

8MyStruct: [ 1, 3.14 ]
16 16
```

Works with user types

```
struct MyStruct
{
  int i;
  double d;
  char t[100];
};
std::ostream& operator<<(std::ostream& os, const MyStruct& s)
{
  os << "[ " << s.i << ", " << s.d << " ]";
  return os;
}
void f()
try
{
  MyStruct s{1, 3.14};
  a = s;
  std::cout << a.type().name() << ": " << std::any_cast<MyStruct>(a) << '\n';
  std::cout << sizeof(a) << " " << sizeof(s) << '\n';
}
catch (std::bad_any_cast& e)
{
   std::cout << e.what() << '\n';
}

8MyStruct: [ 1, 3.14 ]
16 120
```

any_cast has multiple specializations

```
void f()
{
  std::any a = 42;

  if ( a.has_value() )
  {
    std::cout << a.type().name() << '\n';

    int *ip = std::any_cast<int>(&a);   // not any_cast<int*> !!!
    std::cout << *ip << '\n';
  }
  a.reset();

  If ( ! a.has_value() )
    std::cout << a.type().name() << '\n';
}
```

```
1
v
```

any_cast has multiple specializations

```
void f()
{
  std::any a = 42;

  if ( a.has_value() )
  {
    std::cout << a.type().name() << '\n';

    int *ip = std::any_cast<int*>(&a);   // ERROR
    std::cout << *ip << '\n';
  }
  a.reset();

  If ( ! a.has_value() )
    std::cout << a.type().name() << '\n';
}
```

```
37: error: cannot convert int** to int* in initialization
```

Optional (C++17)

- Maybe monad implementation
- Replaces return types like std::pair<T,bool>
- Optional contains value
  - Initialized/assigned with value of T
  - Initialized/assigned with optional<T> which contains value
- Optional does not contain value
  - Default initialized or initialized with value of std::nullopt_t
  - Initialized/assigned with optional<T> which does not contain value
- If optional<T> contains a value, than it is allocated as T
  - Not a pointer based heap storage
- Convertible to bool: true if contains value
- No optional reference

std::optional

```
std::optional<double> convert( const std::string& s)
try
{
    return std::stoi(s);  // C++11
}
catch (std::invalid_argument e)
{
    return {};
}
catch (std::out_of_range e)
{
    return {};
}

int main()
{
    double d = convert("-3.14e-5").value_or(0.0);
}
```

Use of optional

```
void f(bool b1)
{
  std::optional<int> opt1;
  std::cout << opt1.value_or(-1) << '\n';
  try
  {
    std::cout << opt1.value() << '\n';
  }
  catch( std::bad_optional_access& e)
  {
    std::cerr << e.what() << '\n';
  }
  opt1 = b1 ? std::optional<int>(42) : std::nullopt;

  std::cout << opt1.value_or(-1) << '\n';
  if ( opt1 )
  {
    std::cout << opt1.value() << '\n';
    *opt1 = 2;   // access contained data, also ->  exists
    int i = opt1.value();
    std::cout << i << '\n';
  }
}
```

```
-1
bad optional access
42
42
2
```

Use of pointers

```
void f(bool b1)
{
  std::optional<std::string> opt2;
  *opt2 = "Hello";

  std::cout << *opt2 << '\n';
  std::cout << std::boolalpha << opt2.has_value() << '\n';

  std::cout << opt2.value_or("no value") << '\n';
  std::string s = *std::move(opt2);

  std::cout << s << ", " << opt2->size() << '\n';
}
```

```
Hello
false
no value
Hello, 0
```

Variant (C++17)

- Type-safe union with automatic discriminant
- Holds one of the alternative types or no value
  - No value state is hard to achieve
- Variant is not allowed to allocate dynamic memory
  - The contained object representation is inside the variant object
- Can hold the same type more than once
  - Can hold the type with different cv qualifier
- Default initialized variant hold the first alternative (index() == 0)
  - If it is unambiguous
- Not allowed to hold a reference or an array

std::variant

```
std::variant<int, std::string, double>  v;

v = 55;

try
{
  std::cout << std::get<double>(v);
}
catch (std::bad_variant_access e)
{
  std::err << “not a double”;
}


// supports non-POD types (unions does not)
```

std::variant

```
void f()
{
    std::variant<int, float> v, w;
    v = 42; // v contains int
    int i = std::get<int>(v);
    std::cout << i << '\n';
    std::cout << std::get<0>(v) << '\n';
    w = std::get<0>(v); // same effect as the previous line
    w = v;              // same effect as the previous line

 // std::get<double>(v); // error: no double in [int, float]
 // std::get<3>(v);      // error: valid index values are 0 and 1

    try
    {
      std::get<float>(w); // w contains int, not float: will throw
    }
    catch (std::bad_variant_access& e)
    {
      std::cerr << e.what() << '\n';
    }
}
```

```
42
42
Unexpected index
```

std::variant

```
void f()
{
    std::variant<int, float> v, w;
    v = 42; // v contains int
    int i = std::get<int>(v);
    std::cout << i << '\n';
    std::cout << std::get<0>(v) << '\n';
    w = std::get<0>(v); // same effect as the previous line
    w = v;              // same effect as the previous line

    if ( auto pval = std::get_if<int>(&v) )  // pointer to variant!
    {
      std::cout << *pval << '\n';
    }
    else
    {
      std::cerr << "Other type" << '\n';
    }
}
```

```
42
42
Unexpected index
```

std::variant

```
void f()
{
    std::variant<std::string, double> s1("Hello");      // conversion works
    std::variant<std::string const char *> s2("Hello"); // choose const char *

    s1 = "Hallo";  // conversion when non-ambigous

    std::cout << std::boolalpha
              << "variant holds double? "
              << std::holds_alternative<double>(s1) << '\n'
              << "variant holds string? "
              << std::holds_alternative<std::string>(s1) << '\n';
         //   << std::holds_alternative<int>(s1) << '\n';  // compile error
}

variant holds double? false
variant holds string? true
```

std::visit

```
// vis is a callable
// The return type is deduced from the returned expression as if by decltype


template <class R, class Visitor, class... Variants>
constexpr R visit(Visitor&&, Variants&&...);      // since C++20
```

Applies the vis visitor to the variants vars:

```
std::invoke(std::forward<Visitor>(vis), std::get<is>(std::forward<Variants>(vars))...)
```

std::visit

```
using var_t = std::variant<int, long, double, std::string>; // the variant to visit
template<class T> struct always_false : std::false_type {}; // helper type for the visitor

int main()
{
    std::vector<var_t> vec = {10, 15l, 1.5, "hello"};
    for(auto& v: vec)
    {
        // 1. void visitor, only called for side-effects (here, for I/O)
        std::visit([](auto&& arg){std::cout << arg;}, v);

        // 2. value-returning visitor, demonstrates the idiom of returning another variant
        var_t w = std::visit([](auto&& arg) -> var_t {return arg + arg;}, v);

        // 3. type-matching visitor: a lambda that handles each type differently
        std::cout << ". After doubling, variant holds ";
        std::visit([](auto&& arg) {
            using T = std::decay_t<decltype(arg)>;
            if constexpr (std::is_same_v<T, int>)
                std::cout << "int with value " << arg << '\n';
            else if constexpr (std::is_same_v<T, long>)
                std::cout << "long with value " << arg << '\n';
            else if constexpr (std::is_same_v<T, double>)
                std::cout << "double with value " << arg << '\n';
            else if constexpr (std::is_same_v<T, std::string>)
                std::cout << "std::string with value " << std::quoted(arg) << '\n';
            else
                static_assert(always_false<T>::value, "non-exhaustive visitor!");
        }, w);
    }
}
```

std::visit

```
using var_t = std::variant<int, long, double, std::string>; // the variant to visit

// helper type for the visitor
template<class... Ts> struct overloaded : Ts... { using Ts::operator()...; };
template<class... Ts> overloaded(Ts...) -> overloaded<Ts...>;

int main()
{
    std::vector<var_t> vec = {10, 15L, 1.5, "hello"};

    for (auto& v: vec)
    {
        // type-matching visitor: a class with 3 overloaded operator()'s
        std::visit(overloaded {
            [](auto arg) { std::cout << arg << ' '; },
            [](double arg) { std::cout << std::fixed << arg << ' '; },
            [](const std::string& arg) { std::cout << std::quoted(arg) << ' '; },
        }, v);
    }

}
```

```
10 15 1.500000 “hello”
```

string_view (C++17)

- basic_string_view<char>
- Can refer a constant contiguous char-like sequence
- Therefore iterator and const_iterator are the same types
- Typical implementation is

```
struct string_view
{
   CharT *ptr;
   size_t len;
};
```

String_view example

```
#include <iostream>
#include <string_view>

int main()
{
  std::string s =  "Hello world";
  std::string_view sv = s;
  std::string_view sv1(s.data(),5);
  std::cout << sv1 << '\n';
  sv = sv.substr(6);
  std::cout << sv << '\n';
  std::cout << sv[2] << '\n';
  // sv[2] = 'R';  // ERROR, cannot modify characters
  std::string_view sv2 = sv.substr(1);
  std::cout << sv2 << '\n';
  std::string_view sv3 = sv.substr(-1);

}
```

```
Hello
world
r
orld
terminate called after throwing an instance of 'std::out_of_range'
  what():  basic_string_view::substr: __pos (which is 18446744073709551615) > this->size()
(which is 5)
Aborted (core dumped)
```

String_view is not owner

```
#include <iostream>
#include <string_view>

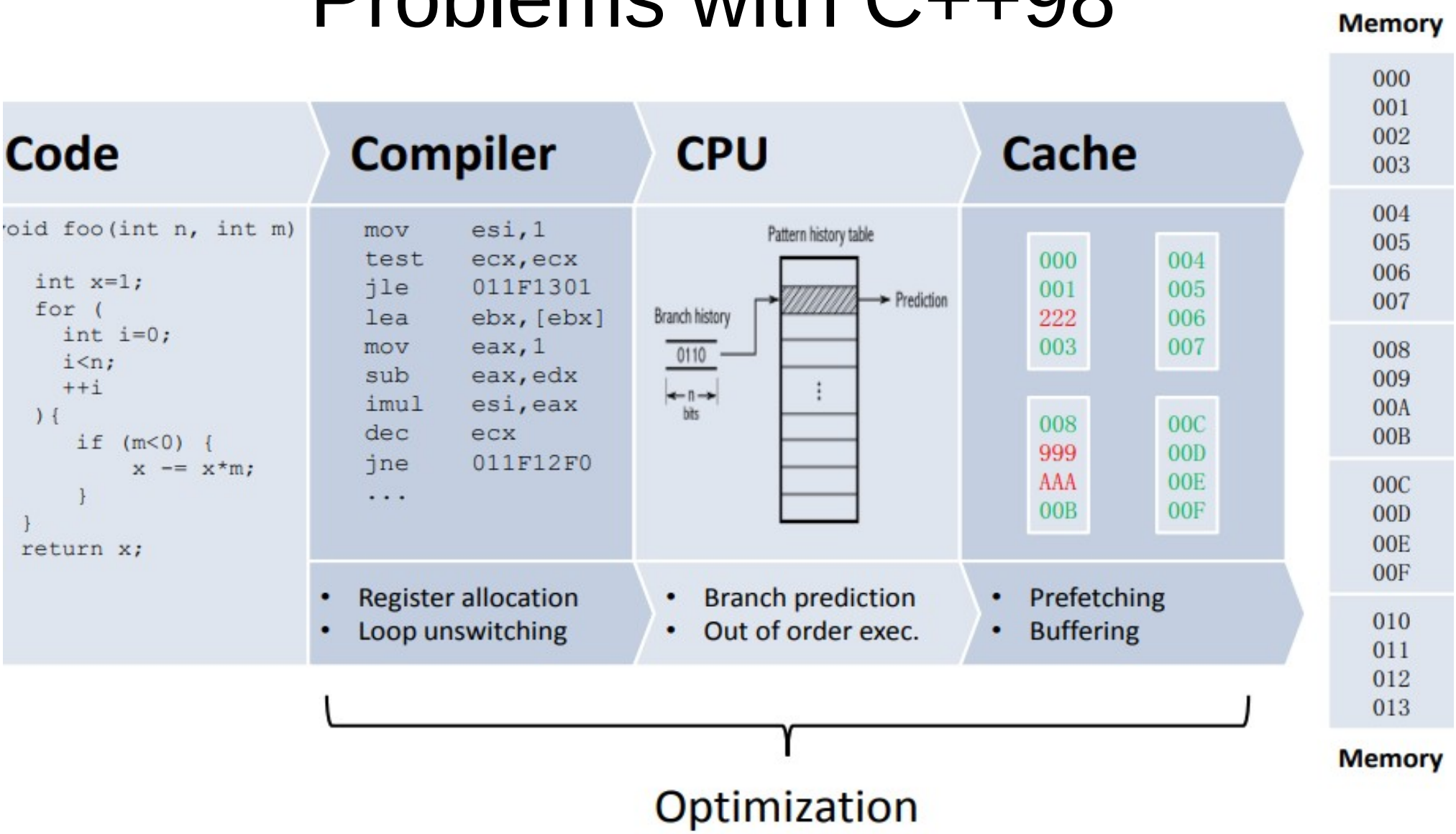
int main()
{
  std::string_view sv5;
  {
    sv5 = std::string{"Hello world"};
    std::string_view sv6 = "Hello world"s;
    std::cout << sv6[3] << '\n';
  }
  std::cout << sv5[3] << '\n';
}
```

Concurrent programming in C++11

Multithreading is just one damn thing after, before, or simultaneous with another. --Andrei Alexandrescu

- Problems with C++98 memory model
- Double-checked locking pattern
- C++11 memory model
- Atomics
- Std::thread
- Mutex/Lock
- Conditional variable
- Future/Promise/Async

Problems with C++98



Valentin Ziegler, Fabio Fracassi C++ Memory Model (Meeting C++ Berlin, 2014)
https://www.think-cell.com/en/career/talks/pdf/think-cell_talk_memorymodel.pdf

Problems with C++98

```
int X = 0;
int Y = 0;

// thread 1                          // thread 2
int r1 = X;                          int r2 = Y;
if ( 1 == r1 )                       if ( 1 == r2 )
    Y = 1;                               X = 1;

// can it be at the end of execution  r1 == r2 == 1 ?
```

Problems with C++98

```
struct S { char a; char b; };
struct S s = { 'a', 'b' };

// thread 1                          // thread 2
s.a = 'x';                           s.b = 'y';

// thread 1 may compiled:            // thread 2 may be compiled:
struct S tmp = x;                    struct S tmp = x;
tmp.a = 'x';                         tmp.b = 'y';
s = tmp;                             s = tmp;

// can the result be { 'a', 'y' } or { 'x', 'b' }?
```

Problems with C++98

Hans Böhm: Threads cannot be implemented as a library
https://dl.acm.org/doi/10.1145/1065010.1065042

Francesco Zappa Nardelli EuroLLVM 2015
https://llvm.org/devmtg/2015-04/slides/CConcurrency_EuroLLVM2015.pdf

Singleton pattern

```
// in singleton.h:
class Singleton
{
public:
    static Singleton *instance();
    void other_method();
    // other methods ...
private:
    static Singleton *pinstance;
};

// in singleton.cpp:
Singleton *Singleton::pinstance = 0;

Singleton *Singleton::instance()
{
    if ( 0 == pinstance )
    {
        pinstance = new Singleton;  // lazy initialization
    }
    return pinstance;
}

// Usage:

   Singleton::istance()-> other_method();
```

Thread safe singleton construction

```
// in singleton.h:
class Singleton
{
public:
    static Singleton *instance();
    void other_method();
    // other methods ...
private:
    static Singleton *pinstance;
    static Mutex      lock_;
};

// in singleton.cpp:
Singleton *Singleton::pinstance = 0;

Singleton *Singleton::instance()
{
    Guard<Mutex> guard(lock_);  // constructor acquires lock_: not efficient
    // this is now the critical section
    if ( 0 == pinstance )
    {
        pinstance = new Singleton;  // lazy initialization
    }
    return pinstance;
} // destructor releases lock_
```

Thread safe singleton construction?

```
// in singleton.h:
class Singleton
{
public:
    static Singleton *instance();
    void other_method();
    // other methods ...
private:
    static Singleton *pinstance;
    static Mutex      lock_;
};

// in singleton.cpp:
Singleton *Singleton::pinstance = 0;

Singleton *Singleton::instance()
{
    // this is now the critical section
    if ( 0 == pinstance )
    {
        Guard<Mutex> guard(lock_);  // constructor acquires lock_: too late!
        // this is now the critical section
        pinstance = new Singleton;  // lazy initialization
    }
    return pinstance;
} // destructor releases lock_
```

Double checked locking pattern

```
Singleton *Singleton::instance()
{
    if ( 0 == pinstance )
    {
        Guard<Mutex> guard(lock_);  // constructor acquires lock_
        // this is now the critical section

        if ( 0 == pinstance )   // re-check pinstance
        {
            pinstance = new Singleton;  // lazy initialization
        }
    }                               // destructor releases lock_
    return pinstance;
}
```

```
Singleton::istance()-> other_method();  // does not lock when not necessary
```

Double checked locking pattern

```
Singleton *Singleton::instance()
{
    if ( 0 == pinstance )
    {
        Guard<Mutex> guard(lock_);  // constructor acquires lock_
        // this is now the critical section

        if ( 0 == pinstance )   // re-check pinstance
        {
            pinstance = new Singleton;  // lazy initialization
        }
    }                               // destructor releases lock_
    return pinstance;
}
```

```
Singleton::istance()-> other_method();  // does not lock when not necessary
```

Meyers and Alexandrescu: C++ and the Perils of Double-Checked Locking:
https://www.aristeia.com/Papers/DDJ_Jul_Aug_2004_revised.pdf

Problems with DCLP

```
if ( 0 == pinstance )
{
      // ...
      pinstance = new Singleton;  // atomic?
      // ...
}
return pinstance;

// might use half-initialized pointer value
Singleton::istance()-> other_method();
```

- Pointer assignment may not be atomic
  - We can observe a not null but still invalid pointer value

New expression

```
pinstance = new Singleton;  // how this is compiled?
```

- New expression include many steps
  - (1) Allocation space with ::operator new()
  - (2) Run of constructor
  - (3) Returning the pointer

- If the compiler does (1) + (3) and leaves (2) as the last step the pointer points to uninitialized memory area

Observable behavior in C++98

```
void foo()
{
    int x = 0, y = 0;        // (1)
    x = 5;                   // (2)
    y = 10;                  // (3)
    printf( "%d,%d", x, y); // (4)
}
```

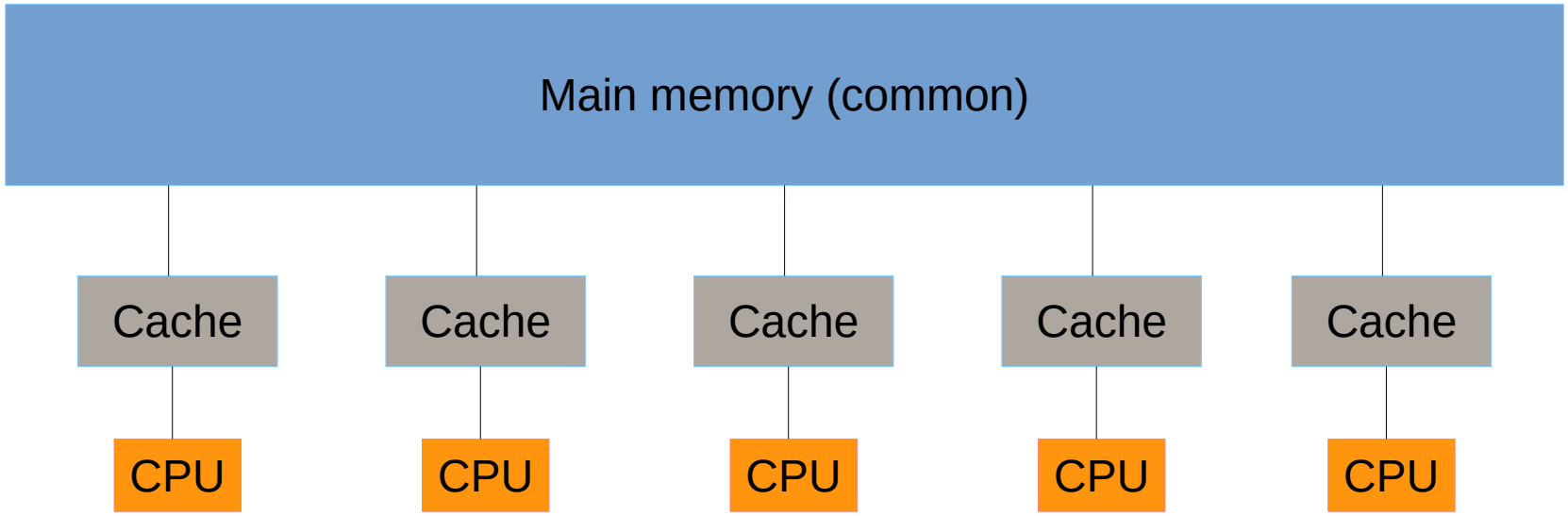
- What is visible for the outer word
  - I/O operations
  - Read/write volatile objects
- Defined by a singled-threaded mind

Sequence point

```
if ( 0 == pinstance )   // re-check pinstance
{
    // pinstance = new Singleton;
    Singleton *temp = operator new( sizeof(Singleton) );
    new (temp) Singleton;  // run the constructor
    pinstance = temp;
}
```

- The compiler can completely optimize out temp
- Even if we are using volatile temp we have issues

Modern hardware architecture



Singleton pattern

```
Singleton *Singleton::instance()
{
    Singleton *temp = pInstance;    // read pInstance

    Acquire();    // prevent visibility of later memory operations
                  // from moving up from this point

    if ( 0 == temp )
    {
        Guard<Mutex> guard(lock_);
        // this is now the critical section

        if ( 0 == pinstance )   // re-check pinstance
        {
            temp = new Singleton;

            Release();  // prevent visibility of earlier memory operations
                        // from moving down from this point

            pinstance = temp;   // write pInstance
        }
    }
    return pinstance;
}
```

C++11 memory model

- Describes the interactions of threads through memory
- Describes well defined behavior
- Constraints compiler for code generation

- C++ memory model contract
  - Programmer ensures that the program has no data race
  - System guarantees sequentially consistent execution

Terminology

- Only minimal progress guaranties are given on threads:
  - unblocked threads will make progress
  - implementation should ensure that writes in a thread should be visible in other threads "in a finite amount of time".
- The A happens before B relationship:
  - A is sequenced before B or
  - A inter-thread **happens before** B

    == there is a **synchronization point** between A and B
- Synchronization point:
  - thread creation sync with start of thread execution
  - thread completion sync with the return of join()
  - unlocking a mutex sync with the next locking of that mutex

Terminology

- Memory location
  - an object of scalar type
  - a maximal sequence of adjacent bit-fields all having non-zero width
- Data race

  A program contains **data race** if contains two actions in different threads, at least one is not "atomic" **and** neither happens before the other.
- Two threads of execution can update and access separate memory locations without interfering each others

Terminology

- Memory location
  - an object of scalar type
  - a maximal sequence of adjacent bit-fields all having non-zero width
- Data race == undefined behavior

  A program contains **data race** if contains two actions in different threads, at least one is not "atomic" **and** neither happens before the other.
- Two threads of execution can update and access separate memory locations without interfering each others

Sequential consistency

- Sequential consistent (default behavior)
  - Leslie Lamport, 1979
  - Each threads are executed in sequential order
  - The operations of each thread appear in this sequence for the other threads in that order

Sequential consistency

```
std::mutex m;
Data d;
bool flag = false;


// thread 1                |  // thread 2
void Produce()             |  void Consume()
{                          |  {
                           |
                           |
  d = result;              |
  flag = true;             |
                           |
                           |
                           |    bool ready = flag;
                           |
                           |
                           |    if ( ready )  use(d);
}                          |  }
```

Sequential consistency

```
std::mutex m;
Data d;
bool flag = false;


// thread 1                |  // thread 2
void Produce()             |  void Consume()
{                          |  {
                           |
                           |
  d = result;              |
  flag = true;             |
                           |
                           |
                           |    bool ready = flag;
                           |
                           |
                           |    if ( ready )  use(d);
}                          |  }
```

Data race!

Sequential consistency

```
std::mutex m;
Data d;
bool flag = false;


// thread 1                |  // thread 2
void Produce()             |  void Consume()
{                          |  {
  m.lock();                |
  d = result;              |
  flag = true;             |
  m.unlock();              |
                           |    m.lock();
                           |    bool ready = flag;
                           |    m.unlock();
                           |
                           |    if ( ready )  use(d);
}                          |  }
```

Sequential consistency

```
std::mutex m;
Data d;
bool flag = false;
```

```
// thread 1                  |  // thread 2
void Produce()               |  void Consume()
{                            |  {
  m.lock();                  |    bool ready;
  d = result;                |
  flag = true;               |
  m.unlock();                |
                             |    m.lock();
Synchronized with            |    bool ready = flag;
                             |    m.unlock();
                             |
                             |    if ( ready )  use(d);
}                            |  }
```

Sequential consistency

```
std::mutex m;
Data d;
bool flag = false;
```

```
// thread 1                  |  // thread 2
void Produce()               |  void Consume()
{                            |  {
  m.lock();                  |    bool ready;
  d = result;                |
  flag = true;               |
  m.unlock();                |
                             |    m.lock();
                             |    bool ready = flag;
                             |    m.unlock();
                             |
                             |    if ( ready ) use(d);
}                            |  }
```

Happens before

Synchronized with

Sequential consistency

```
std::mutex m;
Data d;
bool flag = false;
```

```
// thread 1                  |  // thread 2
void Produce()               |  void Consume()
{                            |  {
  m.lock();                  |    bool ready;
  d = result;                |
  flag = true;               |
  m.unlock();                |
                             |    m.lock();
                             |    bool ready = flag;
                             |    m.unlock();
                             |
                             |    if ( ready )  use(d);
}                            |  }
```

Synchronized with

Happens before

Sequential consistency

```
std::mutex m;
Data d;
bool flag = false;

// thread 1
void Produce()
{
  m.lock();
  d = result;
  flag = true;
  m.unlock();

}
```

```
// start of thread 2
void Consume()
{
  bool ready;

  m.lock();
  bool ready = flag;
  m.unlock();

  if ( ready )  use(d);
}
```

Synchronized with

Happens before

Sequential consistency

```
std::mutex m;
Data d;
std::atomic<bool> flag = false;


// thread 1                |        // thread 2
void Produce()             |  void Consume()
{                          |  {
  ~~m.lock();~~            |    bool ready;
  d = result;              |
  flag.store(true);        |
  ~~m.unlock();~~          |
                           |    ~~m.lock();~~
                           |    bool ready = flag.load();
                           |    ~~m.unlock();~~
                           |
                           |    if ( ready )  use(d);
}                          |  }
```

Sequential consistency

```
std::mutex m;
Data d;
std::atomic<bool> flag = false;
```

```
// thread 1                  |        // thread 2
void Produce()               |   void Consume()
{                            |   {
  m.lock();                  |     bool ready;
  d = result;                |
  flag.store(true);          |
  m.unlock();                |
                             |     m.lock();
                             |     bool ready = flag.load();
                             |     m.unlock();
                             |
                             |     if ( ready ) use(d);
}                            |   }
```

C++11 memory model

```
int x, y;

// thread 1        |        // thread 2
x = 1;             |        cout << y << ", ";
y = 2;             |        cout << x << endl;
```

```
In C++03  not even Undefined Behavior

In C++11  Undefined Behavior
```

C++11 memory model

```
std::atomic<int> x, y;

// thread 1        |        // thread 2
x.store(1);        |        cout << y.load() << ", ";
y.store(2);        |        cout << x.load() << endl;


// Equivalent to:

int x, y;
mutex x_mutex, y_mutex;

// thread 1        |        // thread 2
x_mutex.lock()     |        y_mutex.lock();
x = 1;             |        cout << y << ", ";
x_mutex.unlock()   |        y_mutex.unlock();
y_mutex.lock()     |        x_mutex.lock();
y = 2;             |        cout << x << endl;
y_mutex.unlock()   |        x_mutex.unlock();
```

C++11 memory model (default)

```
std::atomic<int> x, y;
x.store(0); y.store(0);

// thread 1        |        // thread 2
x.store(1);        |        cout << y.load() << ", ";
y.store(2);        |        cout << x.load() << endl;
```

Result can be:

```
0 0
2 1
0 1
// never prints: 2 0
```

Sequential consistency: atomics == atomic load/store + ordering

Memory ordering

- memory_order_seq_cst (default)
- memory_order_consume
- memory_order_acquire
- memory_order_release
- memory_order_acq_rel
- memory_order_relaxed

X86/x86_64 does not require additional instructions to implement acquire-release ordering

Relaxed memory order

- Each memory location has a total modification order
  - But this may be not observable directly
- Memory operations performed by
  - The same thread and
  - On the same memory location

  are not reordered with respect of modification order

Relaxed memory order

```
std::atomic<int> x, y;

// relaxed
// thread 1                          |  // thread 2
x.store(1, memory_order_relaxed); |  cout << y.load(memory_order_relaxed) << ", ";
y.store(2, memory_order_relaxed); |  cout << x.load(memory_order_relaxed) << endl;

// Defined, atomic, but not ordered, result may be:
0 0
2 1
0 1
2 0
```

Relaxed memory order

```
// read-modify-write

std::atomic<int> x;

// relaxed
// thread 1
int i = x.load();                          // current value of x

While ( ! x.compare_exchange_weak( i,   // expected value of x, i is updated on failure
                                   i+1,  // desired value of x
                                   memory_order_relaxed
                                  )
      ) ; // try it again on failure



// equivalent solution

x.fetch_add( 1, memory_order_relaxed);
```

Acquire – release memory order

- A store-release operation **synchronizes** with all load-acquire operations reading a stored value
- Operations preceding the store-release in the releasing thread **happens before** operations following the load-acquire
- On some platforms acquire-release is cheaper than sequention consistency

```
std::mutex m; Data d;
std::atomic<bool> flag = false;

// thread 1                      |  // thread 2
void Produce()                   |  void Consume()
{                                |  {
  d = result;                    |
  flag.store(true,               |    bool ready =
   memory_order_release);        |         flag.load(memory_order_acquire);
                                 |    if ( ready )  use(d);
}                                |  }
```

Acquire – release memory order

- A store-release operation **synchronizes** with all load-acquire operations reading a stored value
- Operations preceding the store-release in the releasing thread **happens before** operations following the load-acquire
- On some platforms acquire-release is cheaper than sequention consistency

```
std::mutex m; Data d;
std::atomic<bool> flag = false;

// thread 1                      |  // thread 2
void Produce()                   |  void Consume()
{                                |  {
  d = result;                    |
  flag.store(true,               |    bool ready =
   memory_order_release);        |      flag.load(memory_order_acquire);
                                 |    if ( ready )  use(d);
}                                |  }
```

Acquire – release memory order

- A store-release operation **synchronizes** with all load-acquire operations reading a stored value
- Operations preceding the store-release in the releasing thread **happens before** operations following the load-acquire
- On some platforms acquire-release is cheaper than sequention consistency

```
std::mutex m; Data d;
std::atomic<bool> flag = false;

// thread 1                          |  // thread 2
void Produce()                       |  void Consume()
{                                    |  {
  d = result;                        |
  flag.store(true,                   |    bool ready =
   memory_order_release);            |      flag.load(memory_order_acquire);
                                     |    if ( ready ) use(d);
}                                    |  }
```

Acquire – release memory order

- A store-release operation **synchronizes** with all load-acquire operations reading a stored value
- Operations preceding the store-release in the releasing thread **happens before** operations following the load-acquire
- On some platforms acquire-release is cheaper than sequention consistency

```
std::mutex m; Data d;
std::atomic<bool> flag = false;

// thread 1                          |  // thread 2
void Produce()                       |  void Consume()
{                                    |  {
  d = result;                        |
  flag.store(true,                   |    bool ready =
   memory_order_release);            |    flag.load(memory_order_acquire);
                                     |    if ( ready ) use(d);
}                                    |  }
```

Acquire – release memory order

```
// acquire-release
// thread 1                          |  // thread 2
x.store(1, memory_order_release); |  cout << y.load(memory_order_acquire) << ", ";
y.store(2, memory_order_release); |  cout << x.load(memory_order_acquire) << endl;

// In C++11 Defined and the result can be:
0 0
2 1
0 1
// never prints: 2 0, but can be faster than strict ordering.
// results may be different in more complex programs
```

Consume – release memory order

- Operations preceding the store-release in the releasing thread **happens before** an operation X in the consuming thread where X has a **data dependency** on the loaded value

```
int d = 0;
std::atomic<int *> ptr = std::nullptr;

// thread 1                          |  // thread 2
void Produce()                       |  void Consume()
{                                    |  {
  d = 42                             |    int *p;
  ptr.store(&d,                      |
   memory_order_release);            |    if (p= ptr.load(memory_order_consume))
                                     |    {
                                     |      assert ( 42 == *p );
                                     |
                                     |      assert ( 42 ==  d );
                                     |    }
}                                    |  }
```

Consume – release memory order

- Operations preceding the store-release in the releasing thread **happens before** an operation X in the consuming thread where X has a **data dependency** on the loaded value

```
int d = 0;
std::atomic<int *> ptr = std::nullptr;

// thread 1                          |  // thread 2
void Produce()                       |  void Consume()
{                                    |  {
  d = 42                             |    int *p;
  ptr.store(&d,                      |
   memory_order_release);            |    if (p= ptr.load(memory_order_consume))
                                     |    {
                                     |      assert ( 42 == *p );
                                     |
                                     |      assert ( 42 ==  d );
                                     |    }
}                                    |  }
```

Consume – release memory order

- Operations preceding the store-release in the releasing thread **happens before** an operation X in the consuming thread where X has a **data dependency** on the loaded value

```
int d = 0;
std::atomic<int *> ptr = std::nullptr;

// thread 1                          |  // thread 2
void Produce()                       |  void Consume()
{                                    |  {
  d = 42                             |    int *p;
  ptr.store(&d,                      |
   memory_order_release);            |    if (p= ptr.load(memory_order_consume))
                                     |    {
                                     |      assert ( 42 == *p );  // ok
                                     |
                                     |      assert ( 42 ==  d );
                                     |    }
}                                    |  }
```

Consume – release memory order

- Operations preceding the store-release in the releasing thread **happens before** an operation X in the consuming thread where X has a **data dependency** on the loaded value

```
int d = 0;
std::atomic<int *> ptr = std::nullptr;

// thread 1                          |  // thread 2
void Produce()                       |  void Consume()
{                                    |  {
  d = 42                             |    int *p;
  ptr.store(&d,                      |
   memory_order_release);            |    if (p= ptr.load(memory_order_consume))
                                     |    {
                                     |      assert ( 42 == *p );   // ok
                                     |
                                     |      assert ( 42 == d );   // DATA RACE
                                     |    }
}                                    |  }
```

std::thread

```
class thread
{
public:
  typedef native_handle ...;
  typedef id ...;

  thread() noexept;                  // does not represent a thread
  thread( thread&& other) noexept; // move constructor
  ~thread();                         // if joinable() calls std::terminate()

  template <typename Function, typename... Args> // copies args to thread local
  explicit thread( Function&& f, Arg&&... args); // then execute f with args

  thread(const thread&) = delete;  // no copy
  thread& operator=(thread&& other) noexept;  // move
  void swap( thread& other);  // swap

  bool joinable() const; // thread object owns a physical thread
  void join();           // blocks current thread until *this finish
  void detach();         // separates physical thread from the thread object

  std::thread::id get_id() const;               // std::this_thread
  static unsigned int hardware_concurrency();  // supported concurrent threads
  native_handle_type native_handle();           // e.g. thread id
};
```

Usage of std::thread

```
void f( int i, const std::string&);
{
    std::cout << "Hello concurrent world" << std::endl;
}

int main()
{
    int i = 3;
    std::string s("Hello");

    // Will copy both i and s
    // We can prevent the copy by using reference wrapper
    // std::thread t( f, std::ref(i), std::ref(s));
    std::thread t( f, i, s);

    // if the thread destructor runs and the thread is joinable, than
    // std::system_error will be thrown.
    // Use join() or detach() to avoid that.
    t.join();

    return 0;
}
```

Issue with join()

- If the thread destructor called when the thread is still *joinable* std::system_error will be thrown
- Alternatives are not really feasible:
- Implicit join:
  - The destructor waits until the thread execution is completed
  - Hard-to detect performance issues
- Implicit detach
  - The destructor may run, but the underlying thread is still under execution
  - We may destroy resources still used by the thread
- Scoped_thread or thread_strategy parameters

Scoped thread

```
class scoped_thread   // Anthony Williams
{
    std::thread t;
public:
    explicit scoped_thread(std::thread t_): t(std::move(t_))
    {
        if(!t.joinable())
            throw std::logic_error(“No thread”);
    }
    ~scoped_thread()
    {
        t.join();
    }
    scoped_thread(scoped_thread const&)=delete;
    scoped_thread& operator=(scoped_thread const&)=delete;
};

struct func;

void f()
{
    int some_local_state;
    scoped_thread t(std::thread(func(some_local_state)));
    do_something_in_current_thread();
}
```

Usage of std::thread

```
struct func
{
    int& i;
    func(int& i_) : i (i_) { }

    void operator()()
    {
        for(unsigned int j=0; j < 1000000; ++j)
        {
            do_something(i);  // i refers to a destroyed variable
        }
    }
};

void oops()
{
    int some_local_state=0;

    func my_func(some_local_state);

    std::thread my_thread(my_func);

    my_thread.detach();  // don't wait the thread to finish
}  // i is destroyed, but the thread is likely still running..
```

std::thread works with containers

```
void do_work(unsigned id);

void f()
{
    std::vector<std::thread> threads;
    for(unsigned i=0;i<20;++i)
    {
        threads.push_back(std::thread(do_work,i));
    }
    std::for_each(threads.begin(),threads.end(),
                  std::mem_fn(&std::thread::join));
}
```

std::thread works with containers

```
// std::thread::id identifiers returned by std::this_thread::get_id()
// it returns std::thread::id() if there is no associated thread.
std::thread::id master_thread;
void some_core_part_of_algorithm()
{
    if(std::this_thread::get_id()==master_thread)
    {
         do_master_thread_work();
    }
    do_common_work();
}


// gives a hint about the available cores. Be aware of
// “oversubscription", i.e. using more threads than cores we have.
std::thread::hardware_concurency()
```

Syncronization objects: mutex

```
#include <mutex>

void f()
{
    std::mutex m;
    int sh;  // shared data
    // ...
    m.lock();
    // manipulate shared data:
    sh+=1;
    m.unlock();
}


void g()
{
    std::mutex m;
    int sh;  // shared data
    // ...
    if ( m.try_lock() )
    {
      // manipulate shared data:
      sh+=1;
      m.unlock();
    }
}
```

```
// Recursive mutex
std::recursive_mutex m;
int sh;  // shared data

void h(int i)
{
    // ...
    m.lock();
    // manipulate shared data:
    sh+=1;
    if (--i>0) f(i);
    m.unlock();
    // ...
}
```

Syncronization objects: timed mutex

```
void f1()
{
    std::timed_mutex m;
    int sh;  // shared data
    // ...
    if (m.try_lock_for(std::chrono::seconds(10)))
    {
      // manipulate shared data:
      sh+=1;
      m.unlock();
    }
    else
      // we didn't get the mutex; do something else
}
void f2()
{
    std::timed_mutex m;
    int sh;  // shared data
    // ...
    if (m.try_lock_until(midnight))
    {
      // manipulate shared data:
      sh+=1;
      m.unlock();
    }
    else
      // we didn't get the mutex; do something else
}
```

RAII support

```
#include <list>
#include <mutex>
#include <algorithm>

std::list<int> l;
std::mutex     m;

void add_to_list(int value);
{
    // lock acquired - with RAII style lock management
    std::lock_guard< std::mutex > guard(m);
    l.push_back(value);
}   // lock released
```

Pointers or references pointing out from the guarded area may be an issue!

Can this go dead-locked?

```
bool operator<( T const& lhs, T const& rhs)
{
    if ( &lhs == &rhs )
      return false;

    std::lock_guard< std::mutex > guard(lhs.m)
    std::lock_guard< std::mutex > guard(rhs.m)

    return lhs.data < rhs.data;
}
```

Can this go dead-locked?

```
bool operator<( T const& lhs, T const& rhs)
{
    if ( &lhs == &rhs )
      return false;

    std::lock_guard< std::mutex > guard(lhs.m)
    std::lock_guard< std::mutex > guard(rhs.m)

    return lhs.data < rhs.data;
}



// thread1            |            thread2

    a < b             |             b < a
```

Correct solution

```
bool operator<( T const& lhs, T const& rhs)
{
    if ( &lhs == &rhs )
        return false;

    // std::lock - lock two or more mutexes
    std::lock( lhs.m, rhs.m);
    std::lock_guard< std::mutex >  lock_lhs( lhs.m, std::adopt_lock);
    std::lock_guard< std::mutex >  lock_rhs( rhs.m, std::adopt_lock);

    return lhs.data < rhs.data;
}

// attempts to lock in unspecified order
template <class Lockable1, class Lockable2, class Lockable3, ...>
void std::lock( Lockable1 m1, Lockable2 m2, Lockable3 m3, ...);

// attempts to lock in left-to-right order
// returns -1 on success, otherwise the index of first failed
template <class Lockable1, class Lockable2, class Lockable3, ...>
int std::try_lock( Lockable1 m1, Lockable2 m2, Lockable3 m3, ...);
```

Unique_lock with defer_lock

```
bool operator<( T const& lhs, T const& rhs)
{
    if ( &lhs == &rhs )
        return false;

    // std::unique_locks constructed with defer_lock can be locked
    // manually, by using lock() on the lock object ...
    std::unique_lock< std::mutex >  lock_lhs( lhs.m, std::defer_lock);
    std::unique_lock< std::mutex >  lock_rhs( rhs.m, std::defer_lock);
    // lock_lhs.owns_lock() now false

    // ... or passing to std::lock
    std::lock( lock_lhs, lock_rhs);  // designed to avoid dead-lock
    // also there is an unlock() memberfunction

    // lock_lhs.owns_lock() now true
    return lhs.data < rhs.data;
}
```

Unique_lock only moveable

```
std::unique_lock<std::mutex> get_lock()
{
    extern std::mutex some_mutex;
    std::unique_lock<std::mutex> lk(some_mutex);
    prepare_data();
    return lk; // same as std::move(lk),
               // return does not require std::move
}

void process_data()
{
    std::unique_lock<std::mutex> lk(get_lock());
    do_something();
}
```

Shared_lock in C++14

```
std::shared_timed_mutex m;
my_data d;

void reader()
{
    std::shared_lock<std::shared_timed_mutex>  rl(m);
    read_only(d);
}

void writer()
{
    std::lock_guard<std::shared_timed_mutex>   wl(m);
    write(d);
}
```

Use of shared_timed_mutex may have worse performance

Mutex management

```
lock_guard
   C++11: Simple scoped wrapper around a mutex
          Non-copyable, non-movable

unique_lock
   C++11: Simple scoped wrapper around a mutex
          Non-copyable,
          Movable: unique_lock(unique_lock&&) operator=(unique_lock&&)
          unlock()

shared_lock
   C++14: lock the mutex in shared mode e.g shared_timed_mutex (c++14)
          Non-copyable, movable

scoped_lock
   C++17: variadic template class RAII to own one or more mutexes
          Non-copyable, owning multiple mutexes with std::lock()
```

Concurrent singleton

```
template <typename T>
class MySingleton
{
public:
    std::shared_ptr<T> instance()
    {
        std::call_once( resource_init_flag, init_resource);
        return resource_ptr;
    }
private:
    void init_resource()
    {
        resource_ptr.reset( new T(...) );
    }
    std::shared_ptr<T> resource_ptr;
    std::once_flag     resource_init_flag; // can't be moved or copied
};
```

Meyers singleton

```
// Meyers singleton:
// C++11 guaranties: local static is initialized in a thread safe way
//
class MySingleton;
MySingleton& MySingletonInstance()
{
    static MySingleton _instance;
    return _instance;
}
```

Spin lock

```
bool flag;   // waiting for this flag
std::mutex m;

void wait_for_flag()
{
    std::unique_lock<std::mutex> lk(m);
    while(!flag)
    {
        lk.unlock();
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        lk.lock();
    }
}
```

Condition variable

```
std::mutex                 my_mutex;
std::queue< data_t >       my_queue;
std::conditional_variable data_cond;  // conditional variable

void producer()
{
    while ( more_data_to_produce() )
    {
        const data_t data = produce_data();
        std::lock_guard< std::mutex > prod_lock(my_mutex);  // guard the push
        my_queue.push(data);
        data_cond.notify_one(); // notify the waiting thread to evaluate cond.
    }
}

void consumer()
{
    while ( true )
    {
        std::unique_lock< std::mutex > cons_lock(my_mutex); // not lock_guard
        data_cond.wait(cons_lock,                   // returns if lamdba returns true
                    [&my_queue]{return !my_queue.empty();}); // else unlocks and waits
        data_t data = my_queue.front();  // lock is hold here to protect pop...
        my_queue.pop();
        cons_lock.unlock();              // ... until here
        consume_data(data);
    }
}
```

Condition variable

- During the wait the condition variable may check the condition any time
- But under the protection of the mutex and returns immediately if condition is true.

- Spurious wake: wake up without notification from other thread. Undefined times and frequency -> better to avoid functions with side effect (e.g. using a counter in lambda to check how many notifications were is bad)

Condition variable

```
std::mutex                 my_mutex;
std::queue< data_t >       my_queue;
std::conditional_variable data_cond;  // conditional variable

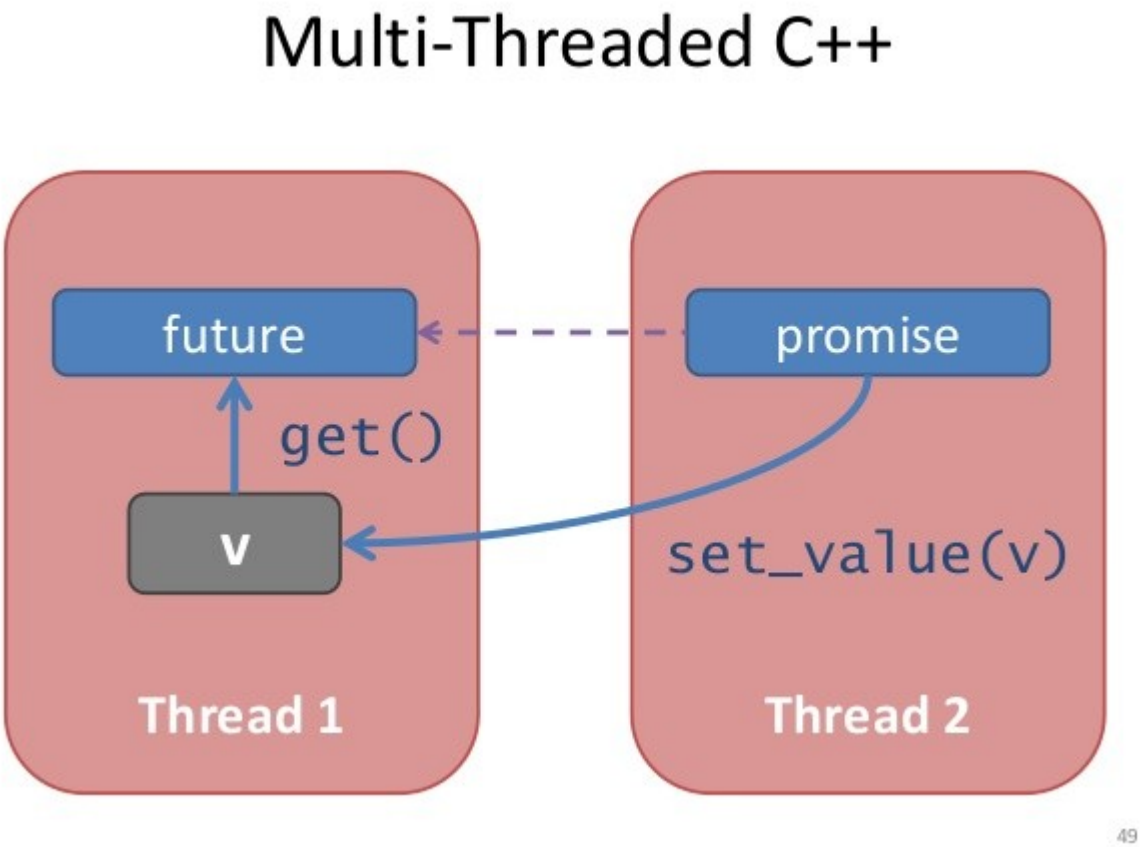
void producer()
{
    while ( more_data_to_produce() )
    {
        const data_t data = produce_data();
        { // more optimal: release lock before notify
          std::lock_guard< std::mutex > prod_lock(my_mutex);  // guard the push
          my_queue.push(data);
        }
        data_cond.notify_one(); // notify the waiting thread to evaluate cond.
    }
}

void consumer()
{
    while ( true )
    {
        std::unique_lock< std::mutex > cons_lock(my_mutex); // not lock_guard
        data_cond.wait(cons_lock,                    // returns if lamdba returns true
                    [&my_queue]{return !my_queue.empty();}); // else unlocks and waits
        data_t data = my_queue.front();  // lock is hold here to protect pop...
        my_queue.pop();
        cons_lock.unlock();              // ... until here
        consume_data(data);
    }
}
```

Future

- 1976 Daniel P. Friedman and David Wise: promise
- 1977 Henry Baker and Carl Hewitt: future
- Future: a read-only placeholder view of a variable or exception
- Promise: a writeable, single assignment container (to set the future)
- Communication channel: promise → future
- std::future the
  - Only instance to refer the async event
  - Move-only
- std::shared_future
  - Multiple instances referring to the same event
  - Copiable
  - All instances will be ready on the same time

Future-Promise



std::async

```
#include <future>
#include <iostream>

int f(int);
void do_other_stuff();

int main()
{
    std::future<int> the_answer = std::async(f,1);
    do_other_stuff();
    std::cout<< "The answer is " << the_answer.get() << std::endl;
}

// The std::async() executes the task either in a new thread or on get()

// starts in a new thread
auto fut1 = std::async(std::launch::async, f, 1);
// run in the same thread on wait() or get()
auto fut2 = std::async(std::launch::deferred, f, 2);
// default: implementation chooses
auto fut3 = std::async(std::launch::deferred | std::launch::async, f, 3);
// default: implementation chooses
auto fut4 = std::async(f, 4);

// If no wait() or get() is called, then the task may not be executed at all.
```

std::async

```
// from cppreference.com
#include <iostream>
#include <future>
#include <thread>
#include <chrono>

int main()
{
    std::future<int> future = std::async(std::launch::async, [](){
        std::this_thread::sleep_for(std::chrono::seconds(3));
        return 8;
    });

    std::cout << "waiting...\n";
    std::future_status status;
    do {
        status = future.wait_for(std::chrono::seconds(1));
        if (status == std::future_status::deferred) {
            std::cout << "deferred\n";
        } else if (status == std::future_status::timeout) {
            std::cout << "timeout\n";
        } else if (status == std::future_status::ready) {
            std::cout << "ready!\n";
        }
    } while (status != std::future_status::ready);

    std::cout << "result is " << future.get() << '\n';
}
```

Exceptions

```
double square_root(double x)
{
    if ( x < 0 )
    {
        throw std::out_of_range("x<0");
    }
    return sqrt(x);
}

int main()
{
    std::future<double> fut = std::async( square_root, -1);
    // do something else...
    double res = fut.get(); // f becomes ready on exception and rethrows
}                           // exception object could be a copy of original
```

Exceptions

```
void asyncFun( std::promise<int> myPromise)
{
    int result;
    try
    {
        // calculate the result
        myPromise.set_value(result);
    }
    catch ( ... )
    {
        myPromise.set_exception(std::current_exception());
    }
}

// In the calling thread:
int main()
{
    std::promise<int> intPromise;
    std::future<int> intFuture = intPromise.getFuture();
    std::thread t(asyncFun, std::move(intPromise));

    // do other stuff here, while asyncFun is working

    int result = intFuture.get();  // may throw MyException
    return 0;
}
```

par_algorithms (C++17)

- Based on Intel's Threading Building Blocks (TBB)
- Extends STL algorithms with execution policy
  - std::execution::seq Sequential execution
  - std::execution::par Parallel execution
  - std::execution::par_unseq Parallel SIMD execution
  - std::execution::unseq Sequential SIMD execution
- These policies are permissions not obligations. Implementation may choose what can be parallelized
- Minimal requirement: forward iterator
- The programmer's task to ensure that element access functions will not cause dead lock or data race
- In case of paralellization and vectorization access must not use any blocking synchronization

Parallel STL

```
// Example from Stroustrup

template<class T, class V>
struct Accum     // simple accumulator function object
{
    T* b;
    T* e;
    V val;
    Accum(T* bb, T* ee, const V& vv) : b{bb}, e{ee}, val{vv} {}
    V operator() () { return std::accumulate(b,e,val); }
};

double comp(vector<double>& v)   // spawn many tasks if v is large enough
{
    if (v.size()<10000) return std::accumulate(v.begin(),v.end(),0.0);

    auto f0 {async(Accum{&v[0],&v[v.size()/4],0.0})};
    auto f1 {async(Accum{&v[v.size()/4],&v[v.size()/2],0.0})};
    auto f2 {async(Accum{&v[v.size()/2],&v[v.size()*3/4],0.0})};
    auto f3 {async(Accum{&v[v.size()*3/4],&v[v.size()],0.0})};

    return f0.get()+f1.get()+f2.get()+f3.get();
}
```

Parallel STL

```
// Example from cppreference

template<class T, class V>
struct Accum     // simple accumulator function object
{
    T* b;
    T* e;
    V val;
    Accum(T* bb, T* ee, const V& vv) : b{bb}, e{ee}, val{vv} {}
    V operator() () { return std::accumulate(b,e,val); }
};

double comp(vector<double>& v)
{
    // non-deterministic if binary_op is not associative or not commutative
    double res = std::reduce(std::execution::par, v.begin(), v.end(), 0.0);
    return res;
}
```

Vectorization

```
std::vector<int> v {1,2, ... };

int sum { std::accumulate(v.begin(), v.end(), 0) };


int sum = 0;
for ( size_t i = 0; i < v.size(); ++i)
{
    sum += v[i];
}



int sum = 0;
for ( size_t i = 0; i < v.size() / 4; i+=4)
{
    sum += v[i] + v[i+1] + v[i+2] + v[i+3];   // most CPU supports this
}
// handle if (v.size()/4) is not 0
```

accumulate() vs reduce()

```
#include <iostream>
#include <vector>

int main()
{
  std::vector<long long> v1;
  for ( int i = 0; i < 10; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4}); // creates 50 elements
  }

  long long sum = 0;
  for ( std::size_t i = 0; i < v1.size(); ++i) // summa x^2  x in [0..49]
  {
    sum += v1[i]*v1[i];
  }

  std::cout << sum << '\n';

  return 0;
}
```

```
$ ./a.out
300
```

accumulate() vs reduce()

```
#include <iostream>
#include <numeric>
#include <vector>


std::vector<long long> v1;
auto sqrsum = [] (auto s, auto val) { return s + val * val; };

int main()
{
  for ( int i = 0; i < 10; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4});  // creates 50 elements
  }

  auto sum1 = std::accumulate(v1.begin(), v1.end(), 0LL, sqrsum);  // classical STL



  std::cout << sum1 << '\n';
  return 0;
}


$ ./a.out
300
```

accumulate() vs reduce()

```
#include <iostream>
#include <numeric>
#include <vector>


std::vector<long long> v1;
auto sqrsum = [] (auto s, auto val) { return s + val * val; };

int main()
{
  for ( int i = 0; i < 10; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4});  // creates 50 elements
  }
  // accumulate is guaranteed left associative
  auto sum1 = std::accumulate(v1.begin(), v1.end(), 0LL, sqrsum);  // classical STL



  std::cout << sum1 << '\n';
  return 0;
}


$ ./a.out
300
```

accumulate() vs reduce()

```
#include <iostream>
#include <numeric>
#include <vector>
#include <execution>

std::vector<long long> v1;
auto sqrsum = [] (auto s, auto val) { return s + val * val; };

int main()
{
  for ( int i = 0; i < 10; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4});
  }
  // accumulate is guaranteed left associative
  auto sum1 = std::accumulate(v1.begin(), v1.end(), 0LL, sqrsum);
  // reduce can work parallel
  auto sum2 = std::reduce(std::execution::par, v1.begin(), v1.end(), 0LL, sqrsum);

  std::cout << sum1 << “, “ << sum2 << '\n';
  return 0;
}
```

```
$ ./a.out
300, 300
```

accumulate() vs reduce()

```
#include <iostream>
#include <numeric>
#include <vector>
#include <execution>

std::vector<long long> v1;
auto sqrsum = [] (auto s, auto val) { return s + val * val; };

int main()
{
  for ( int i = 0; i < 1000; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4});
  }
  // accumulate is guaranteed left associative
  auto sum1 = std::accumulate(v1.begin(), v1.end(), 0LL, sqrsum);
  // reduce can work parallel
  auto sum2 = std::reduce(std::execution::par, v1.begin(), v1.end(), 0LL, sqrsum);

  std::cout << sum1 << “, “ << sum2 << '\n';
  return 0;
}
```

```
$ ./a.out
30000, 30000
```

accumulate() vs reduce()

```
#include <iostream>
#include <numeric>
#include <vector>
#include <execution>

std::vector<long long> v1;
auto sqrsum = [] (auto s, auto val) { return s + val * val; };

int main()
{
  for ( int i = 0; i < 1000000; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4});
  }
  // accumulate is guaranteed left associative
  auto sum1 = std::accumulate(v1.begin(), v1.end(), 0LL, sqrsum);
  // reduce can work parallel
  auto sum2 = std::reduce(std::execution::par, v1.begin(), v1.end(), 0LL, sqrsum);

  std::cout << sum1 << “, “ << sum2 << '\n';
  return 0;
}
```

```
$ ./a.out
30000000, 59820950156796
```

accumulate() vs reduce()

```
#include <iostream>
#include <numeric>
#include <vector>
#include <execution>

std::vector<long long> v1;
auto sqrsum = [] (auto s, auto val) { return s + val * val; };  // not commutative

int main()
{
  for ( int i = 0; i < 1000000; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4});
  }
  // accumulate is guaranteed left associative
  auto sum1 = std::accumulate(v1.begin(), v1.end(), 0LL, sqrsum);
  // reduce can work parallel
  auto sum2 = std::reduce(std::execution::par, v1.begin(), v1.end(), 0LL, sqrsum);

  std::cout << sum1 << “, “ << sum2 << '\n';
  return 0;
}
```

```
$ ./a.out
30000000, 59820950156796
```

accumulate() vs reduce()

```
#include <iostream>
#include <numeric>
#include <vector>
#include <execution>
#include <functional>
std::vector<long long> v1;
auto sqrsum = [] (auto s, auto val) { return s + val * val; };

int main()
{
  for ( int i = 0; i < 1000000; ++i)
  {
    v1.insert( v1.end(), {0,1,2,3,4});
  }
  // accumulate is guaranteed left associative
  auto sum1 = std::accumulate(v1.begin(), v1.end(), 0LL, sqrsum);
  auto sum2 = std::transform_reduce(std::execution::par,  // map-reduce
          v1.begin(), v1.end(), 0LL,
          std::plus<>(),
          [](auto v) { return v*v; });
  std::cout << sum1 << “, “ << sum2 << '\n';
  return 0;
}

$ ./a.out
30000000, 30000000
```

C++20

- resumable functions
  - async ... wait
- continuation
  - then()
  - when_any()
  - when_all()
- transactional memory – ???

- Critics on C++ concurrency:

  Bartosz Milewski's blog: Broken promises - C++0x futures
  http://bartoszmilewski.com/2009/03/03/broken-promises-c0x-futures/

  MeetingC++ - Hartmut Kaiser: Plain Threads are the GOTO of todays computing https://www.youtube.com/watch?v=4OCUEgSNIAY

Compiling, linking issues

- Inclusion nightmare and compile time
- How to reduce included files
- PIMPL
- Fast PIMPL
- Template code blow
- Linking C and C++ together
- Order of linking issues
- Static initialization/destruction problem

Header files

- Good for defining interface
- Good for breaking circular dependencies
- Necessary for templates
- Increase compile time
- Break OO principles

Header files

```
#include <iostream>
#include <ostream>
#include <list>
// none of A, B, C, D, E are templates
// Only A and C have virtual functions

#include "a.h"      // class A
#include "b.h"      // class B
#include "c.h"      // class C
#include "d.h"      // class D
#include "e.h"      // class E

class X : public A, private B
{
public:
        X( const C&);
    B   f(int, char*);
    C   f(int, C);
    C&  g(B);
    E   h(E);
    virtual std::ostream& print(std::ostream&) const;
private:
    std::list<C>    clist_;
    D               d_;
};
```

Header files

- Remove <iostream>. People automatically include <iostream>, even if input functions never used.
- Replace <ostream> with <iosfwd>. Parameters and return types only need to be forward declared. Because ostream is basic_ostream<char> template, it is not enough to declare.
- Replace "e.h" with forward declaration of class E.
- Leave "a.h" and "b.h": we need a full declaration of the base classes in case of inheritance. The compiler must know the size of bases, whether functions are virtual or not.
- Leave "c.h" and "d.h": list<C> and D are private data members of X.

Header files v2

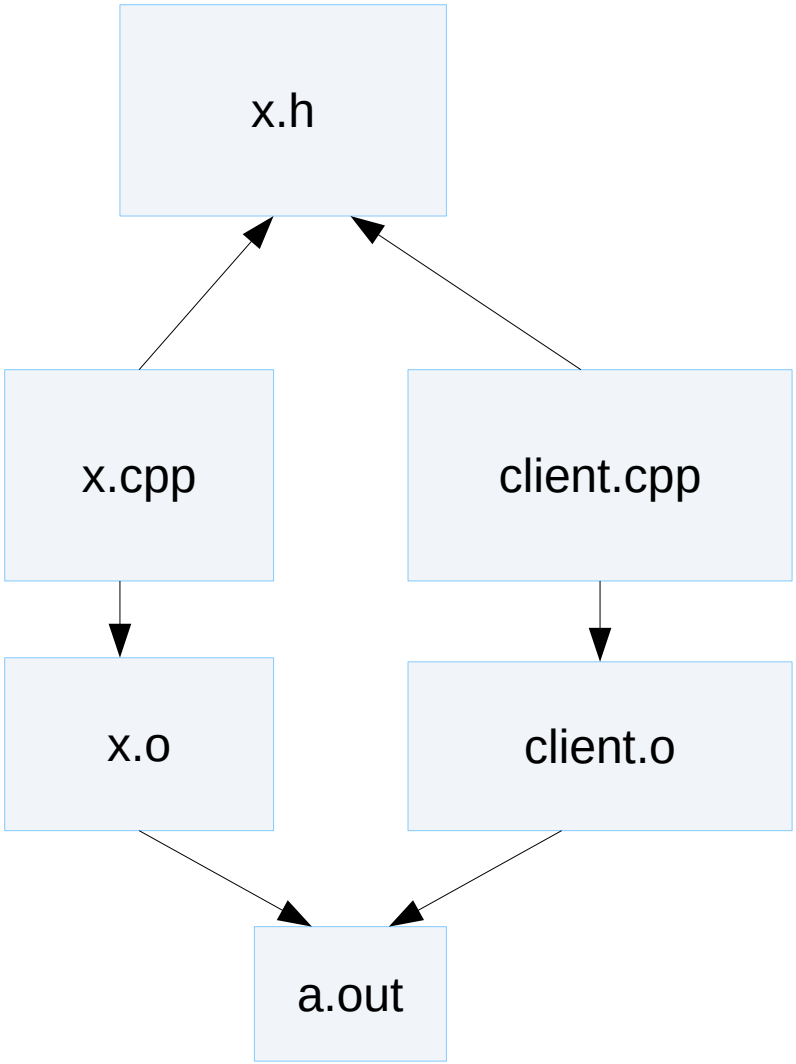
```
#include <iosfwd>
#include <list>
// none of A, B, C, D, E are templates
// Only A and C have virtual functions

#include "a.h"      // class A
#include "b.h"      // class B
// #include “c.h”   // class C
#include "d.h"      // class D

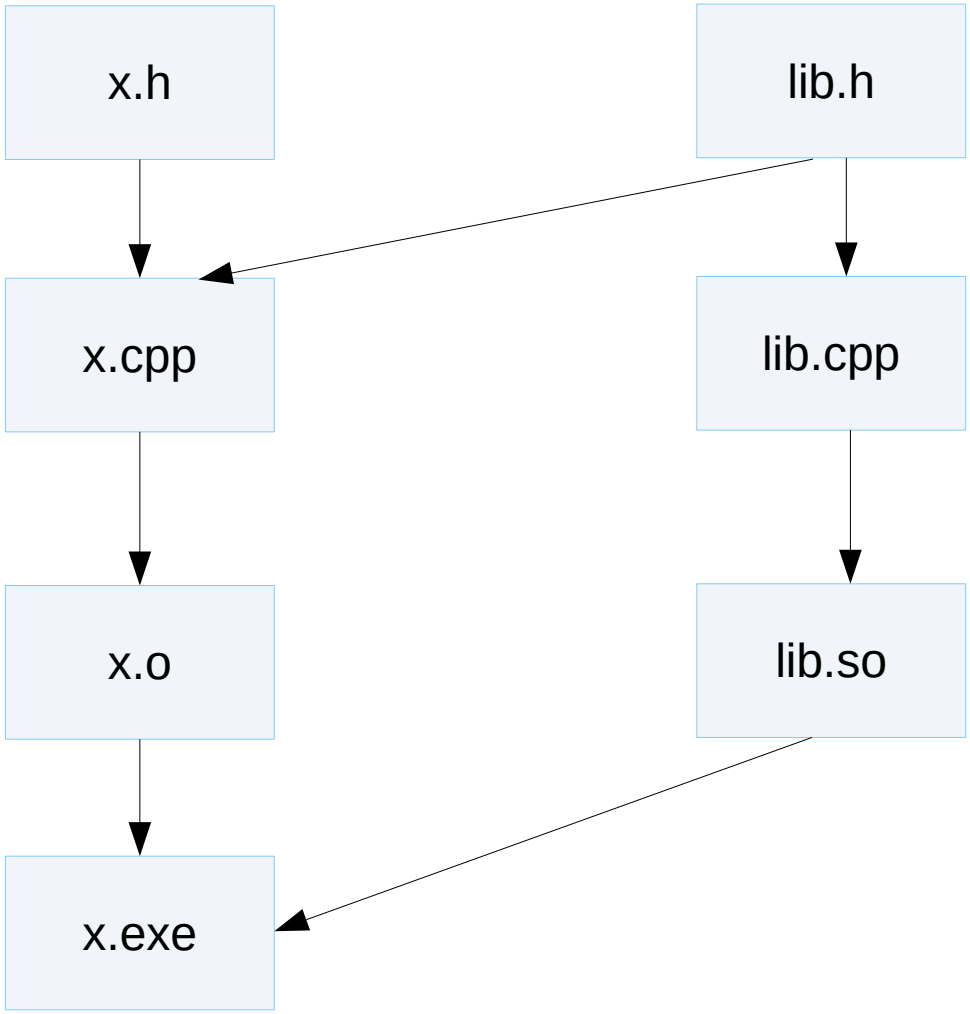
class C; class E;  // forward declaration

class X : public A, private B
{
public:
        X( const C&);
    B   f(int, char*);
    C   f(int, C);
    C&  g(B);
    E   h(E);
    virtual std::ostream& print(std::ostream&) const;
private:
    std::list<C>    clist_;
    D               d_;
};
```

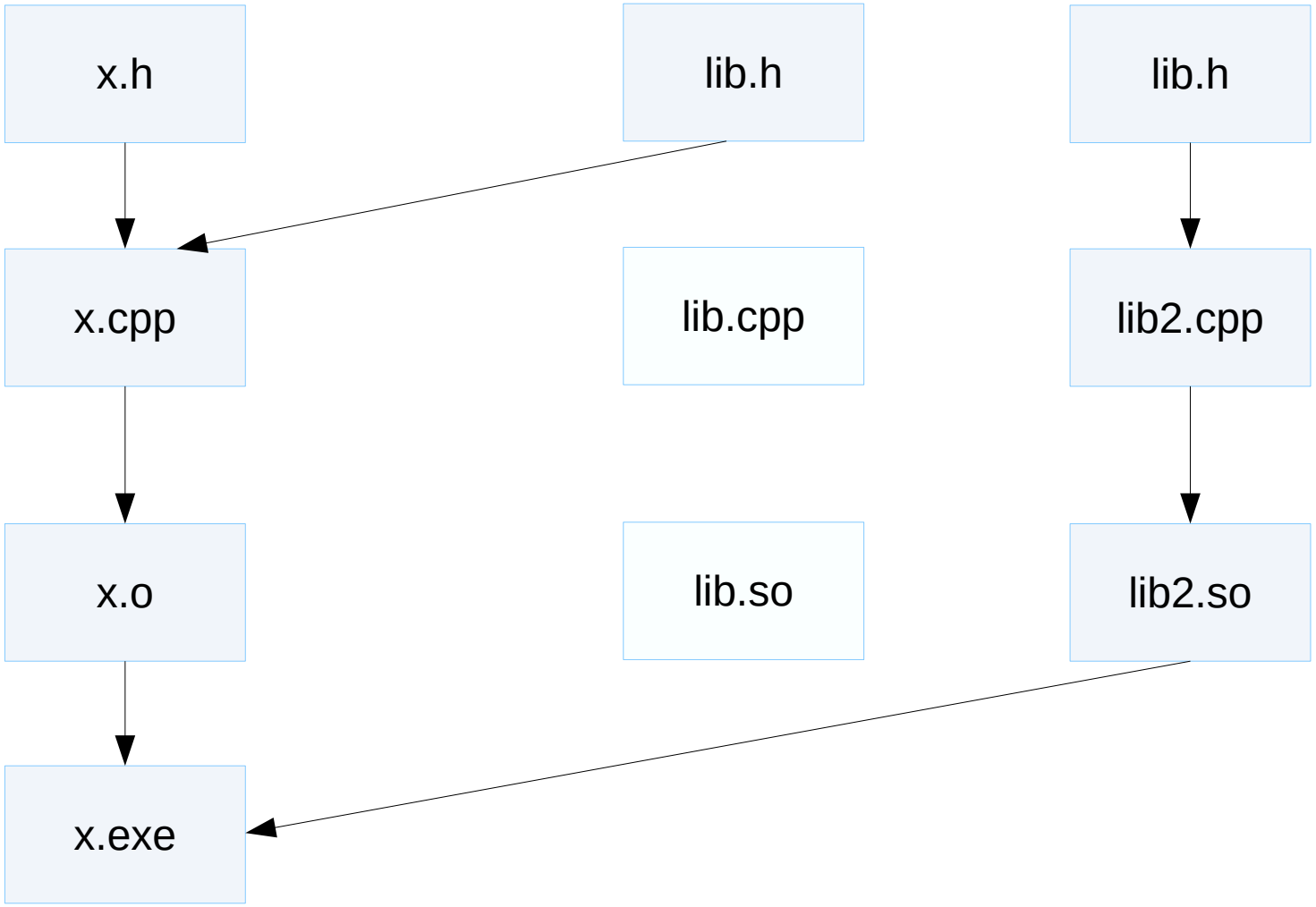
PIMPL



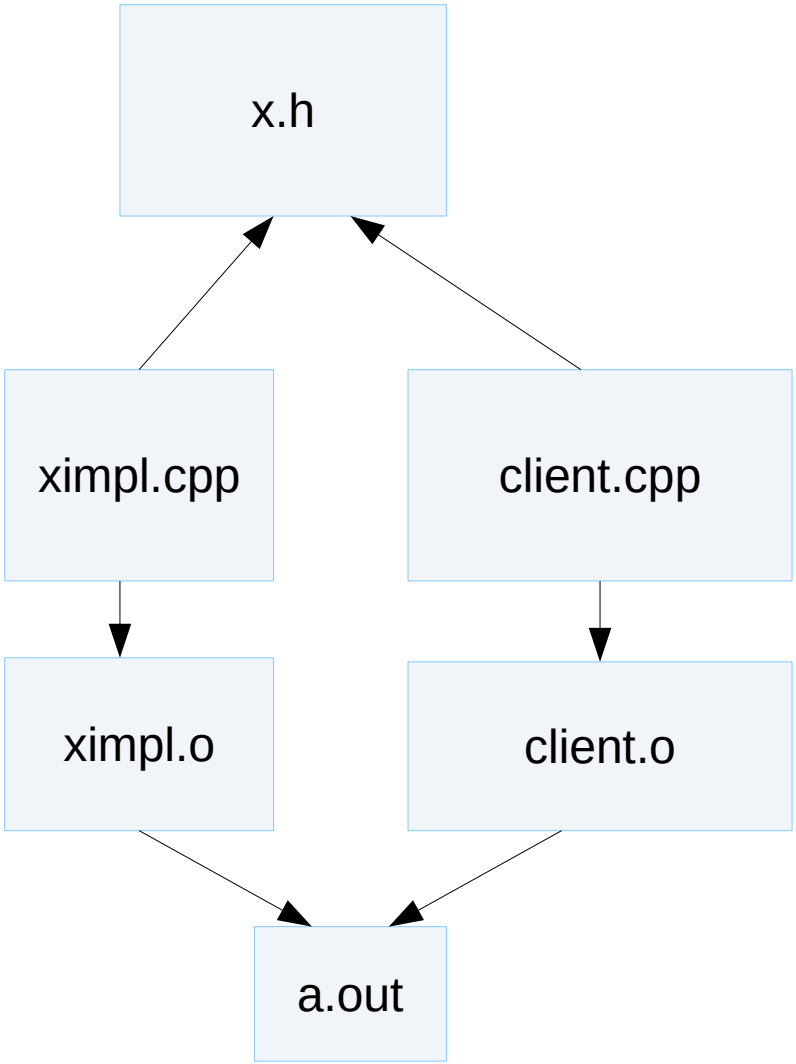
Binary compatibility



Binary compatibility



PIMPL



PIMPL

```
// file x.h
class X
{
    // public and protected members
private:
    // pointer to forward declared implementation class
    struct Ximpl;
    XImpl *pimpl_;  // opaque pointer
};
```

```
// file ximpl.cpp
struct XImpl   // not neccessary to declare as "class"
{
    // private members; fully hidden
    // can be changed without recompiling clients
};
```

PIMPL

```
#include <iosfwd>
#include "a.h"      // class A
#include "b.h"      // class B
class C;
class E;
class X : public A, private B
{
public:
        X( const C&);
    B   f(int, char*);
    C   f(int, C);
    C&  g(B);
    E   h(E);
    virtual std::ostream& print(std::ostream&) const;
private:
    struct Ximpl;
    XImpl *pimpl_; // opaque pointer to forward-declared class
};


// file x.cpp
#include "x.h"
#include "c.h"      // class C
#include "d.h"      // class D
#include <list>
struct Ximpl
{
    std::list<C>    clist_;  // class list<C> and D is hidden
    D               d_;
};
```

PIMPL with unique_ptr

```
// file x.h
class X
{
public:
    X();
private:
    // pointer to forward declared class
    struct Ximpl;
    std::unique_ptr<Ximpl> pimpl_;  // opaque pointer
};
```

```
// file ximpl.cpp
struct X::XImpl   // not neccessary to declare as "class"
{
    X::X()
    // can be changed without recompiling clients
};


X::X() : pimpl_(std::make_unique<XImpl>()) {}
```

PIMPL with unique_ptr

```
#include “x.h”

X  xObj;   // Error: incomplete type!
```

PIMPL with unique_ptr

```
// file x.h
class X
{
public:
    X();
    // ~X() { } compiler generated
private:
    // pointer to forward declared class
    struct Ximpl;
    std::unique_ptr<Ximpl> pimpl_;  // opaque pointer
};

// file ximpl.cpp
struct X::XImpl   // not necessary to declare as "class"
{
    X::X()
    // can be changed at without recompiling clients
};

X::X() : pimpl_(std::make_unique<XImpl>()) {}
```

PIMPL with unique_ptr

```
// file x.h
class X
{
public:
    X();
    ~X();  // declaration only!
private:
    // pointer to forward declared class
    struct Ximpl;
    std::unique_ptr<Ximpl> pimpl_;  // opaque pointer
};

// file ximpl.cpp
struct X::XImpl   // not necessary to declare as "class"
{
    X::X()
    // can be changed at without recompiling clients
};

X::X() : pimpl_(std::make_unique<XImpl>()) {}
X::~X() {}  // definition, also works: X::~X() = default
```

PIMPL with unique_ptr

```
// file x.h
class X
{
public:
    X();
    ~X();
    X(X&& rhs);
    X& operator=(X&& rhs);
private:
    // pointer to forward declared class
    struct Ximpl;
    std::unique_ptr<Ximpl> pimpl_;  // opaque pointer
};

// file ximpl.cpp
X::X() : pimpl_(std::make_unique<XImpl>()) {}
X::~X() = default;
X::X(X&& rhs) = default;
X& X::operator=(X&& rhs) = default;
```

PIMPL with unique_ptr

```
// file x.h
class X
{
public:
    X();
    ~X();
    X(X&& rhs);
    X& operator=(X&& rhs);
    X(const X& rhs);
    X& operator=(const X& rhs);
private:
    // pointer to forward declared class
    struct Ximpl;
    std::unique_ptr<Ximpl> pimpl_;  // opaque pointer
};

// file ximpl.cpp
X::X(const X& rhs) : pimpl_(std::make_unique<XImpl>(*rhs.pImpl_) {}
X& X::operator=(const X& rhs){ *pImpl_ = *rhs.pImpl_; return *this; }
```

Removing inheritance(?)

```
#include <iosfwd>
#include "a.h"      // class A
class B;
class C;
class E;
class X : public A  // ,private B
{
public:
        X( const C&);
    B   f(int, char*);
    C   f(int, C);
    C&  g(B);
    E   h(E);
    virtual std::ostream& print(std::ostream&) const;
private:
    struct Ximpl;
    XImpl *pimpl_; // opaque pointer to forward-declared class
};
// file x.cpp
#include "x.h"
#include "b.h"      // class B
#include "c.h"      // class C
#include "d.h"      // class D
#include <list>
struct Ximpl
{
    B               b_;  // delegate
    std::list<C>    clist_;
    D               d_;
};
```

Fast PIMPL

```
// file x.h
class X
{
  // public and protected members
private:
  static const size_t XImplSize = 128;  // estimation
  char ximpl_[XImplSize]; // instead of opaque pointer
};

// file ximpl.cpp
struct XImpl   // not neccessary to declare as "class"
{
  Ximpl::Ximpl(X *tp) : _self(tp) {
    static_assert (XImplSize >= sizeof(XImpl));
    // ...
  };
  X *_self; // might be different than XImpl::this
};
X::X()  { new (ximpl_) XImpl(this); }
X::~X() { (reinterpret_cast<XImpl*>(ximpl_)->~XImpl(); }
```

Fast PIMPL

```
// file x.h
class X
{
  // public and protected members
private:
  static const size_t XImplSize = 128;
  alignas(std::max_align_t) char ximpl_[XImplSize];
};

// file ximpl.cpp
struct XImpl   // not neccessary to declare as "class"
{
  Ximpl::Ximpl(X *tp) : _self(tp) {
    static_assert (XImplSize >= sizeof(XImpl));
    // ...
  };
  X *_self; // might be different than XImpl::this
};
X::X()  { new (ximpl_) XImpl(this); }
X::~X() { (reinterpret_cast<XImpl*>(ximpl_)->~XImpl(); }
```

Template code blow

- Templates are instantiated on request
- Each different template argument type creates new specialization
  - Good: we can specialize for types
  - Bad: Code is generated for all different arguments
  - All template member functions are templates

Template code blow

```
template <class T>
class matrix
{
public:
    int get_cols() const { return cols_; }
    int get_rows() const { return rows_; }
private:
    int cols_;
    int rows_;
    T *elements_;
};

matrix<int>     mi;
matrix<double>  md;
matrix<long>    ml;
```

Template code blow

```
class matrix_base
{
public:
    int get_cols() const { return cols_; }
    int get_rows() const { return rows_; }
protected:
    int cols_;
    int rows_;
};

template <class T>
class matrix : public matrix_base
{
private:
    T *elements_;
};
```

Using C and C++ together

- Object model is (almost) the same
- C++ programs regularly call C libraries
- Issues:
  - Virtual inheritance
  - Virtual functions (pointer to vtbl)
  - Mangled name vs C linkage name

Using C and C++ together

```
//
// C++ header for C/C++ files:
//
#ifdef __cplusplus
extern "C"
{
#endif
    int     f(int);
    double  g(double, int);
    // …
#ifdef __cplusplus
}
#endif
```

Place of instantiation

```
#include <iostream>
#include <algorithm>

// which swap?
template<typename T>
struct Test{
     void operator()(T& lhs,T& rhs){
         std::swap(lhs,rhs);
     }
};
struct MyT { };
namespace std {
     inline void swap(MyT& lhs,MyT& rhs){
         std::cout << "MySwap" << std::endl;
     }
}
```

```
int main()
{
     MyT t1,t2;
     Test<MyT>{}(t1,t2);
}

$ ./a.out
$
```

Place of instantiation

```
#include <iostream>
#include <algorithm>

// from here
template<typename T>
struct Test{
     void operator()(T& lhs,T& rhs){
         std::swap(lhs,rhs);
     }
};
struct MyT { };
namespace std {
     inline void swap(MyT& lhs,MyT& rhs){
         std::cout << "MySwap" << std::endl;
     }
}
// to here
template<typename T>
struct Test{
     void operator()(T& lhs,T& rhs){
         std::swap(lhs,rhs);
     }
};
int main()
{
     MyT t1,t2;
     Test<MyT>{}(t1,t2);
}

$ ./a.out
MySwap
$
```

Correct use of swap

```
#include <iostream>
#include <algorithm>


struct MyT { };

namespace std {  // do not extend std namespace
     inline void swap(MyT& lhs,MyT& rhs){
         std::cout << "MySwap" << std::endl;
     }
}
// works here
template<typename T>
struct Test{
     void operator()(T& lhs,T& rhs){
         using std::swap;   // introduce std::swap into the sscope
         swap(lhs,rhs);     // use ADL to select the better swap
     }
};




int main()
{
     MyT t1,t2;
     Test<MyT>{}(t1,t2);
}

$ ./a.out
MySwap
$
```

Correct use of swap

```
#include <iostream>
#include <algorithm>


struct MyT { };

namespace std {  // do not extend std namespace
     inline void swap(MyT& lhs,MyT& rhs){
         std::cout << "MySwap" << std::endl;
     }
}
// works here
template<typename T>
struct Test{
     void operator()(T& lhs,T& rhs){
         using std::swap;   // introduce std::swap into the sscope
         swap(lhs,rhs);     // use ADL to select the better swap
     }
};
inline void swap(MyT& lhs,MyT& rhs){
    std::cout << "MySwap" << std::endl;
}

int main()
{
     MyT t1,t2;
     Test<MyT>{}(t1,t2);
}

$ ./a.out
MySwap
$
```

to_string example

```
#include <string>

template <typename T>
class MyClass
{
public:
  std::string to_string() const;
private:
  std::string to_string_ads(const T &t) const;
};

template <typename T>
std::string MyClass<T>::to_string_ads(const T &t) const  // ADS helper
{
  using std::to_string;
  return to_string(t);
}

int main()
{
  MyClass<double> td;
  MyClass<X>      tx;

  td.to_string(); // calls std::to_string(double);
  tx.to_string(); // calls to_string(const X&);
}
```

Order of linking?

```
// file: t.h
template <class T>
T t(const T&)
{
    return s;
}
```

```
// file: a.cpp
static int s = 2;
#include "t.h"
int g()
{
    return t(1);
}
```

```
// file: b.cpp
static int s = 1;
#include "t.h"
int h()
{
    return t(1);
}
```

```
// file: main.cpp
#include <iostream>
extern int g(); extern int h();
int main()
{
    std::cout << g() << h() << '\n';
    return 0;
}
```

```
$ g++ main.cpp a.cpp b.cpp  &&  ./a.out
```

Order of linking?

```
// file: t.h
template <class T>
T t(const T&)
{
    return s;
}
```

```
// file: a.cpp
static int s = 2;
#include "t.h"
int g()
{
    return t(1);
}
```

```
// file: b.cpp
static int s = 1;
#include "t.h"
int h()
{
    return t(1);
}
```

```
// file: main.cpp
#include <iostream>
extern int g(); extern int h();
int main()
{
    std::cout << g() << h() << '\n';
    return 0;
}
```

```
$ g++ main.cpp a.cpp b.cpp  &&  ./a.out
2 2
```

Order of linking?

```
// file: t.h
template <class T>
T t(const T&)
{
    return s;
}
```

```
// file: a.cpp
static int s = 2;
#include "t.h"
int g()
{
    return t(1);
}
```

```
// file: b.cpp
static int s = 1;
#include "t.h"
int h()
{
    return t(1);
}
```

```
// file: main.cpp
#include <iostream>
extern int g(); extern int h();
int main()
{
    std::cout << g() << h() << '\n';
    return 0;
}
```

```
$ g++ main.cpp b.cpp a.cpp  &&  ./a.out
```

Order of linking?

```
// file: t.h
template <class T>
T t(const T&)
{
    return s;
}
```

```
// file: a.cpp
static int s = 2;
#include "t.h"
int g()
{
    return t(1);
}
```

```
// file: b.cpp
static int s = 1;
#include "t.h"
int h()
{
    return t(1);
}
```

```
// file: main.cpp
#include <iostream>
extern int g(); extern int h();
int main()
{
    std::cout << g() << h() << '\n';
    return 0;
}
```

```
$ g++ main.cpp b.cpp a.cpp  &&  ./a.out
1 1
```

Inline functions

```
// Date.h
class Date
{
public:
    // …
private:
    // …
    int year;
    int month;
    int day;
};

// operators for class Date
inline bool operator<( date d1, date d2)  { ... }
inline bool operator==(date d1,date d2)   { ... }
inline bool operator!=( date d1, date d2) { ... }
inline bool operator<=( date d1, date d2) { ... }
inline bool operator>=( date d1, date d2) { ... }
inline bool operator>( date d1, date d2)  { ... }
```

Inline variables C++17

```
#include <iostream>
void f();
            int g0 = 10;
        inline int g1 = 11;
  static inline int g2 = 12;
  extern inline int g3 = 13;

int main()
{
  std::cout << "&g1 = " << g1 << ", " << &g1 << '\n';
  std::cout << "&g2 = " << g2 << ", " << &g2 << '\n';
  std::cout << "&g3 = " << g3 << ", " << &g3 << '\n';
  f();
  return 0;
}
```

```
#include <iostream>
                int g0 = 20;
        inline int g1 = 21;
  static inline int g2 = 22;
  extern inline int g3 = 23;

void f()
{
  std::cout << "&g1 = " << g1 << ", " << &g1 << '\n';
  std::cout << "&g2 = " << g2 << ", " << &g2 << '\n';
  std::cout << "&g3 = " << g3 << ", " << &g3 << '\n';
}
```

Inline variables C++17

```
#include <iostream>
void f();
            int g0 = 10;
        inline int g1 = 11;
  static inline int g2 = 12;
  extern inline int g3 = 13;

int main()
{
  std::cout << "&g1 = " << g1 << ", " << &g1 << '\n';
  std::cout << "&g2 = " << g2 << ", " << &g2 << '\n';
  std::cout << "&g3 = " << g3 << ", " << &g3 << '\n';
  f();
  return 0;
}
```

/usr/bin/ld: inline2.o:(.data+0x0): multiple definition of `g0'; inline1.o:(.data+0x0): first defined here

```
#include <iostream>
            int g0 = 20;
        inline int g1 = 21;
  static inline int g2 = 22;
  extern inline int g3 = 23;

void f()
{
  std::cout << "&g1 = " << g1 << ", " << &g1 << '\n';
  std::cout << "&g2 = " << g2 << ", " << &g2 << '\n';
  std::cout << "&g3 = " << g3 << ", " << &g3 << '\n';
}
```

Inline variables C++17

```
#include <iostream>
void f();
            int g0 = 10;
        inline int g1 = 11;
  static inline int g2 = 12;
  extern inline int g3 = 13;

int main()
{
  std::cout << "&g1 = " << g1 << ", " << &g1 << '\n';
  std::cout << "&g2 = " << g2 << ", " << &g2 << '\n';
  std::cout << "&g3 = " << g3 << ", " << &g3 << '\n';
  f();
  return 0;
}
```

/usr/bin/ld: inline2.o:(.data+0x0): multiple definition of `g0'; inline1.o:(.data+0x0): first defined here

```
#include <iostream>
            extern int g0 = 20;
            inline int g1 = 21;
  static inline int g2 = 22;
  extern inline int g3 = 23;

void f()
{
  std::cout << "&g1 = " << g1 << ", " << &g1 << '\n';
  std::cout << "&g2 = " << g2 << ", " << &g2 << '\n';
  std::cout << "&g3 = " << g3 << ", " << &g3 << '\n';
}
```

Inline variables C++17

```
#include <iostream>
void f();
            int g0 = 10;
        inline int g1 = 11;
  static inline int g2 = 12;
  extern inline int g3 = 13;

int main()
{
  std::cout << "&g1 = " << g1 << ", " << &g1 << '\n';
  std::cout << "&g2 = " << g2 << ", " << &g2 << '\n';
  std::cout << "&g3 = " << g3 << ", " << &g3 << '\n';
  f();
  return 0;
}
```

```
#include <iostream>
         extern int g0;
         inline int g1 = 21;
  static inline int g2 = 22;
  extern inline int g3 = 23;

void f()
{
  std::cout << "&g1 = " << g1 << ", " << &g1 << '\n';
  std::cout << "&g2 = " << g2 << ", " << &g2 << '\n';
  std::cout << "&g3 = " << g3 << ", " << &g3 << '\n';
}
```

Inline variables C++17

```
#include <iostream>
void f();
            int g0 = 10;
        inline int g1 = 11;
  static inline int g2 = 12;
  extern inline int g3 = 13;

int main()
{
  std::cout << "&g1 = " << g1 << ", " << &g1 << '\n';
  std::cout << "&g2 = " << g2 << ", " << &g2 << '\n';
  std::cout << "&g3 = " << g3 << ", " << &g3 << '\n';
  f();
  return 0;
}
```

```
#include <iostream>
         extern int g0;
         inline int g1 = 21;
  static inline int g2 = 22;
  extern inline int g3 = 23;

void f()
{
  std::cout << "&g1 = " << g1 << ", " << &g1 << '\n';
  std::cout << "&g2 = " << g2 << ", " << &g2 << '\n';
  std::cout << "&g3 = " << g3 << ", " << &g3 << '\n';
}
```

```
&g0 = 10, 0x40406c
&g1 = 21, 0x404060
&g2 = 12, 0x404070
&g3 = 23, 0x404068
&g0 = 10, 0x40406c
&g1 = 21, 0x404060
&g2 = 22, 0x404064
&g3 = 23, 0x404068
```

Inline variables C++17

```
#include <iostream>
void f();
            int g0 = 10;
        inline int g1 = 11;
  static inline int g2 = 12;
  extern inline int g3 = 13;

int main()
{
  std::cout << "&g1 = " << g1 << ", " << &g1 << '\n';
  std::cout << "&g2 = " << g2 << ", " << &g2 << '\n';
  std::cout << "&g3 = " << g3 << ", " << &g3 << '\n';
  f();
  return 0;
}
```

```
#include <iostream>
        extern int g0;
        inline int g1 = 21;
  static inline int g2 = 22;
  extern inline int g3 = 23;

void f()
{
  std::cout << "&g1 = " << g1 << ", " << &g1 << '\n';
  std::cout << "&g2 = " << g2 << ", " << &g2 << '\n';
  std::cout << "&g3 = " << g3 << ", " << &g3 << '\n';
}
```

```
&g0 = 10, 0x40406c
&g1 = 11, 0x404060
&g2 = 12, 0x404070
&g3 = 13, 0x404068
&g0 = 10, 0x40406c
&g1 = 11, 0x404060
&g2 = 22, 0x404064
&g3 = 13, 0x404068
```

Linker symbols

```
$ readelf inline1.o

Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS inline1.cpp
     2: 0000000000000040    11 FUNC    LOCAL  DEFAULT    4 _GLOBAL__sub_I_inline1.cp
     3: 0000000000000004     4 OBJECT  LOCAL  DEFAULT    7 _ZL2g2
     4: 0000000000000000     1 OBJECT  LOCAL  DEFAULT    6 _ZStL8__ioinit
     5: 0000000000000000    59 FUNC    LOCAL  DEFAULT    4 __cxx_global_var_init
     6: 0000000000000000     0 SECTION LOCAL  DEFAULT    2
     7: 0000000000000000     0 SECTION LOCAL  DEFAULT    4
     8: 0000000000000000     0 SECTION LOCAL  DEFAULT    6
     9: 0000000000000000     0 SECTION LOCAL  DEFAULT    7
    10: 0000000000000000     0 SECTION LOCAL  DEFAULT    8
    11: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND _Z1fv
    12: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND _ZNSolsEPKv
    13: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND _ZNSolsEi
    14: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND _ZNSt8ios_base4InitC1Ev
    15: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND _ZNSt8ios_base4InitD1Ev
    16: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND _ZSt4cout
    17: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND _ZStlsISt11char_traitsIcE
    18: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND _ZStlsISt11char_traitsIcE
    19: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND __cxa_atexit
    20: 0000000000000000     0 NOTYPE  GLOBAL HIDDEN   UND __dso_handle
    21: 0000000000000000     4 OBJECT  GLOBAL DEFAULT    7 g0
    22: 0000000000000000     4 OBJECT  WEAK   DEFAULT   10 g1
    23: 0000000000000000     4 OBJECT  WEAK   DEFAULT   12 g3
    24: 0000000000000000   316 FUNC    GLOBAL DEFAULT    2 main
```

Static initialization/destruction

- Matt Godbolt cppcon 18 The Bits between the Bits https://youtu.be/dOfucXtyEsU

```
typedef void (*init_func)(int, char **, char **);

extern init_func __init_array_start[];
extern init_func __init_array_end[];

int __libc_csu_init(int argc, char **argv, char **envp) {
  const size_t size = __init_array_end - __init_array_start;
  for (size_t i = 0; i < size; i++)
    (*__init_array_start[i])(argc, argv, envp);
}
```

Static initialization/destruction

- Static objects inside translation unit constructed in a well-defined order
- No ordering between translation units
- Issues:
  - (1) Constructor of static refers other source's static
  - (2) Destruction order
- Lazy singleton solves (1)
- Schwartz counter solves (1+2)

Schwartz counter

```
// init.h
class InitMngr
{
public:
  InitMngr()  { if ( !count_++ ) init();    }
  ~InitMngr() { if ( !--count_ ) cleanup(); }
  void init();
  void cleanup();
private:
  static long count_;  // one per process
};
namespace { InitMngr initMngr; }   // one per file inclusion


// init.cpp
long InitMngr::count_ = 0;
void InitMngr::init()    {  /* initialization */ }
void InitMngr::cleanup() {  /* cleanup */        }
```